

LẬP TRÌNH PYTHON CHO MÁY HỌC

Từ Python cơ bản đến Machine Learning

Ngô Hoàng Dũng - Lê Hoàng Ân - Nguyễn Thị Hà - Lê Ngọc Ánh Dương

Ngày 14 tháng 11 năm 2026

Lời nói đầu

Giáo trình *Lập trình Python cho Máy học* được biên soạn cho sinh viên đại học năm thứ hai, thứ ba các ngành Công nghệ thông tin, Khoa học dữ liệu và các ngành kỹ thuật có sử dụng dữ liệu. Mục tiêu của giáo trình là giúp người học đi trọn con đường từ những dòng lệnh Python đầu tiên đến việc tự xây dựng, đánh giá và giải thích một mô hình học máy trên dữ liệu thực.

Kiến thức cần có. Người học không cần biết Python trước. Về toán, cần nắm: đại số tuyến tính (vector, ma trận, phép nhân ma trận), giải tích (đạo hàm, đạo hàm riêng, quy tắc dây chuyền) và xác suất thống kê cơ bản (trung bình, phương sai, phân phối chuẩn, công thức Bayes). Những công cụ toán sâu hơn như trị riêng, vector riêng hay phân tích giá trị suy biến được nhắc lại ngay tại chương sử dụng chúng.

Cấu trúc giáo trình

Giáo trình gồm 12 chương, chia thành bốn phần. Trình tự các chương được sắp theo **quan hệ phụ thuộc kiến thức**: mỗi khái niệm được định nghĩa trước khi được sử dụng. Bảng 1 liệt kê nội dung và các chương cần học trước của từng chương.

Vì sao các chương được sắp theo trình tự này

- **Công cụ trước, thuật toán sau.** Phần I trang bị đủ Python, lập trình hướng đối tượng, NumPy, Pandas và Matplotlib để người học đọc hiểu mọi đoạn mã ở các phần sau. Chương 3 giới thiệu sớm giao diện `fit`/`predict`, vì đó là cách mọi mô hình của scikit-learn vận hành.
- **Dữ liệu trước mô hình.** Mọi mô hình đều nhận đầu vào là một ma trận số, nên Phần II mở đầu bằng tiền xử lý dữ liệu. Chương 5 cũng là nơi định nghĩa học máy và thống nhất hệ thống ký hiệu dùng cho toàn bộ giáo trình.
- **Hồi quy trước phân lớp.** Hồi quy tuyến tính là bối cảnh đơn giản nhất để giới thiệu hàm mất mát, Gradient Descent, quá khớp và chính quy hóa. Phân lớp dùng lại toàn bộ bộ máy đó: hồi quy logistic chính là mô hình tuyến tính thêm hàm sigmoid và hàm mất mát entropy chéo.
- **Đánh giá ngay sau hai bài toán có giám sát.** Khi đã có trong tay các mô hình hồi quy và phân lớp, người học mới thấy rõ nhu cầu so sánh chúng một cách khách quan. Chương 8 hệ thống hóa cách chia dữ liệu, các độ đo và cách tìm siêu tham số. Từ Chương 9 trở đi, mọi mô hình đều được đánh giá và tinh chỉnh theo quy trình này.

Chương	Tên chương	Nội dung chính	Cần học trước
Phần I. Nền tảng lập trình Python			
1	Giới thiệu về Python	Lịch sử, cài đặt Python và Anaconda	—
2	Cú pháp Python cơ bản	Kiểu dữ liệu, toán tử, cấu trúc dữ liệu, vòng lặp, hàm	1
3	Lập trình hướng đối tượng	Lớp, kế thừa, giao diện `fit`/`predict`	2
4	NumPy, Pandas và Matplotlib	Mảng, vector hóa, bảng dữ liệu, trực quan hóa	2
Phần II. Dữ liệu và học có giám sát			
5	Tiền xử lý dữ liệu	Khái niệm học máy; làm sạch, mã hóa, chuẩn hóa, ngoại lai; tạo, rút trích và chọn đặc trưng	3, 4
6	Hồi quy	Bình phương tối thiểu, Gradient Descent, quá khớp, chính quy hóa	5
7	Phân lớp	KNN, Naive Bayes, hồi quy logistic, cây quyết định, SVM	6
8	Đánh giá và lựa chọn mô hình	Lỗi tổng quát hóa, kiểm định chéo, độ đo, tìm siêu tham số	6, 7
9	Học kết hợp	Bagging, Random Forest, AdaBoost, Gradient Boosting	7, 8
Phần III. Học không giám sát			
10	Giảm chiều dữ liệu	SVD, PCA, LDA, Kernel PCA, t-SNE	7, 8
11	Phân cụm	K-Means và Mean-shift, Fuzzy C-Means, phân cấp, DBSCAN	5, 10
Phần IV. Thực hành tổng hợp			
12	Thực hành tổng hợp	Tám bài thực hành, gồm Gradient Descent với PyTorch	5–11

Bảng 1: Nội dung và quan hệ phụ thuộc giữa các chương

- **Học kết hợp sau đánh giá.** Bagging giảm phương sai, boosting giảm độ chệch: hai khái niệm được định nghĩa chặt chẽ ở Chương 8. Random Forest được xây dựng trên cây quyết định của Chương 7.
- **Giảm chiều trước phân cụm.** Giảm chiều là chiếc cầu nối giữa hai loại học. LDA dùng nhãn lớp như một bộ phân lớp, còn PCA không dùng nhãn. Ngoài ra, PCA còn là công cụ để trực quan hóa kết quả phân cụm ở Chương 11.
- **Thực hành tổng hợp.** Chương 12 đặt các thuật toán vào quy trình làm việc hoàn chỉnh trên dữ liệu thực. Bài cuối cùng ứng dụng Gradient Descent: ví dụ tính tay ở Chương 6 và 7 được cài đặt lại và kiểm chứng bằng PyTorch.

Giảng viên có thể linh hoạt ở hai chỗ. Người học đã biết Python có thể đọc lướt Phần I. Chương 10 và Chương 11 chỉ phụ thuộc nhau ở phần trực quan hóa, nên có thể dạy theo thứ tự đảo ngược.

Cấu trúc của mỗi chương

Các chương về thuật toán được viết theo cùng một khung:

1. **Mục tiêu**: những gì người học làm được sau chương.
2. **Khởi nguồn**: bối cảnh lịch sử và câu hỏi mà phương pháp ra đời để trả lời.
3. **Lý thuyết và ví dụ tính tay**: công thức được minh họa trên bộ dữ liệu đủ nhỏ để tính bằng giấy bút.
4. **Mã nguồn Python**: cài đặt lại ví dụ tính tay, sau đó dùng thư viện trên dữ liệu thực. Mọi con số trong ví dụ tính tay đều được kiểm chứng bằng mã nguồn đi kèm.
5. **Tóm tắt, bài tập và tài liệu tham khảo**.

Quy ước trình bày

- **Ký hiệu toán học** được thống nhất ở Mục 5.1: n là số mẫu, d là số đặc trưng, $\mathbf{X}$ là ma trận dữ liệu, $\mathbf{y}$ là vector nhãn, $\mathbf{w}$ và b là trọng số và hệ số chặn, $\hat{y}$ là giá trị dự đoán.
- **Số thập phân** trong văn bản dùng dấu phẩy theo chuẩn tiếng Việt ($0{,}1$); trong mã nguồn dùng dấu chấm theo cú pháp Python (`0.1`).
- **Chú thích và chuỗi ký tự trong mã nguồn** được viết tiếng Việt không dấu, để mã chạy được trên mọi môi trường và không lỗi khi in.
- **Thuật ngữ** được dùng bằng tiếng Việt, kèm thuật ngữ tiếng Anh trong ngoặc ở lần xuất hiện đầu tiên.

Nhóm tác giả

Mục lục

Lời nói đầu i

I Nền tảng lập trình Python 1

1 Giới thiệu về Python 2
1.1 Python là gì? . . . 2
1.2 Lịch sử phát triển của Python . . . 2
1.2.1 Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991) . . . 2
1.2.2 Python 1.0 & Chuẩn hóa quản trị (1994 – 1999) . . . 2
1.2.3 Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000) . . . 3
1.2.4 Python 3.0 – "Cuộc đại tái thiết" (Tháng 12/2008) . . . 3
1.2.5 Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay) . . . 3
1.3 Guido van Rossum – tác giả của Python . . . 3
1.4 Tên gọi Python . . . 3
1.5 Hướng dẫn cài đặt Python và Anaconda . . . 3
1.5.1 Cài đặt Python . . . 3
1.5.2 Cài đặt Anaconda/Miniconda . . . 4
1.5.3 Tạo môi trường Conda . . . 5
1.5.4 Làm việc với Jupyter Notebook . . . 5
1.6 Tóm tắt chương . . . 5
1.7 Bài tập . . . 6
Tài liệu tham khảo của chương . . . 6

2 Cú pháp Python cơ bản 7
2.1 Biến và Kiểu dữ liệu . . . 7
2.1.1 Các kiểu dữ liệu cơ bản . . . 7
2.1.2 Chuyển đổi kiểu và định dạng chuỗi . . . 8
2.2 Toán tử trong Python . . . 8
2.2.1 Toán tử cơ bản . . . 9
2.2.2 Toán tử gán mở rộng . . . 9
2.2.3 Toán tử so sánh và toán tử logic . . . 9
2.3 Hàm toán học trong Python . . . 10
2.3.1 Các hằng số toán học . . . 10
2.3.2 Các hàm toán học thường dùng . . . 10
2.4 Cấu trúc dữ liệu nâng cao . . . 10
2.4.1 List (Danh sách) . . . 11
2.4.2 Tuple (Bộ) . . . 11

2.4.3 Dictionary (Từ điển) . . . 11
2.4.4 Set (Tập hợp) . . . 12
2.5 Cấu trúc điều khiển và Vòng lặp . . . 12
2.5.1 Câu lệnh điều kiện if-else . . . 12
2.5.2 Vòng lặp for . . . 13
2.5.3 Vòng lặp while, break và continue . . . 13
2.5.4 List comprehension . . . 14
2.6 Hàm . . . 14
2.6.1 Định nghĩa hàm . . . 15
2.6.2 Tham số mặc định và tham số từ khóa . . . 15
2.6.3 Trả về nhiều giá trị . . . 15
2.6.4 Hàm lambda . . . 16
2.7 Module và thư viện . . . 16
2.8 Tóm tắt chương . . . 16
2.9 Bài tập . . . 17
Tài liệu tham khảo của chương . . . 17

3 Lập trình hướng đối tượng trong Python 19
3.1 Lớp và Đối tượng (Class and Object) . . . 19
3.2 Các phương thức đặc biệt (Magic Methods) . . . 19
3.2.1 Phương thức `__init__` và `__call__` . . . 19
3.3 Tính Kế thừa (Inheritance) . . . 20
3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters) . . . 21
3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn . . . 21
3.6 Mẫu thiết kế Pipeline . . . 22
3.7 Tóm tắt chương . . . 22
3.8 Bài tập . . . 23

4 Xử lý và trực quan hóa dữ liệu (NumPy, Pandas, Matplotlib) 24
4.1 Khởi nguồn của NumPy và Pandas . . . 24
4.2 Thư viện NumPy – Tính toán khoa học . . . 25
4.2.1 Khởi tạo mảng và kiểm tra thuộc tính . . . 25
4.2.2 Thêm, xóa và sửa phần tử . . . 25
4.2.3 Slicing và Boolean Indexing . . . 26
4.2.4 Vector hóa, broadcasting và tham số axis . . . 26
4.2.5 Đại số tuyến tính . . . 27
4.3 Thư viện Pandas – Xử lý và phân tích dữ liệu . . . 28
4.3.1 Tạo và đọc dữ liệu . . . 28
4.3.2 Truy xuất và lọc dữ liệu . . . 28
4.3.3 Dữ liệu thiếu và thống kê mô tả . . . 29
4.3.4 Thay thế và áp dụng hàm . . . 29
4.3.5 Gom nhóm và gộp dữ liệu . . . 30
4.4 Trực quan hóa dữ liệu với Matplotlib . . . 30
4.4.1 Figure, Axes và hai cách viết . . . 30
4.4.2 Bốn biểu đồ cơ bản . . . 31
4.5 SciPy: thống kê và tối ưu hóa . . . 33

4.6 Tóm tắt chương . . . 34
4.7 Bài tập . . . 34
Tài liệu tham khảo của chương . . . 35

II Dữ liệu và học có giám sát 36

5 Tiền xử lý dữ liệu (Data Preprocessing) 37
5.1 Học máy: định nghĩa, phân loại và hệ thống ký hiệu . . . 38
5.1.1 Định nghĩa học máy . . . 38
5.1.2 Ba loại học máy . . . 39
5.1.3 Thuật ngữ và ký hiệu . . . 39
5.1.4 Lộ trình xây dựng một hệ thống học máy . . . 40
5.2 Khởi nguồn và vai trò của tiền xử lý dữ liệu . . . 41
5.2.1 "Rác vào, rác ra" . . . 41
5.2.2 Tiền xử lý trong quy trình khai phá tri thức (KDD) . . . 41
5.2.3 Phân loại các tác vụ tiền xử lý . . . 43
5.2.4 Thư viện scikit-learn và giao diện `fit` / `transform` . . . 44
5.3 Các kiểu dữ liệu và lựa chọn phép biến đổi . . . 44
5.4 Xử lý dữ liệu thiếu (Missing Values) . . . 45
5.4.1 Cơ chế sinh dữ liệu thiếu . . . 45
5.4.2 Simple Imputer: trung bình, trung vị, giá trị phổ biến . . . 46
5.4.3 KNN Imputer và khoảng cách nan-Euclid . . . 47
5.4.4 Iterative Imputer . . . 49
5.5 Mã hóa dữ liệu phân loại . . . 51
5.5.1 Label Encoder . . . 51
5.5.2 One Hot Encoder . . . 53
5.6 Chuẩn hóa thang đo đặc trưng (Feature Scaling) . . . 56
5.6.1 Vì sao cần chuẩn hóa thang đo? . . . 56
5.6.2 Min Max Scaler . . . 56
5.6.3 Max Abs Scaler . . . 56
5.6.4 Standard Scaler (chuẩn hóa z-score) . . . 57
5.6.5 Robust Scaler . . . 57
5.6.6 Ví dụ so sánh bốn bộ chuẩn hóa . . . 57
5.7 Phát hiện ngoại lai (Outlier Detection) . . . 58
5.7.1 Phương pháp một biến: z-score và quy tắc IQR . . . 59
5.7.2 Phương pháp nhiều biến: Local Outlier Factor (LOF) . . . 59
5.8 Kỹ thuật tạo đặc trưng (Feature Engineering) . . . 61
5.8.1 Nhị phân hóa và rời rạc hóa . . . 62
5.8.2 Biến đổi thống kê: logarit và Box-Cox . . . 62
5.8.3 Đặc trưng tương tác và đa thức . . . 63
5.8.4 Đặc trưng thời gian . . . 64
5.9 Rút trích đặc trưng (Feature Extraction) . . . 65
5.9.1 Mô hình túi từ và Count Vectorizer . . . 65
5.9.2 TF-IDF . . . 67
5.10 Lựa chọn đặc trưng (Feature Selection) . . . 70
5.10.1 Phương pháp lọc . . . 70

5.10.2 Phương pháp bao gói: lựa chọn tuần tự lùi (SBS) . . . 71
5.10.3 Phương pháp nhúng: chính quy hóa L1 và rừng ngẫu nhiên . . . 72
5.10.4 Lựa chọn đặc trưng phải nằm trong quy trình kiểm định . . . 74
5.11 Dữ liệu mất cân bằng và SMOTE . . . 74
5.12 Kết hợp các bước: Pipeline và ColumnTransformer . . . 76
5.13 Nghiên cứu tình huống: bộ dữ liệu Ames Housing . . . 77
5.13.1 Giới thiệu bộ dữ liệu . . . 77
5.13.2 Những "cái bẫy" khi đọc dữ liệu . . . 77
5.13.3 Đọc và khảo sát dữ liệu . . . 78
5.13.4 Xây dựng quy trình tiền xử lý hoàn chỉnh . . . 79
5.14 Tóm tắt chương . . . 80
5.15 Bài tập . . . 81
Tài liệu tham khảo của chương . . . 83

6 Hồi quy (Regression) 86
6.1 Khởi nguồn của hồi quy . . . 86
6.1.1 Phương pháp bình phương tối thiểu . . . 86
6.1.2 Vì sao gọi là "hồi quy"? . . . 87
6.2 Hồi quy tuyến tính đơn biến . . . 87
6.2.1 Hàm mất mát và nghiệm bình phương tối thiểu . . . 88
6.3 Hồi quy tuyến tính đa biến . . . 89
6.4 Gradient Descent . . . 90
6.4.1 Ba biến thể: batch, stochastic và mini-batch . . . 92
6.4.2 Ví dụ tính tay: mini-batch Gradient Descent với hai đặc trưng . . . 92
6.4.3 Đánh giá mô hình sau mỗi lần cập nhật . . . 94
6.5 Hồi quy đa thức, quá khớp và chính quy hóa . . . 96
6.5.1 Hồi quy đa thức . . . 96
6.5.2 Quá khớp và thiếu khớp . . . 97
6.5.3 Ridge và Lasso . . . 97
6.6 Hồi quy bền vững và phân tích phần dư . . . 97
6.6.1 RANSAC . . . 98
6.6.2 Hàm mất mát Huber . . . 98
6.6.3 Phân tích phần dư . . . 100
6.7 Hồi quy cho chuỗi thời gian . . . 101
6.8 Nghiên cứu tình huống: dự đoán giá nhà Ames . . . 103
6.9 Tóm tắt chương . . . 104
6.10 Bài tập . . . 105
Tài liệu tham khảo của chương . . . 105

7 Phân lớp (Classification) 107
7.1 Bài toán phân lớp . . . 107
7.2 K láng giềng gần nhất (KNN) . . . 108
7.2.1 KNN có trọng số . . . 109
7.2.2 KNN cho bài toán hồi quy . . . 110
7.3 Naive Bayes . . . 111
7.4 Hồi quy logistic (Logistic Regression) . . . 112

7.4.1 Suy ra hàm mất mát từ nguyên lý hợp lý cực đại . . . 113
7.4.2 Ví dụ: mini-batch Gradient Descent cho phân lớp . . . 114
7.4.3 Chính quy hóa và tham số C . . . 116
7.5 Cây quyết định (Decision Tree) . . . 118
7.5.1 Độ đo độ hỗn tạp . . . 118
7.6 Máy vector hỗ trợ (SVM) . . . 120
7.6.1 Lề cực đại . . . 120
7.6.2 Lề mềm và tham số C . . . 120
7.6.3 Thủ thuật nhân . . . 120
7.7 So sánh các thuật toán . . . 121
7.8 Tóm tắt chương . . . 122
7.9 Bài tập . . . 123
Tài liệu tham khảo của chương . . . 123

8 Đánh giá và lựa chọn mô hình (Validation) 125
8.1 Lỗi tổng quát hóa . . . 125
8.2 Chiến lược chia dữ liệu . . . 126
8.2.1 Hold-out: train / validation / test . . . 126
8.2.2 Kiểm định chéo k-fold . . . 127
8.3 Độ đo cho bài toán phân lớp . . . 128
8.3.1 Ma trận nhầm lẫn . . . 128
8.3.2 Đường ROC và AUC . . . 129
8.4 Độ đo cho bài toán hồi quy . . . 131
8.5 Tìm siêu tham số . . . 131
8.5.1 Đường cong học và đường cong kiểm định . . . 133
8.6 Những sai lầm thường gặp . . . 134
8.7 Tóm tắt chương . . . 134
8.8 Bài tập . . . 135
Tài liệu tham khảo của chương . . . 135

9 Học kết hợp (Ensemble Learning) 137
9.1 Trí tuệ đám đông . . . 137
9.1.1 Độ chệch và phương sai . . . 138
9.2 Bagging và Random Forest . . . 138
9.2.1 Bagging . . . 138
9.2.2 Random Forest . . . 139
9.3 AdaBoost . . . 140
9.4 Gradient Boosting và XGBoost . . . 141
9.4.1 Gradient Boosting . . . 141
9.4.2 XGBoost . . . 142
9.5 Voting và Stacking . . . 143
9.6 Tổng kết so sánh . . . 144
9.7 Tóm tắt chương . . . 144
9.8 Bài tập . . . 144
Tài liệu tham khảo của chương . . . 145

III Học không giám sát 146

10 Giảm chiều dữ liệu (Dimensionality Reduction) 147
10.1 Lời nguyền số chiều . . . 147
10.2 Kiến thức nền: phương sai, hiệp phương sai và trị riêng . . . 148
10.2.1 Phương sai . . . 148
10.2.2 Hiệp phương sai và hệ số tương quan . . . 148
10.2.3 Trị riêng và vector riêng . . . 149
10.3 Phân tích giá trị suy biến (SVD) . . . 150
10.3.1 Lịch sử . . . 150
10.3.2 Tính SVD thông qua $\mathbf{X}^\top\mathbf{X}$ . . . 151
10.3.3 Ứng dụng: nén ảnh và TruncatedSVD . . . 152
10.4 Phân tích thành phần chính (PCA) . . . 154
10.4.1 Lịch sử và ý tưởng . . . 154
10.4.2 Chọn số thành phần . . . 157
10.4.3 PCA trong pipeline học máy . . . 158
10.5 Phân tích biệt thức tuyến tính (LDA) . . . 159
10.6 Giảm chiều phi tuyến: Kernel PCA và t-SNE . . . 161
10.6.1 Kernel PCA . . . 161
10.6.2 t-SNE . . . 162
10.7 Tóm tắt chương . . . 163
10.8 Bài tập . . . 164
Tài liệu tham khảo của chương . . . 165

11 Phân cụm (Clustering) 167
11.1 Bài toán phân cụm . . . 167
11.2 Phân cụm dựa trên tâm: K-Means và Mean-shift . . . 168
11.2.1 Thuật toán K-Means . . . 168
11.2.2 Chọn số cụm K . . . 170
11.2.3 Mean-shift: dịch chuyển về đỉnh mật độ . . . 171
11.3 Phân cụm mềm: Fuzzy C-Means . . . 172
11.4 Phân cụm phân cấp (Hierarchical Clustering) . . . 174
11.5 DBSCAN . . . 175
11.6 So sánh và ứng dụng . . . 177
11.7 Tóm tắt chương . . . 179
11.8 Bài tập . . . 179
Tài liệu tham khảo của chương . . . 180

IV Thực hành tổng hợp 181

12 Thực hành tổng hợp (Labs) 182
12.1 Lab 1 – Hồi quy giá nhà Ames Housing . . . 183
12.1.1 Đọc dữ liệu và tiền xử lý . . . 183
12.1.2 Khảo sát tương quan . . . 183
12.1.3 Huấn luyện và so sánh mô hình . . . 185

12.1.4 Mức độ quan trọng của đặc trưng . . . 187
12.2 Lab 2 – Dự đoán điểm tín dụng: khi dữ liệu không có thông tin . . . 188
12.3 Lab 3 – Nội suy, RobustScaler và tự cài đặt SVD, PCA . . . 189
12.3.1 Nội suy tuyến tính giá trị thiếu . . . 190
12.3.2 RobustScaler tính tay . . . 190
12.3.3 Tự cài đặt SVD: một lỗi chỉ số tinh vi . . . 190
12.3.4 Tự cài đặt PCA: hai lỗi chồng lên nhau . . . 191
12.4 Lab 4 – Dự báo giá cổ phiếu VNM: ảo ảnh $R^2 = 0{,}998$ . . . 191
12.4.1 Đánh giá lại: chia theo thời gian và so sánh với mô hình cơ sở . . . 192
12.5 Lab 5 – Nhận dạng chữ cái viết tay từ điểm ảnh . . . 194
12.5.1 Từ ảnh đến ma trận đặc trưng . . . 194
12.5.2 Kết quả . . . 195
12.6 Lab 6 – Nhận dạng chữ số viết tay . . . 196
12.7 Lab 7 – Phân tích cảm xúc bình luận tiếng Việt . . . 198
12.8 Lab 8 – Ứng dụng Gradient Descent: từ tính tay đến PyTorch . . . 200
12.8.1 Tensor và autograd: kiểm chứng ví dụ tính tay . . . 200
12.8.2 Từ vòng lặp thủ công đến `torch.optim` . . . 203
12.9 Tóm tắt chương . . . 203
12.10 Bài tập . . . 204
Tài liệu tham khảo của chương . . . 205

Phần I

Nền tảng lập trình Python

Chương 1: Giới thiệu về Python

1.1 Python là gì?

Python là một ngôn ngữ lập trình được sử dụng rộng rãi trong các ứng dụng web, phát triển phần mềm, khoa học dữ liệu và máy học (ML). Các nhà phát triển sử dụng Python vì nó hiệu quả, dễ học và có thể chạy trên nhiều nền tảng khác nhau. Phần mềm Python được tải xuống miễn phí, tích hợp tốt với tất cả các loại hệ thống và tăng tốc độ phát triển. (Nguồn: https://aws.amazon.com/vi/what-is/python/)

Python hỗ trợ nhiều mô hình lập trình, trong đó có lập trình thủ tục, lập trình hướng đối tượng và lập trình hàm.

Một chương trình Python đơn giản:

```
print("Hello, Python!")
```

Cú pháp của Python có xu hướng sử dụng khoảng trắng để thể hiện cấu trúc của chương trình. Ví dụ:

```
age = 20
if age >= 18:
    print("Adult")
else:
    print("Under 18")
```

Cách tổ chức này giúp mã nguồn Python có cấu trúc rõ ràng và dễ đọc.

1.2 Lịch sử phát triển của Python

1.2.1 Khởi nguồn & Phiên bản thử nghiệm (1989 – 1991)

Vào cuối năm 1989, khi đang làm việc tại Viện Nghiên cứu Quốc gia về Toán học và Khoa học Máy tính (CWI) ở Hà Lan, Guido van Rossum quyết định viết một ngôn ngữ lập trình mới trong kỳ nghỉ Giáng sinh để làm dự án phụ. Ông muốn tạo ra một ngôn ngữ kết hợp tính đơn giản của ngôn ngữ ABC với khả năng tương tác tốt với hệ điều hành Amoeba.

Tháng 2/1991, Guido đăng tải mã nguồn lên nhóm tin alt.sources với phiên bản Python 0.9.0. Dù chỉ là bản phôi thai, nó đã có những nền móng rất hiện đại: Mô hình OOP, Hệ thống xử lý ngoại lệ, Các kiểu dữ liệu cốt lõi (list, dict, str) và Hệ thống module.

1.2.2 Python 1.0 & Chuẩn hóa quản trị (1994 – 1999)

Tháng 1/1994, Python 1.0 chính thức ra mắt, tích hợp mạnh mẽ các công cụ của Lập trình chức năng (Functional Programming) như lambda, map, filter và reduce.

Năm 1995 (Python 1.2), Guido tiếp tục phát triển Python tại Tập đoàn Sáng kiến Nghiên cứu Quốc gia (CNRI) ở Mỹ, bổ sung khả năng truyền tham số theo tên (Keyword arguments). Trong giai đoạn này, Guido được cộng đồng trao danh hiệu BDFL (Benevolent Dictator for Life).

1.2.3 Python 2.0 & Sự trỗi dậy của Cộng đồng (Tháng 10/2000)

Python 2.0 đánh dấu sự chuyển dịch lớn sang mô hình phát triển mã nguồn mở hoàn toàn. Các tính năng đột phá bao gồm: List Comprehension, Garbage Collector (GC), và ra đời quy trình PEP (Python Enhancement Proposal). Phiên bản Python 2.7 (2010) trở thành phiên bản ổn định nhất của dòng Python 2.

1.2.4 Python 3.0 – ”Cuộc đại tái thiết” (Tháng 12/2008)

Sau nhiều năm phát triển, Python 2 bị tích tụ nhiều lỗi thiết kế. Guido quyết định phát hành Python 3.0 với đặc điểm cốt lõi là Không tương thích ngược (Backwards Incompatible) với Python 2. Những cải tiến quan trọng bao gồm:

- Đồng bộ Unicode cho mọi chuỗi ký tự.
- Thay đổi câu lệnh `print` thành hàm `print()`.
- Tối ưu hóa bộ nhớ bằng cách chuyển các hàm như `range()`, `zip()`, `map()` sang dạng trả về Iterator.

1.2.5 Kỷ nguyên AI/Machine Learning & Khai tử Python 2 (2015 – Nay)

Cú pháp đơn giản giúp Python trở thành lựa chọn hàng đầu trong Khoa học dữ liệu và AI với các thư viện như NumPy, Pandas, Scikit-Learn, TensorFlow, PyTorch.

Ngày 01/01/2020, Python 2 chính thức bị khai tử (EOL), đưa Python 3 trở thành tiêu chuẩn duy nhất. Giai đoạn hiện tại (Python 3.8 – 3.13+) tập trung vào bổ sung các toán tử mới và dự án Faster CPython nhằm tối ưu hóa hiệu năng.

1.3 Guido van Rossum – tác giả của Python

Guido van Rossum là một nhà khoa học máy tính người Hà Lan. Ông là người giữ quyền quyết định tối cao về định hướng kiến trúc của ngôn ngữ (BDFL) cho đến khi từ chức vào năm 2018.

1.4 Tên gọi Python

Tên gọi ”Python” không xuất phát từ loài trăn, mà lấy cảm hứng từ chương trình hài kịch yêu thích của ông trên đài BBC: ”Monty Python’s Flying Circus”.

1.5 Hướng dẫn cài đặt Python và Anaconda

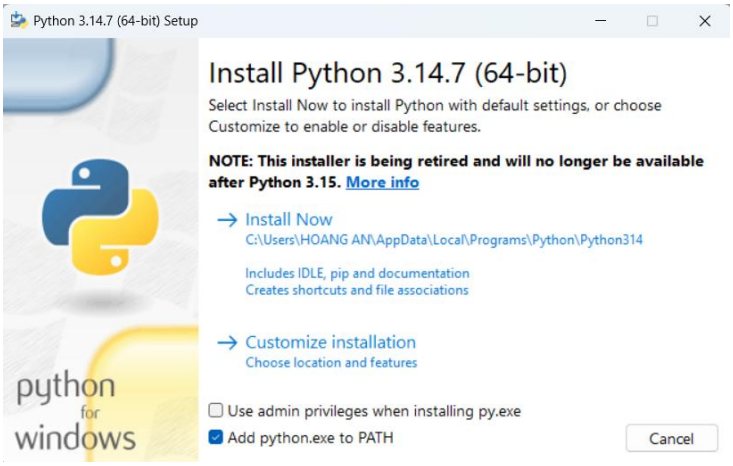
1.5.1 Cài đặt Python

1. Truy cập trang chủ Python: https://www.python.org/downloads/



Hình 1.1: Guido van Rossum - Tác giả sáng lập Python. (Nguồn: Sauria Associates, LLC / FlickrVision)

2. Tải phiên bản cài đặt (Ví dụ: Python 3.14.7 cho Windows).
3. Mở file cài đặt. **Lưu ý quan trọng:** Nhớ tick vào ô ”Add python.exe to PATH” trước khi bấm ”Install Now”.
4. Bấm ”Close” khi cài đặt thành công. Có thể chọn ”Disable path length limit” nếu cần.



Hình 1.2: Tick chọn Add python.exe to PATH trong quá trình cài đặt

1.5.2 Cài đặt Anaconda/Miniconda

Anaconda là một nền tảng phân phối Python và R phục vụ cho khoa học dữ liệu.

1. Truy cập trang tải Anaconda: `https://www.anaconda.com/download`
2. Tải bản cài đặt Anaconda Distribution hoặc Miniconda.
3. Mở file cài đặt, chọn ”Just Me”, chọn đường dẫn lưu (ví dụ: `C:\miniconda3`).

4. Trong phần Advanced Installation Options, có thể tick chọn ”Register Miniconda3 as my default Python”.

5. Nhấn ”Install” và đợi quá trình cài đặt hoàn tất.

1.5.3 Tạo môi trường Conda

Sau khi cài đặt Anaconda/Miniconda, mở Anaconda Prompt và sử dụng lệnh sau để tạo một môi trường ảo (ví dụ tên là `ai`) với phiên bản Python 3.14:

```
conda create -n ai python=3.14 anaconda
```

Xác nhận tiến trình cài đặt các gói bằng cách gõ y (yes) khi được yêu cầu. Sau đó kích hoạt môi trường bằng lệnh `conda activate ai`. Mọi thư viện cài thêm từ lúc này chỉ thuộc về môi trường `ai`, nhờ vậy các dự án khác nhau không xung đột phiên bản thư viện với nhau.

1.5.4 Làm việc với Jupyter Notebook

Các ví dụ và bài thực hành của giáo trình được viết dưới dạng **notebook** (tệp `.ipynb`): một tài liệu tương tác gồm nhiều **ô** (cell), trong đó ô mã chứa lệnh Python, còn ô văn bản (Markdown) chứa phần giải thích. Jupyter bắt nguồn từ dự án IPython do Fernando Pérez khởi xướng năm 2001 và trở thành Project Jupyter năm 2014 (Kluyver và cộng sự, 2016). Notebook cho phép chạy từng đoạn mã và xem ngay kết quả, kể cả biểu đồ, ở bên dưới. Vì vậy notebook rất phù hợp để khám phá dữ liệu và thử nghiệm mô hình.

Listing 1.1: Mở JupyterLab trong môi trường ai

```
conda activate ai
jupyter lab
```

Lệnh `jupyter lab` mở giao diện JupyterLab trên trình duyệt. Một số điểm cần nắm:

- Tổ hợp phím `Shift + Enter` chạy ô hiện tại và chuyển xuống ô kế tiếp.
- Biến tạo ở một ô được giữ trong bộ nhớ của **kernel** và dùng được ở các ô sau, nên thứ tự chạy các ô quyết định kết quả. Trước khi nộp bài, hãy chọn *Restart Kernel and Run All Cells* để chắc chắn notebook chạy đúng từ đầu đến cuối.
- Visual Studio Code và Google Colab cũng mở được tệp `.ipynb` với cách dùng tương tự.

1.6 Tóm tắt chương

Trong chương này, chúng ta đã tìm hiểu:

- Khái niệm cơ bản về Python và ứng dụng của nó.
- Lịch sử hình thành chi tiết qua từng giai đoạn (từ Python 0.9.0 đến Python 3.x).
- Tiểu sử của tác giả Guido van Rossum và nguồn gốc tên gọi thú vị của Python.
- Các bước cài đặt môi trường lập trình cơ bản với Python và Anaconda.
- Cách tạo, kích hoạt môi trường Conda và làm việc với Jupyter Notebook.

1.7 Bài tập

1. Cài đặt Anaconda hoặc Miniconda, tạo và kích hoạt môi trường `ai`, rồi kiểm tra phiên bản Python bằng lệnh `python --version`.

2. Mở JupyterLab và tạo một notebook gồm một ô Markdown giới thiệu bản thân và một ô mã in ra lời chào. Lưu thành tệp `.ipynb`, khởi động lại kernel rồi chạy lại toàn bộ notebook.

3. Trong notebook, chạy lệnh `import this` để đọc "The Zen of Python". Chọn hai nguyên tắc và giải thích ý nghĩa của chúng bằng ví dụ cụ thể.

Tài liệu tham khảo của chương

1. Kluyver, T. et al. (2016). Jupyter Notebooks – a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing*, 87–90. IOS Press.

2. Python Software Foundation. *The Python Tutorial*. https://docs.python.org/3/tutorial/

Chương 2: Cú pháp Python cơ bản

Chương này trình bày các khái niệm nền tảng trong lập trình Python: biến, kiểu dữ liệu, toán tử, cấu trúc dữ liệu, cấu trúc điều khiển, vòng lặp và hàm. Đây là các kiến thức bắt buộc trước khi đi vào xử lý dữ liệu và Machine Learning, vì mọi đoạn mã ở các chương sau đều được ghép từ những thành phần này. Các ví dụ được chọn có chủ đích: mỗi ví dụ là hình thức sơ khai của một thao tác sẽ gặp lại trong học máy, như tính độ chính xác, lưu siêu tham số hay lặp Gradient Descent.
Sau khi học xong chương này, người học có khả năng:

- Khai báo biến, nhận biết và chuyển đổi các kiểu dữ liệu cơ bản; định dạng chuỗi bằng f-string.
- Sử dụng các toán tử số học, toán tử gán, toán tử so sánh và toán tử logic.
- Lựa chọn đúng cấu trúc dữ liệu: list, tuple, dictionary và set.
- Viết cấu trúc rẽ nhánh `if-elif-else`, vòng lặp `for`, `while` và list comprehension.
- Định nghĩa hàm có tham số mặc định, trả về nhiều giá trị; sử dụng hàm `lambda`.
- Nạp thư viện bằng `import` và nhận biết các tên viết tắt quy ước `np`, `pd`, `plt`.

2.1 Biến và Kiểu dữ liệu

Trong Python, biến được khởi tạo tự động khi gán giá trị. Python là ngôn ngữ *dynamically typed* (định kiểu động): kiểu của biến được xác định từ giá trị gán cho nó, không cần khai báo trước.

Định nghĩa 2.1 (Biến trong Python)**.** *Biến là tên gọi đại diện cho một vùng nhớ chứa dữ liệu. Giá trị của biến có thể thay đổi trong quá trình thực thi chương trình.*

2.1.1 Các kiểu dữ liệu cơ bản

Dựa trên nền tảng Python Basic, dưới đây là bảng tổng hợp các kiểu dữ liệu cốt lõi thường xuyên được sử dụng:

Tên	Kiểu dữ liệu	Ví dụ
`str`	Kiểu chuỗi (string)	`a = 'Thien Thanh'`
`int`	Kiểu số nguyên	`a = 79`
`float`	Kiểu số thực	`a = 7.6`
`NoneType`	Kiểu None	`a = None`
`bool`	Kiểu luận lý (True, False)	`a = True`
`list`	Kiểu danh sách	`a = [3, 2, 1, 5]`
`tuple`	Kiểu bộ	`a = (1, 5, 3)`
`dict`	Kiểu từ điển	`b = {'one': 1, 'four': 4}`
`set`	Kiểu tập hợp	`a = {1, 3, 5}`

Bảng 2.1: Các kiểu dữ liệu cơ bản trong Python

Để kiểm tra kiểu dữ liệu của một biến trong Python, ta sử dụng hàm `type()`. Mọi nội dung đứng sau dấu # trên một dòng là **chú thích**, Python bỏ qua khi chạy:

Listing 2.1: Kiểm tra kiểu dữ liệu

```
a = 7
print(type(a))  # Output: <class 'int'>
```

2.1.2 Chuyển đổi kiểu và định dạng chuỗi

Các hàm `int()`, `float()`, `str()`, `bool()` chuyển một giá trị sang kiểu tương ứng. Thao tác này rất thường gặp: dữ liệu đọc từ tệp văn bản luôn ở dạng chuỗi, phải chuyển sang số trước khi tính toán. Để in kết quả, cách gọn nhất là **f-string**: chuỗi có tiền tố `f`, trong đó mỗi biểu thức đặt trong cặp ngoặc nhọn được thay bằng giá trị của nó. Phần sau dấu hai chấm quy định cách hiển thị: `:.3f` là số thực với 3 chữ số thập phân, `:.2%` là phần trăm với 2 chữ số thập phân.

Listing 2.2: Chuyển đổi kiểu và f-string

```
s = "3.75"              # du lieu doc tu tep o dang chuoi
x = float(s)            # chuyen sang so thuc
n = int(x)              # cat bo phan thap phan
print(x + 1, n)         # 4.75 3

name = "KNN"
acc = 0.91234
print(f"Mo hinh {name}: do chinh xac = {acc:.2%}")
# Mo hinh KNN: do chinh xac = 91.23%
print(f"MSE = {0.123456:.3f}")      # MSE = 0.123
```

2.2 Toán tử trong Python

Toán tử là thành phần không thể thiếu để thực hiện các phép tính từ cơ bản đến phức tạp trong các thuật toán.

2.2.1 Toán tử cơ bản

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	`+`	`3 + 7`	`10`
Trừ	`-`	`7 - 2`	`5`
Nhân	`*`	`2 * 7`	`14`
Chia	`/`	`7 / 5`	`1.4`
Chia lấy phần nguyên	`//`	`10 // 4`	`2`
Chia lấy phần dư	`%`	`9 % 5`	`4`
Lấy mũ	`**`	`2 ** 3`	`8`

Bảng 2.2: Các toán tử số học cơ bản

2.2.2 Toán tử gán mở rộng

Để mã nguồn ngắn gọn hơn, Python hỗ trợ các toán tử gán mở rộng kết hợp phép tính và phép gán: `a += 3` tương đương `a = a + 3`.

Tên toán tử	Ký hiệu	Ví dụ	Kết quả
Cộng	`+=`	`a = 7; a += 3`	`10`
Trừ	`-=`	`b = 9; b -= 4`	`5`
Nhân	`*=`	`a = 2; a *= 7`	`14`
Chia	`/=`	`a = 7; a /= 5`	`1.4`
Chia lấy phần nguyên	`//=`	`a = 10; a //= 4`	`2`
Chia lấy phần dư	`%=`	`a = 9; a %= 5`	`4`
Lấy mũ	`**=`	`a = 2; a **= 3`	`8`

Bảng 2.3: Các toán tử gán mở rộng

2.2.3 Toán tử so sánh và toán tử logic

Toán tử so sánh trả về giá trị kiểu `bool`. Cần phân biệt dấu = (phép gán) với == (phép so sánh bằng). Toán tử logic `and`, `or`, `not` kết hợp nhiều điều kiện. Đây là nền tảng của cấu trúc rẽ nhánh ở Mục 2.5 và của phép lọc dữ liệu theo điều kiện ở Chương 4.

Ý nghĩa	Ký hiệu	Ví dụ	Kết quả
Bằng	`==`	`3 == 3.0`	`True`
Khác	`!=`	`2 != 3`	`True`
Lớn hơn, nhỏ hơn	`>`, `<`	`5 > 7`	`False`
Lớn hơn hoặc bằng, nhỏ hơn hoặc bằng	`>=`, `<=`	`4 <= 4`	`True`
Và (đúng khi cả hai đúng)	`and`	`True and False`	`False`
Hoặc (đúng khi ít nhất một vế đúng)	`or`	`True or False`	`True`
Phủ định	`not`	`not True`	`False`
Thuộc về	`in`	`2 in [1, 2, 3]`	`True`

Bảng 2.4: Các toán tử so sánh và toán tử logic

Listing 2.3: Kết hợp điều kiện bằng toán tử logic

```
acc, f1 = 0.92, 0.78
print(acc > 0.9 and f1 > 0.8)       # False
print(acc > 0.9 or f1 > 0.8)        # True
print(not acc > 0.9)                # False
print("svm" in ["knn", "svm"])      # True
```

2.3 Hàm toán học trong Python

Đối với các ứng dụng khoa học dữ liệu, thư viện `math` cung cấp các hằng số và hàm tính toán nâng cao. Để sử dụng, ta cần import thư viện (cơ chế `import` được trình bày ở Mục 2.7):

Listing 2.4: Import thư viện math

```
import math
```

2.3.1 Các hằng số toán học

- Số Euler e (cơ số của logarit tự nhiên): `math.e` $\approx 2{,}718$.
- Số π: `math.pi` $\approx 3{,}14159$.

2.3.2 Các hàm toán học thường dùng

Hàm (Toán học)	Mã lệnh Python	Ví dụ
$\ln x$	`math.log(x)`	`math.log(math.e)` cho `1.0`
$\log_2 x$	`math.log2(x)`	`math.log2(8)` cho `3.0`
$\log_{10} x$	`math.log10(x)`	`math.log10(1000)` cho `3.0`
e^x	`math.exp(x)`	`math.exp(0)` cho `1.0`
$\sqrt{x}$	`math.sqrt(x)`	`math.sqrt(16)` cho `4.0`
a^x	`math.pow(a, x)`	`math.pow(2, 3)` cho `8.0`

Bảng 2.5: Một số hàm toán học từ thư viện math

Các hàm của `math` chỉ nhận **một số**. Để áp dụng một hàm lên cả mảng hàng nghìn số, ta dùng các hàm cùng tên của NumPy như `np.log`, `np.exp` (Chương 4).

2.4 Cấu trúc dữ liệu nâng cao

Đối với Machine Learning, ngoài các kiểu số và chuỗi, ta thường sử dụng bốn cấu trúc dữ liệu có sẵn của Python: List, Tuple, Dictionary và Set. Bảng 2.6 ở cuối mục so sánh bốn cấu trúc này.

2.4.1 List (Danh sách)

List là tập hợp các phần tử có thứ tự và có thể thay đổi (mutable).

Listing 2.5: Thao tác cơ bản trên List

```
# Khoi tao list
features = [1.2, 3.4, 5.1, 2.8]

# Truy cap va thay doi phan tu
features[0] = 0.9
features.append(4.5)

print("List sau khi cap nhat:", features)
```

Chỉ số (index) của list bắt đầu từ 0; chỉ số âm đếm từ cuối. Cú pháp **cắt lát** (slicing) `a[i:j]` lấy các phần tử từ vị trí `i` đến **trước** vị trí `j`. Quy tắc này được giữ nguyên cho mảng NumPy và bảng Pandas ở Chương 4.

Listing 2.6: Chỉ số, cắt lát và các hàm dựng sẵn trên List

```
scores = [7, 9, 4, 10, 6]
print(scores[0], scores[-1])                    # 7 6
print(scores[1:4])                              # [9, 4, 10]
print(len(scores), sum(scores), max(scores))    # 5 36 10
print(sorted(scores))                           # [4, 6, 7, 9, 10]
```

2.4.2 Tuple (Bộ)

Tuple giống list nhưng **không thể thay đổi** (immutable) sau khi tạo. Tuple thường dùng để biểu diễn những giá trị đi cùng nhau và không được sửa, ví dụ kích thước của một bảng dữ liệu: thuộc tính `shape` của mảng NumPy chính là một tuple (n, d). Thao tác **tách tuple** (unpacking) gán từng phần tử cho từng biến.

Listing 2.7: Tuple và phép tách tuple

```
shape = (150, 4)        # (so mau, so dac trung) cua bo du lieu Iris
n, d = shape            # tach tuple
print(n, d)             # 150 4
# shape[0] = 200        # loi TypeError: tuple khong the thay doi
```

2.4.3 Dictionary (Từ điển)

Dictionary lưu các cặp **khóa – giá trị** (key – value) và truy xuất giá trị theo khóa thay vì theo vị trí. Trong học máy, dictionary được dùng để lưu bộ siêu tham số của mô hình, lưu lưới siêu tham số cần thử (Chương 8) hay lưu bảng ánh xạ nhãn chữ sang số (Chương 5).

Listing 2.8: Thao tác trên Dictionary

```
params = {"n_neighbors": 5, "weights": "uniform"}
params["n_neighbors"] = 7           # sua gia tri theo khoa
params["metric"] = "euclidean"      # them cap khoa - gia tri moi
print(params["n_neighbors"])        # 7
```

```
print(list(params.keys()))          # ['n_neighbors', 'weights', 'metric']
for key, value in params.items():   # duyet tung cap khoa - gia tri
    print(key, "=", value)
```

2.4.4 Set (Tập hợp)

Set là tập hợp các phần tử **không trùng lặp** và không có thứ tự. Set thường dùng để tìm các giá trị khác nhau, ví dụ các lớp xuất hiện trong vector nhãn, hoặc để tính giao, hợp giữa hai tập.

Listing 2.9: Tìm các giá trị khác nhau bằng Set

```
labels = ["spam", "ham", "spam", "ham", "spam"]
classes = set(labels)
print(len(classes), sorted(classes))    # 2 ['ham', 'spam']
print({1, 2, 3} & {2, 3, 4})            # {2, 3}   (phep giao)
```

Cấu trúc	Ký hiệu	Có thứ tự	Sửa được	Dùng trong học máy
List	`[ ]`	Có	Có	Danh sách đặc trưng, lịch sử mất mát theo epoch
Tuple	`( )`	Có	Không	Kích thước mảng (n, d), trả về nhiều giá trị
Dictionary	`{k: v}`	Có[1]	Có	Siêu tham số, ánh xạ nhãn
Set	`{ }`	Không	Có	Tập các lớp, tập từ vựng

Bảng 2.6: So sánh bốn cấu trúc dữ liệu có sẵn của Python

2.5 Cấu trúc điều khiển và Vòng lặp

Python dùng **thụt lề** (thường là 4 dấu cách) để xác định khối lệnh thuộc về một câu lệnh điều kiện, vòng lặp hay hàm, thay cho cặp ngoặc nhọn của C hay Java. Thụt lề sai sẽ gây lỗi `IndentationError` hoặc làm chương trình chạy sai logic.

2.5.1 Câu lệnh điều kiện if-else

Listing 2.10: Sử dụng cấu trúc điều kiện

```
score = 0.85

if score >= 0.8:
    print("Mo hinh dat do chinh xac cao")
else:
    print("Can can thiep hoac toi uu them")
```

Khi có nhiều hơn hai trường hợp, dùng thêm `elif`. Python kiểm tra lần lượt từ trên xuống và chỉ thực hiện khối lệnh của **điều kiện đúng đầu tiên**.

[1] Từ Python 3.7, dictionary giữ đúng thứ tự thêm vào của các khóa. Khóa không được trùng nhau.

Listing 2.11: Nhiều nhánh với if-elif-else

```
bmi = 23.5
if bmi < 18.5:
    group = "thieu can"
elif bmi < 25:
    group = "binh thuong"
elif bmi < 30:
    group = "thua can"
else:
    group = "beo phi"
print(group)        # binh thuong
```

Một chuỗi `if-elif` lồng nhau trên các đặc trưng chính là cách một **cây quyết định** đưa ra dự đoán. Khác biệt duy nhất là cây quyết định tự học các ngưỡng như 18,5 hay 25 từ dữ liệu (Chương 7).

2.5.2 Vòng lặp for

Vòng lặp `for` duyệt lần lượt từng phần tử của một dãy. Ba hàm thường đi kèm là: `range(n)` sinh dãy số $0, 1, \ldots, n-1$; `zip(a, b)` duyệt song song hai dãy; `enumerate(a)` trả về đồng thời chỉ số và giá trị.

Listing 2.12: Vòng lặp for với zip, range và enumerate

```
y_true = [1, 0, 1, 1, 0]
y_pred = [1, 0, 0, 1, 1]

correct = 0
for t, p in zip(y_true, y_pred):    # duyet song song hai list
    if t == p:
        correct += 1
print("Do chinh xac:", correct / len(y_true))     # Do chinh xac: 0.6

for epoch in range(3):              # epoch = 0, 1, 2
    print("Epoch", epoch)

for i, model in enumerate(["knn", "svm"]):  # chi so va gia tri
    print(i, model)                 # 0 knn, roi 1 svm
```

Đoạn mã đầu tiên tính **độ chính xác** (accuracy) của một bộ phân lớp: tỉ lệ mẫu có nhãn dự đoán trùng nhãn thật, ở đây là $3/5 = 0{,}6$. Độ đo này được dùng xuyên suốt Chương 7 và được phân tích kỹ ở Chương 8.

2.5.3 Vòng lặp while, break và continue

Vòng lặp `while` lặp lại **chừng nào điều kiện còn đúng**. Nó phù hợp khi chưa biết trước số lần lặp, ví dụ lặp một thuật toán tối ưu cho đến khi hội tụ. Đoạn mã dưới đây tìm điểm cực tiểu của hàm $f(w) = (w-3)^2$: mỗi bước dịch w ngược chiều đạo hàm $f'(w) = 2(w-3)$ một lượng tỉ lệ với hệ số $0{,}1$, và dừng khi bước dịch đủ nhỏ. Đây chính là thuật toán **Gradient Descent**, sẽ được trình bày đầy đủ ở Chương 6.

Listing 2.13: Vòng lặp while: lặp đến khi hội tụ

```
w, lr = 0.0, 0.1        # diem xuat phat va he so buoc
step = 1.0
n_iter = 0
while abs(step) > 1e-4:
    grad = 2 * (w - 3)  # dao ham cua f(w) = (w - 3)^2
    step = lr * grad
    w = w - step
    n_iter += 1
print(n_iter, round(w, 4))      # 40 2.9996
```

Lệnh `break` thoát khỏi vòng lặp ngay lập tức; lệnh `continue` bỏ qua phần còn lại của lượt hiện tại để sang lượt kế tiếp:

Listing 2.14: Điều khiển vòng lặp bằng break và continue

```
for k in [1, 3, 5, 7, 9]:
    if k == 7:
        break           # thoat vong lap
    if k == 3:
        continue        # bo qua k = 3
    print(k)            # in ra 1 roi 5
```

2.5.4 List comprehension

List comprehension tạo một list mới từ một dãy trong **một dòng lệnh**, có thể kèm điều kiện lọc. Cú pháp tổng quát: `[biểu thức for phần_tử in dãy if điều_kiện]`.

Listing 2.15: List comprehension

```
x = [1, 2, 3, 4]
print([v ** 2 for v in x])                  # [1, 4, 9, 16]
print([v for v in x if v % 2 == 0])         # [2, 4]
errors = [(t - p) ** 2 for t, p in zip([3, 5], [2.5, 6])]
print(errors)                               # [0.25, 1]
```

Vòng lặp của Python dễ viết nhưng chậm khi dữ liệu lớn. Ở Chương 4, phần lớn các vòng lặp trên sẽ được thay bằng phép toán **vector hóa** của NumPy, nhanh hơn hàng chục đến hàng trăm lần.

2.6 Hàm

Hàm (function) đóng gói một đoạn mã thành một khối có tên để dùng lại nhiều lần. Hàm được định nghĩa bằng từ khóa `def`, nhận các **tham số** và trả kết quả về bằng `return`. Chuỗi đặt ngay dưới dòng `def` là **docstring**, mô tả công dụng của hàm.

2.6.1 Định nghĩa hàm

Hàm dưới đây tính **sai số bình phương trung bình** (MSE), độ đo cơ bản của bài toán hồi quy ở Chương 6:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_i\right)^2.$$

Listing 2.16: Định nghĩa hàm tính MSE

```
def mse(y_true, y_pred):
    """Tinh sai so binh phuong trung binh."""
    total = 0
    for t, p in zip(y_true, y_pred):
        total += (t - p) ** 2
    return total / len(y_true)

print(mse([3, 5, 2], [2.5, 6, 2]))      # 0.4166666666666667
```

Kiểm tra bằng tay: $\left[(3-2{,}5)^2 + (5-6)^2 + (2-2)^2\right]/3 = (0{,}25 + 1 + 0)/3 \approx 0{,}4167$.

2.6.2 Tham số mặc định và tham số từ khóa

Tham số có thể được gán **giá trị mặc định**, khi đó người gọi hàm có thể bỏ qua nó. Khi gọi hàm, có thể truyền tham số theo **tên** (tham số từ khóa) thay vì theo vị trí. Mọi mô hình của scikit-learn đều nhận siêu tham số theo cách này, ví dụ `KNeighborsClassifier(n_neighbors=5)`.

Listing 2.17: Tham số mặc định và tham số từ khóa

```
def scale(x, factor=1.0, shift=0.0):
    return x * factor + shift

print(scale(10))            # 10.0   (dung gia tri mac dinh)
print(scale(10, 0.5))       # 5.0    (truyen theo vi tri)
print(scale(10, shift=-3))  # 7.0    (truyen theo ten)
```

2.6.3 Trả về nhiều giá trị

Khi `return` nhiều giá trị cách nhau bởi dấu phẩy, Python đóng gói chúng thành một tuple. Bên gọi hàm dùng phép tách tuple để nhận từng giá trị. Hàm `train_test_split` của scikit-learn trả về bốn giá trị theo đúng cách này.

Listing 2.18: Hàm trả về nhiều giá trị

```
def min_max(values):
    return min(values), max(values)

lo, hi = min_max([4, 8, 1, 6])
print(lo, hi)               # 1 8
```

2.6.4 Hàm lambda

Hàm `lambda` là hàm ẩn danh viết trên một dòng, thường dùng làm tham số cho một hàm khác. Ví dụ, khóa sắp xếp của `sorted`, hay hàm áp dụng lên từng phần tử của một cột dữ liệu bằng phương thức `apply` của Pandas (Chương 4).

Listing 2.19: Hàm lambda làm khóa sắp xếp

```
square = lambda v: v ** 2
print(square(4))                                    # 16
results = [("knn", 0.91), ("svm", 0.95), ("tree", 0.88)]
best = sorted(results, key=lambda r: r[1], reverse=True)[0]
print(best)                                         # ('svm', 0.95)
```

2.7 Module và thư viện

Một **module** là một tệp mã Python chứa các hàm, lớp và hằng số. Một **thư viện** (package) gồm nhiều module. Lệnh `import` nạp module vào chương trình theo ba cách:

Listing 2.20: Ba cách import

```
import math                     # nap ca module, goi qua ten module
from math import sqrt, pi       # chi nap mot so ten
import numpy as np              # dat ten viet tat theo quy uoc

print(math.floor(2.7), sqrt(16), round(pi, 2))   # 2 4.0 3.14
print(np.mean([1, 2, 3, 4]))                     # 2.5
```

Cộng đồng Python thống nhất một số tên viết tắt mà mọi tài liệu đều dùng. Giáo trình này tuân theo các quy ước đó:

Câu lệnh	Công dụng	Trình bày ở
`import numpy as np`	Tính toán trên mảng	Chương 4
`import pandas as pd`	Bảng dữ liệu	Chương 4
`import matplotlib.pyplot as plt`	Vẽ biểu đồ	Chương 4
`from sklearn... import ...`	Các mô hình học máy	Chương 5 trở đi
`import torch`	Tự động tính đạo hàm	Lab 8, Chương 12

Bảng 2.7: Tên viết tắt quy ước của các thư viện dùng trong giáo trình

Các thư viện này có sẵn trong bộ cài Anaconda (Chương 1). Nếu dùng bản Python cài riêng, có thể cài thêm bằng lệnh `pip install numpy pandas matplotlib scikit-learn` trong cửa sổ dòng lệnh.

2.8 Tóm tắt chương

Trong chương này, chúng ta đã nắm được:

- Cách khai báo biến, các kiểu dữ liệu cơ bản, chuyển đổi kiểu và định dạng chuỗi bằng f-string.
- Các toán tử số học, toán tử gán mở rộng, toán tử so sánh và toán tử logic.
- Cách sử dụng thư viện `math` cho các hàm toán học trên từng số.
- Bốn cấu trúc dữ liệu: List (có thứ tự, sửa được), Tuple (không sửa được), Dictionary (cặp khóa – giá trị), Set (không trùng lặp).
- Cấu trúc rẽ nhánh `if-elif-else`; vòng lặp `for` với `range`, `zip`, `enumerate`; vòng lặp `while` với `break`, `continue`; list comprehension.
- Định nghĩa hàm bằng `def`, tham số mặc định, tham số từ khóa, trả về nhiều giá trị và hàm `lambda`.
- Nạp thư viện bằng `import` và các tên viết tắt quy ước `np`, `pd`, `plt`.

2.9 Bài tập

1. Cho list điểm `[7.5, 8, 6, 9.5, 5]`. Chỉ dùng vòng lặp `for`, không dùng thư viện, hãy tính điểm trung bình và độ lệch chuẩn $s = \sqrt{\frac{1}{n}\sum_i (x_i - \bar{x})^2}$. In kết quả với hai chữ số thập phân bằng f-string.
2. Viết hàm `accuracy(y_true, y_pred)` trả về tỉ lệ dự đoán đúng. Kiểm tra với hai list ở ví dụ vòng lặp `for` (kết quả phải là $0{,}6$), rồi viết lại thân hàm bằng một dòng list comprehension.
3. Dùng dictionary đếm số lần xuất hiện của mỗi từ trong câu `"the cat and the hat and the bat"`. Đây là ý tưởng của phương pháp Count Vectorizer ở Chương 5.
4. Viết hàm `label_encode(values)` gán cho mỗi giá trị khác nhau một số nguyên theo thứ tự xuất hiện. Hàm trả về hai giá trị: list mã số và dictionary ánh xạ. Ví dụ: `["red", "green", "blue", "red"]` cho `[0, 1, 2, 0]` và `{"red": 0, "green": 1, "blue": 2}`.
5. Sửa vòng lặp `while` tìm cực tiểu của $f(w) = (w-3)^2$ để thử các hệ số bước $0{,}1$; $0{,}5$; $0{,}9$ và $1{,}1$. Thêm điều kiện dừng sau tối đa 1000 bước. Đếm số bước trong từng trường hợp. Với hệ số nào w không tiến về 3? Hãy giải thích bằng cách viết $w - 3$ sau mỗi bước theo $w - 3$ ở bước trước.
6. Dùng list comprehension tạo list bình phương của các số chẵn từ 0 đến 20. Viết lại bằng vòng lặp `for` thông thường và so sánh độ dài hai cách viết.

Tài liệu tham khảo của chương

1. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
2. Python Software Foundation. *The Python Tutorial*. https://docs.python.org/3/tutorial/

3. Sarkar, D., Bali, R., & Sharma, T. (2018). *Practical Machine Learning with Python*, Chương 2. Apress.

Chương 3: Lập trình hướng đối tượng trong Python

Lập trình hướng đối tượng (Object-Oriented Programming - OOP) là phương pháp luận lập trình đóng vai trò then chốt trong việc thiết kế các thư viện Machine Learning hiện đại như Scikit-Learn, PyTorch và TensorFlow. Chương này trình bày các khái niệm OOP cốt lõi và hướng dẫn cách ứng dụng OOP để tự đóng gói một mô hình Machine Learning.

3.1 Lớp và Đối tượng (Class and Object)

Định nghĩa 3.1 (Lớp và Đối tượng)**.** ***Lớp (Class)*** *là khuôn mẫu định nghĩa các thuộc tính (dữ liệu) và phương thức (hành vi).* ***Đối tượng (Object)*** *là một thể hiện cụ thể được khởi tạo từ lớp đó.*

Ví dụ 3.1. *Khai báo một lớp cơ bản lưu trữ mẫu dữ liệu:*

Listing 3.1: Khai báo Class và tạo Object trong Python

```
class DataSample:
    def __init__(self, feature_name, value):
        self.feature_name = feature_name  # Thuoc tinh
        self.value = value

    def display_info(self):               # Phuong thuc
        print(f"Feature: {self.feature_name}, Value: {self.value}")

# Khoi tao doi tuong
sample = DataSample(feature_name="Age", value=25)
sample.display_info()
```

3.2 Các phương thức đặc biệt (Magic Methods)

Các phương thức có cú pháp hai dấu gạch dưới (`__method__`) giúp tùy biến hành vi của đối tượng trong Python.

3.2.1 Phương thức `__init__` và `__call__`

- `__init__`: Hàm khởi tạo (constructor), tự động thực thi khi tạo mới một đối tượng.

– `__call__`: Cho phép gọi trực tiếp một đối tượng giống như một hàm, thường dùng khi thiết kế các Layer trong Neural Network.

Listing 3.2: Sử dụng phương thức __call__

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, x):
        return x * self.factor

# Su dung nhu mot ham
double = Multiplier(factor=2)
print(double(5))  # Output: 10
```

3.3 Tính Kế thừa (Inheritance)

Tính kế thừa cho phép tạo một lớp mới dựa trên lớp đã có, giúp tái sử dụng mã nguồn và chuẩn hóa giao diện của các bộ biến đổi dữ liệu (Transformers).

Listing 3.3: Kế thừa lớp BaseTransformer trong tiền xử lý dữ liệu

```
class BaseTransformer:
    def fit(self, arr):
        raise NotImplementedError("Phuong thuc fit() bat buoc phai duoc trien
    khai o lop con.")

    def transform(self, arr):
        raise NotImplementedError("Phuong thuc transform() bat buoc phai trien
    khai o lop con.")

    def fit_transform(self, arr):
        self.fit(arr)
        return self.transform(arr)

class MinMaxScaler(BaseTransformer):
    def fit(self, arr):
        self.min_ = min(arr)
        self.max_ = max(arr)
        return self

    def transform(self, arr):
        return [(x - self.min_) / (self.max_ - self.min_) for x in arr]

    def inverse_transform(self, arr):
        return [x * (self.max_ - self.min_) + self.min_ for x in arr]
```

3.4 Quản lý thuộc tính: Siêu tham số (Hyperparameters) và Tham số học được (Learned Parameters)

Trong thiết kế Machine Learning, ta cần phân biệt rõ giữa siêu tham số (cấu hình trước khi huấn luyện) và tham số học được (được sinh ra sau khi gọi hàm `fit`).

Listing 3.4: Bắt lỗi an toàn khi quản lý tham số học được

```
class MinMaxScaler(BaseTransformer):
    def __init__(self, feature_range=(0, 1)):
        self.feature_range = feature_range  # Hyperparameter

    def fit(self, arr):
        self.min = float(min(arr))          # Learned Parameter
        self.max = float(max(arr))          # Learned Parameter
        return self

    def transform(self, arr):
        # Kiem tra an toan: Dam bao mo hinh da duoc fit()
        if not hasattr(self, 'min') or not hasattr(self, 'max'):
            raise RuntimeError("Loi: Ban phai goi ham fit() truoc khi goi
   transform()!")

        low, high = self.feature_range
        std = [(x - self.min) / (self.max - self.min) for x in arr]
        return [x * (high - low) + low for x in std]
```

3.5 Thiết kế Custom Estimator chuẩn Scikit-Learn

Trọng tâm ứng dụng OOP trong Machine Learning là đóng gói thuật toán thành một Class có các giao diện chuẩn (`fit`, `predict`, `evaluate`). Dưới đây là triển khai mô hình Hồi quy tuyến tính đơn biến (Simple Linear Regression) bằng công thức đại số.

Listing 3.5: Đóng gói thuật toán Simple Linear Regression

```
class SimpleLinearRegression:
    def __init__(self):
        self.w0 = 0
        self.w1 = 0

    def fit(self, X, y):
        n = len(X)
        mean_x = sum(X) / n
        mean_y = sum(y) / n

        numerator = sum((X[i] - mean_x) * (y[i] - mean_y) for i in range(n))
        denominator = sum((X[i] - mean_x) ** 2 for i in range(n))

        if denominator == 0:
            raise ValueError("Bao loi: Phuong sai cua X bang 0, khong the tinh
   toan.")
```

```
16
17          self.w1 = numerator / denominator
18          self.w0 = mean_y - (self.w1 * mean_x)
19          return self
20
21      def predict(self, X):
22          return [(self.w1 * x_i + self.w0) for x_i in X]
23
24      def evaluate(self, X, y):
25          predictions = self.predict(X)
26          mse = sum((predictions[i] - y[i]) ** 2 for i in range(len(y))) / len(y)
27          return mse
```

3.6 Mẫu thiết kế Pipeline

Pipeline là lớp quản lý chuỗi các bước biến đổi dữ liệu nối tiếp nhau, cho phép tự động hóa toàn bộ quy trình tiền xử lý trước khi đưa vào mô hình dự đoán.

Listing 3.6: Triển khai cấu trúc Pipeline cơ bản

```
1  class Pipeline:
2      def __init__(self, steps):
3          self.steps = steps
4
5      def fit_transform(self, arr):
6          data = arr
7          for transformer in self.steps:
8              data = transformer.fit_transform(data)
9          return data
10
11     def transform(self, arr):
12         data = arr
13         for transformer in self.steps:
14             data = transformer.transform(data)
15         return data
```

3.7 Tóm tắt chương

- Khái niệm Lớp, Đối tượng và tư duy lập trình hướng đối tượng.
- Ứng dụng các Magic Methods (`__init__`, `__call__`) trong xây dựng Framework.
- Nguyên lý Kế thừa giúp tái sử dụng và quản lý mã nguồn linh hoạt.
- Đóng gói thuật toán Machine Learning thành Class theo giao diện tiêu chuẩn.
- Phân biệt rõ ràng siêu tham số và tham số học được thông qua các thuộc tính của đối tượng.

3.8 Bài tập

1. Viết lớp `MeanPredictor` theo giao diện của scikit-learn: `fit(X, y)` lưu trung bình của `y` vào thuộc tính `mean_`, còn `predict(X)` trả về một list gồm `len(X)` giá trị `mean_`. Đây là **mô hình cơ sở** (baseline) mà mọi mô hình hồi quy ở các chương sau phải vượt qua.

2. Viết lớp `MedianPredictor` kế thừa `MeanPredictor` và chỉ ghi đè phương thức `fit` để dùng trung vị. Giải thích vì sao không cần viết lại `predict`.

3. Theo quy ước của scikit-learn, tên của tham số học được kết thúc bằng dấu gạch dưới (`coef_`, `mean_`). Sửa lớp `MinMaxScaler` ở Mục "Quản lý thuộc tính" theo quy ước này, đồng thời xử lý trường hợp mọi giá trị bằng nhau (`max` bằng `min`) để tránh phép chia cho 0.

4. Trong mẫu thiết kế Pipeline, các bước biến đổi chỉ được `fit` trên dữ liệu huấn luyện rồi mới áp dụng cho dữ liệu kiểm tra. Giải thích lý do (gợi ý: hiện tượng rò rỉ dữ liệu ở Chương 5).

Chương 4: Xử lý và trực quan hóa dữ liệu (NumPy, Pandas, Matplotlib)

Trong khoa học dữ liệu và học máy, xử lý dữ liệu đóng vai trò nền tảng. Dữ liệu thực tế thường lộn xộn, thiếu sót hoặc không đúng định dạng. Để giải quyết vấn đề này, Python cung cấp một hệ sinh thái thư viện mạnh mẽ, trong đó nổi bật nhất là NumPy và Pandas. Chương này hướng dẫn cách dùng hai thư viện để biểu diễn, truy xuất, biến đổi và tổng hợp dữ liệu — những thao tác xuất hiện trong mọi chương sau của giáo trình. Phần cuối chương giới thiệu **Matplotlib** để trực quan hóa dữ liệu và **SciPy** cho các phép tính thống kê, tối ưu hóa. Đây là hai công cụ được dùng để vẽ hình và kiểm chứng kết quả ở các chương sau.
Sau khi học xong chương này, người học có khả năng:

- Giải thích vì sao mảng NumPy nhanh hơn list của Python; phân biệt *view* và *copy*.
- Khởi tạo, cắt, lọc và biến đổi mảng; vận dụng vector hóa, broadcasting và tham số `axis`.
- Giải hệ phương trình tuyến tính bằng `np.linalg`.
- Tạo, đọc và truy xuất `DataFrame` bằng `loc`, `iloc` và điều kiện lọc.
- Xử lý dữ liệu thiếu, biến đổi cột, gom nhóm và gộp bảng bằng Pandas.
- Vẽ biểu đồ đường, phân tán, tần suất và biểu đồ hộp bằng Matplotlib theo lối hướng đối tượng; chọn đúng loại biểu đồ cho câu hỏi cần trả lời.
- Dùng SciPy để tính hệ số tương quan, z-score và cực tiểu hóa một hàm mất mát.

4.1 Khởi nguồn của NumPy và Pandas

Python vốn không được thiết kế cho tính toán số: mỗi phần tử của một list là một đối tượng riêng nằm rải rác trong bộ nhớ, và mỗi phép cộng đều phải kiểm tra kiểu dữ liệu. Các thư viện Numeric (1995) và Numarray ra đời để khắc phục điều đó. Năm 2005–2006, Travis Oliphant hợp nhất chúng thành **NumPy** (Numerical Python); bài tổng quan của Harris và cộng sự (2020) trên tạp chí *Nature* ghi nhận NumPy là nền móng của toàn bộ hệ sinh thái Python khoa học, bao gồm SciPy và scikit-learn.

Pandas do Wes McKinney bắt đầu phát triển năm 2008 tại công ty tài chính AQR Capital, nhằm đưa vào Python kiểu làm việc với bảng dữ liệu có nhãn vốn phổ biến trong R và SQL (McKinney, 2010). Pandas được xây dựng trên NumPy: có thể hình dung mỗi cột của một bảng Pandas là một mảng có kèm nhãn.

4.2 Thư viện NumPy – Tính toán khoa học

NumPy cung cấp đối tượng mảng đa chiều `ndarray`: một khối bộ nhớ liền nhau chứa các phần tử **cùng kiểu dữ liệu**. Chính hai đặc điểm này cho phép NumPy thực hiện phép tính bằng các vòng lặp đã biên dịch sẵn, nhanh hơn nhiều so với vòng lặp Python.

4.2.1 Khởi tạo mảng và kiểm tra thuộc tính

Mảng được tạo từ list của Python hoặc bằng các hàm khởi tạo những ma trận đặc biệt. Ba thuộc tính cần kiểm tra đầu tiên là `shape` (kích thước theo từng chiều), `ndim` (số chiều) và `dtype` (kiểu phần tử).

Listing 4.1: Khởi tạo mảng NumPy và kiểm tra thuộc tính

```
import numpy as np

arr1D = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
print("Shape 1D:", arr1D.shape, "| Dtype:", arr1D.dtype)   # (10,) | int64

arr2D = np.array([[1, 2, 3, 4],
                  [5, 6, 7, 8],
                  [9, 10, 11, 12]])
print("Shape 2D:", arr2D.shape, "| ndim:", arr2D.ndim)     # (3, 4) | 2

arr_zeros = np.zeros((3, 4))          # ma tran 3 x 4 toan so 0
arr_ones = np.ones((2, 5))            # ma tran 2 x 5 toan so 1
arr_eye = np.identity(3)              # ma tran don vi 3 x 3
arr_random = np.random.randn(3, 4)    # phan phoi chuan tac N(0, 1)
```

Lưu ý: kiểu `int64` ở trên có thể khác trên máy khác. Với mã nguồn mới, NumPy khuyến nghị tạo số ngẫu nhiên qua `rng = np.random.default_rng(42)` rồi gọi `rng.standard_normal((3, 4))`, nhờ đó kết quả tái lập được.

4.2.2 Thêm, xóa và sửa phần tử

Có thể gán trực tiếp giá trị cho một phần tử. Các hàm `append`, `insert`, `delete` thì **không sửa mảng gốc** mà trả về một mảng mới. Vì kích thước của `ndarray` là cố định, mỗi lần gọi đều phải cấp phát và sao chép toàn bộ dữ liệu; do đó không nên gọi `np.append` trong vòng lặp dài, mà nên gom dữ liệu vào list rồi chuyển thành mảng một lần ở cuối.

Listing 4.2: Thêm, xóa và sửa phần tử mảng

```
arr1D[0] = 99                                 # sua truc tiep mang goc

arr_appended = np.append(arr1D, [11, 12])     # them vao cuoi -> mang moi
arr_inserted = np.insert(arr1D, 2, 999)       # chen 999 vao vi tri 2 -> mang moi
arr_deleted = np.delete(arr1D, 0)             # xoa phan tu dau -> mang moi
arr1D[0] = 1                                  # tra lai gia tri ban dau
```

4.2.3 Slicing và Boolean Indexing

Slicing lấy một phần của mảng theo cú pháp `bắt_đầu:kết_thúc` trên từng chiều (không gồm vị trí kết thúc). **Boolean Indexing** dùng một mảng điều kiện đúng/sai để lọc phần tử; đây là cách lọc dữ liệu thông dụng nhất.

Listing 4.3: Slicing và lọc dữ liệu bằng điều kiện

```
sub_arr = arr2D[:2, :3]              # 2 dong dau, 3 cot dau
print(sub_arr)                       # [[1 2 3]
                                     #  [5 6 7]]

city_data = np.random.randn(5, 3)
pos_values = city_data[city_data > 0]    # mang 1 chieu cac gia tri duong
print("Cac gia tri > 0:", pos_values)

city_data[city_data > 0] = 0             # thay moi gia tri duong bang 0
print(city_data)
```

View và copy. Đây là cái bẫy phổ biến nhất của NumPy: slicing trả về một **view**, tức một "cửa sổ" nhìn vào cùng vùng nhớ, chứ không sao chép dữ liệu. Sửa view đồng nghĩa với sửa mảng gốc. Muốn dữ liệu độc lập, phải gọi `.copy()`. Ngược lại, Boolean Indexing luôn trả về bản sao.

Listing 4.4: Slicing tạo view, không sao chép dữ liệu

```
v = arr1D[:3]
v[0] = 99
print(arr1D[:3])     # [99  2  3]  -> mang goc bi thay doi!
arr1D[0] = 1

c = arr1D[:3].copy()
c[0] = 99
print(arr1D[:3])     # [1 2 3]     -> mang goc giu nguyen
```

4.2.4 Vector hóa, broadcasting và tham số axis

Vector hóa (vectorization) là viết phép tính trên cả mảng thay vì lặp qua từng phần tử: mã nguồn ngắn hơn, gần với công thức toán hơn, và nhanh hơn rất nhiều. Khi thử nhân đôi một triệu số trên máy của nhóm tác giả, vòng lặp Python mất khoảng 320 ms, còn phép tính NumPy khoảng 5 ms; con số cụ thể tùy máy.

Listing 4.5: So sánh vòng lặp Python với vector hóa

```
x = np.arange(1_000_000, dtype=float)
y1 = [i * 2 for i in x.tolist()]     # vong lap Python: cham
y2 = x * 2                           # vector hoa: nhanh
```

Broadcasting là quy tắc cho phép tính toán giữa hai mảng khác kích thước: so sánh các chiều từ phải sang trái, hai chiều tương thích khi bằng nhau hoặc một trong hai bằng 1; chiều bằng 1 được "kéo giãn" cho khớp. Tham số `axis` chỉ chiều bị "gộp lại" trong phép tổng hợp: `axis=0` tính theo từng cột, `axis=1` tính theo từng dòng.

Ví dụ 4.1 (Chuẩn hóa tâm bằng broadcasting)**.** *Với* `arr2D` *có kích thước* $(3, 4)$*, trung bình theo từng cột* `arr2D.mean(axis=0)` *có kích thước* $(4,)$ *và bằng* $(5, 6, 7, 8)$*. Phép trừ* `arr2D - arr2D.mean(axis=0)` *tự động trừ vector này khỏi cả ba dòng. Đây chính là bước chuẩn hóa tâm của* `StandardScaler` *và PCA ở các chương sau.*

Listing 4.6: Tổng hợp theo axis và broadcasting

```
print(arr2D.sum(axis=0))      # theo cot:  [15 18 21 24]
print(arr2D.sum(axis=1))      # theo dong: [10 26 42]
print(arr2D.mean())           # toan bo mang: 6.5

print(arr2D - arr2D.mean(axis=0))
# [[-4. -4. -4. -4.]
#  [ 0.  0.  0.  0.]
#  [ 4.  4.  4.  4.]]

print(arr2D.reshape(4, 3).shape, arr2D.T.shape)   # (4, 3) (4, 3)
```

4.2.5 Đại số tuyến tính

Mô-đun `np.linalg` cung cấp các phép tính đại số tuyến tính: định thức, nghịch đảo, trị riêng, SVD và giải hệ phương trình. Toán tử `@` (hoặc `np.dot`) là phép nhân ma trận, còn $*$ là phép nhân **từng phần tử**.

Ví dụ 4.2 (Giải hệ phương trình tuyến tính)**.** *Giải hệ*

$$\begin{cases} 7x + 5y - 3z = 16 \\ 3x - 5y + 2z = -8 \\ 5x + 3y - 7z = 0 \end{cases}$$

Viết dưới dạng $\mathbf{Ax} = \mathbf{b}$*, với* $\det \mathbf{A} = 256 \neq 0$ *nên hệ có nghiệm duy nhất* $(x, y, z) = (1, 3, 2)$*. Kiểm tra phương trình đầu:* $7 \cdot 1 + 5 \cdot 3 - 3 \cdot 2 = 16$*.*

Listing 4.7: Giải hệ phương trình bằng np.linalg.solve

```
A = np.array([[7, 5, -3],
              [3, -5, 2],
              [5, 3, -7]])
b = np.array([16, -8, 0])

x = np.linalg.solve(A, b)
print("Nghiem x, y, z:", x)                       # [1. 3. 2.]
print("Kiem tra:", np.allclose(np.dot(A, x), b))   # True
```

Nên dùng `np.linalg.solve` thay vì tính `np.linalg.inv(A) @ b`: cách này nhanh hơn và ổn định số học hơn. So sánh hai số thực phải dùng `np.allclose` chứ không dùng `==`, vì phép tính dấu phẩy động luôn có sai số làm tròn.

4.3 Thư viện Pandas – Xử lý và phân tích dữ liệu

Pandas cung cấp hai cấu trúc dữ liệu: `Series` (một chiều, có nhãn) và `DataFrame` (bảng hai chiều, mỗi cột có thể mang một kiểu dữ liệu khác nhau). Đây là khác biệt lớn so với `ndarray`, và là lý do Pandas phù hợp với dữ liệu dạng bảng ngoài thực tế.

4.3.1 Tạo và đọc dữ liệu

`DataFrame` có thể được tạo từ list các dictionary, từ list các list kèm tên cột, hoặc đọc từ tệp bằng `read_csv()`. Hàm `head()` và `tail()` cho xem nhanh các dòng đầu và cuối. Ví dụ dưới đây dùng **số liệu mô phỏng** của một mã cổ phiếu, chỉ nhằm minh họa, không phải giá giao dịch thật.

Listing 4.8: Tạo DataFrame, ghi và đọc tệp CSV

```
import pandas as pd

data_dict = [
    {"city": "Delhi", "data": 1000},
    {"city": "Bangalore", "data": 2000},
    {"city": "Mumbai", "data": 1500},
]
df_dict = pd.DataFrame(data_dict)

data_list = [[1, "Apple"], [2, "Banana"]]
df_list = pd.DataFrame(data_list, columns=["ID", "Fruit"])

data_vnm = {
    "Date": ["10-01-2023", "10-02-2023", "10-03-2023", "10-04-2023",
             "10-05-2023", "10-06-2023", "10-07-2023"],
    "Ticker": ["VNM"] * 7,
    "Open": [75.5, 76.0, 75.8, 77.2, 76.5, 78.0, 77.8],
    "Close": [76.0, 75.5, 77.0, 76.5, 78.0, 77.5, 78.2],
    "Volume": [12000, 15000, 13500, 18000, 16000, 20000, 19500],
}
df_mock = pd.DataFrame(data_vnm)
df_mock.to_csv("vinamilk_stock.csv", index=False)   # ghi ra tep CSV

df_stock = pd.read_csv("vinamilk_stock.csv")        # doc lai tu tep
print(df_stock.tail(3))
#          Date Ticker  Open  Close  Volume
# 4  10-05-2023    VNM  76.5   78.0   16000
# 5  10-06-2023    VNM  78.0   77.5   20000
# 6  10-07-2023    VNM  77.8   78.2   19500
```

4.3.2 Truy xuất và lọc dữ liệu

Hai cách truy xuất quan trọng là `loc` (theo **nhãn**) và `iloc` (theo **vị trí số nguyên**). Điểm dễ nhầm: lát cắt của `iloc` không gồm vị trí cuối, còn lát cắt của `loc` **có gồm** nhãn cuối. Điều kiện

lọc hoạt động giống Boolean Indexing của NumPy; khi kết hợp nhiều điều kiện phải dùng `&`, `|` và đặt mỗi điều kiện trong ngoặc đơn.

Listing 4.9: Truy xuất bằng iloc, loc và điều kiện lọc

```
print(df_stock.iloc[:2, :3])      # 2 dong dau, 3 cot dau

high = df_stock.loc[df_stock["Volume"] > 16000, ["Date", "Close"]]
print(high)
#          Date  Close
# 3  10-04-2023   76.5
# 5  10-06-2023   77.5
# 6  10-07-2023   78.2
```

4.3.3 Dữ liệu thiếu và thống kê mô tả

`isna().sum()` đếm số ô thiếu (`NaN`) theo từng cột; `describe()` và `info()` mô tả nhanh bộ dữ liệu. Các hàm thống kê của Pandas mặc định **bỏ qua** `NaN`, nên số đếm `count` của cột thiếu sẽ nhỏ hơn số dòng.

Listing 4.10: Phát hiện và điền dữ liệu thiếu

```
df = pd.DataFrame({
    "price": [100, 200, np.nan],
    "quantity_purchased": [10, 20, 32],
    "user_type": ["A", "B", "C"],
    "user_class": ["X", "Y", "X"],
})
print(df.isna().sum())           # cot price thieu 1 gia tri
print(df.describe())             # price: count = 2, mean = 150

mean_price = df["price"].mean()  # 150.0
df["price"] = df["price"].fillna(mean_price)
```

Lưu ý về phiên bản: cách viết cũ `df["price"].fillna(value, inplace=True)` vốn thao tác trên một bản sao của cột, nên từ Pandas 3 không còn thay đổi `df`. Hãy luôn gán lại kết quả cho cột như trên. Các kỹ thuật điền giá trị thiếu có cơ sở hơn được trình bày ở Chương 5.

4.3.4 Thay thế và áp dụng hàm

`replace()` thay giá trị theo một bảng ánh xạ; `apply()` áp dụng một hàm cho từng phần tử của cột. Khi có thể, nên ưu tiên phép tính vector hóa: `df["price"] * 0.9` cho cùng kết quả với `apply` dưới đây nhưng nhanh hơn.

Listing 4.11: Biến đổi cột bằng replace và apply

```
df["user_type"] = df["user_type"].replace({"A": "Premium", "B": "Standard"})
df["discounted_price"] = df["price"].apply(lambda x: x * 0.9)
print(df)
#    price  quantity_purchased user_type user_class  discounted_price
# 0  100.0                  10   Premium          X              90.0
# 1  200.0                  20  Standard          Y             180.0
# 2  150.0                  32         C          X             135.0
```

4.3.5 Gom nhóm và gộp dữ liệu

`groupby()` thực hiện quy trình **tách – áp dụng – kết hợp** (split–apply–combine; Wickham, 2011): tách các dòng thành nhóm theo giá trị của một cột, tính toán trên từng nhóm, rồi ghép kết quả. `concat()` nối các bảng theo chiều dọc hoặc chiều ngang. `merge()` ghép hai bảng theo cột khóa chung, tương tự phép `JOIN` của SQL.

Listing 4.12: Gom nhóm bằng groupby, gộp bảng bằng concat và merge

```
grouped = df.groupby("user_class")["quantity_purchased"].agg(["sum", "mean", "
    count"])
print(grouped)
#              sum  mean  count
# user_class
# X             42  21.0      2
# Y             20  20.0      1

head_tail = pd.concat([df.head(1), df.tail(1)], ignore_index=True)   # noi theo
     dong

discounts = pd.DataFrame({"user_class": ["X", "Y", "Z"],
                          "discount": [0.1, 0.2, 0.3]})
merged = pd.merge(df[["user_class", "price"]], discounts,
                  on="user_class", how="left")
print(merged)
#   user_class  price  discount
# 0          X  100.0       0.1
# 1          Y  200.0       0.2
# 2          X  150.0       0.1
```

Với `how="left"`, mọi dòng của bảng bên trái được giữ lại; dòng `Z` của bảng bên phải bị bỏ vì không có dòng nào khớp.

4.4 Trực quan hóa dữ liệu với Matplotlib

Bảng số cho biết giá trị, còn biểu đồ cho thấy **hình dạng** của dữ liệu: xu hướng, nhóm, ngoại lai, độ lệch. Trước khi xây dựng bất kỳ mô hình nào, người làm học máy luôn vẽ dữ liệu để kiểm tra các giả định. Bộ tứ của Anscombe (1973) là minh chứng kinh điển: bốn tập dữ liệu có cùng trung bình, cùng phương sai, cùng hệ số tương quan và cùng đường hồi quy, nhưng khi vẽ ra thì có hình dạng hoàn toàn khác nhau.

Matplotlib do John D. Hunter viết năm 2003, ban đầu để hiển thị tín hiệu điện não đồ, với cú pháp mô phỏng MATLAB (Hunter, 2007). Ngày nay đây là thư viện vẽ đồ thị nền tảng của Python. Các công cụ cấp cao hơn như thư viện seaborn hay phương thức `plot` của Pandas đều vẽ thông qua Matplotlib.

4.4.1 Figure, Axes và hai cách viết

Một hình vẽ của Matplotlib gồm một **Figure** (khung hình) chứa một hoặc nhiều **Axes** (vùng vẽ, mỗi vùng có hệ trục riêng). Có hai cách viết lệnh vẽ:

– **Cách viết pyplot**: gọi trực tiếp `plt.plot(...)`, `plt.xlabel(...)`. Các lệnh tác động lên vùng vẽ đang được chọn. Cách này ngắn gọn, phù hợp để xem nhanh một biểu đồ.

– **Cách viết hướng đối tượng**: tạo khung hình và các vùng vẽ bằng `fig, ax = plt.subplots()`, rồi gọi phương thức của từng vùng vẽ như `ax.plot(...)`, `ax.set_xlabel(...)`. Cách này rõ ràng khi có nhiều biểu đồ con, và được dùng trong giáo trình.

Biểu đồ	Phương thức	Câu hỏi biểu đồ trả lời
Đường	`ax.plot`	Đại lượng thay đổi thế nào theo thời gian hay theo số vòng lặp (giá cổ phiếu, mất mát theo epoch)?
Phân tán	`ax.scatter`	Hai biến có quan hệ với nhau không? Các lớp có tách biệt nhau không?
Tần suất	`ax.hist`	Một biến phân phối ra sao: lệch về một phía, nhiều đỉnh hay có đuôi dài?
Hộp	`ax.boxplot`	Phân phối khác nhau thế nào giữa các nhóm? Điểm nào là ngoại lai?
Cột	`ax.bar`	Một đại lượng khác nhau thế nào giữa các hạng mục (độ chính xác của các mô hình)?
Ảnh, ma trận	`ax.imshow`	Hiển thị ảnh, ma trận tương quan, ma trận nhầm lẫn.

Bảng 4.1: Chọn loại biểu đồ theo câu hỏi cần trả lời

4.4.2 Bốn biểu đồ cơ bản

Đoạn mã dưới đây vẽ bốn biểu đồ trên cùng một khung hình 2×2. Biểu đồ đường dùng lại bảng `df_stock` tạo ở Mục 4.3.1. Ba biểu đồ còn lại dùng dữ liệu mô phỏng 100 sinh viên gồm số giờ tự học mỗi tuần và điểm thi. Điểm thi được sinh theo quan hệ tuyến tính với số giờ tự học, cộng thêm nhiễu ngẫu nhiên. Đây chính là dạng dữ liệu của bài toán hồi quy ở Chương 6. Nếu chuyển điểm thành nhãn đậu hoặc rớt, ta có bài toán phân lớp của Chương 7.

Listing 4.13: Bốn biểu đồ cơ bản với Matplotlib

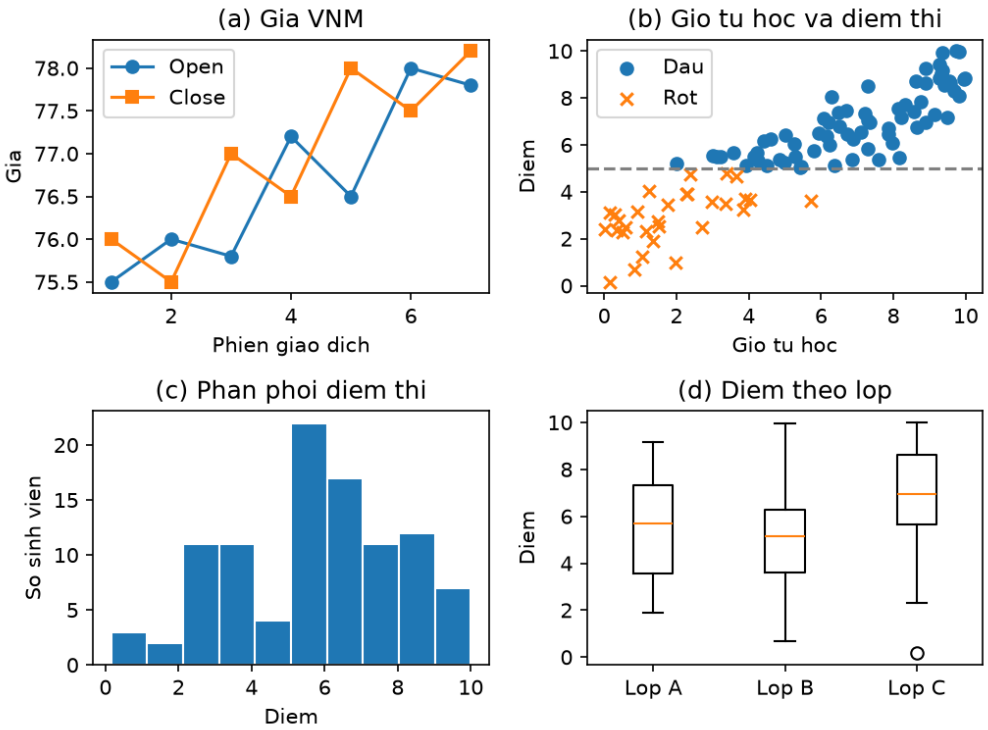
```
import matplotlib.pyplot as plt

rng = np.random.default_rng(0)
hours = rng.uniform(0, 10, 100)                       # gio tu hoc moi tuan
score = np.clip(2 + 0.7 * hours + rng.normal(0, 1, 100), 0, 10)
passed = score >= 5                                   # dau neu diem >= 5

fig, axes = plt.subplots(2, 2, figsize=(7, 5.2))      # khung hinh 2 x 2

ax = axes[0, 0]                                       # (a) bieu do duong
session = range(1, len(df_stock) + 1)
ax.plot(session, df_stock["Open"], marker="o", label="Open")
ax.plot(session, df_stock["Close"], marker="s", label="Close")
ax.set(title="(a) Gia VNM", xlabel="Phien giao dich", ylabel="Gia")
ax.legend()
```

```
16
17 ax = axes[0, 1]                                    # (b) bieu do phan tan
18 ax.scatter(hours[passed], score[passed], label="Dau")
19 ax.scatter(hours[~passed], score[~passed], marker="x", label="Rot")
20 ax.axhline(5, color="gray", linestyle="--")        # nguong diem 5
21 ax.set(title="(b) Gio tu hoc va diem thi",
22        xlabel="Gio tu hoc", ylabel="Diem")
23 ax.legend()
24
25 ax = axes[1, 0]                                    # (c) bieu do tan suat
26 ax.hist(score, bins=10, edgecolor="white")
27 ax.set(title="(c) Phan phoi diem thi",
28        xlabel="Diem", ylabel="So sinh vien")
29
30 ax = axes[1, 1]                                    # (d) bieu do hop
31 ax.boxplot([score[:35], score[35:70], score[70:]])
32 ax.set_xticks([1, 2, 3], ["Lop A", "Lop B", "Lop C"])
33 ax.set(title="(d) Diem theo lop", ylabel="Diem")
34
35 fig.tight_layout()                                 # tranh chu de len nhau
36 fig.savefig("bon_bieu_do.png", dpi=150)            # luu ra tep anh
37 plt.show()
```



Hình 4.1: Kết quả của đoạn mã trên: (a) biểu đồ đường, (b) biểu đồ phân tán tô màu theo nhãn, (c) biểu đồ tần suất, (d) biểu đồ hộp. Chữ trên hình lấy từ chuỗi ký tự trong mã nguồn nên được viết không dấu.

Một số điểm cần lưu ý trong đoạn mã:

- `axes` là một mảng NumPy kích thước 2×2 chứa các vùng vẽ; `axes[0, 1]` là vùng ở dòng 0, cột 1.
- `hours[passed]` là phép lọc Boolean indexing ở Mục 4.2.3; toán tử `~` đảo ngược mảng logic, nên `hours[~passed]` lấy các sinh viên rớt.
- `ax.set(...)` đặt nhiều thuộc tính cùng lúc, tương đương gọi lần lượt `set_title`, `set_xlabel` và `set_ylabel`.
- Hộp trong biểu đồ (d) trải từ tứ phân vị Q_1 đến Q_3, đường giữa hộp là trung vị; hai "râu" kéo dài tối đa $1{,}5$ lần khoảng tứ phân vị. Điểm nằm ngoài râu được vẽ riêng. Đó chính là quy tắc IQR phát hiện ngoại lai ở Chương 5.
- Trong Jupyter Notebook, biểu đồ hiện ngay dưới ô lệnh. Khi chạy tệp `.py`, lệnh `plt.show()` mở cửa sổ hiển thị.

Vẽ nhanh từ Pandas. DataFrame có sẵn phương thức `plot` gọi Matplotlib bên dưới: `df_stock.plot(y=["Open", "Close"])` vẽ biểu đồ đường, còn `df_stock.hist()` vẽ biểu đồ tần suất cho mọi cột số. Thư viện **seaborn** (Waskom, 2021) bổ sung các biểu đồ thống kê cấp cao, chẳng hạn bản đồ nhiệt của ma trận tương quan dùng trong các bài thực hành ở Chương 12.

4.5 SciPy: thống kê và tối ưu hóa

SciPy (Scientific Python) ra đời năm 2001 và được xây dựng trên NumPy (Virtanen và cộng sự, 2020). Thư viện này gom các thuật toán khoa học kinh điển thành nhiều module con. Hai module hữu ích nhất cho học máy là `scipy.stats` cho thống kê và `scipy.optimize` cho tối ưu hóa.

Listing 4.14: Tương quan, z-score và cực tiểu hóa hàm mất mát với SciPy

```
from scipy import stats, optimize

x = np.array([2.0, 4.0, 6.0, 8.0])
y = np.array([3.1, 4.9, 7.2, 8.8])
r, p_value = stats.pearsonr(x, y)              # he so tuong quan Pearson
print(round(r, 4))                             # 0.9978
print(stats.zscore([2, 4, 6, 8, 30]).round(2))
# [-0.78 -0.59 -0.39 -0.2   1.96]

def loss(params):                              # MSE cua y = w*x + b
    w, b = params
    return np.mean((y - (w * x + b)) ** 2)

res = optimize.minimize(loss, x0=[0.0, 0.0])   # bat dau tu w = b = 0
print(res.x.round(3))                          # [0.97 1.15]
```

Ba kết quả trên sẽ được gặp lại ở các chương sau:

- Hệ số tương quan $r = 0{,}9978$ rất gần 1, cho thấy x và y có quan hệ tuyến tính rất chặt. Phân tích tương quan được dùng để lựa chọn đặc trưng ở Chương 5.

– Trong cột $(2, 4, 6, 8, 30)$, giá trị 30 khác hẳn các giá trị còn lại nhưng chỉ có $z = 1{,}96$, chưa vượt ngưỡng 3 thường dùng để phát hiện ngoại lai. Chương 5 giải thích hiện tượng "che lấp" này.

– Hàm `minimize` tìm được $w = 0{,}97$ và $b = 1{,}15$ chỉ từ định nghĩa của hàm mất mát. Đây đúng là nghiệm bình phương tối thiểu: $w = \sum_i (x_i - \bar{x})(y_i - \bar{y}) / \sum_i (x_i - \bar{x})^2 = 19{,}4/20 = 0{,}97$ và $b = \bar{y} - w\bar{x} = 6 - 0{,}97 \times 5 = 1{,}15$. Chương 6 xây dựng công thức này, cùng thuật toán Gradient Descent để tự tìm cực tiểu.

4.6 Tóm tắt chương

– `ndarray` chứa các phần tử cùng kiểu trong vùng nhớ liền nhau, nên tính toán rất nhanh; hãy kiểm tra `shape`, `ndim`, `dtype`.

– Slicing trả về view, muốn dữ liệu độc lập phải dùng `.copy()`; `append`, `insert`, `delete` trả về mảng mới.

– Ưu tiên vector hóa và broadcasting thay vì vòng lặp; `axis=0` tính theo cột, `axis=1` tính theo dòng.

– `DataFrame` là bảng có nhãn; `loc` truy xuất theo nhãn, `iloc` theo vị trí.

– Phát hiện dữ liệu thiếu bằng `isna()`, điền bằng `fillna()` và gán lại kết quả cho cột.

– `groupby` tổng hợp theo nhóm; `concat` nối bảng; `merge` ghép bảng theo khóa.

– Matplotlib: một Figure chứa các Axes. Tạo bằng `fig, ax = plt.subplots()` rồi gọi phương thức của `ax`. Chọn loại biểu đồ theo câu hỏi cần trả lời (Bảng 4.1); luôn vẽ dữ liệu trước khi xây dựng mô hình.

– SciPy bổ sung thống kê (`scipy.stats`) và tối ưu hóa (`scipy.optimize`). Cực tiểu hóa một hàm mất mát chính là cách các mô hình học máy được huấn luyện.

4.7 Bài tập

1. Tạo ma trận 5×5 các số ngẫu nhiên chuẩn tắc với `default_rng(0)`. Thay mọi giá trị âm bằng 0, rồi tính tổng theo từng dòng và từng cột.

2. Giải thích kết quả in ra của đoạn lệnh `a = np.arange(5); b = a[1:3]; b[0] = 100; print(a)`. Sửa đoạn lệnh để `a` không bị thay đổi.

3. Không dùng vòng lặp, chuẩn hóa z-score từng cột của `arr2D`: trừ trung bình cột rồi chia cho độ lệch chuẩn cột. Kiểm tra lại bằng `StandardScaler`.

4. Giải hệ $2x + y = 5$, $x - 3y = -1$ bằng tay và bằng `np.linalg.solve`.

5. Với `df_stock`, thêm cột `Change` $=$ `Close` $-$ `Open`, rồi lọc các phiên có `Change` dương và `Volume` trên 15000.

6. Tạo bảng đơn hàng gồm mã khách và số tiền, bảng khách hàng gồm mã khách và thành phố. Dùng `merge` rồi `groupby` để tính tổng tiền theo thành phố. So sánh kết quả của `how="inner"` và `how="left"` khi có đơn hàng không tìm thấy khách.

7. Vẽ trên cùng một vùng vẽ đường giá `Close` của `df_stock` và đường trung bình trượt 3 phiên `df_stock["Close"].rolling(3).mean()`. Thêm tiêu đề, nhãn trục và chú giải. Vì sao đường trung bình trượt thiếu hai điểm đầu?

8. Với dữ liệu giờ tự học và điểm thi ở Mục 4.4, tính hệ số tương quan bằng `stats.pearsonr`, rồi dùng `optimize.minimize` tìm đường thẳng $y = wx + b$ có MSE nhỏ nhất. Vẽ đường thẳng đó lên biểu đồ phân tán. So sánh w tìm được với hệ số $0{,}7$ đã dùng để sinh dữ liệu.

Tài liệu tham khảo của chương

1. Anscombe, F. J. (1973). Graphs in Statistical Analysis. *The American Statistician*, 27(1), 17–21.

2. Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

3. Hunter, J. D. (2007). Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering*, 9(3), 90–95.

4. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

5. McKinney, W. (2022). *Python for Data Analysis* (3rd ed.). O'Reilly Media.

6. Sarkar, D., Bali, R., & Sharma, T. (2018). *Practical Machine Learning with Python*, Chương 2–3. Apress.

7. Virtanen, P. et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261–272.

8. Waskom, M. L. (2021). seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60), 3021.

9. Wickham, H. (2011). The Split-Apply-Combine Strategy for Data Analysis. *Journal of Statistical Software*, 40(1), 1–29.

Phần II

Dữ liệu và học có giám sát

Chương 5: Tiền xử lý dữ liệu (Data Preprocessing)

Các thuật toán học máy (Machine Learning – ML) cổ điển như hồi quy tuyến tính, KNN, K-Means, SVM hay PCA đều được xây dựng trên nền tảng đại số tuyến tính và giải tích: chúng nhận đầu vào là **các vector số thực** và thực hiện phép cộng, phép nhân, phép tính khoảng cách giữa các vector đó. Trong khi đó, dữ liệu ngoài đời thực lại rất đa dạng: tên màu sắc, tên khu phố, đoạn văn bản bình luận, mức đánh giá "tốt – trung bình – kém", diện tích nhà tính bằng feet vuông bên cạnh số phòng tắm chỉ từ 0 đến 4… Khoảng cách giữa "dữ liệu thô" và "dữ liệu mà mô hình hiểu được" chính là đối tượng nghiên cứu của chương này.

> *Nguyên tắc cốt lõi:* Mô hình ML không làm việc được trực tiếp trên văn bản hay nhãn chữ — **mọi thứ đều phải được biến đổi thành số**, và phải biến đổi sao cho **không làm sai lệch ý nghĩa** của dữ liệu.

Sau khi học xong chương này, người học có khả năng:

- Phát biểu định nghĩa hình thức của học máy, phân biệt ba loại học máy và sử dụng thống nhất hệ thống thuật ngữ, ký hiệu của giáo trình.
- Trình bày được nguồn gốc lịch sử và vị trí của tiền xử lý dữ liệu trong quy trình khai phá tri thức (KDD) và CRISP-DM.
- Phân biệt các kiểu dữ liệu (định danh, thứ bậc, số, văn bản) và chọn phép biến đổi phù hợp cho từng kiểu.
- Xử lý dữ liệu thiếu bằng **Simple Imputer**, **KNN Imputer** (khoảng cách nan-Euclid) và **Iterative Imputer**.
- Mã hóa dữ liệu phân loại bằng **Label Encoder** và **One Hot Encoder**, hiểu rõ ưu nhược điểm của từng cách.
- Chuẩn hóa thang đo đặc trưng bằng **Min Max Scaler**, **Max Abs Scaler**, **Standard Scaler** và **Robust Scaler**.
- Phát hiện ngoại lai bằng z-score, quy tắc IQR và **Local Outlier Factor**.
- Phân biệt ba công đoạn tạo, rút trích và lựa chọn đặc trưng; rút trích đặc trưng từ văn bản bằng **Count Vectorizer** và **TF-IDF Vectorizer**, tự tính tay được ma trận TF-IDF.
- Xử lý dữ liệu mất cân bằng bằng **SMOTE** và đặt bước lấy mẫu đúng vị trí trong pipeline.
- Xây dựng quy trình tiền xử lý hoàn chỉnh, tránh rò rỉ dữ liệu, trên bộ dữ liệu thực tế **Ames Housing**.

5.1 Học máy: định nghĩa, phân loại và hệ thống ký hiệu

Đây là chương đầu tiên của giáo trình đi vào học máy, vì vậy trước khi bàn đến dữ liệu, cần thống nhất *học máy là gì*, *có những loại bài toán nào* và *các đại lượng được ký hiệu ra sao*. Hệ thống ký hiệu dưới đây được dùng xuyên suốt các chương sau.

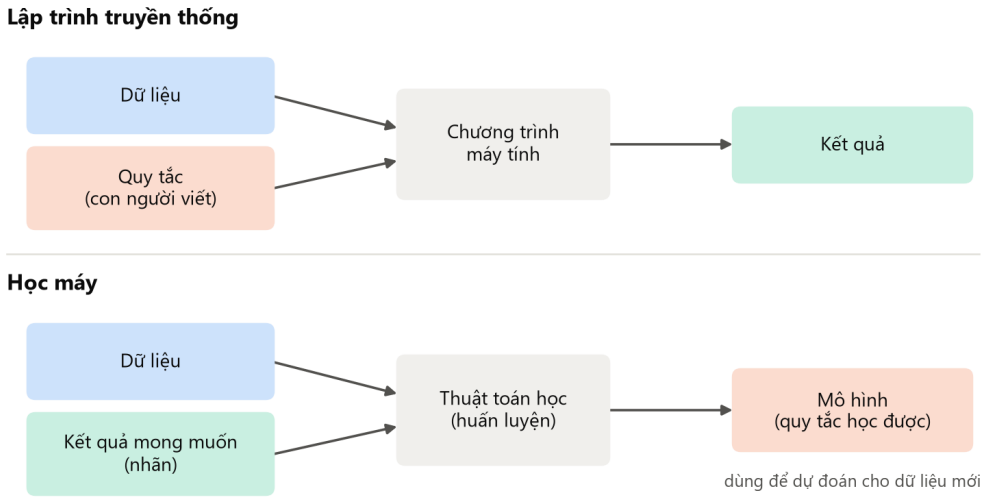
5.1.1 Định nghĩa học máy

Thuật ngữ *machine learning* xuất hiện trong công trình của Arthur Samuel (1959) tại IBM về chương trình chơi cờ đam tự cải thiện qua các ván đấu với chính nó. Quan niệm của Samuel thường được tóm lược là: học máy là lĩnh vực nghiên cứu nhằm *cho máy tính khả năng học mà không cần được lập trình một cách tường minh*. Gần bốn thập kỷ sau, Tom Mitchell (1997) đưa ra định nghĩa hình thức được dùng rộng rãi nhất hiện nay:

Định nghĩa 5.1 (Học máy – Mitchell, 1997)**.** *Một chương trình máy tính được gọi là* ***học*** *từ kinh nghiệm E đối với một lớp tác vụ T và một độ đo hiệu năng P, nếu hiệu năng của nó trên các tác vụ thuộc T, đo bằng P, được cải thiện nhờ kinh nghiệm E.*

Ví dụ 5.1 (Bộ lọc thư rác theo định nghĩa của Mitchell)**.** ***Tác vụ*** *T: phân loại một email là thư rác hay thư thường.* ***Kinh nghiệm*** *E: tập email đã được người dùng gán nhãn.* ***Độ đo*** *P: tỉ lệ email được phân loại đúng, hoặc chính xác hơn là precision và recall (Chương 8). Chương trình "học" nếu càng nhận nhiều email có nhãn thì P càng tăng.*

Định nghĩa này làm rõ điểm khác biệt căn bản so với lập trình truyền thống (Sarkar và cộng sự, 2018, Chương 1). Trong lập trình truyền thống, con người viết **quy tắc** và máy tính áp dụng quy tắc lên dữ liệu để sinh kết quả. Trong học máy, con người cung cấp **dữ liệu cùng kết quả mong muốn**, còn thuật toán tự rút ra quy tắc dưới dạng một **mô hình**. Cách tiếp cận này đặc biệt có giá trị khi quy tắc quá phức tạp để viết tay (nhận dạng chữ viết), thay đổi theo thời gian (hành vi gian lận), hoặc chưa ai biết trước (phân khúc khách hàng).



Hình 5.1: Lập trình truyền thống (dữ liệu + quy tắc → kết quả) so với học máy (dữ liệu + kết quả → mô hình).

5.1.2 Ba loại học máy

Căn cứ vào loại "kinh nghiệm" E mà thuật toán nhận được, học máy được chia thành ba loại chính (Raschka và cộng sự, 2022, Chương 1):

Loại	Dữ liệu và mục tiêu	Bài toán tiêu biểu	Trong giáo trình
Học có giám sát (supervised)	Dữ liệu có nhãn $(\mathbf{x}, y)$; học ánh xạ $\mathbf{x} \mapsto y$ để dự đoán cho dữ liệu mới	Phân lớp (y rời rạc), hồi quy (y liên tục)	Chương 6, 7, 9
Học không giám sát (unsupervised)	Chỉ có $\mathbf{x}$, không có nhãn; khám phá cấu trúc ẩn	Phân cụm, giảm chiều, phát hiện bất thường	Chương 10, 11
Học tăng cường (reinforcement)	Tác tử tương tác với môi trường, nhận **phần thưởng**; học chính sách hành động	Chơi cờ, điều khiển robot	Ngoài phạm vi

Bảng 5.1: Ba loại học máy và vị trí của chúng trong giáo trình

Giữa hai loại đầu còn có **học bán giám sát** (semi-supervised): chỉ một phần nhỏ dữ liệu có nhãn, phần lớn không có nhãn — tình huống rất phổ biến vì gán nhãn tốn kém. Ngoài ra, có thể phân loại theo hai tiêu chí khác (Sarkar và cộng sự, 2018): theo cách nhận dữ liệu, gồm **học theo lô** (batch learning, huấn luyện một lần trên toàn bộ dữ liệu) và **học trực tuyến** (online learning, cập nhật dần khi dữ liệu mới đến); theo cách tổng quát hóa, gồm **học dựa trên mẫu** (instance-based, như KNN: ghi nhớ dữ liệu và so sánh) và **học dựa trên mô hình** (model-based, như hồi quy tuyến tính: rút ra tham số rồi bỏ dữ liệu).

5.1.3 Thuật ngữ và ký hiệu

Thuật ngữ	Tiếng Anh (đồng nghĩa)	Ý nghĩa
Mẫu	sample, instance, observation, example	Một dòng của bảng dữ liệu
Đặc trưng	feature, attribute, predictor, input variable	Một cột dùng làm đầu vào
Nhãn, biến mục tiêu	label, target, response, output	Đại lượng cần dự đoán
Mô hình	model, hypothesis	Hàm f ánh xạ $\mathbf{x}$ thành dự đoán $\hat{y}$
Tham số	parameter, weight	Đại lượng học từ dữ liệu (trọng số $\mathbf{w}$, hệ số chặn b)
Siêu tham số	hyperparameter	Đại lượng chọn trước khi huấn luyện (K của KNN, độ sâu của cây)
Hàm mất mát	loss function, cost function	Đo mức sai khác giữa dự đoán và giá trị thật; huấn luyện là cực tiểu hóa hàm này

Bảng 5.2: Thuật ngữ cơ bản của học máy

Quy ước ký hiệu dùng trong toàn bộ giáo trình:

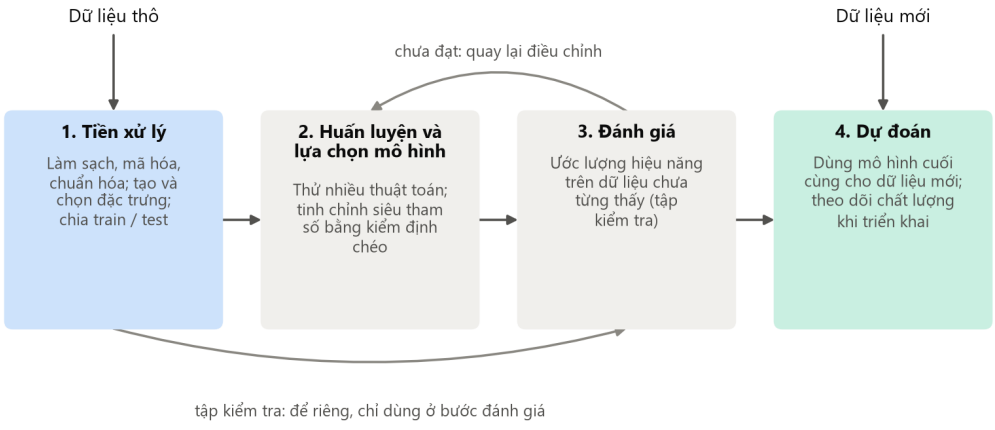
- n: số mẫu; d: số đặc trưng.
- $\mathbf{X} \in \mathbb{R}^{n \times d}$: **ma trận dữ liệu**, mỗi dòng là một mẫu, mỗi cột là một đặc trưng — đúng với cấu trúc của `DataFrame` trong pandas và quy ước `X` của scikit-learn.
- $\mathbf{x}_i \in \mathbb{R}^d$: mẫu thứ i (dòng i của $\mathbf{X}$); x_{ij}: giá trị đặc trưng j của mẫu i.
- $\mathbf{y} \in \mathbb{R}^n$: vector nhãn; y_i: nhãn của mẫu i; $\hat{y}_i$: giá trị **dự đoán**.
- Vector được viết in đậm và hiểu là vector cột; $\mathbf{w}^\top \mathbf{x} = \sum_j w_j x_j$ là tích vô hướng.

Ví dụ, bộ dữ liệu hoa Iris (Fisher, 1936) có $n = 150$ mẫu, $d = 4$ đặc trưng (chiều dài, chiều rộng của đài hoa và cánh hoa), nên $\mathbf{X} \in \mathbb{R}^{150 \times 4}$ và $\mathbf{y} \in \{0, 1, 2\}^{150}$ ứng với ba loài hoa.

5.1.4 Lộ trình xây dựng một hệ thống học máy

Mọi dự án học máy có giám sát đều đi qua bốn giai đoạn (Raschka và cộng sự, 2022). Các chương của giáo trình lần lượt trang bị công cụ cho từng giai đoạn:

1. **Tiền xử lý** (Chương 5): làm sạch, mã hóa, chuẩn hóa, tạo và lựa chọn đặc trưng; chia dữ liệu thành tập huấn luyện và tập kiểm tra.
2. **Huấn luyện mô hình** (Chương 6, 7 và 9): thử nhiều thuật toán thuộc các họ khác nhau.
3. **Đánh giá và lựa chọn mô hình** (Chương 8): tinh chỉnh siêu tham số bằng kiểm định chéo, rồi ước lượng hiệu năng trên dữ liệu chưa từng thấy bằng tập kiểm tra. Chương này được đặt ngay sau hồi quy và phân lớp để mọi mô hình ở các chương sau đều được đánh giá theo cùng một quy trình.
4. **Dự đoán**: dùng mô hình cuối cùng cho dữ liệu mới, theo dõi chất lượng khi triển khai.



Hình 5.2: Lộ trình xây dựng hệ thống học máy: tiền xử lý → huấn luyện và lựa chọn mô hình → đánh giá → dự đoán.

Không có thuật toán tốt nhất cho mọi bài toán. Định lý *không có bữa trưa miễn phí* (No Free Lunch) của Wolpert (1996) chỉ ra rằng, tính trung bình trên mọi bài toán có thể có, không thuật toán học nào vượt trội hơn thuật toán nào. Một thuật toán chỉ tốt hơn trên những bài toán phù hợp với các giả định ngầm của nó. Hệ quả thực hành, được cả ba tài liệu tham khảo chính nhấn mạnh: **luôn so sánh nhiều mô hình** (kể cả một mô hình cơ sở đơn giản) bằng quy trình đánh giá đúng đắn, thay vì tin vào một thuật toán "thần kỳ". Domingos (2012) bổ sung một nhận xét đúc kết từ thực tiễn: yếu tố quyết định thành bại của phần lớn dự án học máy không phải thuật toán mà là **đặc trưng** được sử dụng — đó là lý do chương này dài nhất giáo trình.

5.2 Khởi nguồn và vai trò của tiền xử lý dữ liệu

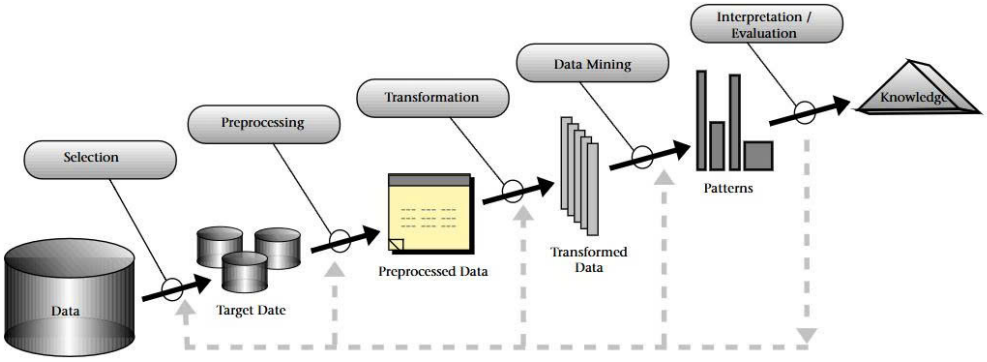
5.2.1 "Rác vào, rác ra"

Ngay từ những ngày đầu của ngành máy tính, giới lập trình viên đã lưu truyền nguyên lý *Garbage In, Garbage Out* (GIGO – "rác vào, rác ra"), thường được gán cho George Fuechsel, một kỹ sư IBM vào cuối thập niên 1950. Nguyên lý này nói rằng một chương trình dù đúng đến đâu cũng chỉ cho ra kết quả tốt khi dữ liệu đầu vào tốt. Đối với học máy, nguyên lý này còn nghiêm trọng hơn: mô hình *học* trực tiếp từ dữ liệu, nên mọi sai lệch, nhiễu hay cách mã hóa không phù hợp đều được mô hình "học thuộc" và khuếch đại thành dự đoán sai.

5.2.2 Tiền xử lý trong quy trình khai phá tri thức (KDD)

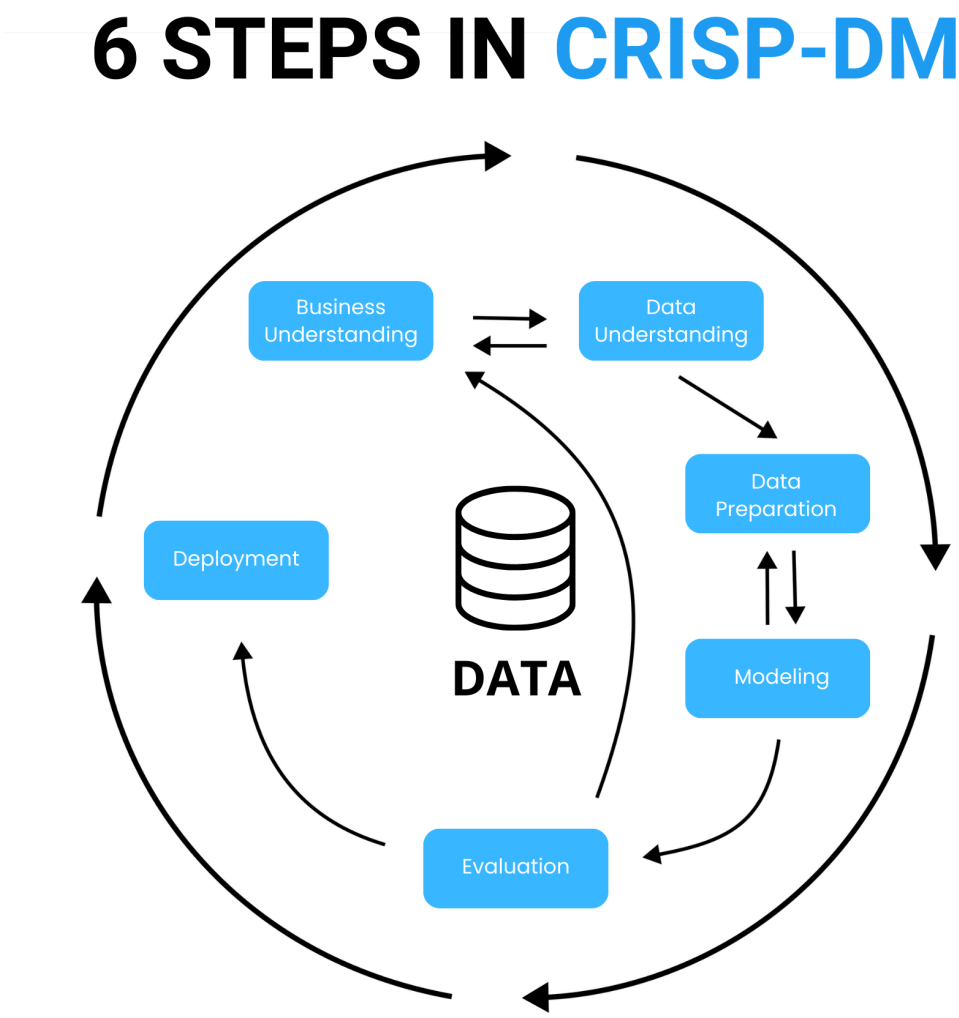
Năm 1996, Fayyad, Piatetsky-Shapiro và Smyth công bố bài báo kinh điển *From Data Mining to Knowledge Discovery in Databases* trên tạp chí *AI Magazine*, chính thức hóa quy trình **khám phá tri thức từ cơ sở dữ liệu** (Knowledge Discovery in Databases – KDD). Trong quy trình này, "khai phá dữ liệu" (tức bước chạy thuật toán) chỉ là **một** bước, đứng sau hàng loạt bước chuẩn bị:

1. **Lựa chọn** (Selection): chọn tập dữ liệu, biến quan tâm.
2. **Tiền xử lý** (Preprocessing): làm sạch, xử lý nhiễu, xử lý giá trị thiếu.
3. **Biến đổi** (Transformation): mã hóa, chuẩn hóa, giảm chiều.
4. **Khai phá dữ liệu** (Data Mining): áp dụng thuật toán.
5. **Diễn giải / Đánh giá** (Interpretation/Evaluation).



Hình 5.3: Quy trình KDD theo Fayyad và cộng sự (1996). Tiền xử lý và biến đổi dữ liệu đứng trước bước khai phá dữ liệu.(https://onlinelibrary.wiley.com/doi/epdf/10.1609/aimag.v17i3.1230)

Cùng thời điểm, một liên minh công nghiệp gồm DaimlerChrysler, SPSS và NCR phát triển chuẩn **CRISP-DM** (Cross-Industry Standard Process for Data Mining, công bố bản 1.0 năm 2000) gồm sáu pha: *Hiểu nghiệp vụ – Hiểu dữ liệu – Chuẩn bị dữ liệu – Mô hình hóa – Đánh giá – Triển khai*. Pha *Chuẩn bị dữ liệu* được nhiều tài liệu thực hành ước tính chiếm phần lớn thời gian của một dự án (con số thường được trích dẫn là khoảng 60–80%). Cuốn *Data Preparation for Data Mining* của Dorian Pyle (1999) là một trong những chuyên khảo đầu tiên dành riêng cho chủ đề này.



Hình 5.4: Sáu pha của CRISP-DM. Pha "Chuẩn bị dữ liệu" là nơi diễn ra phần lớn các kỹ thuật trong chương này.(https://rdsic.edu.vn/blog/blog-6/1549734giai-doan-nao-trong-crisp-dm-xem-xet-cac-dinh-dang-du-lieu-cua-nguon-du-lieu-trong-a-to-vi-cb.html

5.2.3 Phân loại các tác vụ tiền xử lý

Theo Han, Kamber và Pei (*Data Mining: Concepts and Techniques*), tiền xử lý dữ liệu gồm bốn nhóm tác vụ chính:

Nhóm tác vụ	Nội dung
Làm sạch dữ liệu	Xử lý giá trị thiếu (**Simple / KNN / Iterative Imputer**), dữ liệu nhiễu, **ngoại lai** (IQR, LOF), dữ liệu sai định dạng.
Tích hợp dữ liệu	Gộp dữ liệu từ nhiều nguồn, xử lý trùng lặp và mâu thuẫn.
Biến đổi dữ liệu	**Mã hóa** (Label/One Hot Encoder), **vector hóa văn bản** (Count/TF-IDF), **chuẩn hóa thang đo** (các Scaler).
Rút gọn dữ liệu	Giảm số chiều (PCA, SVD – xem chương Giảm chiều dữ liệu), giảm số mẫu.

Bảng 5.3: Bốn nhóm tác vụ tiền xử lý dữ liệu

Chương này đi theo đúng trình tự của quy trình KDD: trước hết là nhóm **làm sạch dữ liệu** (điền giá trị thiếu, phát hiện ngoại lai), sau đó là nhóm **biến đổi dữ liệu** — nhóm kỹ thuật mà gần như mọi bài toán ML đều phải dùng — rồi đến kỹ thuật **cân bằng dữ liệu** SMOTE, và kết thúc bằng một nghiên cứu tình huống trên dữ liệu thực.

5.2.4 Thư viện scikit-learn và giao diện `fit` / `transform`

Toàn bộ kỹ thuật trong chương được cài đặt bằng thư viện **scikit-learn**. Dự án bắt đầu năm 2007 như một dự án Google Summer of Code của David Cournapeau, sau đó được nhóm nghiên cứu tại INRIA (Pháp) phát triển và phát hành công khai năm 2010; bài báo giới thiệu chính thức của Pedregosa và cộng sự đăng trên *Journal of Machine Learning Research* năm 2011.

Điểm mạnh của scikit-learn là **giao diện thống nhất** (Buitinck và cộng sự, 2013): mọi bộ biến đổi (transformer) đều có ba phương thức:

Phương thức	Ý nghĩa
`fit(X)`	**Học** các tham số từ dữ liệu (từ điển nhãn, min/max, trung bình, độ lệch chuẩn…).
`transform(X)`	**Áp dụng** các tham số đã học để biến đổi dữ liệu.
`fit_transform(X)`	Gộp hai bước trên (chỉ dùng trên tập huấn luyện).
`inverse_transform(X)`	Biến đổi ngược về giá trị gốc (khi phép biến đổi khả nghịch).

Bảng 5.4: Giao diện chung của các bộ biến đổi trong scikit-learn

Định nghĩa 5.2 (Rò rỉ dữ liệu – Data leakage)**.** *Rò rỉ dữ liệu xảy ra khi thông tin từ tập kiểm tra (test) được sử dụng, trực tiếp hay gián tiếp, trong quá trình huấn luyện. Với tiền xử lý, lỗi điển hình là gọi* `fit` *trên* ***toàn bộ*** *dữ liệu trước khi chia train/test. Quy tắc đúng:* ***chỉ*** `fit` ***trên tập huấn luyện****, sau đó dùng* `transform` *cho cả tập huấn luyện và tập kiểm tra.*

5.3 Các kiểu dữ liệu và lựa chọn phép biến đổi

Trước khi chọn kỹ thuật, cần xác định **thang đo** của từng biến. Stevens (1946) phân biệt bốn thang đo: định danh (nominal), thứ bậc (ordinal), khoảng (interval) và tỉ lệ (ratio). Trong thực hành ML, ta gom thành các nhóm sau:

Kiểu dữ liệu	Đặc điểm	Ví dụ	Phép biến đổi
Định danh (nominal)	Các giá trị chỉ khác nhau, **không có thứ tự**	màu sắc, khu phố	One Hot Encoder
Thứ bậc (ordinal)	Có **thứ tự** nhưng khoảng cách không rõ ràng	kém < TB < tốt	Ordinal Encoder
Nhãn đầu ra (target)	Biến cần dự đoán trong phân lớp	"spam"/"ham"	Label Encoder
Số (numeric)	Có giá trị và khoảng cách thực	diện tích, giá	Các Scaler
Văn bản (text)	Chuỗi từ có độ dài thay đổi	bình luận, tài liệu	Count / TF-IDF Vectorizer

Bảng 5.5: Kiểu dữ liệu và kỹ thuật tiền xử lý tương ứng

5.4 Xử lý dữ liệu thiếu (Missing Values)

Dữ liệu thiếu xuất hiện trong hầu hết các bộ dữ liệu thực tế: người được khảo sát bỏ trống câu hỏi, cảm biến mất tín hiệu, dữ liệu ghép từ nhiều nguồn không khớp nhau… Trong NumPy và pandas, giá trị thiếu được biểu diễn bằng `NaN` (*Not a Number* — một giá trị đặc biệt của chuẩn số thực dấu phẩy động IEEE 754). Phần lớn mô hình của scikit-learn (hồi quy tuyến tính, SVM, KNN, PCA…) sẽ **báo lỗi** khi gặp `NaN`, vì vậy xử lý dữ liệu thiếu là bước bắt buộc trước khi huấn luyện.

5.4.1 Cơ chế sinh dữ liệu thiếu

Donald Rubin (1976) đặt nền móng lý thuyết cho bài toán dữ liệu thiếu khi phân biệt ba **cơ chế thiếu**:

Cơ chế	Ý nghĩa	Ví dụ
MCAR – Missing Completely At Random	Việc thiếu hoàn toàn ngẫu nhiên, không phụ thuộc vào biến nào	Phiếu khảo sát bị thất lạc ngẫu nhiên
MAR – Missing At Random	Việc thiếu chỉ phụ thuộc vào các biến **quan sát được**	Người trẻ ít khai thu nhập hơn (tuổi đã biết)
MNAR – Missing Not At Random	Việc thiếu phụ thuộc vào chính **giá trị bị thiếu**	Người thu nhập rất cao từ chối khai thu nhập

Bảng 5.6: Ba cơ chế thiếu dữ liệu theo Rubin (1976)

Xóa các dòng có ô trống (`df.dropna()`) chỉ an toàn khi dữ liệu thiếu kiểu MCAR và chiếm tỉ lệ nhỏ; trong các trường hợp còn lại, xóa dòng vừa lãng phí dữ liệu vừa làm **lệch** mẫu. Giải pháp phổ biến hơn là **điền giá trị thiếu** (imputation). scikit-learn cung cấp ba bộ điền với mức độ phức tạp tăng dần: `SimpleImputer`, `KNNImputer` và `IterativeImputer`. Giống mọi bộ biến đổi khác, bộ điền phải được `fit` **chỉ trên tập huấn luyện**.

5.4.2 Simple Imputer: trung bình, trung vị, giá trị phổ biến

Cách đơn giản nhất là thay mọi ô thiếu của một cột bằng một **thống kê của chính cột đó**, tính trên các giá trị quan sát được:

- `strategy="mean"` – trung bình cộng: phù hợp với dữ liệu gần đối xứng, ít ngoại lai.
- `strategy="median"` – trung vị: bền vững với ngoại lai và dữ liệu lệch.
- `strategy="most_frequent"` – giá trị xuất hiện nhiều nhất (mode): dùng được cho cả cột chữ.
- `strategy="constant"` – một hằng số do người dùng chọn (ví dụ `"NA"` hoặc 0).

Ví dụ 5.2 (Điền bằng trung bình và trung vị)**.** *Hai cột dữ liệu, mỗi cột thiếu một giá trị:*

$$A = (7,\ 3,\ 4,\ 6,\ 2,\ ?), \qquad B = (6,\ 8,\ 9,\ 8,\ 7,\ ?).$$

Cách 1 – theo trung bình:

$$\bar{A} = \frac{7+3+4+6+2}{5} = \frac{22}{5} = 4{,}4, \qquad \bar{B} = \frac{6+8+9+8+7}{5} = \frac{38}{5} = 7{,}6.$$

Cách 2 – theo trung vị: *sắp xếp các giá trị quan sát được rồi chọn phần tử ở giữa:* $A \to (2, 3, \mathbf{4}, 6, 7)$*, điền 4;* $B \to (6, 7, \mathbf{8}, 8, 9)$*, điền 8.*

Khi số giá trị quan sát được là ***chẵn****, trung vị là trung bình cộng của hai phần tử ở giữa. Ví dụ với* $(6, 9, 7, 3, 2, 5)$*: sắp xếp thành* $(2, 3, \mathbf{5}, \mathbf{6}, 7, 9)$*, trung vị bằng* $\frac{5+6}{2} = 5{,}5$*.*

Listing 5.1: SimpleImputer với ví dụ trên

```
import numpy as np
from sklearn.impute import SimpleImputer

X = np.array([[7, 6], [3, 8], [4, 9], [6, 8], [2, 7], [np.nan, np.nan]])
for s in ["mean", "median", "most_frequent"]:
    imp = SimpleImputer(strategy=s).fit(X)
    print(s, imp.statistics_)
# mean          [4.4 7.6]
# median        [4.  8. ]
# most_frequent [2.  8. ]  (cot A khong co gia tri lap -> lay gia tri nho nhat)
print(np.median([6, 9, 7, 3, 2, 5]))     # 5.5
```

Hạn chế. Điền bằng một hằng số làm **giảm phương sai** của cột và **làm yếu tương quan** giữa các cột, vì mọi ô thiếu nhận cùng một giá trị bất kể các thuộc tính còn lại của bản ghi. Tham số `add_indicator=True` bổ sung cột nhị phân đánh dấu ô nào đã được điền — bản thân sự kiện "bị thiếu" nhiều khi cũng mang thông tin có ích cho mô hình.

5.4.3 KNN Imputer và khoảng cách nan-Euclid

Troyanskaya và cộng sự (2001) đề xuất điền giá trị thiếu trong dữ liệu biểu hiện gen bằng các **láng giềng gần nhất**: hai bản ghi giống nhau ở các cột quan sát được thì nhiều khả năng cũng giống nhau ở cột bị thiếu. Thuật toán KNN được trình bày đầy đủ ở Chương 7; ở đây chỉ cần ý tưởng: *giá trị điền là trung bình của cột đó trên k bản ghi gần nhất*.

Khó khăn là chính các bản ghi cũng có ô trống, nên không tính được khoảng cách Euclid thông thường. Ta dùng khoảng cách **nan-Euclid**: chỉ cộng trên những tọa độ mà **cả hai** điểm đều có giá trị, rồi nhân với một hệ số để bù cho các tọa độ bị bỏ qua.

Định nghĩa 5.3 (Khoảng cách nan-Euclid)**.** *Với hai vector $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$ có thể chứa giá trị thiếu, gọi P là tập các tọa độ mà cả hai vector cùng có giá trị:*

$$d_{nan}(\mathbf{x}, \mathbf{y}) = \sqrt{\frac{d}{|P|} \sum_{j \in P} (x_j - y_j)^2}, \tag{5.1}$$

*trong đó d là **tổng số chiều** của bài toán và $|P|$ là **số chiều tính được**.*

Ví dụ 5.3 (Tính khoảng cách nan-Euclid)**.**

- $\mathbf{x} = (5, \mathit{nan})$ *và* $\mathbf{y} = (4, 5)$*: chỉ tọa độ thứ nhất tính được, $d = 2$, $|P| = 1$:*

$$d_{nan} = \sqrt{\tfrac{2}{1}(5-4)^2} = \sqrt{2} \approx 1{,}414.$$

- $\mathbf{A} = (3, \mathit{nan}, \mathit{nan}, 6)$ *và* $\mathbf{B} = (1, \mathit{nan}, 4, 5)$*: tính được tọa độ 1 và 4, $d = 4$, $|P| = 2$:*

$$d_{nan} = \sqrt{\tfrac{4}{2}\left[(3-1)^2 + (6-5)^2\right]} = \sqrt{10} \approx 3{,}162.$$

Ví dụ 5.4 (Chạy tay KNN Imputer với $k = 3$)**.** *Điền ba ô thiếu trong Bảng 5.7.*

STT	x_1	x_2	x_3
1	7	6	4
2	**?**	4	5
3	1	5	**?**
4	3	9	7
5	8	**?**	8

Bảng 5.7: Dữ liệu ví dụ cho KNN Imputer

Bước 1 – xác định X và y. Để điền x_1 ở dòng 2, coi cột x_1 là "nhãn" y, hai cột còn lại (x_2, x_3) là đặc trưng X. Điểm cần dự đoán là $\mathbf{q} = (4, 5)$; các dòng được làm láng giềng phải **có** giá trị x_1 (dòng 1, 3, 4, 5).

Bước 2 – tính khoảng cách nan-Euclid trên hai cột (x_2, x_3), $d = 2$:

$$\begin{aligned}
d(\mathbf{q}, \text{dòng } 1) &= \sqrt{(4-6)^2 + (5-4)^2} = \sqrt{5} \approx 2{,}236, \\
d(\mathbf{q}, \text{dòng } 3) &= \sqrt{\tfrac{2}{1}(4-5)^2} = \sqrt{2} \approx 1{,}414 && \text{(dòng 3 thiếu } x_3\text{)}, \\
d(\mathbf{q}, \text{dòng } 4) &= \sqrt{(4-9)^2 + (5-7)^2} = \sqrt{29} \approx 5{,}385, \\
d(\mathbf{q}, \text{dòng } 5) &= \sqrt{\tfrac{2}{1}(5-8)^2} = \sqrt{18} \approx 4{,}243 && \text{(dòng 5 thiếu } x_2\text{)}.
\end{aligned}$$

Bước 3 – lấy trung bình của $k = 3$ láng giềng gần nhất (dòng 3, 1, 5): $x_1 = \frac{1+7+8}{3} \approx 5{,}333$.

Làm tương tự cho hai ô còn lại, ta được bảng tổng hợp:

Ô cần điền	Điểm q	Khoảng cách đến các dòng	3 láng giềng	Giá trị điền
x_1, dòng 2	$(x_2, x_3) = (4, 5)$	$d_1 = 2{,}236$; $d_3 = 1{,}414$; $d_4 = 5{,}385$; $d_5 = 4{,}243$	3, 1, 5	$\frac{1+7+8}{3} \approx 5{,}333$
x_3, dòng 3	$(x_1, x_2) = (1, 5)$	$d_1 = 6{,}083$; $d_2 = 1{,}414$; $d_4 = 4{,}472$; $d_5 = 9{,}899$	2, 4, 1	$\frac{5+7+4}{3} \approx 5{,}333$
x_2, dòng 5	$(x_1, x_3) = (8, 8)$	$d_1 = 4{,}123$; $d_2 = 4{,}243$; $d_3 = 9{,}899$; $d_4 = 5{,}099$	1, 2, 4	$\frac{6+4+9}{3} \approx 6{,}333$

Bảng 5.8: Kết quả chạy tay KNN Imputer ($k = 3$)

Lưu ý.

- Hệ số $d/|P|$ phải được áp dụng **mỗi khi** có tọa độ bị thiếu. Quên hệ số này — chẳng hạn viết $d(\mathbf{q}, \text{dòng } 5) = \sqrt{(5-8)^2} = 3$ thay vì $\sqrt{18} \approx 4{,}243$ — là lỗi hay gặp khi tính tay. Ở ví dụ này lỗi đó tình cờ không làm đổi tập láng giềng, nhưng trong trường hợp tổng quát sẽ cho kết quả sai.
- `KNNImputer` của scikit-learn đo khoảng cách trên **toàn bộ** 3 cột. Tọa độ đang cần điền của dòng nhận vốn bị thiếu nên tự động bị loại, và hệ số trở thành $3/|P|$ thay vì $2/|P|$. Mọi khoảng cách từ dòng nhận vì thế cùng được nhân với $\sqrt{3/2}$, nên thứ tự láng giềng và giá trị điền **không đổi**.
- KNN Imputer dựa trên khoảng cách nên cần **chuẩn hóa thang đo** (mục 5.6) trước khi dùng; tham số `weights="distance"` cho láng giềng gần hơn trọng số lớn hơn.

Listing 5.2: Khoảng cách nan-Euclid và KNNImputer

```
import numpy as np
from sklearn.impute import KNNImputer
from sklearn.metrics.pairwise import nan_euclidean_distances

nan = np.nan
print(nan_euclidean_distances([[5, nan]], [[4, 5]]))                    #
    [[1.4142]]
print(nan_euclidean_distances([[3, nan, nan, 6]], [[1, nan, 4, 5]]))  #
    [[3.1623]]

X = np.array([[7, 6, 4], [nan, 4, 5], [1, 5, nan], [3, 9, 7], [8, nan, 8]])
print(KNNImputer(n_neighbors=3).fit_transform(X).round(3))
# [[7.    6.    4.   ]
#  [5.333 4.    5.   ]
#  [1.    5.    5.333]
#  [3.    9.    7.   ]
#  [8.    6.333 8.   ]]
```

5.4.4 Iterative Imputer

Simple Imputer bỏ qua quan hệ giữa các cột; KNN Imputer khai thác quan hệ đó một cách cục bộ. **Iterative Imputer** đi xa hơn: *mô hình hóa mỗi cột có giá trị thiếu như một hàm hồi quy của các cột còn lại*. Ý tưởng này bắt nguồn từ kỹ thuật **MICE** (Multivariate Imputation by Chained Equations) của van Buuren và Groothuis-Oudshoorn (2011), vốn phát triển từ khung *điền khuyết bội* (multiple imputation) của Rubin (1987). Trong scikit-learn, lớp `IterativeImputer` vẫn được xếp vào nhóm thử nghiệm, nên cần khai báo `from sklearn.experimental import enable_iterative_imputer` trước khi dùng.

Thuật toán:

1. **Khởi tạo**: điền tạm mọi ô thiếu bằng trung bình cột.
2. Lần lượt với từng cột x_j có ô thiếu: coi x_j là biến mục tiêu y, các cột còn lại (với giá trị đang điền tạm) là X; huấn luyện mô hình hồi quy trên các dòng mà x_j được quan sát; dự đoán và **cập nhật** các ô thiếu của x_j.
3. Lặp lại bước 2 cho đến khi các giá trị điền ổn định hoặc đạt số vòng tối đa `max_iter`.

Bộ hồi quy ở bước 2 có thể là bất kỳ mô hình nào. Trong ví dụ dưới đây ta dùng hồi quy tuyến tính, giải bằng phương trình chuẩn $\boldsymbol{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ (Chương 6); cột đầu tiên của $\mathbf{X}$ toàn số 1 ứng với hệ số chặn β_0.

Ví dụ 5.5 (Chạy tay một vòng Iterative Imputer)**.** *Điền ba ô thiếu trong Bảng 5.9 theo thứ tự cột* $x_1 \to x_2 \to x_3$.

STT	x_1	x_2	x_3
1	3	7	1
2	**?**	4	8
3	6	3	7
4	5	**?**	4
5	1	9	**?**

Bảng 5.9: Dữ liệu ví dụ cho Iterative Imputer

Bước 1 – khởi tạo bằng trung bình cột:

$$\bar{x}_1 = \frac{3+6+5+1}{4} = 3{,}75, \qquad \bar{x}_2 = \frac{7+4+3+9}{4} = 5{,}75, \qquad \bar{x}_3 = \frac{1+8+7+4}{4} = 5.$$

Bước 2 – dự đoán x_1 từ (x_2, x_3). Dùng bốn dòng có x_1 (dòng 4 đang mang giá trị tạm $x_2 = 5{,}75$, dòng 5 mang $x_3 = 5$):

$$\mathbf{X} = \begin{pmatrix} 1 & 7 & 1 \\ 1 & 3 & 7 \\ 1 & 5{,}75 & 4 \\ 1 & 9 & 5 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 3 \\ 6 \\ 5 \\ 1 \end{pmatrix}, \quad \mathbf{X}^\top\mathbf{X} = \begin{pmatrix} 4 & 24{,}75 & 17 \\ 24{,}75 & 172{,}06 & 96 \\ 17 & 96 & 91 \end{pmatrix}, \quad \mathbf{X}^\top\mathbf{y} = \begin{pmatrix} 15 \\ 76{,}75 \\ 70 \end{pmatrix},$$

$$(\mathbf{X}^\top\mathbf{X})^{-1} \approx \begin{pmatrix} 5{,}9563 & -0{,}5735 & -0{,}5077 \\ -0{,}5735 & 0{,}0693 & 0{,}0340 \\ -0{,}5077 & 0{,}0340 & 0{,}0700 \end{pmatrix}, \qquad \boldsymbol{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} \approx \begin{pmatrix} 9{,}78895 \\ -0{,}90153 \\ -0{,}10841 \end{pmatrix}.$$

Thay dòng 2 ($x_2 = 4$, $x_3 = 8$): $x_1 = 9{,}78895 - 0{,}90153 \times 4 - 0{,}10841 \times 8 \approx 5{,}3155$.
Bước 3 – dự đoán x_2 từ (x_1, x_3), thế giá trị vừa điền $x_1 = 5{,}3155$ vào dòng 2 (dòng 5 vẫn mang $x_3 = 5$):

$$\mathbf{X} = \begin{pmatrix} 1 & 3 & 1 \\ 1 & 5{,}3155 & 8 \\ 1 & 6 & 7 \\ 1 & 1 & 5 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 7 \\ 4 \\ 3 \\ 9 \end{pmatrix} \Rightarrow \boldsymbol{\beta} \approx \begin{pmatrix} 10{,}4972 \\ -1{,}1578 \\ -0{,}0598 \end{pmatrix}.$$

Thay dòng 4 ($x_1 = 5$, $x_3 = 4$): $x_2 = 10{,}4972 - 1{,}1578 \times 5 - 0{,}0598 \times 4 \approx 4{,}469$.
Bước 4 – dự đoán x_3 từ (x_1, x_2) với các giá trị vừa điền (tính với đầy đủ chữ số):

$$\mathbf{X} = \begin{pmatrix} 1 & 3 & 7 \\ 1 & 5{,}3155 & 4 \\ 1 & 6 & 3 \\ 1 & 5 & 4{,}4689 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 8 \\ 7 \\ 4 \end{pmatrix} \Rightarrow \boldsymbol{\beta} \approx \begin{pmatrix} 9{,}7807 \\ 0{,}3487 \\ -1{,}4001 \end{pmatrix}.$$

Thay dòng 5 ($x_1 = 1$, $x_2 = 9$): $x_3 = 9{,}7807 + 0{,}3487 \times 1 - 1{,}4001 \times 9 \approx -2{,}47$.

Kết thúc vòng thứ nhất: $x_1 \approx 5{,}32$; $x_2 \approx 4{,}47$; $x_3 \approx -2{,}47$.

Phân tích kết quả: bài toán điều kiện xấu. Giá trị $x_3 \approx -2{,}47$ nằm **ngoài** miền quan sát $[1;\ 8]$ của cột x_3 — một dấu hiệu cảnh báo. Nguyên nhân nằm ở bốn dòng dùng để hồi quy ở bước 4: tổng $x_1 + x_2$ của chúng lần lượt là 10; $9{,}32$; 9; $9{,}47$, tức hai đặc trưng gần như phụ thuộc tuyến tính (**đa cộng tuyến**). Khi đó $\mathbf{X}^\top\mathbf{X}$ gần suy biến: số điều kiện của nó vào khoảng $1{,}8 \times 10^6$ và phần tử lớn nhất của $(\mathbf{X}^\top\mathbf{X})^{-1}$ lên tới khoảng 9505. Hệ quả là nghiệm cực kỳ nhạy với sai số làm tròn ở các bước trước:

x_1 **(dòng 2)**	x_2 **(dòng 4)**	β_0	β_1	β_2	x_3 **(dòng 5)**
5,31553… (đủ chữ số)	4,46889…	9,781	0,349	$-1{,}400$	$-2{,}47$
5,316	4,469	8,852	0,460	$-1{,}316$	$-2{,}53$
5,32	4,47	1,242	1,375	$-0{,}624$	$-3{,}00$
5,306	4,47	32,894	$-2{,}430$	$-3{,}501$	$-1{,}04$
5,3445	4,47	$-41{,}058$	6,445	3,224	$-5{,}59$

Bảng 5.10: Độ nhạy của bước 4 khi các giá trị điền trước đó bị làm tròn

Chỉ cần x_1 lệch $0{,}01$ là hệ số chặn β_0 đã nhảy từ khoảng 9,8 lên 32,9 hay xuống -41. Vì vậy, khi tính tay mỗi người làm tròn một kiểu sẽ cho những hệ số rất khác nhau, dù **không ai tính sai phép tính**. Lặp thêm nhiều vòng cũng không giúp gì: với `max_iter=10`, giá trị điền của x_3 còn trôi tới khoảng $-28{,}5$.

Bài học thực hành:

1. Luôn **kiểm tra tính hợp lý** của giá trị được điền (so với miền quan sát, ý nghĩa nghiệp vụ).
2. Giới hạn miền giá trị bằng `min_value`, `max_value`.
3. Ưu tiên bộ hồi quy **có điều chuẩn** — mặc định của `IterativeImputer` là `BayesianRidge` — vì hệ số ổn định hơn khi có đa cộng tuyến (xem Ridge ở Chương 6). Tuy vậy, điều chuẩn không thay thế được bước kiểm tra: trên chính ví dụ này, `BayesianRidge` không giới hạn miền vẫn cho $x_3 \approx -2{,}51$.

4. Chỉ dùng Iterative Imputer khi có **đủ dữ liệu** để ước lượng quan hệ giữa các cột; với bốn dòng như ví dụ, Simple Imputer hoặc KNN Imputer an toàn hơn.

Listing 5.3: IterativeImputer tái hiện ví dụ tính tay

```
import numpy as np
from sklearn.experimental import enable_iterative_imputer  # bat tinh nang thu nghiem
from sklearn.impute import IterativeImputer
from sklearn.linear_model import LinearRegression

nan = np.nan
X = np.array([[3, 7, 1], [nan, 4, 8], [6, 3, 7], [5, nan, 4], [1, 9, nan]])

# Mot vong, hoi quy tuyen tinh, thu tu cot x1 -> x2 -> x3 (giong tinh tay)
imp = IterativeImputer(estimator=LinearRegression(), initial_strategy="mean",
                       imputation_order="roman", max_iter=1)
print(imp.fit_transform(X).round(4))
# dong 2: x1 = 5.3155 ; dong 4: x2 = 4.4689 ; dong 5: x3 = -2.4715

# Mac dinh BayesianRidge, 10 vong, gioi han mien gia tri theo du lieu quan sat
lo, hi = np.nanmin(X, axis=0), np.nanmax(X, axis=0)
imp = IterativeImputer(max_iter=10, min_value=lo, max_value=hi, random_state=0)
print(imp.fit_transform(X).round(3))
# dong 2: x1 = 4.833 ; dong 4: x2 = 4.5 ; dong 5: x3 = 1.0 (bi chan duoi)
```

Bộ điền	Ý tưởng	Ưu điểm	Nhược điểm
SimpleImputer	Thống kê của chính cột (mean, median, mode)	Nhanh, đơn giản, ổn định	Bỏ qua quan hệ giữa các cột; làm giảm phương sai
KNNImputer	Trung bình của k láng giềng theo khoảng cách nan-Euclid	Tận dụng sự tương đồng giữa các bản ghi	Chậm với dữ liệu lớn; cần chuẩn hóa thang đo
IterativeImputer	Hồi quy lặp từng cột theo các cột còn lại (MICE)	Khai thác quan hệ đa biến	Chậm; có thể ngoại suy phi thực tế; cần kiểm tra kết quả

Bảng 5.11: So sánh ba bộ điền giá trị thiếu

5.5 Mã hóa dữ liệu phân loại

5.5.1 Label Encoder

Định nghĩa 5.4 (Label Encoding)**.** *Cho tập K nhãn phân biệt $\mathcal{C} = \{c_0, c_1, \ldots, c_{K-1}\}$. Label Encoding là một song ánh*

$$\phi : \mathcal{C} \to \{0, 1, \ldots, K-1\}, \qquad \phi(c_k) = k, \tag{5.2}$$

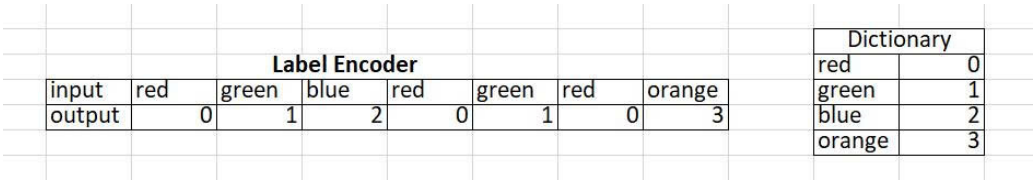
*gán cho mỗi nhãn một số nguyên duy nhất. Kết quả là một **từ điển** (dictionary) ánh xạ nhãn $\to$ số.*

Ví dụ 5.6. *Cho dãy đầu vào gồm 7 phần tử:* `red`, `green`, `blue`, `red`, `green`, `red`, `orange`. *Duyệt dãy từ trái sang phải, mỗi khi gặp một nhãn **mới** ta cấp cho nó số tiếp theo:*

Input	red	green	blue	red	green	red	orange
Output	0	1	2	0	1	0	3

Dictionary	
red	0
green	1
blue	2
orange	3

Bảng 5.12: Label Encoding theo thứ tự xuất hiện



Hình 5.5: Minh họa Label Encoder trên bảng tính: dòng input, dòng output và từ điển ánh xạ.

Để hiểu bản chất thuật toán, ta tự cài đặt Label Encoder chỉ bằng các cấu trúc đã học ở chương Python cơ bản: **list, vòng lặp `for` và dictionary**.

Listing 5.4: Tự cài đặt Label Encoder bằng list, for và dictionary

```
data = ["red", "green", "blue", "red", "green", "red", "orange"]

# Buoc 1: xay dung tu dien nhan -> so (theo thu tu xuat hien)
dictionary = {}
for label in data:
    if label not in dictionary:
        dictionary[label] = len(dictionary)
print(dictionary)
# {'red': 0, 'green': 1, 'blue': 2, 'orange': 3}

# Buoc 2: ma hoa du lieu
encoded = [dictionary[label] for label in data]
print(encoded)          # [0, 1, 2, 0, 1, 0, 3]

# Buoc 3: giai ma (inverse) bang tu dien dao
inverse = {v: k for k, v in dictionary.items()}
print([inverse[v] for v in encoded])
```

Trong scikit-learn, lớp `LabelEncoder` làm việc tương tự nhưng có một điểm khác quan trọng: các nhãn được **sắp xếp theo thứ tự từ điển (alphabet)** trước khi đánh số, chứ không theo thứ tự xuất hiện.

Listing 5.5: Label Encoder với scikit-learn

```
from sklearn.preprocessing import LabelEncoder

data = ["red", "green", "blue", "red", "green", "red", "orange"]
le = LabelEncoder()
```

```
encoded = le.fit_transform(data)

print(le.classes_)    # ['blue' 'green' 'orange' 'red']
print(encoded)        # [3 1 0 3 1 3 2]
print(le.inverse_transform([0, 3]))   # ['blue' 'red']
```

Hạn chế của Label Encoder. Label Encoder tạo ra một **thứ tự giả** giữa các nhãn. Với từ điển ở Bảng 5.12, khoảng cách Euclid giữa các màu là:

$$d(\text{red}, \text{green}) = |0 - 1| = 1, \qquad d(\text{red}, \text{orange}) = |0 - 3| = 3. \tag{5.3}$$

Nghĩa là mô hình sẽ "hiểu" rằng màu đỏ giống màu xanh lá hơn màu cam ba lần — một kết luận hoàn toàn vô nghĩa, chỉ xuất phát từ việc ta đánh số tùy ý. Với các thuật toán dựa trên khoảng cách như **KNN, K-Means** hoặc các mô hình tuyến tính, đây là nguồn sai lệch nghiêm trọng.

> **Lưu ý chuẩn mực:** Trong scikit-learn, `LabelEncoder` được thiết kế để mã hóa **biến mục tiêu** y (nhãn lớp trong bài toán phân lớp). Với các **đặc trưng đầu vào** có thứ bậc, dùng `OrdinalEncoder` (cho phép tự khai báo thứ tự); với đặc trưng định danh, dùng `OneHotEncoder`.

Listing 5.6: Ordinal Encoder với thứ tự do người dùng khai báo

```
from sklearn.preprocessing import OrdinalEncoder

quality = [["Po"], ["TA"], ["Gd"], ["Ex"], ["TA"], ["Fa"]]
oe = OrdinalEncoder(categories=[["Po", "Fa", "TA", "Gd", "Ex"]])
print(oe.fit_transform(quality).ravel())
# [0. 2. 3. 4. 2. 1.]   -> Po < Fa < TA < Gd < Ex
```

5.5.2 One Hot Encoder

Ý tưởng mã hóa "một bit bật" (*one-hot*) xuất phát từ thiết kế mạch số, nơi mỗi trạng thái của máy trạng thái được biểu diễn bằng đúng một flip-flop ở mức 1. Trong thống kê, kỹ thuật tương đương là **biến giả** (dummy variable), được Suits (1957) phổ biến cho hồi quy với biến phân loại.

Định nghĩa 5.5 (One Hot Encoding)**.** *Với tập K nhãn $\mathcal{C} = \{c_0, \ldots, c_{K-1}\}$, One Hot Encoding ánh xạ nhãn c_k thành vector đơn vị thứ k trong $\mathbb{R}^K$:*

$$\psi(c_k) = \mathbf{e}_k = (0, \ldots, 0, \underbrace{1}_{\text{vị trí } k}, 0, \ldots, 0) \in \{0, 1\}^K. \tag{5.4}$$

*Một cột phân loại được thay bằng K cột nhị phân; mỗi hàng có **đúng một** số 1.*

Ưu điểm then chốt: mọi cặp nhãn khác nhau đều **cách đều nhau**:

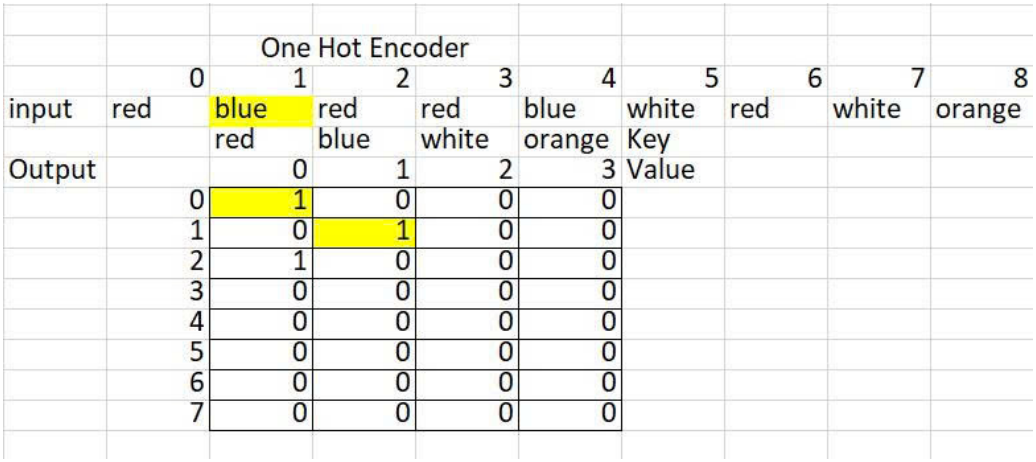
$$\|\mathbf{e}_i - \mathbf{e}_j\|_2 = \sqrt{2}, \qquad \forall\, i \neq j, \tag{5.5}$$

nên không còn thứ tự giả. Đây là lý do One Hot Encoder là lựa chọn mặc định cho đặc trưng định danh khi dùng **K-Means, KNN**, SVM hay hồi quy tuyến tính.

Ví dụ 5.7. *Cho dãy 9 phần tử (chỉ số 0–8):* `red, blue, red, red, blue, white, red, white, orange`. *Từ điển theo thứ tự xuất hiện là {red: 0, blue: 1, white: 2, orange: 3}, do đó mỗi phần tử trở thành một vector 4 chiều:*

Chỉ số	Input	red (0)	blue (1)	white (2)	orange (3)
0	red	1	0	0	0
1	blue	0	1	0	0
2	red	1	0	0	0
3	red	1	0	0	0
4	blue	0	1	0	0
5	white	0	0	1	0
6	red	1	0	0	0
7	white	0	0	1	0
8	orange	0	0	0	1

Bảng 5.13: Ma trận One Hot (9×4) cho dãy màu sắc



Hình 5.6: Minh họa One Hot Encoder trên bảng tính: từ điển (Key – Value) và ma trận đầu ra; ô được tô vàng là vị trí số 1 tương ứng với nhãn của từng hàng.

Listing 5.7: Tự cài đặt One Hot Encoder bằng list, for và dictionary

```
data = ["red", "blue", "red", "red", "blue",
        "white", "red", "white", "orange"]

# Buoc 1: tu dien (Key -> Value)
dictionary = {}
for label in data:
    if label not in dictionary:
        dictionary[label] = len(dictionary)
K = len(dictionary)          # so cot = so nhan phan biet = 4

# Buoc 2: moi phan tu -> vector co dung mot so 1
one_hot = []
for label in data:
```

```
    row = [0] * K
    row[dictionary[label]] = 1
    one_hot.append(row)

for i, row in enumerate(one_hot):
    print(i, data[i], row)
# 0 red   [1, 0, 0, 0]
# 1 blue  [0, 1, 0, 0]
# ...
# 8 orange [0, 0, 0, 1]
```

Listing 5.8: One Hot Encoder với scikit-learn và pandas

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder

data = np.array(["red", "blue", "red", "red", "blue",
                 "white", "red", "white", "orange"]).reshape(-1, 1)

ohe = OneHotEncoder(handle_unknown="ignore")
X = ohe.fit_transform(data)       # ket qua la ma tran thua (sparse)
print(ohe.get_feature_names_out())
# ['x0_blue' 'x0_orange' 'x0_red' 'x0_white']   (sap xep alphabet)
print(X.toarray()[:3])
# [[0. 0. 1. 0.]
#  [1. 0. 0. 0.]
#  [0. 0. 1. 0.]]

# Nhan moi chua tung gap -> vector toan 0 thay vi bao loi
print(ohe.transform([["green"]]).toarray())   # [[0. 0. 0. 0.]]

# Cach nhanh voi pandas
print(pd.get_dummies(pd.Series(data.ravel()), dtype=int).head(3))
```

Một số vấn đề cần lưu ý.

- **Bẫy biến giả** (dummy variable trap): K cột one-hot luôn có tổng bằng 1, tức phụ thuộc tuyến tính hoàn toàn với cột hệ số chặn. Với hồi quy tuyến tính không chính quy hóa, nên bỏ một cột bằng `OneHotEncoder(drop="first")` hoặc `pd.get_dummies(..., drop_first=True)`.
- **Bùng nổ số chiều**: biến có hàng nghìn giá trị (mã bưu chính, mã sản phẩm) sinh ra hàng nghìn cột. Khi đó cần gộp các nhãn hiếm, dùng ma trận thưa hoặc các kỹ thuật như target encoding, hashing.
- **Nhãn chưa từng gặp** ở tập kiểm tra: dùng `handle_unknown="ignore"` để tránh lỗi khi triển khai.

Tiêu chí	Label / Ordinal Encoder	One Hot Encoder
Số cột sinh ra	1	K (số nhãn)
Tạo thứ tự giả	Có	Không
Phù hợp với	Biến mục tiêu; đặc trưng thứ bậc; mô hình cây	Đặc trưng định danh; KNN, K-Means, SVM, mô hình tuyến tính
Nhược điểm	Sai lệch khoảng cách	Tăng số chiều, ma trận thưa

Bảng 5.14: So sánh Label Encoder và One Hot Encoder

5.6 Chuẩn hóa thang đo đặc trưng (Feature Scaling)

5.6.1 Vì sao cần chuẩn hóa thang đo?

Xét hai đặc trưng của một căn nhà: diện tích lô đất (hàng nghìn đến hàng trăm nghìn feet vuông) và số phòng tắm (0 đến 4). Trong khoảng cách Euclid

$$d(\mathbf{x}, \mathbf{z}) = \sqrt{(x_1 - z_1)^2 + (x_2 - z_2)^2}, \tag{5.6}$$

chênh lệch diện tích sẽ áp đảo hoàn toàn chênh lệch số phòng tắm, dù trên thực tế số phòng tắm có thể quan trọng không kém. Chuẩn hóa thang đo đưa mọi đặc trưng về cùng một "đơn vị" để:

- Các thuật toán dựa trên **khoảng cách** (KNN, K-Means, SVM với nhân RBF) đối xử công bằng giữa các đặc trưng.
- **Gradient Descent** hội tụ nhanh hơn, vì mặt hàm mất mát bớt bị kéo dài theo một chiều.
- **PCA** không bị chi phối bởi đặc trưng có phương sai lớn chỉ do đơn vị đo.
- Các mô hình có **chính quy hóa** (Ridge, Lasso) phạt các hệ số một cách công bằng.

Ngược lại, các mô hình dựa trên **cây quyết định** (Decision Tree, Random Forest, XGBoost) chỉ so sánh ngưỡng trên từng đặc trưng nên **không** cần chuẩn hóa.

Ký hiệu một cột đặc trưng là $x = (x_1, \ldots, x_n)$. Bốn bộ chuẩn hóa phổ biến trong scikit-learn được định nghĩa như sau.

5.6.2 Min Max Scaler

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)} \in [0, 1]. \tag{5.7}$$

Tổng quát hơn, với `feature_range=(a, b)`: $x''_i = a + x'_i(b - a)$. Phép biến đổi giữ nguyên hình dạng phân phối, nhưng **rất nhạy với ngoại lai** vì min và max chính là các giá trị cực trị.

5.6.3 Max Abs Scaler

$$x'_i = \frac{x_i}{\max_j |x_j|} \in [-1, 1]. \tag{5.8}$$

Max Abs Scaler không dịch chuyển dữ liệu (không trừ đi giá trị nào), nên **giữ nguyên các số 0**. Đây là lựa chọn phù hợp cho **ma trận thưa** như ma trận TF-IDF (Mục 5.9.2): trừ đi trung bình sẽ biến mọi số 0 thành khác 0 và phá vỡ tính thưa.

5.6.4 Standard Scaler (chuẩn hóa z-score)

$$z_i = \frac{x_i - \mu}{\sigma}, \qquad \mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}. \tag{5.9}$$

Sau biến đổi, cột có trung bình 0 và độ lệch chuẩn 1. Lưu ý scikit-learn dùng độ lệch chuẩn **tổng thể** (chia cho n, tức `ddof=0`), khác với `pandas.Series.std()` mặc định chia cho $n-1$. Standard Scaler là lựa chọn mặc định cho hồi quy tuyến tính, hồi quy logistic, SVM, PCA và mạng nơ-ron.

5.6.5 Robust Scaler

$$x_i' = \frac{x_i - \text{median}(x)}{\text{IQR}(x)}, \qquad \text{IQR} = Q_3 - Q_1, \tag{5.10}$$

trong đó Q_1, Q_3 là các phân vị 25% và 75%. Trung vị và khoảng tứ phân vị **gần như không bị ảnh hưởng bởi ngoại lai**, nên Robust Scaler phù hợp với dữ liệu lệch hoặc có giá trị bất thường (cách phát hiện ngoại lai được trình bày ở Mục 5.7).

5.6.6 Ví dụ so sánh bốn bộ chuẩn hóa

Ví dụ 5.8. *Cho cột* $x = (2, 4, 6, 8, 30)$*, trong đó* 30 *là một ngoại lai. Các đại lượng cần thiết:* $\min = 2$, $\max = 30$, $\max|x| = 30$, $\mu = 10$, $\sigma = \sqrt{104} \approx 10{,}198$, $\text{median} = 6$, $Q_1 = 4$, $Q_3 = 8$, $\text{IQR} = 4$.

x	MinMax	MaxAbs	Standard	Robust
2	0.0000	0.0667	-0.7845	-1.0
4	0.0714	0.1333	-0.5883	-0.5
6	0.1429	0.2000	-0.3922	0.0
8	0.2143	0.2667	-0.1961	0.5
30	1.0000	1.0000	1.9612	6.0

Bảng 5.15: Kết quả của bốn bộ chuẩn hóa trên cột có ngoại lai

Quan sát Bảng 5.15: với Min Max Scaler, bốn giá trị "bình thường" bị ép vào đoạn $[0;\ 0{,}21]$ — gần như mất khả năng phân biệt — chỉ vì một ngoại lai. Robust Scaler giữ các giá trị bình thường trải đều trong $[-1;\ 0{,}5]$ và đẩy ngoại lai ra xa (6,0), giúp ngoại lai **dễ nhận diện** thay vì làm méo phần còn lại.

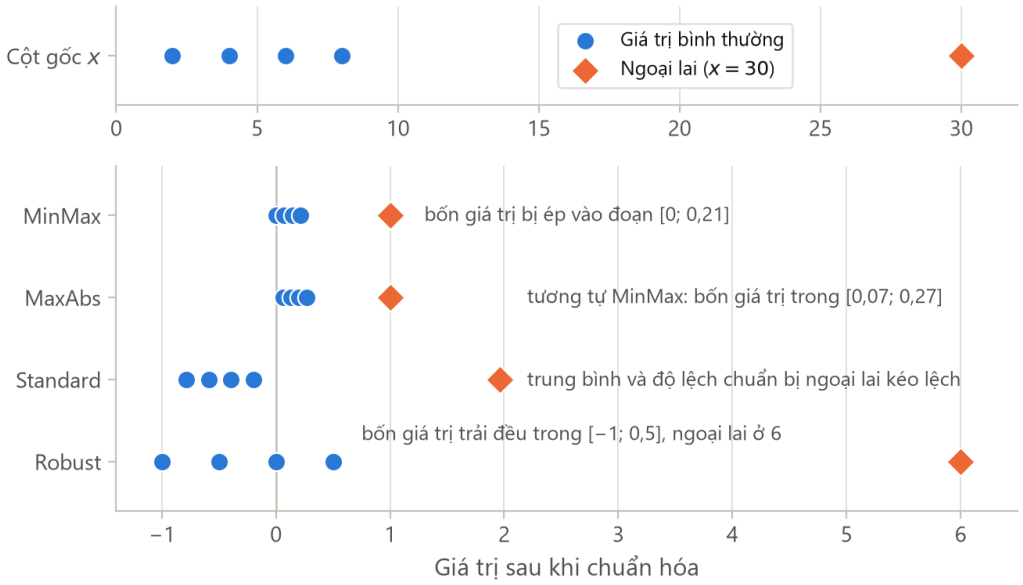
Listing 5.9: So sánh bốn bộ chuẩn hóa trong scikit-learn

```
import numpy as np
from sklearn.preprocessing import (MinMaxScaler, MaxAbsScaler,
                                   StandardScaler, RobustScaler)

x = np.array([2, 4, 6, 8, 30], dtype=float).reshape(-1, 1)

for scaler in [MinMaxScaler(), MaxAbsScaler(),
               StandardScaler(), RobustScaler()]:
```

```
    print(type(scaler).__name__, scaler.fit_transform(x).ravel().round(4))
# MinMaxScaler   [0.     0.0714 0.1429 0.2143 1.    ]
# MaxAbsScaler   [0.0667 0.1333 0.2    0.2667 1.    ]
# StandardScaler [-0.7845 -0.5883 -0.3922 -0.1961  1.9612]
# RobustScaler   [-1.   -0.5  0.   0.5  6. ]
```



Hình 5.7: Năm giá trị của cột $x = (2, 4, 6, 8, 30)$ trước và sau khi áp dụng bốn bộ chuẩn hóa (số liệu của Bảng 5.15).

Bộ chuẩn hóa	Miền giá trị	Bền với ngoại lai	Khi nào dùng
MinMaxScaler	$[0, 1]$	Không	Cần miền cố định (ảnh, mạng nơ-ron), dữ liệu ít ngoại lai
MaxAbsScaler	$[-1, 1]$	Không	Ma trận thưa (TF-IDF), dữ liệu đã tâm ở 0
StandardScaler	Không giới hạn	Trung bình	Mặc định cho mô hình tuyến tính, SVM, PCA
RobustScaler	Không giới hạn	Có	Dữ liệu lệch, nhiều ngoại lai

Bảng 5.16: Tóm tắt lựa chọn bộ chuẩn hóa

5.7 Phát hiện ngoại lai (Outlier Detection)

Theo định nghĩa kinh điển của Hawkins (1980), **ngoại lai** (outlier) là *"một quan sát lệch khỏi các quan sát khác nhiều đến mức gợi nghi ngờ rằng nó được sinh ra bởi một cơ chế khác"*. Ngoại lai có thể là lỗi nhập liệu (năm xây garage 2207 trong Ames Housing), lỗi đo đạc, hoặc là hiện tượng có thật nhưng hiếm (một giao dịch gian lận). Vì vậy **phát hiện** và **xử lý** là hai quyết định khác nhau: chỉ loại bỏ khi chắc chắn đó là lỗi; trong các trường hợp khác có thể giữ nguyên, biến đổi (lấy logarit), cắt ngưỡng (winsorizing), hoặc dùng mô hình và bộ chuẩn hóa bền vững như `RobustScaler`.

5.7.1 Phương pháp một biến: z-score và quy tắc IQR

Quy tắc z-score. Coi điểm có $|z| = |x - \bar{x}|/s > 3$ là ngoại lai. Điểm yếu của quy tắc này là chính ngoại lai kéo lệch $\bar{x}$ và làm s phình to (**hiệu ứng che lấp** – masking). Với cột $x = (2, 4, 6, 8, 30)$ đã dùng ở Mục 5.6: $\bar{x} = 10$, $s = 10{,}20$ nên $z(30) = 1{,}96 < 3$ — giá trị 30 **không** bị phát hiện. Thực ra có thể chứng minh $|z| \le \sqrt{n-1}$ (Shiffler, 1988), nên với $n = 5$ không quan sát nào có thể vượt ngưỡng 3.

Quy tắc IQR (Tukey, 1977). Dựa trên các tứ phân vị Q_1, Q_3 và khoảng tứ phân vị $\text{IQR} = Q_3 - Q_1$; điểm nằm ngoài khoảng

$$\big[\, Q_1 - 1{,}5\,\text{IQR},\;\; Q_3 + 1{,}5\,\text{IQR} \,\big] \tag{5.11}$$

là ngoại lai — đây cũng chính là "râu" của biểu đồ hộp (boxplot). Với cùng dữ liệu: $Q_1 = 4$, $Q_3 = 8$, $\text{IQR} = 4$, khoảng hợp lệ là $[-2;\ 14]$, nên 30 bị phát hiện. Tứ phân vị bền vững với ngoại lai nên quy tắc IQR không bị che lấp.

Listing 5.10: z-score và quy tắc IQR

```
import numpy as np

x = np.array([2, 4, 6, 8, 30])
z = (x - x.mean()) / x.std()
print(z.round(2))                                   # [-0.78 -0.59 -0.39
    -0.2   1.96]
q1, q3 = np.percentile(x, [25, 75]); iqr = q3 - q1
print(x[(x < q1 - 1.5 * iqr) | (x > q3 + 1.5 * iqr)])  # [30]
```

5.7.2 Phương pháp nhiều biến: Local Outlier Factor (LOF)

Trong không gian nhiều chiều, một điểm có thể "bình thường" trên từng biến riêng lẻ nhưng bất thường khi xét đồng thời. Breunig, Kriegel, Ng và Sander (2000) đề xuất **LOF** (Local Outlier Factor): so sánh **mật độ cục bộ** của một điểm với mật độ của các láng giềng; điểm nằm trong vùng thưa hơn hẳn các láng giềng của nó là ngoại lai. LOF cùng tư tưởng "dựa trên mật độ" với thuật toán phân cụm DBSCAN (Chương 11); hai công trình có chung hai tác giả là Hans-Peter Kriegel và Jörg Sander.

Định nghĩa 5.6 (Các đại lượng của LOF)**.** *Với số láng giềng k và $N_k(p)$ là tập k láng giềng gần nhất của điểm p:*

- ***k-khoảng cách**: k-dist(p) là khoảng cách từ p đến láng giềng thứ k của nó.*
- ***Khoảng cách tiếp cận** (reachability distance):*

$$\textit{reach-dist}_k(p, o) = \max\{\textit{k-dist}(o),\ d(p, o)\}.$$

- ***Mật độ tiếp cận cục bộ** (local reachability density):*

$$\textit{lrd}_k(p) = \left(\frac{1}{k} \sum_{o \in N_k(p)} \textit{reach-dist}_k(p, o) \right)^{-1}.$$

– ***Hệ số ngoại lai cục bộ***:

$$LOF_k(p) = \frac{1}{k} \sum_{o \in N_k(p)} \frac{lrd_k(o)}{lrd_k(p)}.$$

$LOF \approx 1$: mật độ tương đương các láng giềng (điểm bình thường); $LOF \gg 1$: điểm thưa hơn hẳn láng giềng (ngoại lai).

Phép lấy max trong khoảng cách tiếp cận có tác dụng **làm trơn**: các điểm nằm sâu trong cùng một cụm dày đặc có khoảng cách tiếp cận bằng nhau, nhờ đó LOF không dao động vì những chênh lệch khoảng cách rất nhỏ.

Ví dụ 5.9 (Tính tay LOF với $k = 3$)**.** *Cho 5 điểm $A(0,0)$, $B(1,0)$, $C(0,1)$, $D(1,1)$ và $E(5,4)$ với ma trận khoảng cách Euclid trong Bảng 5.17.*

	A	B	C	D	E
A	0	1	1	1,414	6,403
B	1	0	1,414	1	5,657
C	1	1,414	0	1	5,831
D	1,414	1	1	0	5
E	6,403	5,657	5,831	5	0

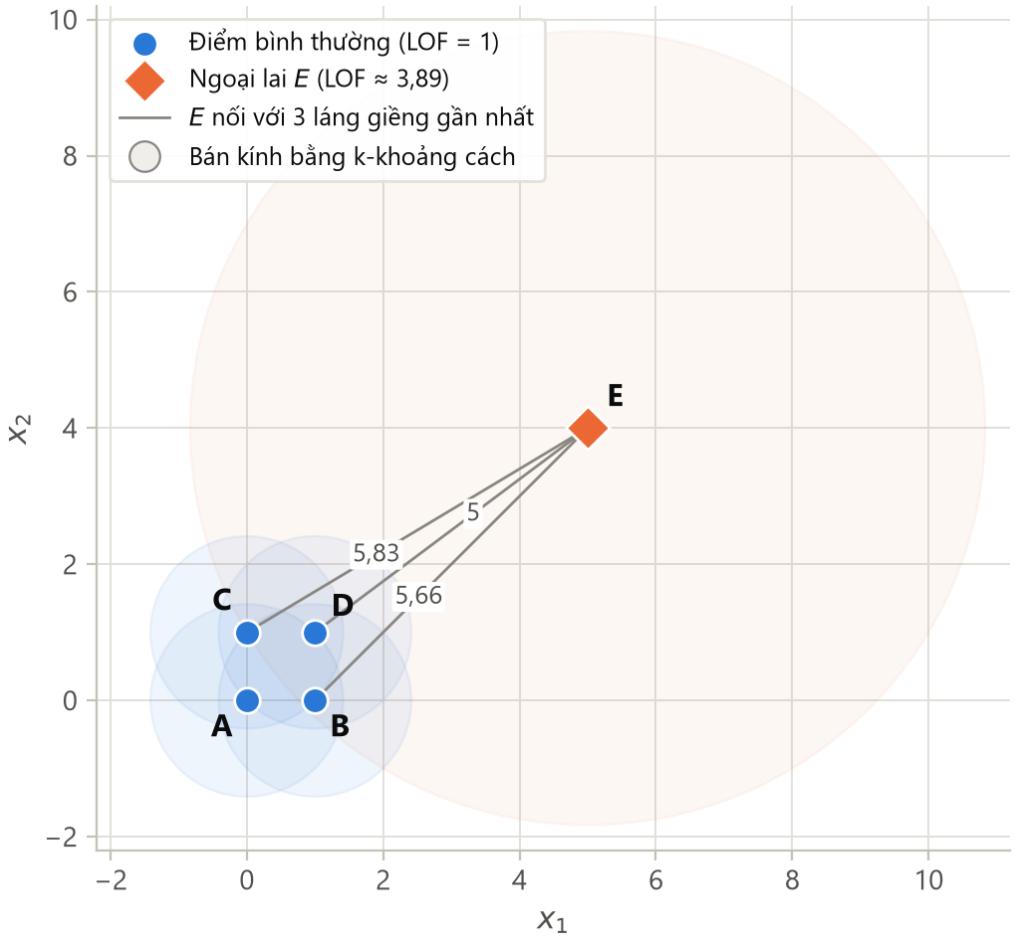
Bảng 5.17: Khoảng cách giữa 5 điểm của ví dụ LOF

Bước 1 – k-khoảng cách. Mỗi điểm A, B, C, D có hai láng giềng ở khoảng cách 1 và một láng giềng ở khoảng cách $\sqrt{2}$, nên k-dist $= 1{,}414$. Ba láng giềng của E là D, B, C, nên k-dist$(E) = 5{,}831$.
Bước 2 – khoảng cách tiếp cận. Chẳng hạn reach-dist$(A, B) = \max\{\text{k-dist}(B),\ d(A, B)\} = \max\{1{,}414;\ 1\} = 1{,}414$: dù A chỉ cách B một đơn vị, khoảng cách tiếp cận vẫn là $1{,}414$. Với E, các khoảng cách thật đều lớn hơn k-khoảng cách của láng giềng nên được giữ nguyên.
Bước 3 – mật độ và hệ số LOF được tổng hợp trong Bảng 5.18.

Điểm	**3 láng giềng**	**reach-dist**	**TB reach-dist**	**lrd**	**LOF**	**Nhãn**
A	B, C, D	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
B	A, D, C	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
C	A, D, B	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
D	B, C, A	1,414; 1,414; 1,414	1,414	0,7071	1,000	1
E	D, B, C	5; 5,657; 5,831	5,496	0,1820	**3,886**	-1

Bảng 5.18: Bảng tính LOF ($k = 3$); nhãn theo quy ước của scikit-learn: 1 là điểm bình thường, -1 là ngoại lai

Chẳng hạn LOF$(E) = \dfrac{(0{,}7071 + 0{,}7071 + 0{,}7071)/3}{0{,}1820} \approx 3{,}886 \gg 1$, nên E là ngoại lai; các điểm còn lại có LOF $= 1$. Với `contamination="auto"`, scikit-learn coi điểm có LOF $> 1{,}5$ là ngoại lai.



Hình 5.8: Minh họa LOF: bốn điểm A, B, C, D tạo thành vùng dày đặc; điểm E nằm trong vùng thưa với LOF $\approx 3{,}89$.

Listing 5.11: Local Outlier Factor với scikit-learn

```
import numpy as np
from sklearn.neighbors import LocalOutlierFactor

X = np.array([[0, 0], [1, 0], [0, 1], [1, 1], [5, 4]])
lof = LocalOutlierFactor(n_neighbors=3)
print(lof.fit_predict(X))                       # [ 1  1  1  1 -1]
print(-lof.negative_outlier_factor_.round(4))   # [1.     1.     1.     1.
    3.8862]
```

Để phát hiện ngoại lai trên **dữ liệu mới** (sau khi đã huấn luyện), đặt tham số `novelty=True` cho `LocalOutlierFactor`. Một lựa chọn khác mở rộng tốt cho dữ liệu lớn là `IsolationForest` (Liu, Ting & Zhou, 2008): ngoại lai là những điểm bị cô lập sau rất ít lần chia ngẫu nhiên.

5.8 Kỹ thuật tạo đặc trưng (Feature Engineering)

Kỹ thuật tạo đặc trưng là quá trình vận dụng tri thức chuyên môn để tạo ra, từ dữ liệu thô, những đặc trưng giúp thuật toán học tốt hơn (Sarkar và cộng sự, 2018, Chương 4; Liu, 2019,

Chương 9). Đặc trưng có hai nguồn: **đặc trưng gốc** lấy trực tiếp từ dữ liệu, và **đặc trưng dẫn xuất** được tính từ các đặc trưng gốc — chẳng hạn "tuổi" tính từ "ngày sinh", hay "giá trên mỗi mét vuông" tính từ giá và diện tích. Các bộ mã hóa và bộ chuẩn hóa ở các mục trước là những trường hợp riêng của kỹ thuật tạo đặc trưng; mục này bổ sung các kỹ thuật dành cho dữ liệu số và dữ liệu thời gian. Với dữ liệu thô không có dạng bảng như văn bản và ảnh, đặc trưng được rút trích bằng các kỹ thuật ở Mục 5.9.

5.8.1 Nhị phân hóa và rời rạc hóa

Nhị phân hóa (binarization) biến một đặc trưng số thành 0/1 theo một ngưỡng θ: $x' = 1$ nếu $x \geq \theta$, ngược lại $x' = 0$. Kỹ thuật này hữu ích khi điều quan trọng chỉ là *có hay không*, chẳng hạn "đã từng mua hàng" thay vì "số lần mua hàng". Với thu nhập (USD/tháng) $(600, 800, 1000, 1200, 1500, 2000, 2500, 4000, 8000, 20000)$ và ngưỡng 1500, ta được $(0, 0, 0, 0, 1, 1, 1, 1, 1, 1)$. Lưu ý: lớp `Binarizer` của scikit-learn dùng phép so sánh **lớn hơn hẳn** ($x > \theta$).

Rời rạc hóa (binning, discretization) chia miền giá trị thành các khoảng và thay mỗi giá trị bằng chỉ số khoảng chứa nó. Có ba chiến lược chính:

- **Khoảng đều** (fixed-width): chỉ số khoảng $= \lfloor x/h \rfloor$ với độ rộng h.
- **Khoảng theo phân vị** (adaptive, quantile): các mốc là phân vị của dữ liệu, nên mỗi khoảng chứa số mẫu xấp xỉ bằng nhau — thích hợp cho dữ liệu lệch.
- **Khoảng theo tri thức chuyên môn**: mốc do chuyên gia quy định, ví dụ các nhóm tuổi.

Ví dụ 5.10 (Rời rạc hóa tuổi)**.** *Tuổi của 8 khách hàng:* $(12, 25, 37, 41, 58, 63, 70, 19)$.

- *Khoảng đều* $h = 20$*:* $\lfloor 12/20 \rfloor = 0$*,* $\lfloor 25/20 \rfloor = 1$*, ... cho* $(0, 1, 1, 2, 2, 3, 3, 0)$.
- *Theo tứ phân vị (`pd.qcut(q=4)`): các mốc* 12*;* 23,5*;* 39*;* 59,25*;* 70*, mỗi khoảng chứa đúng 2 mẫu; nhãn khoảng cũng là* $(0, 1, 1, 2, 2, 3, 3, 0)$.
- *Theo chuyên môn, mốc* $(0, 18]$*,* $(18, 35]$*,* $(35, 60]$*,* $(60, 120]$*: trẻ em, thanh niên, trung niên, trung niên, trung niên, cao tuổi, cao tuổi, thanh niên.*

Rời rạc hóa giúp mô hình tuyến tính nắm bắt quan hệ phi tuyến (mỗi khoảng có một hệ số riêng sau khi mã hóa One Hot) và giảm ảnh hưởng của ngoại lai; cái giá phải trả là mất thông tin chi tiết bên trong mỗi khoảng. scikit-learn cung cấp `KBinsDiscretizer` với ba chiến lược `"uniform"`, `"quantile"`, `"kmeans"`.

5.8.2 Biến đổi thống kê: logarit và Box-Cox

Nhiều đại lượng thực tế như thu nhập, giá nhà, diện tích lô đất có phân phối **lệch phải**: đa số giá trị nhỏ, một số ít rất lớn. Các biến đổi thuộc họ **biến đổi lũy thừa** (power transform) làm phân phối cân đối hơn, ổn định phương sai, và giúp các mô hình giả định phân phối chuẩn hoạt động tốt hơn (Sarkar và cộng sự, 2018, Chương 4).

- **Biến đổi logarit** $x' = \ln x$ (hoặc $\ln(1 + x)$ khi có giá trị 0): "nén" các giá trị lớn và "giãn" các giá trị nhỏ.

– **Biến đổi Box-Cox** (Box & Cox, 1964), áp dụng cho $x > 0$:

$$x^{(\lambda)} = \begin{cases} \dfrac{x^{\lambda} - 1}{\lambda} & \lambda \neq 0, \\ \ln x & \lambda = 0, \end{cases} \tag{5.12}$$

trong đó λ được ước lượng bằng hợp lý cực đại sao cho dữ liệu sau biến đổi gần chuẩn nhất. Biến đổi logarit là trường hợp riêng $\lambda = 0$.

– **Biến đổi Yeo-Johnson** (Yeo & Johnson, 2000): mở rộng Box-Cox cho cả giá trị 0 và giá trị âm.

Dữ liệu thu nhập (10 giá trị ở trên)	**Hệ số bất đối xứng (skewness)**
Dữ liệu gốc	2,10
Sau biến đổi logarit	0,80
Sau biến đổi Box-Cox ($\hat{\lambda} \approx -0{,}42$)	0,16

Bảng 5.19: Hiệu quả của biến đổi lũy thừa; skewness bằng 0 ứng với phân phối đối xứng

Listing 5.12: Nhị phân hóa, rời rạc hóa và biến đổi lũy thừa

```
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.preprocessing import KBinsDiscretizer, PowerTransformer

income = np.array([600, 800, 1000, 1200, 1500, 2000, 2500, 4000, 8000,
    20000.0])
print((income >= 1500).astype(int))                  # nhi phan hoa

age = np.array([12, 25, 37, 41, 58, 63, 70, 19])
print(np.floor(age / 20).astype(int))                 # khoang deu h = 20
print(pd.qcut(age, q=4, labels=False))                # theo tu phan vi
print(pd.cut(age, bins=[0, 18, 35, 60, 120],
             labels=["tre em", "thanh nien", "trung nien", "cao tuoi"]))
kb = KBinsDiscretizer(n_bins=4, encode="ordinal", strategy="quantile")
print(kb.fit_transform(age.reshape(-1, 1)).ravel())

print(stats.skew(income), stats.skew(np.log(income)))     # 2.10  0.80
x_bc, lam = stats.boxcox(income)
print(round(lam, 2), stats.skew(x_bc))                    # -0.42  0.16
pt = PowerTransformer(method="yeo-johnson")   # mac dinh chuan hoa luon ve TB
    0, DLC 1
income_t = pt.fit_transform(income.reshape(-1, 1))
```

5.8.3 Đặc trưng tương tác và đa thức

Mô hình tuyến tính chỉ cộng dồn ảnh hưởng riêng lẻ của từng đặc trưng. Khi ảnh hưởng của đặc trưng này phụ thuộc vào đặc trưng khác — chẳng hạn diện tích làm tăng giá

nhà mạnh hơn khi chất lượng xây dựng cao — cần bổ sung **đặc trưng tương tác** $x_j x_k$. `PolynomialFeatures(degree=p)` sinh mọi đơn thức có bậc không quá p; với d đặc trưng, số cột tạo ra (kể cả cột hằng số 1) là $\binom{d+p}{p}$. Với 5 đặc trưng và $p = 2$, đó là $\binom{7}{2} = 21$ cột — chính là cách Lab 1 (Chương 12) nâng R^2 từ 0,78 lên 0,85. Vì số cột tăng theo cấp d^p, đặc trưng đa thức thường được dùng kèm chính quy hóa hoặc lựa chọn đặc trưng.

5.8.4 Đặc trưng thời gian

Một cột ngày giờ chưa thể đưa trực tiếp vào mô hình nhưng chứa nhiều thông tin có thể khai thác (Sarkar và cộng sự, 2018, Chương 4):

- **Thành phần lịch**: năm, quý, tháng, ngày trong tuần, cuối tuần hay không, giờ trong ngày.
- **Mã hóa chu kỳ**: tháng 12 và tháng 1 liền nhau nhưng mã số 12 và 1 lại cách xa nhau. Biểu diễn tháng m bằng cặp $\left(\sin\frac{2\pi m}{12}, \cos\frac{2\pi m}{12}\right)$ đặt 12 tháng lên một đường tròn, nhờ đó tháng 12 nằm cạnh tháng 1.
- **Đặc trưng trễ và cửa sổ trượt** cho chuỗi thời gian: giá trị của các phiên trước (Lab 4), trung bình trượt, độ lệch chuẩn trượt (**độ biến động**), tỉ suất sinh lời. Khi dự báo giá cổ phiếu, Liu (2019, Chương 9) xây dựng trung bình giá theo tuần, tháng, năm, tỉ số giữa các trung bình đó, và độ lệch chuẩn của giá và khối lượng giao dịch.

Nguyên tắc vàng của đặc trưng thời gian: chỉ được dùng thông tin **đã biết tại thời điểm dự báo**. Liu (2019) nhấn mạnh rằng giá cao nhất, thấp nhất và khối lượng của *chính ngày cần dự báo* không được đưa vào đặc trưng, vì chúng chỉ được biết khi phiên đóng cửa. Trong pandas, phép `shift(1)` bảo đảm mỗi đặc trưng chỉ dùng dữ liệu đến hết phiên hôm trước.

Listing 5.13: Tạo đặc trưng thời gian và đặc trưng cửa sổ trượt

```
# Du lieu minh hoa: 300 phien giao dich gia lap, chi so la ngay giao dich
idx = pd.bdate_range("2023-01-02", periods=300)     # cac ngay lam viec
rng = np.random.default_rng(0)
df = pd.DataFrame({"Close": 75 * np.exp(np.cumsum(rng.normal(0, 0.01, 300))),
                   "Volume": rng.integers(10000, 20000, 300)}, index=idx)

df["month"] = df.index.month
df["dayofweek"] = df.index.day_of_week              # 0 = thu Hai
df["month_sin"] = np.sin(2 * np.pi * df["month"] / 12)
df["month_cos"] = np.cos(2 * np.pi * df["month"] / 12)

close = df["Close"]
df["ret_1"] = close.pct_change().shift(1)                  # ti suat sinh loi
    phien truoc
df["ma_5"] = close.rolling(5).mean().shift(1)              # trung binh 1 tuan
df["ma_21"] = close.rolling(21).mean().shift(1)            # trung binh 1 thang
df["ratio_5_21"] = df["ma_5"] / df["ma_21"]                # xu huong ngan /
    trung han
df["vol_21"] = close.pct_change().rolling(21).std().shift(1)   # do bien dong
df = df.dropna()        # bo cac dong dau chua du du lieu cho cua so
```

5.9 Rút trích đặc trưng (Feature Extraction)

Kỹ thuật tạo đặc trưng ở Mục 5.8 bổ sung cột mới từ các cột số sẵn có. Nhiều loại dữ liệu lại hoàn toàn không có sẵn cột số nào: văn bản là chuỗi ký tự có độ dài tùy ý, ảnh là một lưới điểm ảnh. **Rút trích đặc trưng** (feature extraction) là quá trình biến dữ liệu thô như vậy thành một vector số có số chiều cố định, đồng thời giữ lại thông tin hữu ích cho bài toán. Rút trích đặc trưng khác với lựa chọn đặc trưng ở Mục 5.10. Lựa chọn chỉ giữ lại một tập con các cột đã có, còn rút trích tạo ra các cột hoàn toàn mới. Trong scikit-learn, các công cụ rút trích cho văn bản nằm trong module `sklearn.feature_extraction`.

Dữ liệu thô	Phương pháp rút trích	Đặc trưng thu được	Trình bày ở
Văn bản	Túi từ (Count Vectorizer), TF-IDF	Tần suất hoặc trọng số của từng từ	Mục này
Ảnh	Làm phẳng lưới điểm ảnh	Độ sáng của từng điểm ảnh	Lab 5, Lab 6 (Chương 12)
Dữ liệu số nhiều chiều	PCA, LDA	Tổ hợp tuyến tính của các cột gốc	Chương 10

Bảng 5.20: Các hướng rút trích đặc trưng theo loại dữ liệu

Phần còn lại của mục trình bày chi tiết hướng rút trích phổ biến nhất trong tiền xử lý: **vector hóa văn bản**.

5.9.1 Mô hình túi từ và Count Vectorizer

Ý tưởng biểu diễn một văn bản bằng **tần suất các từ** có từ rất sớm: nhà ngôn ngữ học Zellig Harris (1954) trong *Distributional Structure* đã nhắc đến "túi từ" (bag of words), và Hans Peter Luhn (1957) tại IBM đề xuất dùng tần suất từ để đánh chỉ mục tài liệu tự động. Năm 1975, Salton, Wong và Yang hệ thống hóa thành **mô hình không gian vector** (Vector Space Model): mỗi văn bản là một vector trong không gian có số chiều bằng kích thước từ vựng.

Định nghĩa 5.7 (Count Vectorizer)**.** *Cho kho văn bản (corpus) $D = \{d_1, \ldots, d_N\}$ và từ vựng $V = \{t_1, \ldots, t_M\}$ gồm mọi từ xuất hiện trong D. Count Vectorizer biểu diễn văn bản d bằng vector*

$$\mathbf{x}_d = \big(\mathrm{tf}(t_1, d), \mathrm{tf}(t_2, d), \ldots, \mathrm{tf}(t_M, d)\big), \tag{5.13}$$

trong đó $\mathrm{tf}(t, d)$ (term frequency) là số lần từ t xuất hiện trong d. Cả kho văn bản trở thành ma trận văn bản – từ (document-term matrix) kích thước $N \times M$.

Ví dụ 5.11. *Xét kho văn bản gồm 3 câu:*

- d_1*: "the cats are in the house"*
- d_2*: "the dogs are in the house and outside"*
- d_3*: "the cats and dogs are friends"*

Từ vựng (sắp xếp alphabet) gồm $M = 9$ từ: and, are, cats, dogs, friends, house, in, outside, the.

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	1	1	0	0	1	1	0	2
d_2	1	1	0	1	0	1	1	1	2
d_3	1	1	1	1	1	0	0	0	1
$\mathrm{df}(t)$	2	3	2	2	1	2	2	1	3

Bảng 5.21: Ma trận đếm từ (Count Vectorizer) và tần suất văn bản $\mathrm{df}(t)$

Listing 5.14: Count Vectorizer với scikit-learn

```
from sklearn.feature_extraction.text import CountVectorizer

corpus = [
    "the cats are in the house",
    "the dogs are in the house and outside",
    "the cats and dogs are friends",
]
cv = CountVectorizer()
X = cv.fit_transform(corpus)

print(cv.vocabulary_)
# {'the': 8, 'cats': 2, 'are': 1, 'in': 6, 'house': 5,
#  'dogs': 3, 'and': 0, 'outside': 7, 'friends': 4}
print(X.toarray())
# [[0 1 1 0 0 1 1 0 2]
#  [1 1 0 1 0 1 1 1 2]
#  [1 1 1 1 1 0 0 0 1]]
```

Mặc định `CountVectorizer` chuyển chữ về chữ thường và tách từ bằng biểu thức chính quy `(?u)\b\w\w+\b`, tức **bỏ qua các từ chỉ có 1 ký tự**. Các tham số hay dùng: `stop_words` (loại từ dừng), `ngram_range=(1,2)` (lấy thêm cặp từ), `min_df`/`max_df` (loại từ quá hiếm/quá phổ biến).

Chuẩn hóa L2. Văn bản dài có số đếm lớn hơn chỉ vì dài hơn. Để so sánh công bằng, mỗi vector được chia cho chuẩn Euclid của nó:

$$\hat{\mathbf{x}}_d = \frac{\mathbf{x}_d}{\|\mathbf{x}_d\|_2}, \qquad \|\mathbf{x}_d\|_2 = \sqrt{\textstyle\sum_j x_{dj}^2}. \tag{5.14}$$

Với d_1: $\|\mathbf{x}_{d_1}\|_2^2 = 1+1+1+1+2^2 = 8$, nên các số 1 trở thành $1/\sqrt{8} \approx 0{,}3536$ và số 2 trở thành $2/\sqrt{8} \approx 0{,}7071$. Tương tự $\|\mathbf{x}_{d_2}\|_2 = \sqrt{10}$ và $\|\mathbf{x}_{d_3}\|_2 = \sqrt{6}$:

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	0.3536	0.3536	0	0	0.3536	0.3536	0	0.7071
d_2	0.3162	0.3162	0	0.3162	0	0.3162	0.3162	0.3162	0.6325
d_3	0.4082	0.4082	0.4082	0.4082	0.4082	0	0	0	0.4082

Bảng 5.22: Ma trận đếm sau khi chuẩn hóa L2 theo từng hàng

5.9.2 TF-IDF

Count Vectorizer có một nhược điểm: những từ xuất hiện ở **mọi** văn bản (như "the", "are") có số đếm cao nhưng gần như không giúp phân biệt văn bản. Năm 1972, Karen Spärck Jones đề xuất trong bài báo *A statistical interpretation of term specificity and its application in retrieval* (*Journal of Documentation*) rằng trọng số của một từ nên tỉ lệ nghịch với số văn bản chứa nó — đó là **tần suất văn bản nghịch đảo** (Inverse Document Frequency – IDF). Salton và Buckley (1988) sau đó tổng kết các biến thể trọng số TF-IDF, trở thành chuẩn mực của ngành truy hồi thông tin suốt nhiều thập kỷ.

Định nghĩa 5.8 (TF-IDF)**.** *Với N là số văn bản và* $\mathrm{df}(t)$ *là số văn bản chứa từ t, trọng số TF-IDF của từ t trong văn bản d là*

$$\textit{tf-idf}(t,d) = \mathrm{tf}(t,d) \times \mathrm{idf}(t). \tag{5.15}$$

Dạng kinh điển của IDF là $\mathrm{idf}(t) = \ln \dfrac{N}{\mathrm{df}(t)}$*. Thư viện scikit-learn dùng hai biến thể có cộng thêm 1 để từ xuất hiện ở mọi văn bản không bị triệt tiêu hoàn toàn:*

$$\textit{Không làm trơn (}\texttt{smooth_idf=False}\textit{)}: \quad \mathrm{idf}(t) = 1 + \ln \frac{N}{\mathrm{df}(t)}, \tag{5.16}$$

$$\textit{Làm trơn (}\texttt{smooth_idf=True}\textit{, mặc định)}: \quad \mathrm{idf}(t) = 1 + \ln \frac{N+1}{\mathrm{df}(t)+1}. \tag{5.17}$$

Công thức làm trơn tương đương việc thêm một "văn bản ảo" chứa mọi từ đúng một lần, giúp tránh phép chia cho 0.

Ví dụ 5.12 (Tính tay IDF)**.** *Với $N = 3$ và* df *ở Bảng 5.21, chỉ có ba giá trị* $\mathrm{df} \in \{1,2,3\}$*:*

$\mathrm{df}(t)$	**Các từ**	**IDF không làm trơn (5.16)**	**IDF làm trơn (5.17)**
1	friends, outside	$1 + \ln 3 = 2{,}0986$	$1 + \ln 2 = 1{,}6931$
2	and, cats, dogs, house, in	$1 + \ln 1{,}5 = 1{,}4055$	$1 + \ln \frac{4}{3} = 1{,}2877$
3	are, the	$1 + \ln 1 = 1$	$1 + \ln 1 = 1$

Bảng 5.23: Giá trị IDF cho kho 3 văn bản

Nhân từng cột của ma trận đếm (Bảng 5.21) với IDF tương ứng, ta được ma trận TF-IDF (chưa chuẩn hóa). Chẳng hạn với từ "cats" trong d_1: $\mathrm{tf} = 1$, $\mathrm{df} = 2$ nên tf-idf $= 1 \times 1{,}2877 = 1{,}2877$ (làm trơn); với "the" trong d_1: $2 \times 1 = 2$.

	and	are	cats	dogs	friends	house	in	outside	the
IDF làm trơn (`smooth_idf=True`*)*									
d_1	0	1	1.2877	0	0	1.2877	1.2877	0	2
d_2	1.2877	1	0	1.2877	0	1.2877	1.2877	1.6931	2
d_3	1.2877	1	1.2877	1.2877	1.6931	0	0	0	1
IDF không làm trơn (`smooth_idf=False`*)*									
d_1	0	1	1.4055	0	0	1.4055	1.4055	0	2
d_2	1.4055	1	0	1.4055	0	1.4055	1.4055	2.0986	2
d_3	1.4055	1	1.4055	1.4055	2.0986	0	0	0	1

Bảng 5.24: Ma trận TF-IDF chưa chuẩn hóa (`norm=None`)

	and	are	cats	dogs	friends	house	in	outside	the
d1	0	1	1	0	0	1	1	0	2
d2	1	1	0	1	0	1	1	1	2
d3	1	1	1	1	1	0	0	0	1
df(t)	2	3	2	2	1	2	2	1	3
idf(t,D) smooth	1.287682	1	1.287682	1.287682	1.693147	1.287682	1.287682	1.693147	1
d1	0	1	1.2877	0	0	1.2877	1.2877	0	2
d2	1.2877	1	0	1.2877	0	1.2877	1.2877	1.6931	2
d3	1.2877	1	1.2877	1.2877	1.6931	0	0	0	1
	and	are	cats	dogs	friends	house	in	outside	the
d1	0	1	1	0	0	1	1	0	2
d2	1	1	0	1	0	1	1	1	2
d3	1	1	1	1	1	0	0	0	1
d1^2	0	1	1	0	0	1	1	0	4
d2^2	1	1	0	1	0	1	1	1	4
d3^2	1	1	1	1	1	0	0	0	1
	0	0.353553	0.353553	0	0	0.353553	0.353553	0	0.707107
	0.316228	0.316228	0	0.316228	0	0.316228	0.316228	0.316228	0.632456
	0.408248	0.408248	0.408248	0.408248	0.408248	0	0	0	0.408248

Hình 5.9: Tính TF-IDF trên bảng tính (ảnh chụp từ tệp `Cluster.xlsx`). Phần trên: ma trận đếm, df(t), idf(t, D) làm trơn và ma trận TF-IDF. Phần dưới: chuẩn hóa L2 ma trận đếm — bình phương từng phần tử, rồi chia mỗi dòng cho căn của tổng bình phương.

	and	are	cats	dogs	friends	house	in	outside	the
d1	0	1	1	0	0	1	1	0	2
d2	1	1	0	1	0	1	1	1	2
d3	1	1	1	1	1	0	0	0	1
df	2	3	2	2	1	2	2	1	3
idf	1.405465	1	1.405465	1.405465	2.098612	1.405465	1.405465	2.098612	1
tf-idf	0	1	1.405465	0	0	1.405465	1.405465	0	2
	1.405465	1	0	1.405465	0	1.405465	1.405465	2.098612	2
	1.405465	1	1.405465	1.405465	2.098612	0	0	0	1

Hình 5.10: Ma trận TF-IDF với IDF không làm trơn (giá trị 1,405465 và 2,098612) — khớp với kết quả `smooth_idf=False` của scikit-learn. Các ô đánh dấu là những từ có df lớn nhất (*are*, *the*) và nhỏ nhất (*friends*, *outside*).

Chuẩn hóa L2 và độ tương đồng cosine. Mặc định `TfidfVectorizer` còn chuẩn hóa L2 từng hàng. Áp dụng cho ma trận làm trơn ở Bảng 5.24:

	and	are	cats	dogs	friends	house	in	outside	the
d_1	0	0.3166	0.4077	0	0	0.4077	0.4077	0	0.6333
d_2	0.3382	0.2626	0	0.3382	0	0.3382	0.3382	0.4447	0.5252
d_3	0.4105	0.3188	0.4105	0.4105	0.5397	0	0	0	0.3188

Bảng 5.25: Ma trận TF-IDF mặc định của scikit-learn (`smooth_idf=True`, `norm="l2"`)

Vì mỗi hàng có độ dài 1, tích vô hướng giữa hai hàng chính là **độ tương đồng cosine**:

$$\cos(d_i, d_j) = \hat{\mathbf{x}}_{d_i} \cdot \hat{\mathbf{x}}_{d_j}. \tag{5.18}$$

Ta có $\cos(d_1, d_2) \approx 0{,}6915$, $\cos(d_1, d_3) \approx 0{,}4702$, $\cos(d_2, d_3) \approx 0{,}5288$: hai câu cùng nói về "in the house" (d_1, d_2) giống nhau nhất — phù hợp với trực giác.

Listing 5.15: TF-IDF Vectorizer với các cấu hình khác nhau

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

corpus = [
    "the cats are in the house",
    "the dogs are in the house and outside",
    "the cats and dogs are friends",
]

# 1) Chi dem tu (tuong duong CountVectorizer)
v = TfidfVectorizer(use_idf=False, smooth_idf=False, norm=None)
print(v.fit_transform(corpus).toarray())

# 2) TF-IDF khong lam tron, chua chuan hoa -> 1.4055, 2.0986
v = TfidfVectorizer(use_idf=True, smooth_idf=False, norm=None)
print(v.fit_transform(corpus).toarray().round(4))

# 3) TF-IDF lam tron, chua chuan hoa -> 1.2877, 1.6931
v = TfidfVectorizer(use_idf=True, smooth_idf=True, norm=None)
print(v.fit_transform(corpus).toarray().round(4))
print(v.idf_.round(4))
# [1.2877 1.     1.2877 1.2877 1.6931 1.2877 1.2877 1.6931 1.    ]

# 4) Cau hinh mac dinh: lam tron + chuan hoa L2
v = TfidfVectorizer()
X = v.fit_transform(corpus)
print(X.toarray().round(4))
print(cosine_similarity(X).round(4))
# [[1.     0.6915 0.4702]
#  [0.6915 1.     0.5288]
#  [0.4702 0.5288 1.    ]]
```

Cấu hình	`use_idf`	`smooth_idf`	`norm`
Đếm từ thuần túy	False	–	None
Đếm từ chuẩn hóa L2	False	–	`"l2"`
TF-IDF thô	True	False	None
TF-IDF mặc định	True	True	`"l2"`

Bảng 5.26: Các cấu hình của `TfidfVectorizer` dùng trong chương

5.10 Lựa chọn đặc trưng (Feature Selection)

Tạo thêm đặc trưng làm tăng thông tin, nhưng cũng làm tăng số chiều. Những đặc trưng **không liên quan** hoặc **dư thừa** làm mô hình dễ quá khớp, tốn chi phí tính toán, khó diễn giải, và đặc biệt gây hại cho các thuật toán dựa trên khoảng cách như KNN (lời nguyền số chiều, Chương 10). Có hai hướng giảm số chiều (Raschka và cộng sự, 2022, Chương 4):

- **Lựa chọn đặc trưng**: giữ lại một **tập con** các cột gốc, nên vẫn giữ nguyên ý nghĩa của từng cột — nội dung của mục này.
- **Rút trích đặc trưng**: tạo các cột **mới** từ dữ liệu gốc (Mục 5.9); với dữ liệu số, các cột mới là tổ hợp của cột gốc (PCA, LDA — Chương 10).

Tìm tập con tốt nhất bằng vét cạn là bất khả thi: d đặc trưng có $2^d - 1$ tập con khác rỗng, tức 8191 tập con khi $d = 13$ và khoảng 10^{15} tập con khi $d = 50$. Guyon và Elisseeff (2003) phân các phương pháp thực tế thành ba họ:

Họ	Nguyên lý	Ví dụ	Ưu / nhược
Lọc (filter)	Chấm điểm từng đặc trưng bằng một thống kê, **độc lập với mô hình**	Ngưỡng phương sai, kiểm định F, χ^2, thông tin tương hỗ	Rất nhanh / bỏ qua tương tác và dư thừa
Bao gói (wrapper)	Tìm kiếm tập con, đánh giá bằng **hiệu năng của chính mô hình**	SBS, SFS, RFE	Chính xác / tốn kém
Nhúng (embedded)	Lựa chọn diễn ra **trong quá trình huấn luyện**	Chính quy hóa L1, độ quan trọng của rừng ngẫu nhiên	Cân bằng / phụ thuộc mô hình

Bảng 5.27: Ba họ phương pháp lựa chọn đặc trưng (Guyon & Elisseeff, 2003)

Để minh họa, ta dùng bộ dữ liệu **Wine** của UCI — cũng là bộ dữ liệu Raschka và cộng sự (2022) dùng cho chủ đề này: 178 mẫu rượu vang thuộc 3 giống nho (59, 71 và 48 mẫu), mô tả bởi 13 đặc trưng hóa học như nồng độ cồn (`alcohol`), `flavanoids`, `proline`, cường độ màu (`color_intensity`)… Dữ liệu được chia phân tầng 70/30 thành 124 mẫu huấn luyện và 54 mẫu kiểm tra (`random_state=0`).

5.10.1 Phương pháp lọc

Ngưỡng phương sai. Đặc trưng gần như hằng số không mang thông tin phân biệt. `VarianceThreshold` loại các cột có phương sai nhỏ hơn một ngưỡng. Với một đặc trưng nhị phân nhận giá trị 1 với tỉ lệ p, phương sai bằng $p(1-p)$; ngưỡng $0{,}8 \times 0{,}2 = 0{,}16$ vì thế loại các cột mà một giá trị chiếm trên 80% số mẫu.

Ví dụ 5.13 (Ngưỡng phương sai)**.** *Ma trận* $\mathbf{X} = \begin{pmatrix} 0 & 2 & 0 & 3 \\ 0 & 1 & 4 & 3 \\ 0 & 1 & 1 & 3 \\ 1 & 1 & 0 & 3 \end{pmatrix}$ *có phương sai các cột là* $(0{,}1875;\ 0{,}1875;\ 2{,}6875;\ 0)$. *Cột 1 là cột nhị phân với* $p = 0{,}25$*, nên* $p(1-p) = 0{,}1875$*. Với ngưỡng 0,2, chỉ cột 3 được giữ lại; cột 4 là hằng số nên chắc chắn bị loại.*

Vì phương sai phụ thuộc đơn vị đo, ngưỡng phương sai chỉ nên dùng cho các cột cùng thang đo, chẳng hạn các cột nhị phân.

Kiểm định đơn biến. Mỗi đặc trưng được chấm điểm theo mức độ liên hệ với nhãn: **thống kê F của ANOVA** (`f_classif`) cho đặc trưng số và nhãn phân loại; χ^2 (`chi2`) cho đặc trưng không âm như tần suất từ; **thông tin tương hỗ** (`mutual_info_classif`) đo được cả quan hệ phi tuyến. `SelectKBest` giữ lại k đặc trưng điểm cao nhất. Trên Wine, năm đặc trưng có điểm F cao nhất là `alcohol`, `flavanoids`, `hue`, `od280/od315` và `proline`; năm đặc trưng có thông tin tương hỗ cao nhất là `flavanoids` (0,760), `od280/od315` (0,589), `proline` (0,546), `color_intensity` (0,510) và `alcohol` (0,465). Hai tiêu chí đồng thuận ở bốn trên năm đặc trưng. Điểm yếu chung của phương pháp lọc là xét **từng đặc trưng riêng lẻ**: hai đặc trưng tương quan mạnh với nhau đều được điểm cao dù đặc trưng thứ hai không mang thêm thông tin, còn hai đặc trưng chỉ có ích *khi đi cùng nhau* thì đều bị bỏ sót.

5.10.2 Phương pháp bao gói: lựa chọn tuần tự lùi (SBS)

Thuật toán lựa chọn tuần tự lùi (Sequential Backward Selection – SBS) là một thuật toán **tham lam**: tại mỗi bước, nó chọn phương án tốt nhất cục bộ thay vì xét mọi tổ hợp (Raschka và cộng sự, 2022, Chương 4; xem so sánh các biến thể trong Ferri và cộng sự, 1994). Gọi J là hàm tiêu chí, chẳng hạn độ chính xác kiểm định chéo của mô hình trên tập đặc trưng đang xét:

1. Khởi tạo $k = d$ với tập đặc trưng đầy đủ X_d.
2. Tìm đặc trưng x^- mà việc loại bỏ nó làm **giảm hiệu năng ít nhất**: $x^- = \arg\max_{x \in X_k} J(X_k \setminus \{x\})$.
3. Loại bỏ: $X_{k-1} = X_k \setminus \{x^-\}$, gán $k \leftarrow k - 1$.
4. Dừng nếu k bằng số đặc trưng mong muốn; ngược lại quay lại bước 2.

Đi từ d xuống 1, SBS chỉ đánh giá $d + (d-1) + \cdots + 2 = \frac{d(d+1)}{2} - 1$ tập con — 90 tập con với $d = 13$, so với 8191 nếu vét cạn. Biến thể **lựa chọn tuần tự tiến** (SFS) xuất phát từ tập rỗng và thêm dần; **loại bỏ đặc trưng đệ quy** (Recursive Feature Elimination – RFE; Guyon và cộng sự, 2002) rẻ hơn nữa vì loại đặc trưng có hệ số hoặc độ quan trọng nhỏ nhất thay vì huấn luyện lại cho mọi khả năng.

k	**Tập đặc trưng do SBS chọn**	**CV (train)**	**Test**
13	toàn bộ	0,944	0,963
8	alcohol, ash, alcalinity_of_ash, magnesium, flavanoids, color_intensity, hue, proline	0,976	0,982
5	alcohol, ash, flavanoids, hue, proline	0,984	0,982
3	alcohol, flavanoids, proline	0,968	0,889
2	flavanoids, proline	0,927	0,870
1	flavanoids	0,783	0,704

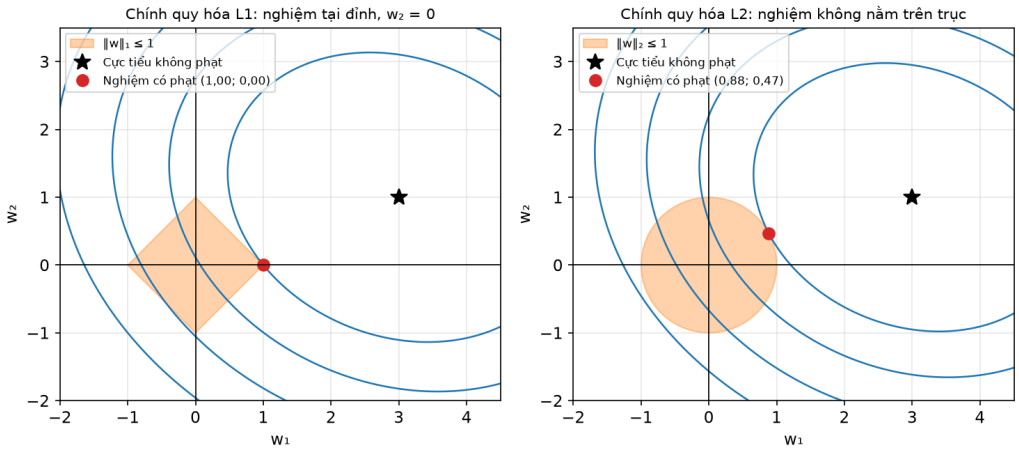
Bảng 5.28: SBS cho KNN ($K = 5$, đã chuẩn hóa) trên Wine: độ chính xác kiểm định chéo 5-fold trên tập huấn luyện và trên tập kiểm tra

Bảng 5.28 cho hai nhận xét. Thứ nhất, **bỏ bớt đặc trưng có thể cải thiện mô hình**: chỉ với 5 trên 13 đặc trưng, KNN đạt độ chính xác kiểm tra 0,982, cao hơn mức 0,963 khi dùng đủ 13

đặc trưng, vì các đặc trưng không liên quan làm méo khoảng cách. Thứ hai, bỏ quá nhiều thì mô hình **thiếu khớp**. Cột CV là ước lượng **lạc quan**, vì chính nó đã được dùng để chọn tập con; ước lượng khách quan là cột Test, nơi tập kiểm tra chỉ được dùng một lần.

5.10.3 Phương pháp nhúng: chính quy hóa L1 và rừng ngẫu nhiên

Chính quy hóa L1. Như đã thấy với Lasso (Tibshirani, 1996; Chương 6), số hạng phạt $\lambda\|\mathbf{w}\|_1$ đẩy nhiều hệ số về **đúng bằng 0**. Về mặt hình học, cực tiểu hàm mất mát với ràng buộc $\|\mathbf{w}\|_1 \leq t$ có miền ràng buộc là hình thoi với các đỉnh nằm trên trục tọa độ, nên đường đồng mức của hàm mất mát thường chạm miền ràng buộc tại một đỉnh, nơi một số tọa độ bằng 0. Ràng buộc L2 có miền hình tròn, không có đỉnh, nên hệ số chỉ nhỏ đi chứ hiếm khi bằng 0 (Raschka và cộng sự, 2022, Chương 4).



Hình 5.11: Diễn giải hình học: miền ràng buộc L1 (hình thoi) cho nghiệm thưa tại đỉnh, miền ràng buộc L2 (hình tròn) thì không.

C (nghịch đảo mức phạt)	**Số hệ số khác 0 (trên 39)**	**Độ chính xác test**
0,01	0	0,389
0,05	6	0,944
0,1	7	0,982
0,5	13	1,000
1	14	1,000
10	18	1,000

Bảng 5.29: Hồi quy logistic với phạt L1 trên Wine (3 lớp $\times$ 13 đặc trưng $=$ 39 hệ số)

Phạt quá mạnh ($C = 0{,}01$) đưa mọi hệ số về 0: mô hình chỉ còn đoán lớp đông nhất, đạt độ chính xác $21/54 \approx 0{,}389$. Chỉ cần 7 hệ số khác 0, mô hình đã đạt 0,982. Một lưu ý về phiên bản: từ scikit-learn 1.8, tham số `penalty` bị khai tử; phạt L1 được khai báo bằng `l1_ratio=1.0` với `solver="saga"`. Các phiên bản cũ hơn dùng `penalty="l1"`.

Độ quan trọng từ rừng ngẫu nhiên. Rừng ngẫu nhiên (Breiman, 2001; Chương 9) đo độ quan trọng của một đặc trưng bằng **mức giảm độ hỗn tạp trung bình** của mọi nút chia theo đặc trưng đó, trên mọi cây. Với 500 cây trên Wine, thứ hạng là `proline` (0,185), `flavanoids`

(0,175), `color_intensity` (0,144), `od280/od315` (0,136), `alcohol` (0,119), `hue` (0,059)… Thước đo này có hai hạn chế cần biết (Strobl và cộng sự, 2007): nó thiên vị các đặc trưng liên tục hoặc có nhiều giá trị khác nhau, và các đặc trưng tương quan với nhau phải "chia nhau" độ quan trọng. Độ quan trọng hoán vị (`permutation_importance`) khắc phục được hạn chế thứ nhất. Lớp `SelectFromModel` biến bất kỳ mô hình nào có `coef_` hoặc `feature_importances_` thành một bộ lựa chọn đặc trưng.

Đối chiếu kết quả. Ba đặc trưng `proline`, `flavanoids` và `alcohol` xuất hiện trong nhóm đầu ở **mọi** phương pháp — lọc, bao gói lẫn nhúng. Sự đồng thuận giữa các phương pháp có nguyên lý khác nhau là bằng chứng đáng tin cậy hơn kết quả của bất kỳ phương pháp riêng lẻ nào.

Listing 5.16: Ba họ phương pháp lựa chọn đặc trưng trên bộ dữ liệu Wine

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import (VarianceThreshold, SelectKBest,
    f_classif,
                                       mutual_info_classif,
    SequentialFeatureSelector)

X, y = load_wine(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y,
                                          random_state=0)

# 1) Loc: kiem dinh F va thong tin tuong ho
skb = SelectKBest(f_classif, k=5).fit(X_tr, y_tr)
print(list(X.columns[skb.get_support()]))
mi = pd.Series(mutual_info_classif(X_tr, y_tr, random_state=0), index=X.columns
    )
print(mi.sort_values(ascending=False).head(5).round(3))

# 2) Bao goi: SBS cho KNN
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
sbs = SequentialFeatureSelector(knn, n_features_to_select=5,
                                direction="backward", cv=5).fit(X_tr, y_tr)
cols = list(X.columns[sbs.get_support()])
print(cols, knn.fit(X_tr[cols], y_tr).score(X_te[cols], y_te))   # 0.9815

# 3) Nhung: phat L1 (scikit-learn >= 1.8) va rung ngau nhien
l1 = make_pipeline(StandardScaler(),
                   LogisticRegression(l1_ratio=1.0, C=0.1, solver="saga",
    max_iter=50000))
l1.fit(X_tr, y_tr)
print("He so khac 0:", (l1[-1].coef_ != 0).sum(), "/", l1[-1].coef_.size)
rf = RandomForestClassifier(n_estimators=500, random_state=1).fit(X_tr, y_tr)
```

```
print(pd.Series(rf.feature_importances_, index=X.columns)
        .sort_values(ascending=False).head(5).round(3))
```

5.10.4 Lựa chọn đặc trưng phải nằm trong quy trình kiểm định

Đây là một trong những lỗi phương pháp luận nghiêm trọng và phổ biến nhất. Ambroise và McLachlan (2002) phân tích lại các nghiên cứu phân loại ung thư từ dữ liệu biểu hiện gen (vài chục mẫu, hàng nghìn gen) và chỉ ra rằng nhiều tỉ lệ lỗi gần 0% được công bố là do **chọn gen trên toàn bộ dữ liệu trước rồi mới kiểm định chéo**. Khi đó các mẫu kiểm định đã góp phần vào việc chọn đặc trưng: đây là rò rỉ dữ liệu dưới dạng **thiên lệch lựa chọn** (selection bias). Cách làm đúng là đặt bước lựa chọn vào `Pipeline`, để nó được thực hiện lại trên phần huấn luyện của từng fold:

Listing 5.17: Lựa chọn đặc trưng đúng cách: bên trong Pipeline

```
pipe = make_pipeline(StandardScaler(), SelectKBest(f_classif, k=5),
                     KNeighborsClassifier(n_neighbors=5))
print(cross_val_score(pipe, X_tr, y_tr, cv=5).mean())   # uoc luong khong thien
    lech
```

5.11 Dữ liệu mất cân bằng và SMOTE

Trong nhiều bài toán phân lớp, lớp quan trọng nhất lại là lớp **hiếm**: giao dịch gian lận (thường dưới 1%), bệnh hiếm, thiết bị sắp hỏng. Mô hình huấn luyện trên dữ liệu như vậy có xu hướng "bỏ qua" lớp thiểu số mà vẫn đạt accuracy cao — *nghịch lý accuracy* sẽ được phân tích ở Chương 8. Có ba nhóm giải pháp:

- **Ở mức thuật toán**: gán trọng số lớn hơn cho lỗi trên lớp thiểu số, ví dụ đặt tham số `class_weight` bằng `"balanced"`.
- **Giảm mẫu lớp đa số** (undersampling): bỏ bớt mẫu của lớp đông — đơn giản nhưng mất thông tin.
- **Tăng mẫu lớp thiểu số** (oversampling): nhân bản ngẫu nhiên các mẫu hiếm dễ gây quá khớp, vì mô hình gặp đi gặp lại đúng những điểm đó.

SMOTE (Synthetic Minority Over-sampling Technique), do Chawla, Bowyer, Hall và Kegelmeyer công bố năm 2002, khắc phục nhược điểm của việc nhân bản: thay vì sao chép, SMOTE **tổng hợp mẫu mới** nằm trên đoạn thẳng nối một mẫu thiểu số với một láng giềng cùng lớp của nó.

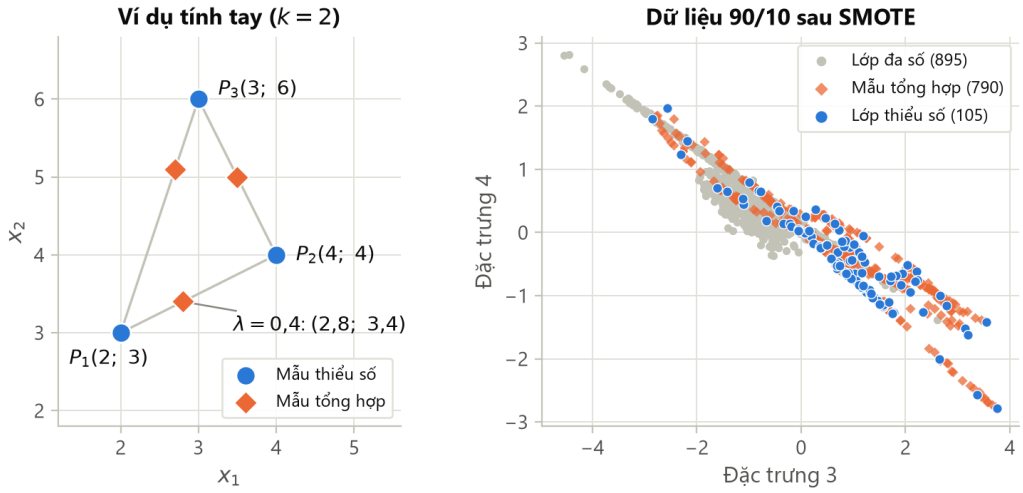
Định nghĩa 5.9 (Sinh mẫu SMOTE)**.** *Với mỗi mẫu thiểu số $\mathbf{x}_i$: tìm k láng giềng gần nhất **trong lớp thiểu số**; chọn ngẫu nhiên một láng giềng $\mathbf{x}_{nn}$ và một số $\lambda \sim U(0,1)$; mẫu mới là*

$$\mathbf{x}_{\text{mới}} = \mathbf{x}_i + \lambda\,(\mathbf{x}_{nn} - \mathbf{x}_i). \tag{5.19}$$

Ví dụ 5.14 (Tạo một mẫu SMOTE)**.** *Lớp thiểu số gồm $P_1(2,3)$, $P_2(4,4)$, $P_3(3,6)$; chọn $k=2$. Hai láng giềng của P_1 là P_2 ($d=\sqrt{5}\approx 2{,}236$) và P_3 ($d=\sqrt{10}\approx 3{,}162$). Giả sử lần rút ngẫu nhiên chọn được P_2 và $\lambda = 0{,}4$:*

$$\mathbf{x}_{\text{mới}} = (2,3) + 0{,}4 \times \big[(4,4)-(2,3)\big] = (2,3) + (0{,}8;\ 0{,}4) = (2{,}8;\ 3{,}4).$$

Điểm mới nằm trên đoạn P_1P_2, bên trong vùng của lớp thiểu số, và không trùng với mẫu nào đã có.



Hình 5.12: SMOTE sinh mẫu tổng hợp trên đoạn thẳng nối các mẫu thiểu số với láng giềng cùng lớp. Trái: ví dụ tính tay; phải: dữ liệu 90/10 của đoạn code bên dưới, chiếu lên hai đặc trưng có điểm F cao nhất.

SMOTE được cài đặt trong thư viện `imbalanced-learn` (Lemaître, Nogueira & Aridas, 2017), cài bằng lệnh `pip install imbalanced-learn`. Các biến thể đáng chú ý: **Borderline-SMOTE** (Han và cộng sự, 2005) chỉ sinh mẫu gần biên quyết định; **ADASYN** (He và cộng sự, 2008) sinh nhiều mẫu hơn ở vùng khó học; **SMOTE-NC** xử lý dữ liệu có cột định danh.

Listing 5.18: SMOTE với imbalanced-learn

```
from collections import Counter
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE

X, y = make_classification(n_samples=1000, n_features=4, n_informative=2,
                           weights=[0.9, 0.1], random_state=42)
print(Counter(y))                       # lop 0: 895, lop 1: 105
X_res, y_res = SMOTE(k_neighbors=5, random_state=42).fit_resample(X, y)
print(Counter(y_res))                   # lop 0: 895, lop 1: 895
```

Nguyên tắc vàng: chỉ áp dụng SMOTE trên **tập huấn luyện**. Nếu sinh mẫu trước khi chia dữ liệu, các mẫu tổng hợp được nội suy từ chính những điểm sẽ rơi vào tập kiểm tra, khiến kết quả đánh giá lạc quan giả tạo (rò rỉ dữ liệu). Thư viện `imbalanced-learn` cung cấp `Pipeline` riêng, trong đó bước lấy mẫu **chỉ chạy khi `fit`**, tức chỉ trên phần huấn luyện của mỗi fold kiểm định chéo:

Listing 5.19: So sánh các cách xử lý mất cân bằng trong Pipeline của imbalanced-learn

```
from imblearn.pipeline import Pipeline as ImbPipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_validate

models = {
```

```
    "khong xu ly": ImbPipeline([("scale", StandardScaler()),
                                ("clf", LogisticRegression())]),
    "class_weight": ImbPipeline([("scale", StandardScaler()),
                                 ("clf", LogisticRegression(class_weight="
    balanced"))]),
    "smote": ImbPipeline([("scale", StandardScaler()),
                          ("smote", SMOTE(random_state=42)),   # chi chay khi
    fit
                          ("clf", LogisticRegression())]),
}
for name, m in models.items():
    r = cross_validate(m, X, y, cv=5, scoring=["recall", "precision", "f1"])
    print(name, {k[5:]: round(v.mean(), 3) for k, v in r.items() if k.
    startswith("test")})
```

Cách xử lý	Recall	Precision	F1
Không xử lý	0,486	0,807	0,589
`class_weight="balanced"`	0,838	0,437	0,572
SMOTE trong pipeline	0,838	0,441	0,575

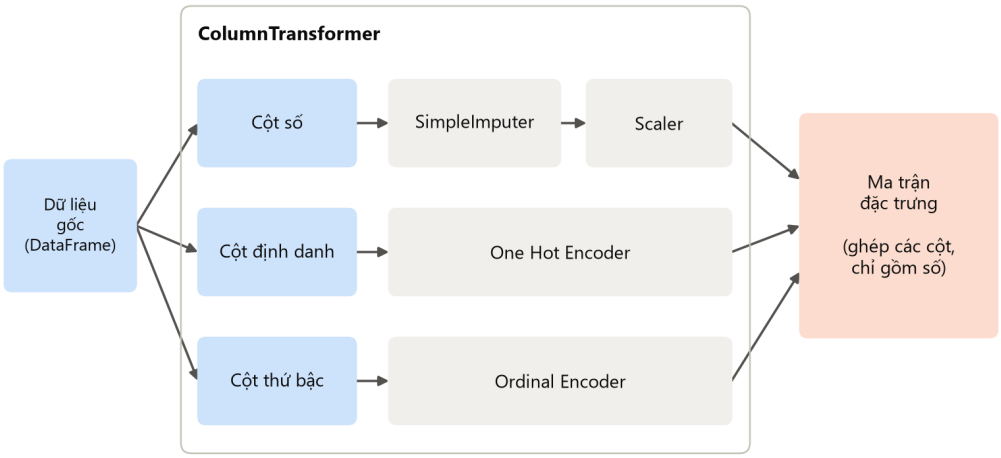
Bảng 5.30: Hồi quy logistic trên dữ liệu tỉ lệ 90/10, kiểm định chéo 5-fold; lớp dương là lớp thiểu số

Kết quả cho một bài học quan trọng: SMOTE giúp mô hình tìm ra nhiều mẫu thiểu số hơn hẳn (recall tăng từ 0,486 lên 0,838), nhưng phải trả giá bằng nhiều báo động nhầm hơn (precision giảm từ 0,807 xuống 0,441); F1 gần như không đổi, và hiệu quả tương đương cách đơn giản `class_weight="balanced"`. SMOTE vì vậy **không phải phép màu**: nó dịch chuyển điểm cân bằng giữa precision và recall, và nên được chọn khi **bỏ sót lớp thiểu số đắt hơn báo động nhầm** (sàng lọc bệnh, phát hiện gian lận). Việc so sánh phải dựa trên độ đo phù hợp với bài toán (Chương 8), không phải accuracy.

5.12 Kết hợp các bước: Pipeline và ColumnTransformer

Dữ liệu thực tế thường có **nhiều kiểu cột cùng lúc**. scikit-learn cung cấp:

- `ColumnTransformer`: áp dụng các bộ biến đổi khác nhau cho các nhóm cột khác nhau rồi ghép kết quả.
- `Pipeline`: xâu chuỗi các bước tiền xử lý và mô hình thành một đối tượng duy nhất. Khi gọi `pipeline.fit(X_train, y_train)`, mọi bước `fit` chỉ nhìn thấy tập huấn luyện — đây là cách tự động **tránh rò rỉ dữ liệu**.



Hình 5.13: Sơ đồ ColumnTransformer: cột số đi qua SimpleImputer + Scaler, cột định danh qua One Hot Encoder, cột thứ bậc qua Ordinal Encoder, sau đó ghép lại thành ma trận đặc trưng.

5.13 Nghiên cứu tình huống: bộ dữ liệu Ames Housing

5.13.1 Giới thiệu bộ dữ liệu

Bộ dữ liệu **Ames Housing** do Dean De Cock (Đại học Truman State) công bố năm 2011 trên *Journal of Statistics Education*, như một lựa chọn thay thế hiện đại cho bộ dữ liệu giá nhà Boston kinh điển. Dữ liệu ghi nhận các giao dịch bán nhà ở thành phố Ames, bang Iowa (Hoa Kỳ) trong giai đoạn 2006–2010, lấy từ Văn phòng Định giá Tài sản của thành phố.

File `data/AmesHousing.txt` (phân tách bằng ký tự tab) gồm **2930 dòng** và **82 cột**: hai cột định danh bản ghi (`Order`, `PID`), biến mục tiêu `SalePrice` (giá bán, USD) và 79 biến mô tả căn nhà. Đây là bộ dữ liệu lý tưởng cho chương này vì nó chứa **đủ mọi kiểu dữ liệu** đã học:

Kiểu	**Ví dụ cột**	**Giá trị mẫu**
Định danh	`MS Zoning`, `Neighborhood`, `Street`, `Bldg Type`	RL, RM, FV; NAmes, Gilbert; Pave, Grvl
Định danh mã hóa bằng số	`MS SubClass`	020, 060, 120 (mã loại nhà, **không phải** đại lượng)
Thứ bậc	`Exter Qual`, `Kitchen Qual`, `Bsmt Qual`	Po < Fa < TA < Gd < Ex
Số rời rạc	`Full Bath`, `Garage Cars`, `Yr Sold`	0–4; 0–5; 2006–2010
Số liên tục	`Lot Area`, `Gr Liv Area`, `SalePrice`	`Lot Area` tối đa 215245 ft^2

Bảng 5.31: Các kiểu dữ liệu trong Ames Housing

5.13.2 Những "cái bẫy" khi đọc dữ liệu

Quan sát kỹ file dữ liệu cho thấy nhiều vấn đề rất điển hình mà người làm ML phải xử lý:

1. **"NA" không phải lúc nào cũng là thiếu dữ liệu.** Ở các cột `Alley`, `Fence`, `Pool QC`, `Fireplace Qu`, `Garage Type`, `Bsmt Qual`…, chuỗi `NA` mang nghĩa *"không có"* (không có hẻm, không có hàng rào, không có garage). Nếu đọc bằng `pd.read_csv` mặc định, pandas sẽ biến tất cả `NA` thành `NaN` và ta **đánh mất thông tin**. Trong khi đó, ô **rỗng** (ví dụ `Lot Frontage` ở nhiều dòng, `Mas Vnr Type` ở một số dòng) mới thực sự là dữ liệu thiếu.

2. **Khoảng trắng thừa:** cột `Sale Type` có giá trị `"WD "` (có dấu cách cuối). Không làm sạch thì `"WD"` và `"WD "` bị coi là hai nhãn khác nhau khi One Hot.

3. **Số nhưng là phân loại:** `MS SubClass` (020, 060…) là mã loại nhà; dùng trực tiếp như số sẽ gây thứ tự giả giống hệt vấn đề của Label Encoder.

4. **Lỗi nhập liệu:** có bản ghi với `Garage Yr Blt` = 2207 — một năm không thể xảy ra (nhiều khả năng là 2007).

5. **Ngoại lai:** một vài căn có `Gr Liv Area` trên 4000 ft^2 nhưng giá bán thấp bất thường (giao dịch bán từng phần). Chính De Cock khuyến nghị loại các căn nhà trên 4000 ft^2 khi dùng dữ liệu cho giảng dạy. Cột `Lot Area` cũng lệch phải rất mạnh (tối đa 215245 ft^2).

5.13.3 Đọc và khảo sát dữ liệu

Listing 5.20: Đọc Ames Housing và giữ nguyên ý nghĩa của "NA"

```
import pandas as pd

# keep_default_na=False: KHONG bien "NA" thanh NaN
# na_values=[""]       : chi o rong moi la du lieu thieu
df = pd.read_csv("data/AmesHousing.txt", sep="\t",
                 keep_default_na=False, na_values=[""])
print(df.shape)                          # (2930, 82)

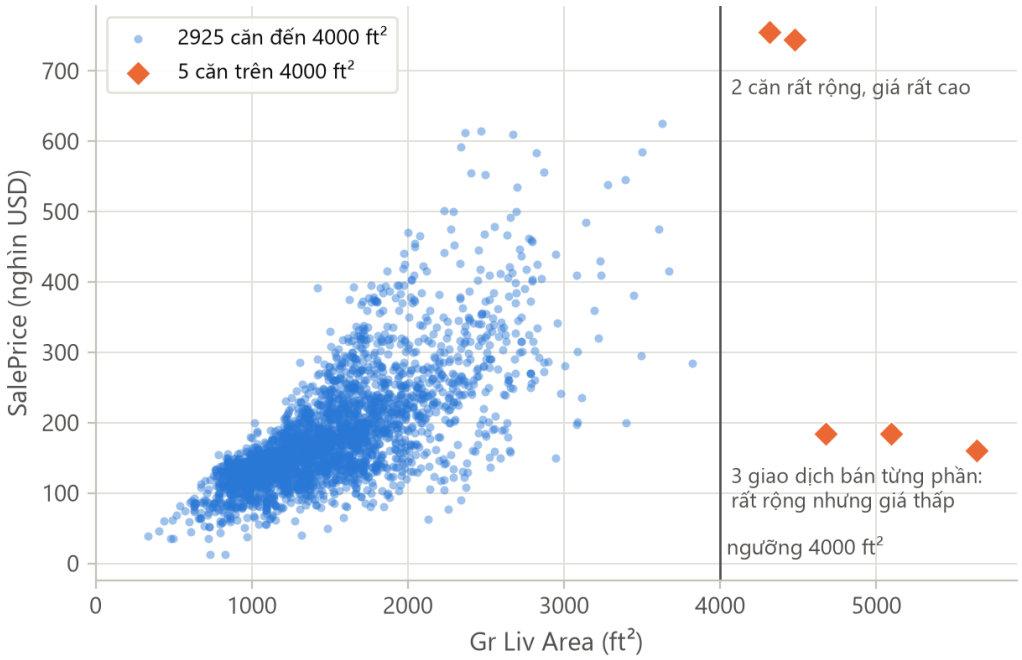
# Lam sach khoang trang thua o cac cot chu
obj_cols = df.select_dtypes("object").columns
df[obj_cols] = df[obj_cols].apply(lambda s: s.str.strip())

# MS SubClass la ma phan loai -> doi sang kieu chuoi
df["MS SubClass"] = df["MS SubClass"].astype(str)

# Thong ke du lieu thieu that su
missing = df.isna().sum()
print(missing[missing > 0].sort_values(ascending=False))

# Sua loi nhap lieu va loai ngoai lai theo khuyen nghi cua De Cock
df.loc[df["Garage Yr Blt"] > 2010, "Garage Yr Blt"] = None
df = df[df["Gr Liv Area"] <= 4000]
print(df.shape)

print(df["Exter Qual"].value_counts())
print(df[["Lot Area", "Gr Liv Area", "SalePrice"]].describe())
```



Hình 5.14: Biểu đồ phân tán `Gr Liv Area` – `SalePrice` của Ames Housing; các điểm bên phải ngưỡng 4000 ft^2 là ngoại lai. Trong đó 3 căn giá thấp là giao dịch bán từng phần (`Sale Condition` = Partial).

5.13.4 Xây dựng quy trình tiền xử lý hoàn chỉnh

Ta chọn một tập con đặc trưng đại diện cho từng kiểu dữ liệu và áp dụng đúng kỹ thuật đã học cho từng nhóm:

Listing 5.21: Pipeline tiền xử lý cho Ames Housing

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import (OneHotEncoder, OrdinalEncoder,
                                   StandardScaler, RobustScaler)

nominal = ["MS Zoning", "Neighborhood", "Bldg Type", "MS SubClass",
           "Alley", "Garage Type"]
quality = ["Exter Qual", "Kitchen Qual", "Bsmt Qual", "Fireplace Qu"]
numeric = ["Gr Liv Area", "Total Bsmt SF", "Garage Cars",
           "Full Bath", "Year Built", "Overall Qual"]
skewed  = ["Lot Area", "Lot Frontage"]

# "NA" o cac cot chat luong = khong co -> muc thap nhat
levels = ["NA", "Po", "Fa", "TA", "Gd", "Ex"]

X = df[nominal + quality + numeric + skewed]
y = np.log(df["SalePrice"])      # log giup giam do lech cua gia

```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

preprocess = ColumnTransformer([
    ("nom", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore")),
    ]), nominal),
    ("ord", Pipeline([
        ("impute", SimpleImputer(strategy="constant", fill_value="NA")),
        ("ordinal", OrdinalEncoder(categories=[levels] * len(quality))),
    ]), quality),
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), numeric),
    ("skew", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", RobustScaler()),
    ]), skewed),
])

# CHI fit tren tap huan luyen, transform cho ca hai tap
X_train_t = preprocess.fit_transform(X_train)
X_test_t  = preprocess.transform(X_test)
print(X_train_t.shape, X_test_t.shape)
print(preprocess.get_feature_names_out()[:10])
```

Ma trận `X_train_t` thu được chỉ gồm các số thực, đã đồng nhất thang đo và không còn giá trị thiếu — sẵn sàng cho các mô hình hồi quy ở chương tiếp theo. Khi ghép `preprocess` với một mô hình trong cùng một `Pipeline`, toàn bộ quy trình có thể được đánh giá bằng kiểm định chéo mà không lo rò rỉ dữ liệu:

Listing 5.22: Ghép tiền xử lý với mô hình trong một Pipeline

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import cross_val_score

model = Pipeline([("prep", preprocess), ("reg", LinearRegression())])
scores = cross_val_score(model, X_train, y_train, cv=5, scoring="r2")
print(scores.mean().round(3))
```

5.14 Tóm tắt chương

– Học máy là quá trình cải thiện hiệu năng P trên tác vụ T nhờ kinh nghiệm E (Mitchell, 1997); có ba loại chính: có giám sát, không giám sát và tăng cường. Dữ liệu được ký hiệu thống nhất là $\mathbf{X} \in \mathbb{R}^{n \times d}$ và $\mathbf{y} \in \mathbb{R}^n$. Theo định lý No Free Lunch, không có thuật toán tốt nhất cho mọi bài toán.

– Tiền xử lý dữ liệu là bước nền tảng trong quy trình KDD (Fayyad và cộng sự, 1996) và pha Chuẩn bị dữ liệu của CRISP-DM; nguyên lý "rác vào, rác ra" nhắc rằng chất lượng

mô hình bị giới hạn bởi chất lượng dữ liệu.

– Dữ liệu thiếu có ba cơ chế MCAR, MAR, MNAR (Rubin, 1976). **Simple Imputer** điền bằng trung bình/trung vị/mode; **KNN Imputer** lấy trung bình k láng giềng theo khoảng cách nan-Euclid $\sqrt{\frac{d}{|P|}\sum_{j\in P}(x_j - y_j)^2}$; **Iterative Imputer** hồi quy lặp từng cột theo các cột còn lại — mạnh nhưng có thể ngoại suy phi thực tế khi dữ liệu ít hoặc đa cộng tuyến, nên luôn phải kiểm tra giá trị điền.

– Ngoại lai: z-score dễ bị che lấp; quy tắc IQR của Tukey bền vững hơn; **LOF** so sánh mật độ cục bộ của một điểm với láng giềng, LOF $\gg 1$ là ngoại lai.

– Mô hình ML chỉ làm việc với số: dữ liệu chữ phải được mã hóa sao cho **không tạo ra quan hệ giả**.

– **Label Encoder** ánh xạ nhãn $\rightarrow$ số nguyên, gọn nhẹ nhưng tạo thứ tự giả; dùng cho biến mục tiêu. **Ordinal Encoder** dùng cho đặc trưng thứ bậc.

– **One Hot Encoder** biến mỗi nhãn thành một vector đơn vị, mọi nhãn cách đều $\sqrt{2}$; phù hợp với KNN, K-Means, mô hình tuyến tính.

– **Count Vectorizer** biểu diễn văn bản bằng số lần xuất hiện của từ; **TF-IDF** (Spärck Jones, 1972) giảm trọng số của các từ phổ biến; chuẩn hóa L2 cho phép đo độ tương đồng cosine bằng tích vô hướng.

– Bốn bộ chuẩn hóa **MinMax, MaxAbs, Standard, Robust** khác nhau về miền giá trị, khả năng giữ tính thưa và độ bền với ngoại lai.

– **Tạo đặc trưng**: nhị phân hóa, rời rạc hóa (khoảng đều, phân vị, chuyên môn), biến đổi lũy thừa (logarit, Box-Cox, Yeo-Johnson) cho dữ liệu lệch, đặc trưng tương tác và đa thức, đặc trưng thời gian (mã hóa chu kỳ, cửa sổ trượt với `shift(1)` để không dùng thông tin tương lai).

– **Lựa chọn đặc trưng** gồm ba họ: lọc (ngưỡng phương sai, F, χ^2, thông tin tương hỗ), bao gói (SBS, SFS, RFE) và nhúng (L1, độ quan trọng của rừng ngẫu nhiên). Bước lựa chọn phải nằm trong pipeline để tránh thiên lệch lựa chọn.

– **SMOTE** sinh mẫu thiểu số bằng nội suy $\mathbf{x}_i + \lambda(\mathbf{x}_{nn} - \mathbf{x}_i)$; chỉ áp dụng trên tập huấn luyện (dùng Pipeline của imbalanced-learn) và đánh giá bằng recall, precision, F1 thay vì accuracy.

– Luôn `fit` trên tập huấn luyện; dùng `Pipeline` và `ColumnTransformer` để quy trình gọn gàng và tránh rò rỉ dữ liệu.

5.15 Bài tập

1. Cho dãy gồm các phần tử `cat`, `dog`, `fish`, `dog`, `cat`, `bird`. Hãy (a) tự cài đặt Label Encoder theo thứ tự xuất hiện; (b) so sánh với kết quả của `LabelEncoder` trong scikit-learn và giải thích vì sao khác nhau.

2. Với dãy ở bài 1, tự cài đặt One Hot Encoder bằng list, for và dictionary. Chứng minh rằng khoảng cách Euclid giữa hai vector one-hot khác nhau bất kỳ luôn bằng $\sqrt{2}$.

3. Thêm câu d_4 = "the birds are outside" vào kho văn bản ở mục TF-IDF. Tính tay $\mathrm{df}(t)$ và $\mathrm{idf}(t)$ (làm trơn) cho các từ *are, outside, the, birds*, sau đó kiểm tra bằng `TfidfVectorizer`.

4. Dùng bảng tính (Excel/Google Sheets) dựng lại toàn bộ Bảng 5.21–5.25. Gợi ý: hàm `LN`, `SQRT`, `SUMSQ`.

5. Cho cột $x = (10, 12, 11, 13, 12, 100)$. Tính tay kết quả của bốn bộ chuẩn hóa và nhận xét bộ nào phù hợp nhất. Vì sao?

6. Giải thích vì sao gọi `StandardScaler().fit(X)` trên toàn bộ dữ liệu trước khi chia train/test là rò rỉ dữ liệu. Thiết kế thí nghiệm trên Ames Housing để minh họa.

7. Với Ames Housing: đếm số lượng giá trị `NA` ("không có") và ô rỗng (thiếu thật sự) trong từng cột; bổ sung các cột `Heating QC`, `Garage Finish`, `Foundation` vào pipeline với kỹ thuật mã hóa phù hợp.

8. Cho cột $(5, 8, ?, 6, 100, 7)$. Điền giá trị thiếu bằng trung bình và bằng trung vị. Cách nào hợp lý hơn? Vì sao? (Đáp số: 25,2 và 7.)

9. Tính khoảng cách nan-Euclid giữa $(1, \mathrm{nan}, 3, 4)$ và $(2, 5, \mathrm{nan}, 6)$, rồi kiểm tra bằng `nan_euclidean_distances`.

10. Với Bảng 5.7, chạy tay lại KNN Imputer với $k = 2$, sau đó với $k = 3$ và trọng số $1/d$. So sánh với `KNNImputer(n_neighbors=2)` và `KNNImputer(n_neighbors=3, weights="distance")`.

11. Tiếp tục ví dụ Iterative Imputer thêm một vòng nữa (bắt đầu từ các giá trị điền của vòng 1), rồi so sánh với `IterativeImputer(estimator=LinearRegression(), imputation_order="roman", max_iter=2)`. Giải thích vì sao giá trị của x_3 không hội tụ về miền $[1;\ 8]$.

12. Thêm điểm $F(2, 2)$ vào ví dụ LOF và tính lại bảng LOF với $k = 3$. Hệ số LOF của E thay đổi thế nào?

13. Trên `load_breast_cancer`, giữ lại toàn bộ lớp lành tính và chỉ 10% lớp ác tính để tạo dữ liệu mất cân bằng. So sánh recall, precision, F1 của hồi quy logistic khi: không xử lý, `class_weight="balanced"`, SMOTE trong pipeline, và SMOTE áp dụng **sai** lên toàn bộ dữ liệu trước khi kiểm định chéo.

14. Theo định nghĩa của Mitchell, hãy xác định T, E, P cho ba bài toán: dự đoán giá nhà Ames, phân khúc khách hàng, và chương trình chơi cờ vua. Bài toán nào là học có giám sát, không giám sát, tăng cường?

15. Với cột `Lot Area` của Ames Housing: vẽ histogram trước và sau biến đổi logarit, Box-Cox, Yeo-Johnson; báo cáo skewness và $\hat{\lambda}$. Rời rạc hóa cột này thành 5 khoảng theo hai chiến lược `"uniform"` và `"quantile"`; vì sao chiến lược thứ nhất cho các khoảng gần như rỗng?

16. Cài đặt thuật toán SBS từ đầu (không dùng `SequentialFeatureSelector`) cho KNN trên Wine và vẽ độ chính xác theo số đặc trưng $k = 1, \ldots, 13$. So sánh tập 5 đặc trưng tìm được với Bảng 5.28.

17. Thí nghiệm thiên lệch lựa chọn: sinh 50 mẫu với 5000 đặc trưng ngẫu nhiên và nhãn ngẫu nhiên. So sánh độ chính xác kiểm định chéo khi (a) chọn 20 đặc trưng bằng `SelectKBest` trên toàn bộ dữ liệu rồi mới kiểm định chéo, (b) đặt `SelectKBest` trong pipeline. Giá trị đúng phải xấp xỉ bao nhiêu?

Tài liệu tham khảo của chương

1. Ambroise, C., & McLachlan, G. J. (2002). Selection bias in gene extraction on the basis of microarray gene-expression data. *Proceedings of the National Academy of Sciences*, 99(10), 6562–6566.

2. Box, G. E. P., & Cox, D. R. (1964). An Analysis of Transformations. *Journal of the Royal Statistical Society, Series B*, 26(2), 211–252.

3. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

4. Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying Density-Based Local Outliers. *Proceedings of ACM SIGMOD 2000*, 93–104.

5. Buitinck, L. et al. (2013). API design for machine learning software: experiences from the scikit-learn project. *ECML PKDD Workshop: Languages for Data Mining and Machine Learning*.

6. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.

7. De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).

8. Domingos, P. (2012). A Few Useful Things to Know about Machine Learning. *Communications of the ACM*, 55(10), 78–87.

9. Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From Data Mining to Knowledge Discovery in Databases. *AI Magazine*, 17(3), 37–54.

10. Ferri, F. J., Pudil, P., Hatef, M., & Kittler, J. (1994). Comparative study of techniques for large-scale feature selection. *Pattern Recognition in Practice IV*, 403–413.

11. Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.

12. Guyon, I., & Elisseeff, A. (2003). An Introduction to Variable and Feature Selection. *Journal of Machine Learning Research*, 3, 1157–1182.

13. Guyon, I., Weston, J., Barnhill, S., & Vapnik, V. (2002). Gene Selection for Cancer Classification using Support Vector Machines. *Machine Learning*, 46, 389–422.

14. Han, J., Kamber, M., & Pei, J. (2011). *Data Mining: Concepts and Techniques* (3rd ed.), Chương 3: Data Preprocessing. Morgan Kaufmann.

15. Han, H., Wang, W.-Y., & Mao, B.-H. (2005). Borderline-SMOTE: A New Over-Sampling Method in Imbalanced Data Sets Learning. *Advances in Intelligent Computing (ICIC 2005)*, LNCS 3644, 878–887.

16. Harris, Z. S. (1954). Distributional Structure. *Word*, 10(2–3), 146–162.

17. Hawkins, D. M. (1980). *Identification of Outliers*. Chapman and Hall.

18. He, H., Bai, Y., Garcia, E. A., & Li, S. (2008). ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. *Proceedings of IJCNN 2008*, 1322–1328.

19. Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning. *Journal of Machine Learning Research*, 18(17), 1–5.

20. Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 413–422.

21. Liu, Y. (H.) (2019). *Python Machine Learning By Example* (2nd ed.). Packt Publishing.

22. Luhn, H. P. (1957). A Statistical Approach to Mechanized Encoding and Searching of Literary Information. *IBM Journal of Research and Development*, 1(4), 309–317.

23. Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.

24. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

25. Pyle, D. (1999). *Data Preparation for Data Mining*. Morgan Kaufmann.

26. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*. Packt Publishing.

27. Rubin, D. B. (1976). Inference and Missing Data. *Biometrika*, 63(3), 581–592.

28. Rubin, D. B. (1987). *Multiple Imputation for Nonresponse in Surveys*. Wiley.

29. Salton, G., Wong, A., & Yang, C. S. (1975). A Vector Space Model for Automatic Indexing. *Communications of the ACM*, 18(11), 613–620.

30. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.

31. Samuel, A. L. (1959). Some Studies in Machine Learning Using the Game of Checkers. *IBM Journal of Research and Development*, 3(3), 210–229.

32. Sarkar, D., Bali, R., & Sharma, T. (2018). *Practical Machine Learning with Python: A Problem-Solver's Guide to Building Real-World Intelligent Systems*. Apress.

33. Shiffler, R. E. (1988). Maximum Z Scores and Outliers. *The American Statistician*, 42(1), 79–80.

34. Spärck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21.

35. Stevens, S. S. (1946). On the Theory of Scales of Measurement. *Science*, 103(2684), 677–680.

36. Strobl, C., Boulesteix, A.-L., Zeileis, A., & Hothorn, T. (2007). Bias in random forest variable importance measures: Illustrations, sources and a solution. *BMC Bioinformatics*, 8, 25.

37. Suits, D. B. (1957). Use of Dummy Variables in Regression Equations. *Journal of the American Statistical Association*, 52(280), 548–551.

38. Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society, Series B*, 58(1), 267–288.

39. Troyanskaya, O., Cantor, M., Sherlock, G., Brown, P., Hastie, T., Tibshirani, R., Botstein, D., & Altman, R. B. (2001). Missing value estimation methods for DNA microarrays. *Bioinformatics*, 17(6), 520–525.

40. Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley.

41. van Buuren, S., & Groothuis-Oudshoorn, K. (2011). mice: Multivariate Imputation by Chained Equations in R. *Journal of Statistical Software*, 45(3), 1–67.

42. Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a Standard Process Model for Data Mining. *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining*.

43. Wolpert, D. H. (1996). The Lack of A Priori Distinctions Between Learning Algorithms. *Neural Computation*, 8(7), 1341–1390.

44. Yeo, I.-K., & Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), 954–959.

Chương 6: Hồi quy (Regression)

Hồi quy (regression) là bài toán **học có giám sát** (supervised learning) trong đó biến cần dự đoán là một **giá trị số liên tục**: giá nhà, nhiệt độ ngày mai, doanh thu tháng tới, lượng điện tiêu thụ… Trong bảng phân công kế hoạch, nhóm thuật toán này gồm **Linear Regression** và mở rộng sang **chuỗi thời gian** (time series). Đây cũng là chương đầu tiên ta sử dụng dữ liệu đã được tiền xử lý ở Chương 5 để huấn luyện một mô hình thực sự.
Sau khi học xong chương này, người học có khả năng:

- Trình bày nguồn gốc lịch sử của phương pháp bình phương tối thiểu và khái niệm "hồi quy".
- Xây dựng mô hình hồi quy tuyến tính đơn biến, đa biến; tự tính tay nghiệm bình phương tối thiểu.
- Hiểu và cài đặt Gradient Descent ở ba dạng batch, stochastic, mini-batch; tính tay một epoch theo từng phần tử và theo dạng ma trận; nhận biết và khắc phục hiện tượng phân kỳ.
- Nhận biết quá khớp và áp dụng hồi quy đa thức, Ridge, Lasso.
- Xử lý ngoại lai bằng hồi quy bền vững (RANSAC, Huber) và chẩn đoán mô hình bằng biểu đồ phần dư.
- Xây dựng mô hình dự báo chuỗi thời gian bằng đặc trưng trễ (lag features).
- Hoàn thiện mô hình dự đoán giá nhà trên bộ dữ liệu Ames Housing.

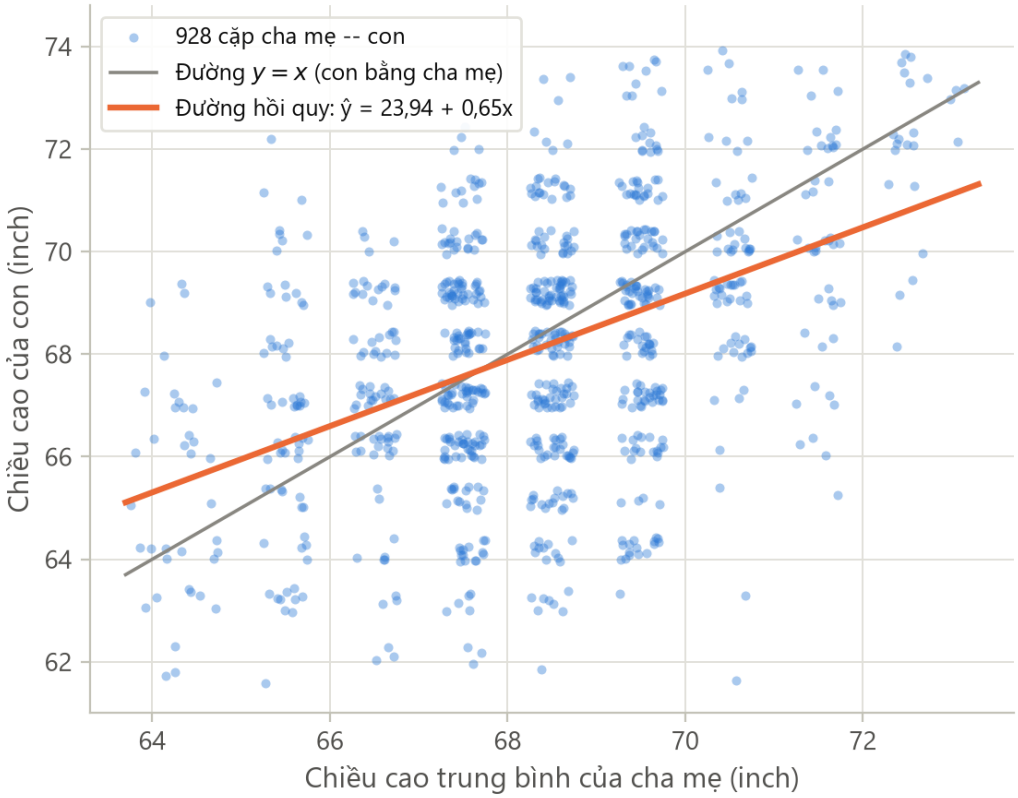
6.1 Khởi nguồn của hồi quy

6.1.1 Phương pháp bình phương tối thiểu

Năm 1805, nhà toán học Pháp Adrien-Marie Legendre công bố *phương pháp bình phương tối thiểu* (méthode des moindres carrés) trong phụ lục của một công trình xác định quỹ đạo sao chổi. Năm 1809, Carl Friedrich Gauss trình bày cùng phương pháp trong *Theoria Motus* và khẳng định đã dùng nó từ năm 1795 để dự đoán vị trí tiểu hành tinh Ceres; cuộc tranh chấp quyền ưu tiên giữa hai ông là một trong những tranh chấp nổi tiếng nhất lịch sử toán học. Gauss còn chỉ ra rằng nếu sai số tuân theo phân phối chuẩn thì nghiệm bình phương tối thiểu chính là ước lượng hợp lý nhất — nền tảng thống kê của hồi quy hiện đại.

6.1.2 Vì sao gọi là "hồi quy"?

Thuật ngữ *regression* đến từ nhà khoa học Anh Francis Galton. Năm 1886, khi nghiên cứu chiều cao của cha mẹ và con cái, Galton nhận thấy con của những cha mẹ rất cao thường thấp hơn cha mẹ, còn con của những cha mẹ rất thấp thường cao hơn — chiều cao có xu hướng "quay về mức trung bình" (*regression towards mediocrity*). Học trò của ông là Karl Pearson sau đó hình thức hóa hệ số tương quan và hồi quy tuyến tính. Ngày nay, "hồi quy" chỉ chung mọi bài toán dự đoán giá trị liên tục, bất kể có hiện tượng "quay về trung bình" hay không.



Hình 6.1: Dữ liệu chiều cao cha mẹ – con cái của Galton (1886), gồm 928 cặp (số liệu từ gói `HistData` của R), và đường hồi quy có hệ số góc 0,65 — nhỏ hơn 1 nên con của cha mẹ rất cao hoặc rất thấp có xu hướng quay về mức trung bình. Các điểm được rải nhẹ vì số liệu gốc đã làm tròn theo khoảng.

6.2 Hồi quy tuyến tính đơn biến

Định nghĩa 6.1 (Hồi quy tuyến tính đơn biến)**.** *Cho tập dữ liệu* $\{(x_i, y_i)\}_{i=1}^{n}$. *Mô hình hồi quy tuyến tính đơn biến giả định*

$$y = w_0 + w_1 x + \varepsilon, \tag{6.1}$$

trong đó w_0 *là hệ số chặn (intercept),* w_1 *là hệ số góc (slope) và* ε *là sai số ngẫu nhiên. Giá trị dự đoán là* $\hat{y} = w_0 + w_1 x$.

6.2.1 Hàm mất mát và nghiệm bình phương tối thiểu

Để chọn (w_0, w_1), ta cực tiểu tổng bình phương sai số (Sum of Squared Errors – SSE):

$$\text{SSE}(w_0, w_1) = \sum_{i=1}^{n} \big(y_i - (w_0 + w_1 x_i)\big)^2. \tag{6.2}$$

Lấy đạo hàm riêng theo w_0, w_1 và cho bằng 0, ta được nghiệm dạng đóng:

$$w_1 = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}, \qquad w_0 = \bar{y} - w_1 \bar{x}. \tag{6.3}$$

Ví dụ 6.1 (Tính tay hồi quy tuyến tính)**.** *Cho 5 điểm dữ liệu:*

i	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	1	2.1	-2	-3.94	7.88	4
2	2	3.9	-1	-2.14	2.14	1
3	3	6.1	0	0.06	0	0
4	4	8.2	1	2.16	2.16	1
5	5	9.9	2	3.86	7.72	4
				Tổng	$S_{xy} = 19.9$	$S_{xx} = 10$

Bảng 6.1: Bảng tính hồi quy tuyến tính đơn biến ($\bar{x} = 3$, $\bar{y} = 6{,}04$)

Theo (6.3): $w_1 = 19{,}9/10 = 1{,}99$ và $w_0 = 6{,}04 - 1{,}99 \times 3 = 0{,}07$. Mô hình thu được là $\hat{y} = 0{,}07 + 1{,}99x$, và dự đoán cho $x = 6$ là $\hat{y} = 12{,}01$. Tổng bình phương sai số còn lại $\text{SSE} = 0{,}071$, tức $\text{MSE} = 0{,}0142$; hệ số xác định $R^2 \approx 0{,}9982$ cho thấy đường thẳng giải thích gần như toàn bộ biến thiên của y.

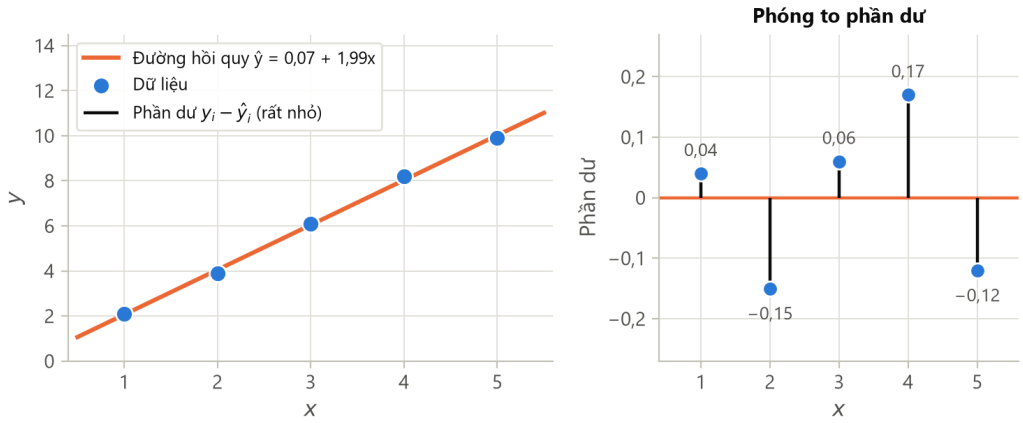
Listing 6.1: Hồi quy tuyến tính: tự tính và dùng scikit-learn

```
import numpy as np
from sklearn.linear_model import LinearRegression

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2.1, 3.9, 6.1, 8.2, 9.9])

# 1) Tu tinh theo cong thuc binh phuong toi thieu
w1 = ((x - x.mean()) * (y - y.mean())).sum() / ((x - x.mean())**2).sum()
w0 = y.mean() - w1 * x.mean()
print(w0, w1)                    # 0.07  1.99

# 2) scikit-learn (X phai la ma tran 2 chieu)
model = LinearRegression()
model.fit(x.reshape(-1, 1), y)
print(model.intercept_, model.coef_)   # 0.07  [1.99]
print(model.predict([[6]]))            # [12.01]
print(model.score(x.reshape(-1, 1), y))  # R^2 = 0.9982
```



Hình 6.2: Năm điểm dữ liệu và đường hồi quy $\hat{y} = 0{,}07 + 1{,}99x$; các đoạn thẳng đứng là phần dư $y_i - \hat{y}_i$. Hình bên phải phóng to các phần dư vì chúng rất nhỏ so với thang đo của y.

6.3 Hồi quy tuyến tính đa biến

Khi có d đặc trưng, mô hình trở thành

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d = \mathbf{w}^\top \tilde{\mathbf{x}}, \tag{6.4}$$

với $\tilde{\mathbf{x}} = (1, x_1, \ldots, x_d)$. Gom n mẫu thành ma trận thiết kế $\mathbf{X} \in \mathbb{R}^{n \times (d+1)}$ (cột đầu toàn số 1), hàm mất mát $\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2$ có nghiệm theo **phương trình chuẩn** (normal equation):

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}. \tag{6.5}$$

Trong thực tế, thư viện không nghịch đảo ma trận trực tiếp mà dùng phân rã QR hoặc SVD (xem chương Giảm chiều dữ liệu) để ổn định số học. Khi các đặc trưng tương quan mạnh với nhau (đa cộng tuyến), $\mathbf{X}^\top \mathbf{X}$ gần suy biến và hệ số trở nên rất bất ổn — đó là động lực của Ridge ở mục sau. Đây cũng là lý do bẫy biến giả (Chương 5) gây lỗi: các cột one-hot cộng lại bằng cột hệ số chặn.

Ví dụ 6.2 (Tính tay phương trình chuẩn với hai đặc trưng)**.** *Cho 4 mẫu trong Bảng 6.2. Tìm mô hình $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$ và dự đoán y tại $x_1 = 3$, $x_2 = 7$.*

bias	x_1	x_2	y
1	7	3	9
1	4	1	15
1	5	6	21
1	9	8	37

Bảng 6.2: Dữ liệu ví dụ phương trình chuẩn; cột bias toàn số 1 ứng với hệ số chặn β_0

Bước 1 – lập các ma trận. Các phần tử của $\mathbf{X}^\top \mathbf{X}$ và $\mathbf{X}^\top \mathbf{y}$ chính là các tổng quen thuộc: $n = 4$, $\sum x_1 = 25$, $\sum x_2 = 18$, $\sum x_1^2 = 171$, $\sum x_1 x_2 = 127$, $\sum x_2^2 = 110$, $\sum y = 82$, $\sum x_1 y = 561$, $\sum x_2 y = 464$:

$$\mathbf{X}^\top \mathbf{X} = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 7 & 4 & 5 & 9 \\ 3 & 1 & 6 & 8 \end{pmatrix} \begin{pmatrix} 1 & 7 & 3 \\ 1 & 4 & 1 \\ 1 & 5 & 6 \\ 1 & 9 & 8 \end{pmatrix} = \begin{pmatrix} 4 & 25 & 18 \\ 25 & 171 & 127 \\ 18 & 127 & 110 \end{pmatrix}, \qquad \mathbf{X}^\top \mathbf{y} = \begin{pmatrix} 82 \\ 561 \\ 464 \end{pmatrix}.$$

Bước 2 – nghịch đảo. Với $\det(\mathbf{X}^\top\mathbf{X}) = 870$, nghịch đảo bằng ma trận phụ hợp chia cho định thức:

$$(\mathbf{X}^\top\mathbf{X})^{-1} = \frac{1}{870}\begin{pmatrix} 2681 & -464 & 97 \\ -464 & 116 & -58 \\ 97 & -58 & 59 \end{pmatrix} \approx \begin{pmatrix} 3{,}0816 & -0{,}5333 & 0{,}1115 \\ -0{,}5333 & 0{,}1333 & -0{,}0667 \\ 0{,}1115 & -0{,}0667 & 0{,}0678 \end{pmatrix}.$$

Kiểm tra nhanh: $\mathbf{X}^\top\mathbf{X}$ đối xứng và xác định dương, nên nghịch đảo của nó cũng **đối xứng** và có mọi phần tử trên **đường chéo dương**. Nếu tính ra một phần tử chéo âm (chẳng hạn $-0{,}0678$ thay vì $0{,}0678$) thì chắc chắn đã có sai sót.

Bước 3 – hệ số:

$$\boldsymbol{\beta} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y} = \frac{1}{870}\begin{pmatrix} 4546 \\ 116 \\ 2792 \end{pmatrix} \approx \begin{pmatrix} 5{,}2253 \\ 0{,}1333 \\ 3{,}2092 \end{pmatrix}.$$

Bước 4 – dự đoán: $\hat{y} = 5{,}2253 + 0{,}1333 \times 3 + 3{,}2092 \times 7 \approx 28{,}09$.

Với chỉ 4 mẫu và 3 tham số, mô hình giải thích được $R^2 \approx 0{,}716$ phương sai của y trên chính tập huấn luyện — ví dụ chỉ nhằm minh họa quy trình tính, không nhằm đánh giá chất lượng mô hình.

Listing 6.2: Kiểm tra ví dụ phương trình chuẩn

```
import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[7, 3], [4, 1], [5, 6], [9, 8]])
y = np.array([9, 15, 21, 37])
Xb = np.c_[np.ones(len(X)), X]                # them cot 1 cho he so chan
print(Xb.T @ Xb)                              # [[4 25 18] [25 171 127] [18 127
    110]]
beta = np.linalg.solve(Xb.T @ Xb, Xb.T @ y)  # on dinh hon np.linalg.inv
print(beta.round(4))                          # [5.2253 0.1333 3.2092]
print(beta @ [1, 3, 7])                       # 28.0897

model = LinearRegression().fit(X, y)
print(model.intercept_, model.coef_, model.predict([[3, 7]]))
print(model.score(X, y))                      # R^2 = 0.7157
```

Diễn giải hệ số. w_j là mức thay đổi trung bình của $\hat{y}$ khi x_j tăng 1 đơn vị và các đặc trưng khác giữ nguyên. Nếu các đặc trưng đã được chuẩn hóa bằng `StandardScaler`, độ lớn $|w_j|$ có thể dùng để so sánh mức ảnh hưởng tương đối giữa các đặc trưng.

Các giả định kinh điển. (1) Quan hệ tuyến tính; (2) sai số độc lập; (3) phương sai sai số không đổi (homoscedasticity); (4) sai số có phân phối chuẩn (cần cho kiểm định thống kê); (5) không có đa cộng tuyến hoàn hảo. Biểu đồ phần dư – giá trị dự đoán là công cụ trực quan để kiểm tra các giả định này.

6.4 Gradient Descent

Với dữ liệu lớn hoặc mô hình không có nghiệm đóng (hồi quy logistic, mạng nơ-ron), ta dùng **Gradient Descent** — phương pháp do Augustin-Louis Cauchy đề xuất từ năm 1847. Ý tưởng:

đứng tại một điểm trên mặt hàm mất mát, đi một bước nhỏ theo hướng **ngược gradient** (hướng giảm nhanh nhất).

Với hàm mất mát $J(w_0, w_1) = \frac{1}{2n}\sum_i (\hat{y}_i - y_i)^2$ và tốc độ học η:

$$\frac{\partial J}{\partial w_0} = \frac{1}{n}\sum_i (\hat{y}_i - y_i), \qquad \frac{\partial J}{\partial w_1} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)\, x_i, \tag{6.6}$$

$$w_0 \leftarrow w_0 - \eta \frac{\partial J}{\partial w_0}, \qquad w_1 \leftarrow w_1 - \eta \frac{\partial J}{\partial w_1}. \tag{6.7}$$

Ví dụ 6.3 (Ba bước Gradient Descent)**.** *Với dữ liệu ở Bảng 6.1, khởi tạo* $w_0 = w_1 = 0$ *và* $\eta = 0{,}05$*:*

Bước	$\partial J/\partial w_0$	$\partial J/\partial w_1$	w_0 mới	w_1 mới
1	-6.0400	-22.1000	0.3020	1.1050
2	-2.4230	-9.0390	0.4232	1.5570
3	-0.9460	-3.7041	0.4704	1.7422
$\vdots$			$\to 0.07$	$\to 1.99$

Bảng 6.3: Gradient Descent hội tụ dần về nghiệm bình phương tối thiểu

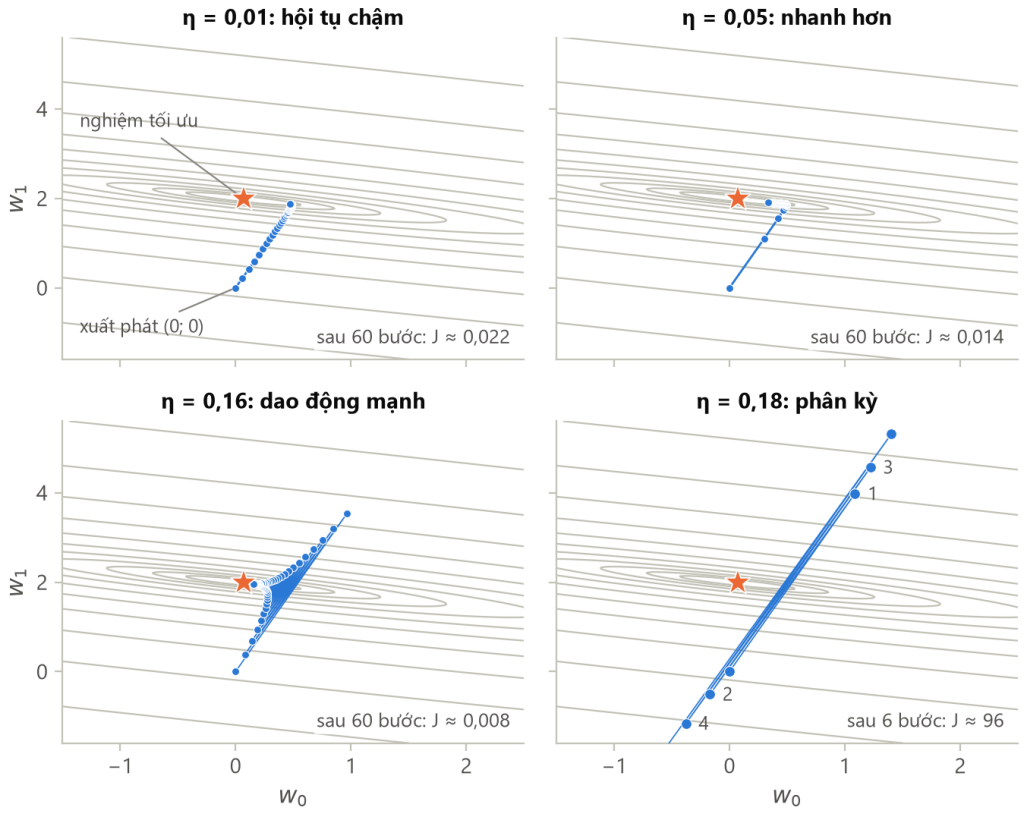
Listing 6.3: Cài đặt Gradient Descent cho hồi quy tuyến tính

```
import numpy as np

x = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([2.1, 3.9, 6.1, 8.2, 9.9])
w0, w1, lr = 0.0, 0.0, 0.05

for step in range(2000):
    error = (w0 + w1 * x) - y
    w0 -= lr * error.mean()
    w1 -= lr * (error * x).mean()
    if step < 3:
        print(step + 1, round(w0, 4), round(w1, 4))
print(round(w0, 4), round(w1, 4))     # ~0.07  ~1.99
```

Chọn tốc độ học. η quá nhỏ: hội tụ rất chậm; η quá lớn: dao động hoặc phân kỳ. Các đặc trưng có thang đo chênh lệch làm mặt hàm mất mát bị "kéo dài", buộc phải chọn η nhỏ — đây là lý do thực tiễn quan trọng nhất của việc chuẩn hóa dữ liệu ở Chương 5. Các biến thể phổ biến: *Stochastic GD* (mỗi bước dùng 1 mẫu, `SGDRegressor`) và *Mini-batch GD* (mỗi bước dùng một nhóm nhỏ mẫu, dùng nhiều trong học sâu).



Hình 6.3: Đường đồng mức của hàm mất mát $J(w_0, w_1)$ và quỹ đạo của Gradient Descent với các tốc độ học khác nhau; ngôi sao là nghiệm tối ưu. Với dữ liệu này, Gradient Descent chỉ hội tụ khi $\eta < 0{,}169$.

6.4.1 Ba biến thể: batch, stochastic và mini-batch

Công thức cập nhật ở trên không đổi; các biến thể chỉ khác nhau ở **số mẫu dùng để ước lượng gradient** trong mỗi bước:

- **Batch Gradient Descent**: mỗi bước dùng toàn bộ n mẫu. Hướng đi chính xác nhất, nhưng mỗi bước tốn kém và cần toàn bộ dữ liệu trong bộ nhớ.
- **Stochastic Gradient Descent** (SGD): mỗi bước dùng 1 mẫu. Mỗi bước rất rẻ nhưng hướng đi nhiễu; phù hợp với học trực tuyến.
- **Mini-batch Gradient Descent**: mỗi bước dùng một **lô** (batch) gồm B mẫu, tham số `batch_size`. Đây là cách dung hòa: gradient ít nhiễu hơn SGD, tận dụng được phép tính ma trận, và là cách huấn luyện mặc định của mạng nơ-ron (thường B từ 32 đến 256).

Một **epoch** là một lượt duyệt hết dữ liệu huấn luyện: với n mẫu và lô B mẫu, mỗi epoch gồm $\lceil n/B \rceil$ bước cập nhật. Batch Gradient Descent và SGD là hai trường hợp riêng của mini-batch, với $B = n$ và $B = 1$.

6.4.2 Ví dụ tính tay: mini-batch Gradient Descent với hai đặc trưng

Ví dụ 6.4 (Mini-batch Gradient Descent cho hồi quy tuyến tính)**.** *Huấn luyện mô hình* $\hat{y} = b + w_1 x_1 + w_2 x_2$ *trên 5 mẫu ở Bảng 6.4 với* `batch_size` $= 3$*: lô 1 gồm 3 mẫu đầu, lô 2 gồm 2 mẫu cuối. Khởi tạo* $b = 0{,}3$*;* $w_1 = 0{,}2$*;* $w_2 = 0{,}1$*; tốc độ học* $\alpha = 0{,}1$.

Lô	bias	x_1	x_2	y (hồi quy)	y (phân lớp)
1	1	1	3	4	0
1	1	2	1	3	0
1	1	1	1	1	0
2	1	7	8	9	1
2	1	6	4	7	1

Bảng 6.4: Dữ liệu ví dụ mini-batch Gradient Descent. Cột bias luôn bằng 1, ứng với hệ số chặn b; cột nhãn phân lớp được dùng ở Chương 7

Hàm mất mát là sai số bình phương trung bình trên một lô gồm n mẫu, và đạo hàm theo từng tham số là

$$L = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2, \tag{6.8}$$

$$\frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i), \qquad \frac{\partial L}{\partial w_j} = \frac{2}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)\,x_{ij}. \tag{6.9}$$

Hệ số 2 xuất hiện vì ở đây dùng $\frac{1}{n}$ thay cho $\frac{1}{2n}$ như mục trước. Hai cách viết chỉ khác nhau một hằng số, tương đương với việc nhân đôi tốc độ học.

Lô 1 – Bước 1: dự đoán. $\hat{y}_1 = 0{,}3\cdot 1 + 0{,}2\cdot 1 + 0{,}1\cdot 3 = 0{,}8$; tương tự $\hat{y}_2 = 0{,}8$ và $\hat{y}_3 = 0{,}6$. Sai lệch $\hat{y} - y = (-3{,}2;\ -2{,}2;\ -0{,}4)$.

Lô 1 – Bước 2: tính gradient ($n = 3$):

$$\begin{aligned}
\frac{\partial L}{\partial b} &= \tfrac{2}{3}\big[(-3{,}2) + (-2{,}2) + (-0{,}4)\big] = \tfrac{2}{3}(-5{,}8) \approx -3{,}8667,\\
\frac{\partial L}{\partial w_1} &= \tfrac{2}{3}\big[1\cdot(-3{,}2) + 2\cdot(-2{,}2) + 1\cdot(-0{,}4)\big] = \tfrac{2}{3}(-8) \approx -5{,}3333,\\
\frac{\partial L}{\partial w_2} &= \tfrac{2}{3}\big[3\cdot(-3{,}2) + 1\cdot(-2{,}2) + 1\cdot(-0{,}4)\big] = \tfrac{2}{3}(-12{,}2) \approx -8{,}1333.
\end{aligned}$$

Lô 1 – Bước 3: cập nhật theo quy tắc $\theta \leftarrow \theta - \alpha\,\partial L/\partial\theta$:

$$b = 0{,}3 + 0{,}3867 = 0{,}6867, \qquad w_1 = 0{,}2 + 0{,}5333 = 0{,}7333, \qquad w_2 = 0{,}1 + 0{,}8133 = 0{,}9133.$$

Mọi gradient đều âm nên mọi trọng số đều tăng — đúng hướng, vì mô hình đang dự đoán thấp hơn giá trị thật.

Lô 2 ($n = 2$, nên hệ số $\frac{2}{n} = 1$), dùng trọng số vừa cập nhật:

$$\begin{aligned}
\hat{y}_4 &= 0{,}6867 + 0{,}7333\cdot 7 + 0{,}9133\cdot 8 = 13{,}1267, & \hat{y}_4 - y_4 &= 4{,}1267,\\
\hat{y}_5 &= 0{,}6867 + 0{,}7333\cdot 6 + 0{,}9133\cdot 4 = 8{,}7400, & \hat{y}_5 - y_5 &= 1{,}7400,
\end{aligned}$$

$$\begin{aligned}
\frac{\partial L}{\partial b} &= 4{,}1267 + 1{,}74 = 5{,}8667,\\
\frac{\partial L}{\partial w_1} &= 7\cdot 4{,}1267 + 6\cdot 1{,}74 = 39{,}3267,\\
\frac{\partial L}{\partial w_2} &= 8\cdot 4{,}1267 + 4\cdot 1{,}74 = 39{,}9733.
\end{aligned}$$

Cập nhật: $b = 0{,}6867 - 0{,}5867 = 0{,}1000$; $w_1 = 0{,}7333 - 3{,}9327 = -3{,}1993$; $w_2 = 0{,}9133 - 3{,}9973 = -3{,}0840$. Hai lô đã đi qua toàn bộ dữ liệu, nên đây là kết thúc epoch thứ nhất.
Về làm tròn: nếu làm tròn trọng số sau lô 1 thành 0,686; 0,73; 0,913 như khi tính tay, lô 2 cho $b = 0{,}1042$; $w_1 = -3{,}1708$; $w_2 = -3{,}0542$. Chênh lệch này chỉ do làm tròn, không phải do phương pháp.

Dạng ma trận. Ghép các mẫu của một lô thành ma trận $\mathbf{X}_B$ (B dòng, cột đầu toàn số 1) và gom tham số thành $\mathbf{w} = (b, w_1, w_2)^\top$. Ba bước trên trở thành ba phép toán ma trận, đúng với mọi số đặc trưng:

$$\hat{\mathbf{y}} = \mathbf{X}_B\mathbf{w}, \qquad \nabla L = \frac{2}{B}\,\mathbf{X}_B^\top(\hat{\mathbf{y}} - \mathbf{y}_B), \qquad \mathbf{w} \leftarrow \mathbf{w} - \alpha\,\nabla L. \tag{6.10}$$

Với lô 1:

$$\begin{pmatrix} 1 & 1 & 3 \\ 1 & 2 & 1 \\ 1 & 1 & 1 \end{pmatrix} \begin{pmatrix} 0{,}3 \\ 0{,}2 \\ 0{,}1 \end{pmatrix} = \begin{pmatrix} 0{,}8 \\ 0{,}8 \\ 0{,}6 \end{pmatrix}, \qquad \frac{2}{3} \begin{pmatrix} 1 & 1 & 1 \\ 1 & 2 & 1 \\ 3 & 1 & 1 \end{pmatrix} \begin{pmatrix} -3{,}2 \\ -2{,}2 \\ -0{,}4 \end{pmatrix} = \begin{pmatrix} -3{,}8667 \\ -5{,}3333 \\ -8{,}1333 \end{pmatrix}.$$

Ma trận thứ hai là $\mathbf{X}_B^\top$: mỗi dòng của nó là một cột đặc trưng, nên một phép nhân tính được $\sum_i (\hat{y}_i - y_i)\, x_{ij}$ cho mọi j cùng lúc.

6.4.3 Đánh giá mô hình sau mỗi lần cập nhật

Để biết mỗi bước cập nhật có thực sự cải thiện mô hình hay không, ta đánh giá mô hình bằng MSE trên cả 5 mẫu sau mỗi lần cập nhật: trước khi học là 16,7; sau lô 1 giảm còn 4,373; nhưng sau lô 2 vọt lên 1016,3. Lô 2 có giá trị đặc trưng lớn (7 và 8) nên gradient lớn (khoảng 39), và với $\alpha = 0{,}1$ bước nhảy đi quá xa điểm cực tiểu: w_1 đổi từ $+0{,}73$ thành $-3{,}20$.

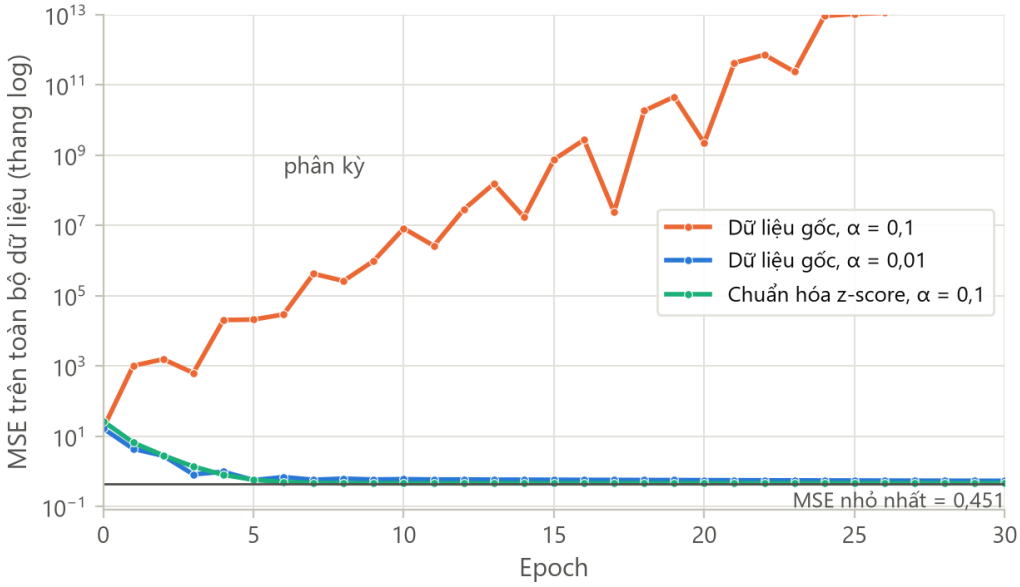
Với hàm mất mát bậc hai, một bước Gradient Descent chỉ làm mất mát giảm ổn định khi

$$\alpha < \frac{2}{\lambda_{\max}}, \tag{6.11}$$

trong đó $\lambda_{\max}$ là trị riêng lớn nhất của ma trận Hessian $\frac{2}{n}\mathbf{X}^\top\mathbf{X}$ — đại lượng đo độ "dốc" nhất của mặt mất mát (xem trị riêng ở Chương 10). Lô 1 có $\lambda_{\max} \approx 11{,}84$, nên cần $\alpha < 0{,}169$: tốc độ học 0,1 an toàn. Lô 2 có $\lambda_{\max} \approx 164{,}5$, nên cần $\alpha < 0{,}0122$: tốc độ học 0,1 lớn gấp khoảng 8 lần ngưỡng. Ngưỡng ổn định phụ thuộc vào thang đo của dữ liệu — đây là cơ sở toán học của lời khuyên chuẩn hóa đặc trưng.

Cấu hình	Epoch 0	Epoch 1	Epoch 2	Epoch 3	Epoch 50
Dữ liệu gốc, $\alpha = 0{,}1$	16,70	1016,33	1560,54	633,12	$\approx 1{,}7 \times 10^{24}$
Dữ liệu gốc, $\alpha = 0{,}01$	16,70	4,39	2,78	0,82	0,507
Chuẩn hóa z-score, $\alpha = 0{,}1$	26,89	6,59	2,85	1,38	0,461

Bảng 6.5: MSE trên toàn bộ 5 mẫu theo epoch. Nghiệm bình phương tối thiểu có MSE nhỏ nhất bằng 0,451; MSE ban đầu của dòng cuối khác vì cùng trọng số khởi tạo nhưng đặc trưng đã đổi thang đo



Hình 6.4: MSE theo epoch của ví dụ mini-batch (thang log): với dữ liệu gốc, $\alpha = 0{,}1$ làm thuật toán phân kỳ; giảm α còn 0,01 hoặc chuẩn hóa đặc trưng đều giúp hội tụ về gần mức nhỏ nhất.

Có hai cách khắc phục: **giảm tốc độ học**, hoặc **chuẩn hóa đặc trưng** (Chương 5) — sau khi chuẩn hóa, cùng $\alpha = 0{,}1$ lại hội tụ nhanh. Trong thực tế còn dùng lịch giảm dần tốc độ học, hoặc các bộ tối ưu thích nghi như Adam (Kingma và Ba, 2015). Ví dụ này cũng không xáo trộn dữ liệu giữa các epoch; ảnh hưởng của việc đó được phân tích ở ví dụ phân lớp (Chương 7).

Listing 6.4: Mini-batch Gradient Descent dạng ma trận cho hồi quy tuyến tính

```
import numpy as np

# Cot dau toan so 1 ung voi he so chan b
X = np.array([[1, 1, 3], [1, 2, 1], [1, 1, 1], [1, 7, 8], [1, 6, 4]], dtype=
    float)
y = np.array([4, 3, 1, 9, 7], dtype=float)

def minibatch_gd(X, y, w, lr=0.1, batch_size=3, epochs=1, verbose=False):
    n = len(y)
    for epoch in range(epochs):
        for start in range(0, n, batch_size):
            Xb = X[start:start + batch_size]
            yb = y[start:start + batch_size]
            y_hat = Xb @ w                                  # buoc 1: du doan
            grad = 2 / len(yb) * Xb.T @ (y_hat - yb)    # buoc 2: gradient cua
    MSE
            w = w - lr * grad                               # buoc 3: cap nhat
            if verbose:
                print("grad", grad.round(4), "-> w", w.round(4))
    return w

def mse(X, w):
    return ((X @ w - y) ** 2).mean()

w0 = np.array([0.3, 0.2, 0.1])                              # b, w1, w2
```

```
w = minibatch_gd(X, y, w0, verbose=True)
# grad [-3.8667 -5.3333 -8.1333] -> w [0.6867 0.7333 0.9133]
# grad [ 5.8667 39.3267 39.9733] -> w [ 0.1    -3.1993 -3.084 ]
print(round(mse(X, w), 2))                                  # 1016.33: phan ky

w = minibatch_gd(X, y, w0, lr=0.01, epochs=50)
print(round(mse(X, w), 3))                                  # 0.507: hoi tu

Xs = X.copy()                                               # chuan hoa z-score hai
    cot dac trung
Xs[:, 1:] = (X[:, 1:] - X[:, 1:].mean(axis=0)) / X[:, 1:].std(axis=0)
w = minibatch_gd(Xs, y, w0, lr=0.1, epochs=50)
print(round(mse(Xs, w), 3))                                 # 0.461: hoi tu
```

Trong đoạn mã trên, công thức gradient được viết tay. Ở Lab 8 (Mục 12.8.1, Chương 12), cùng ví dụ này được chạy lại bằng PyTorch: chỉ khai báo hàm mất mát MSE, máy tự tính gradient và cho đúng từng con số của lời giải tay ở trên.

6.5 Hồi quy đa thức, quá khớp và chính quy hóa

6.5.1 Hồi quy đa thức

Khi quan hệ không tuyến tính, ta tạo thêm đặc trưng $x^2, x^3, \ldots$ rồi vẫn dùng hồi quy tuyến tính (tuyến tính theo *tham số*):

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_p x^p. \tag{6.12}$$

Listing 6.5: Hồi quy đa thức với PolynomialFeatures

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression

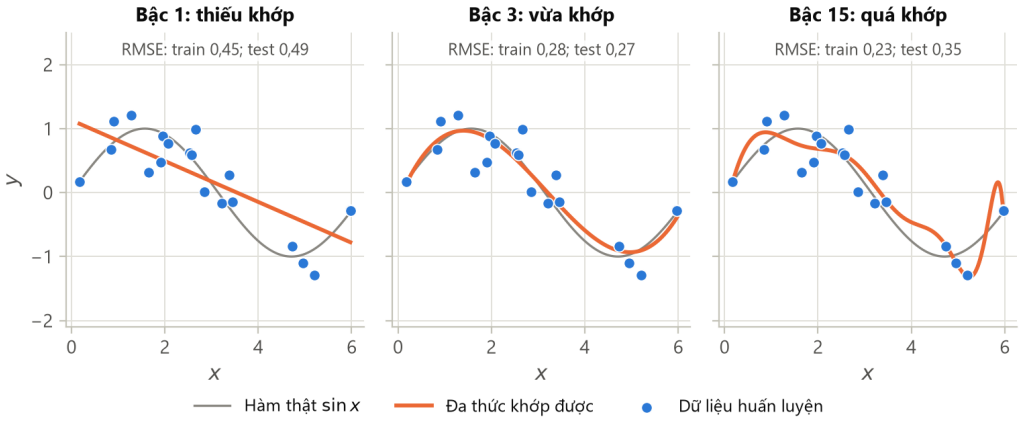
rng = np.random.default_rng(0)              # 40 diem mo phong y = sin(x) +
    nhieu
x = rng.uniform(0, 2 * np.pi, (40, 1))
y_sin = np.sin(x[:, 0]) + rng.normal(0, 0.3, 40)
x_tr, x_te, y_tr, y_te = train_test_split(x, y_sin, random_state=0)

for p in (1, 3, 15):                        # thieu khop, vua khop, qua khop
    m = make_pipeline(PolynomialFeatures(degree=p), StandardScaler(),
                      LinearRegression()).fit(x_tr, y_tr)
    print(p, round(m.score(x_tr, y_tr), 3), round(m.score(x_te, y_te), 3))
# 1 0.337 0.169  |  3 0.821 0.766  |  15 0.853 0.696   (R2 train, R2 test)

poly_model = make_pipeline(PolynomialFeatures(degree=3), StandardScaler(),
                           LinearRegression()).fit(x_tr, y_tr)
```

6.5.2 Quá khớp và thiếu khớp

Bậc p quá thấp, mô hình không nắm bắt được xu hướng (**thiếu khớp** – underfitting); bậc quá cao, mô hình "học thuộc" cả nhiễu, sai số huấn luyện rất nhỏ nhưng dự đoán dữ liệu mới rất kém (**quá khớp** – overfitting). Kết quả của đoạn mã trên cho thấy đúng hai hiện tượng này qua hệ số R^2. Với bậc 1, R^2 thấp trên cả hai tập. Từ bậc 3 lên bậc 15, R^2 trên tập huấn luyện tăng từ 0,821 lên 0,853, nhưng trên tập kiểm tra lại giảm từ 0,766 xuống 0,696. Cách phát hiện và lựa chọn độ phức tạp phù hợp được trình bày chi tiết ở Chương 8.



Hình 6.5: Hồi quy đa thức bậc 1, 3 và 15 trên cùng một tập dữ liệu: thiếu khớp, vừa khớp và quá khớp (20 điểm mô phỏng từ $y = \sin x$ cộng nhiễu; RMSE kiểm tra tính trên 2000 điểm mới).

6.5.3 Ridge và Lasso

Chính quy hóa (regularization) thêm vào hàm mất mát một khoản phạt độ lớn của hệ số:

$$\text{Ridge (Hoerl \& Kennard, 1970)}: \quad \min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \alpha\|\mathbf{w}\|_2^2, \tag{6.13}$$

$$\text{Lasso (Tibshirani, 1996)}: \quad \min_{\mathbf{w}} \tfrac{1}{2n}\|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 + \alpha\|\mathbf{w}\|_1. \tag{6.14}$$

Ridge co các hệ số về gần 0 và có nghiệm đóng $\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X} + \alpha\mathbf{I})^{-1}\mathbf{X}^\top\mathbf{y}$, luôn khả nghịch khi $\alpha > 0$ — khắc phục đa cộng tuyến. Lasso có thể đưa nhiều hệ số về **đúng bằng 0**, tức tự động **chọn đặc trưng**. ElasticNet (Zou & Hastie, 2005) kết hợp cả hai khoản phạt. Tham số α được chọn bằng kiểm định chéo (`RidgeCV`, `LassoCV`). Vì khoản phạt phụ thuộc độ lớn hệ số, **bắt buộc phải chuẩn hóa dữ liệu** trước khi dùng Ridge/Lasso.

6.6 Hồi quy bền vững và phân tích phần dư

Bình phương tối thiểu cực tiểu tổng **bình phương** phần dư, nên một điểm có phần dư bằng 10 đóng góp 100 đơn vị vào hàm mất mát — gấp trăm lần một điểm có phần dư bằng 1. Vì vậy chỉ vài ngoại lai cũng đủ "kéo lệch" toàn bộ đường hồi quy. Nói theo ngôn ngữ thống kê, **điểm sụp đổ** (breakdown point) của OLS bằng 0: chỉ một điểm dữ liệu tồi tệ tùy ý cũng có thể làm ước lượng sai lệch tùy ý (Raschka và cộng sự, 2022, Chương 9).

Ví dụ 6.5 (Ngoại lai kéo lệch đường hồi quy)**.** *Mười điểm được sinh từ quan hệ* $y \approx 2x + 1$ *với* $x = 0, 1, \ldots, 9$ *(có nhiễu nhỏ), cộng thêm ba điểm ngoại lai do lỗi nhập liệu* $(7{,}5;\ 2{,}0)$, $(8{,}5;\ 1{,}0)$, $(9{,}5;\ 3{,}0)$. *Dữ liệu của 13 điểm cho trong Bảng 6.6.*

x	0	1	2	3	4	5	6	7	8	9	7,5	8,5	9,5
y	1,06	2,93	5,32	7,05	8,73	11,18	13,65	15,47	16,65	18,37	**2,0**	**1,0**	**3,0**

Bảng 6.6: Dữ liệu ví dụ hồi quy bền vững; ba cột cuối là ngoại lai

Hồi quy OLS trên cả 13 điểm cho $\hat{y} = 4{,}205 + 0{,}734\,x$: hệ số góc bị kéo từ khoảng 2 xuống còn 0,73, tức ba điểm sai đã làm hỏng mô hình của mười điểm đúng.

6.6.1 RANSAC

RANSAC (RANdom SAmple Consensus) được Fischler và Bolles (1981) đề xuất cho thị giác máy tính, nơi dữ liệu đo đạc chứa rất nhiều điểm sai. Thay vì giảm trọng số ngoại lai, RANSAC **tìm và loại bỏ** chúng (Raschka và cộng sự, 2022, Chương 9):

1. Chọn ngẫu nhiên một tập con nhỏ nhất đủ để khớp mô hình (với đường thẳng là 2 điểm), coi là các điểm "trong" (inlier) giả định, và khớp mô hình.
2. Tính phần dư của mọi điểm còn lại; điểm nào có $|y_i - \hat{y}_i| \le t$ (ngưỡng do người dùng chọn) được thêm vào tập điểm trong — gọi là **tập đồng thuận**.
3. Khớp lại mô hình trên toàn bộ tập điểm trong.
4. Đánh giá sai số của mô hình trên tập điểm trong.
5. Lặp lại một số lần cố định hoặc đến khi đạt ngưỡng hiệu năng; giữ mô hình có tập đồng thuận tốt nhất.

Cần bao nhiêu lần lặp? Gọi ε là tỉ lệ ngoại lai, s là kích thước tập con tối thiểu và p là xác suất mong muốn có ít nhất một lần rút được tập con **toàn điểm trong**. Mỗi lần rút thành công với xác suất $(1-\varepsilon)^s$, nên số lần lặp cần thiết là

$$N = \frac{\ln(1-p)}{\ln\bigl(1-(1-\varepsilon)^s\bigr)}. \tag{6.15}$$

Với ví dụ trên, $\varepsilon = 3/13$, $s = 2$, $p = 0{,}99$: $(1-\varepsilon)^2 = (10/13)^2 \approx 0{,}592$, nên $N = \ln 0{,}01 / \ln 0{,}408 \approx 5{,}14$, tức chỉ cần 6 lần lặp. Mặc định của scikit-learn là 100 lần lặp (`max_trials`).

Với ngưỡng $t = 1{,}5$, RANSAC xác định đúng ba điểm cuối là ngoại lai và cho $\hat{y} = 1{,}185 + 1{,}968\,x$ — **trùng hoàn toàn** với OLS khi chỉ chạy trên mười điểm sạch.

6.6.2 Hàm mất mát Huber

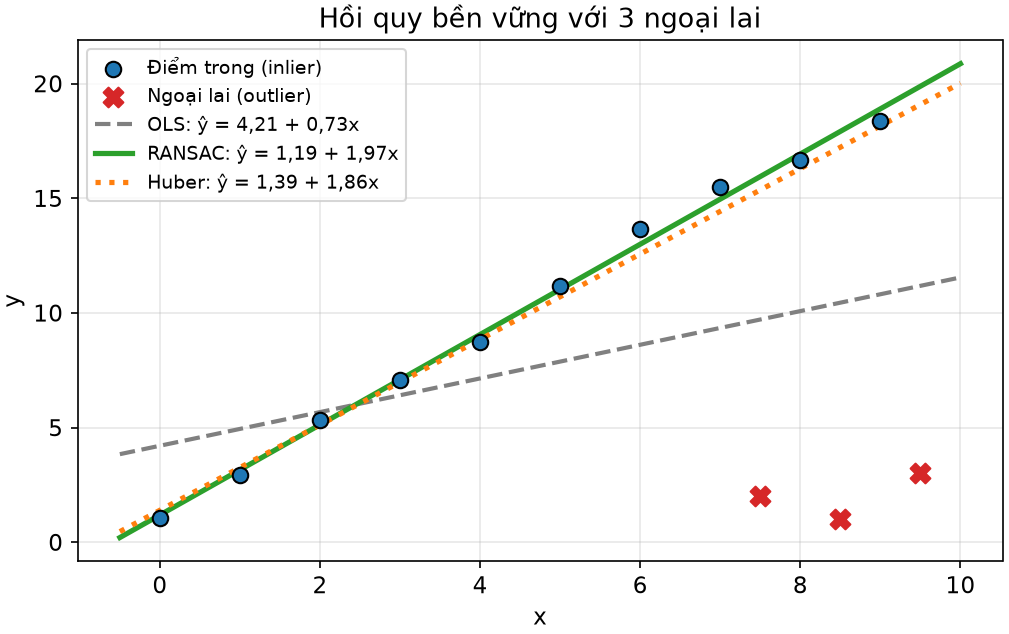
Huber (1964) đề xuất một hàm mất mát lai: bậc hai với phần dư nhỏ, **tuyến tính** với phần dư lớn:

$$L_\delta(r) = \begin{cases} \tfrac{1}{2} r^2 & |r| \le \delta, \\ \delta\bigl(|r| - \tfrac{1}{2}\delta\bigr) & |r| > \delta. \end{cases} \tag{6.16}$$

Vì phần tuyến tính tăng chậm hơn hẳn phần bậc hai, ảnh hưởng của mỗi ngoại lai bị **chặn**. Khác với RANSAC, Huber không loại điểm nào mà chỉ giảm ảnh hưởng của chúng. `HuberRegressor` (tham số `epsilon=1.35` mặc định) cho $\hat{y} = 1{,}392 + 1{,}863\,x$.

Phương pháp	Hệ số góc	Hệ số chặn
Quan hệ sinh dữ liệu	2	1
OLS trên 13 điểm (có ngoại lai)	0,734	4,205
OLS trên 10 điểm sạch (tham chiếu)	1,968	1,185
RANSAC ($t = 1{,}5$)	1,968	1,185
Huber ($\varepsilon = 1{,}35$)	1,863	1,392

Bảng 6.7: So sánh OLS và hai phương pháp hồi quy bền vững



Hình 6.6: Đường OLS bị ba ngoại lai kéo lệch, so với đường RANSAC (điểm trong và ngoại lai được tô màu khác nhau) và đường Huber.

Listing 6.6: OLS, RANSAC và Huber trên dữ liệu có ngoại lai

```
import numpy as np
from sklearn.linear_model import LinearRegression, RANSACRegressor,
    HuberRegressor

rng = np.random.default_rng(0)
x = np.arange(10, dtype=float)
y = 2 * x + 1 + rng.normal(0, 0.5, 10)
x = np.r_[x, 7.5, 8.5, 9.5]                    # them 3 ngoai lai
y = np.r_[y, 2.0, 1.0, 3.0]
X = x.reshape(-1, 1)

ols = LinearRegression().fit(X, y)
ransac = RANSACRegressor(LinearRegression(), residual_threshold=1.5,
                         random_state=0).fit(X, y)
huber = HuberRegressor().fit(X, y)
print(ols.coef_, ols.intercept_)                                  # [0.734]
    4.205
```

```
print(ransac.estimator_.coef_, ransac.estimator_.intercept_)      # [1.968]
    1.185
print(np.where(~ransac.inlier_mask_)[0])                          # [10 11 12]
print(huber.coef_, huber.intercept_)                              # [1.863]
    1.392
```

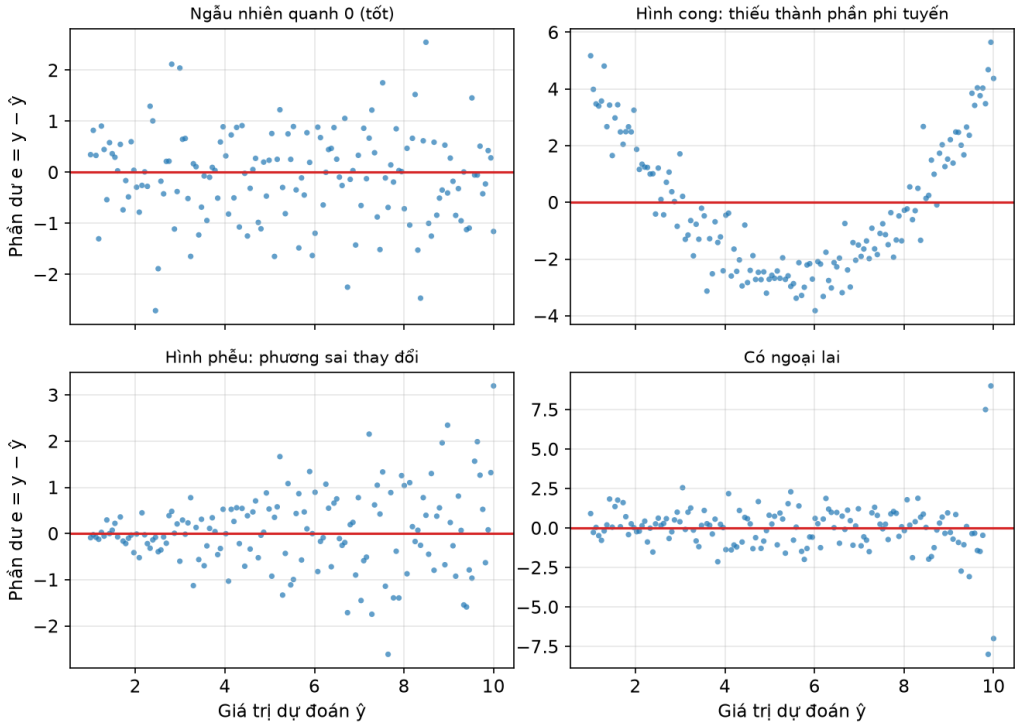
Lưu ý thực hành. Ngưỡng t phụ thuộc thang đo của y; nếu không chỉ định, scikit-learn dùng độ lệch tuyệt đối trung vị (MAD) của y. RANSAC có tính ngẫu nhiên nên cần cố định `random_state`. Cả hai phương pháp đều không thay thế được việc **tìm hiểu nguyên nhân** của ngoại lai (Chương 5): ngoại lai do lỗi nhập liệu thì nên sửa hoặc loại, còn ngoại lai là hiện tượng thật thì có thể chính là điều cần nghiên cứu.

6.6.3 Phân tích phần dư

Với nhiều đặc trưng, không thể vẽ đường hồi quy trong không gian nhiều chiều, nhưng luôn vẽ được **biểu đồ phần dư**: trục hoành là giá trị dự đoán $\hat{y}_i$, trục tung là phần dư $e_i = y_i - \hat{y}_i$ (Raschka và cộng sự, 2022, Chương 9). Với một mô hình tốt, các điểm phân tán **ngẫu nhiên** quanh đường $e = 0$ với độ rộng không đổi. Mọi **khuôn mẫu** trong biểu đồ đều là thông tin mà mô hình chưa nắm bắt được:

Khuôn mẫu	**Chẩn đoán**	**Cách xử lý**
Hình cong (chữ U)	Quan hệ phi tuyến chưa được mô hình hóa	Thêm đặc trưng đa thức, biến đổi đặc trưng, dùng mô hình phi tuyến
Hình phễu (độ rộng tăng theo $\hat{y}$)	Phương sai sai số thay đổi (heteroscedasticity)	Biến đổi logarit biến mục tiêu, như với giá nhà Ames
Vài điểm nằm xa đường 0	Ngoại lai, điểm có ảnh hưởng lớn	Kiểm tra dữ liệu, dùng hồi quy bền vững
Khuôn mẫu theo thứ tự thời gian	Tự tương quan của sai số	Thêm đặc trưng trễ, dùng mô hình chuỗi thời gian

Bảng 6.8: Đọc biểu đồ phần dư



Hình 6.7: Bốn dạng biểu đồ phần dư điển hình: ngẫu nhiên (tốt), hình cong, hình phễu và có ngoại lai.

Listing 6.7: Vẽ biểu đồ phần dư bằng scikit-learn

```
from sklearn.metrics import PredictionErrorDisplay

y_pred = poly_model.predict(x_te)       # mo hinh bac 3 o muc Hoi quy da thuc
PredictionErrorDisplay.from_predictions(y_te, y_pred, kind="
    residual_vs_predicted")
PredictionErrorDisplay.from_predictions(y_te, y_pred, kind="actual_vs_predicted
    ")
```

6.7 Hồi quy cho chuỗi thời gian

Định nghĩa 6.2 (Chuỗi thời gian)**.** *Chuỗi thời gian là dãy quan sát $y_1, y_2, \ldots, y_T$ được ghi nhận theo thứ tự thời gian, thường cách đều (ngày, tháng, quý). Bài toán dự báo là ước lượng $y_{T+1}, y_{T+2}, \ldots$ từ các giá trị quá khứ.*

Các mô hình thống kê kinh điển như ARIMA được Box và Jenkins hệ thống hóa năm 1970. Tuy nhiên, có thể giải bài toán dự báo bằng chính hồi quy tuyến tính thông qua **đặc trưng trễ** (lag features): dùng $y_{t-1}, y_{t-2}, \ldots, y_{t-p}$ làm đầu vào để dự đoán y_t — tương đương mô hình tự hồi quy AR(p). Có thể bổ sung đặc trưng lịch (tháng, thứ trong tuần), trung bình trượt và xu hướng theo thời gian.

Ba nguyên tắc riêng của chuỗi thời gian.

1. **Không xáo trộn dữ liệu** khi chia train/test: tập kiểm tra phải nằm *sau* tập huấn luyện về thời gian.

2. Đặc trưng chỉ được tính từ **quá khứ**; dùng giá trị tương lai là rò rỉ dữ liệu.
3. Kiểm định chéo dùng `TimeSeriesSplit`: các fold huấn luyện luôn đứng trước fold kiểm tra.

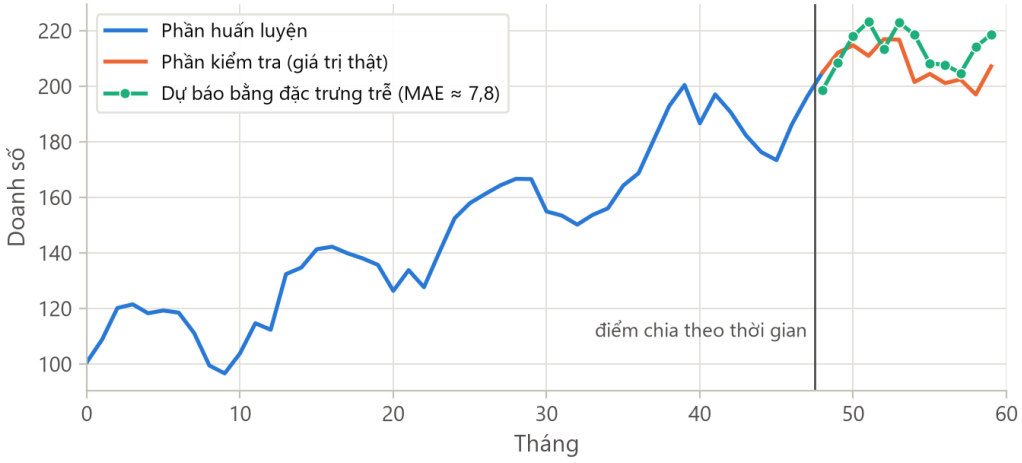
Listing 6.8: Dự báo chuỗi thời gian bằng đặc trưng trễ

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error

# Du lieu mo phong: doanh so thang co xu huong + mua vu + nhieu
rng = np.random.default_rng(0)
t = np.arange(60)
sales = 100 + 2 * t + 15 * np.sin(2 * np.pi * t / 12) + rng.normal(0, 5, 60)
df = pd.DataFrame({"sales": sales})

# Tao dac trung tre va dac trung lich
for lag in [1, 2, 12]:
    df[f"lag_{lag}"] = df["sales"].shift(lag)
df["month"] = t % 12
df["trend"] = t
df = df.dropna()

# Chia theo thoi gian: 12 thang cuoi lam tap kiem tra
train, test = df.iloc[:-12], df.iloc[-12:]
features = ["lag_1", "lag_2", "lag_12", "trend"]
model = LinearRegression().fit(train[features], train["sales"])
pred = model.predict(test[features])
print("MAE:", mean_absolute_error(test["sales"], pred))
```



Hình 6.8: Chuỗi doanh số theo tháng, phần huấn luyện, phần kiểm tra và đường dự báo bằng đặc trưng trễ (kết quả chạy đoạn code trên; dữ liệu mô phỏng).

6.8 Nghiên cứu tình huống: dự đoán giá nhà Ames

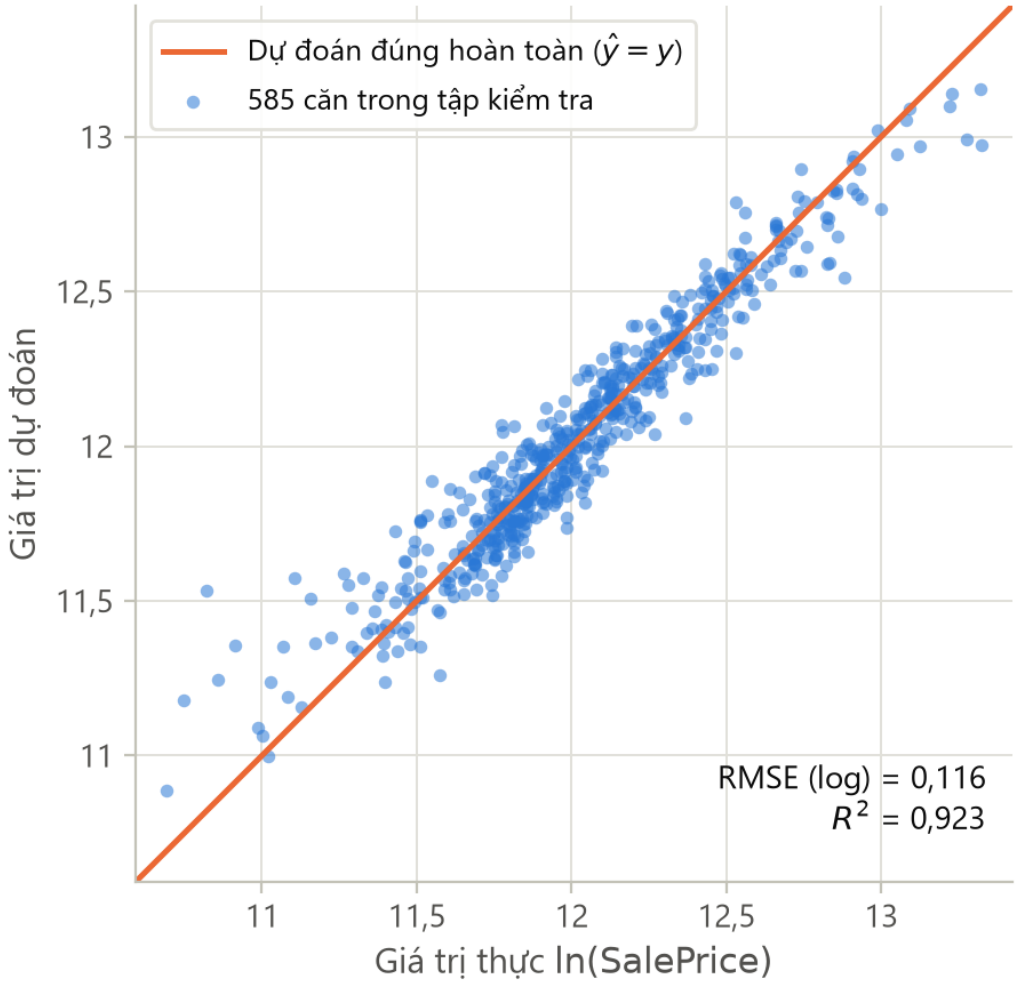
Tiếp nối Chương 5, ta dùng bộ tiền xử lý `preprocess` (ColumnTransformer) đã xây dựng và gắn thêm mô hình hồi quy. Biến mục tiêu được lấy logarit, $y = \ln(\text{SalePrice})$, vì giá nhà lệch phải: sai số trên thang log tương ứng với sai số *tương đối* (phần trăm), công bằng giữa nhà rẻ và nhà đắt.

Listing 6.9: So sánh Linear Regression, Ridge và Lasso trên Ames Housing

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression, RidgeCV, LassoCV
from sklearn.model_selection import cross_val_score
from sklearn.metrics import mean_squared_error

# Can chay truoc ma nguon o muc Nghien cuu tinh huong Ames (Chuong 5):
# muc do tao X_train, X_test, y_train, y_test va bo tien xu ly preprocess
models = {
    "Linear": LinearRegression(),
    "Ridge":  RidgeCV(alphas=np.logspace(-3, 3, 20)),
    "Lasso":  LassoCV(alphas=np.logspace(-4, 0, 20), max_iter=10000),
}
for name, reg in models.items():
    pipe = Pipeline([("prep", preprocess), ("reg", reg)])
    cv = cross_val_score(pipe, X_train, y_train, cv=5,
                         scoring="neg_root_mean_squared_error")
    pipe.fit(X_train, y_train)
    rmse = mean_squared_error(y_test, pipe.predict(X_test)) ** 0.5
    print(f"{name:7s} CV-RMSE={-cv.mean():.4f}  Test-RMSE={rmse:.4f}")

# RMSE tren thang log ~ sai so tuong doi, vd 0.12 ~ khoang 12%
# Doi du doan ve don vi USD:
price_pred = np.exp(pipe.predict(X_test[:5]))
```



Hình 6.9: Biểu đồ giá trị thực – giá trị dự đoán (thang log) của mô hình Ridge trên tập kiểm tra Ames Housing.

6.9 Tóm tắt chương

- Phương pháp bình phương tối thiểu (Legendre 1805, Gauss 1809) là nền tảng của hồi quy; thuật ngữ "hồi quy" bắt nguồn từ nghiên cứu của Galton (1886).
- Hồi quy tuyến tính có nghiệm đóng $w_1 = S_{xy}/S_{xx}$, $w_0 = \bar{y} - w_1\bar{x}$; dạng đa biến $\mathbf{w}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$.
- Gradient Descent cập nhật tham số ngược hướng gradient; tốc độ học và chuẩn hóa dữ liệu quyết định tốc độ hội tụ.
- Mini-batch Gradient Descent cập nhật sau mỗi lô B mẫu: $\hat{\mathbf{y}} = \mathbf{X}_B\mathbf{w}$, $\nabla L = \frac{2}{B}\mathbf{X}_B^\top(\hat{\mathbf{y}} - \mathbf{y}_B)$. Mỗi bước chỉ ổn định khi $\alpha < 2/\lambda_{\max}$, nên đặc trưng có thang đo lớn dễ gây phân kỳ; khắc phục bằng cách giảm α hoặc chuẩn hóa đặc trưng.
- Hồi quy đa thức tăng độ linh hoạt nhưng dễ quá khớp; Ridge và Lasso kiểm soát độ phức tạp, Lasso còn chọn đặc trưng.

– OLS có điểm sụp đổ bằng 0. RANSAC tìm và loại ngoại lai bằng tập đồng thuận, cần $N = \ln(1-p)/\ln(1-(1-\varepsilon)^s)$ lần lặp; mất mát Huber chặn ảnh hưởng của phần dư lớn. Biểu đồ phần dư chẩn đoán quan hệ phi tuyến, phương sai thay đổi, ngoại lai và tự tương quan.

– Với chuỗi thời gian: dùng đặc trưng trễ, chia dữ liệu theo thời gian, không xáo trộn.

6.10 Bài tập

1. Cho $x = (1, 2, 3, 4)$, $y = (3, 5, 7, 10)$. Tính tay w_0, w_1, SSE và R^2; kiểm tra bằng `LinearRegression`.

2. Chứng minh công thức (6.3) bằng cách lấy đạo hàm SSE theo w_0 và w_1.

3. Chạy Gradient Descent cho bài 1 với $\eta \in \{0{,}01;\ 0{,}05;\ 0{,}2\}$. Vẽ giá trị hàm mất mát theo số bước và nhận xét.

4. Với ví dụ mini-batch ở Bảng 6.4, tính tay epoch đầu tiên khi $\alpha = 0{,}01$. MSE trên toàn bộ dữ liệu sau epoch là bao nhiêu? (Đáp số: 4,39.) Lặp lại với `batch_size` $= 1$ (SGD) và `batch_size` $= 5$ (batch); so sánh số bước cập nhật trong một epoch.

5. Sinh dữ liệu $y = \sin(x) +$ nhiễu với 20 điểm. Khớp đa thức bậc 1, 3, 9, 15 và so sánh sai số trên tập huấn luyện và tập kiểm tra.

6. Trên Ames Housing, dùng `LassoCV` và liệt kê 10 đặc trưng có hệ số lớn nhất (theo trị tuyệt đối). Bao nhiêu hệ số bị đưa về 0?

7. Với ví dụ RANSAC (Bảng 6.6), chạy lại với ngưỡng $t \in \{0{,}5;\ 1{,}5;\ 5;\ 15\}$. Ngưỡng nào quá chặt (loại cả điểm đúng), ngưỡng nào quá lỏng (giữ cả ngoại lai)? Tính số lần lặp N cần thiết khi tỉ lệ ngoại lai là 50% và $p = 0{,}999$.

8. Theo Raschka và cộng sự (2022, Chương 9), khớp `SalePrice` theo `Gr Liv Area` của Ames Housing bằng OLS và RANSAC; vẽ các điểm trong, ngoại lai và hai đường hồi quy. Sau đó vẽ biểu đồ phần dư của mô hình đa biến ở mục nghiên cứu tình huống và chẩn đoán theo Bảng 6.8.

9. Vì sao không được dùng `train_test_split(shuffle=True)` cho chuỗi thời gian? Viết lại ví dụ dự báo doanh số với `TimeSeriesSplit`.

Tài liệu tham khảo của chương

1. Box, G. E. P., & Jenkins, G. M. (1970). *Time Series Analysis: Forecasting and Control*. Holden-Day.

2. Cauchy, A.-L. (1847). Méthode générale pour la résolution des systèmes d'équations simultanées. *Comptes Rendus de l'Académie des Sciences*, 25, 536–538.

3. Fischler, M. A., & Bolles, R. C. (1981). Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography. *Communications of the ACM*, 24(6), 381–395.

4. Galton, F. (1886). Regression towards mediocrity in hereditary stature. *Journal of the Anthropological Institute*, 15, 246–263.

5. Gauss, C. F. (1809). *Theoria Motus Corporum Coelestium*. Hamburg: Perthes & Besser.

6. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.), Chương 3. Springer.

7. Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55–67.

8. Huber, P. J. (1964). Robust Estimation of a Location Parameter. *The Annals of Mathematical Statistics*, 35(1), 73–101.

9. Legendre, A.-M. (1805). *Nouvelles méthodes pour la détermination des orbites des comètes*. Paris: Firmin Didot.

10. Liu, Y. (H.) (2019). *Python Machine Learning By Example* (2nd ed.), Chương 9: Stock Price Prediction with Regression Algorithms. Packt Publishing.

11. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*, Chương 9: Predicting Continuous Target Variables with Regression Analysis. Packt Publishing.

12. Tibshirani, R. (1996). Regression Shrinkage and Selection via the Lasso. *Journal of the Royal Statistical Society, Series B*, 58(1), 267–288.

13. Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society, Series B*, 67(2), 301–320.

Chương 7: Phân lớp (Classification)

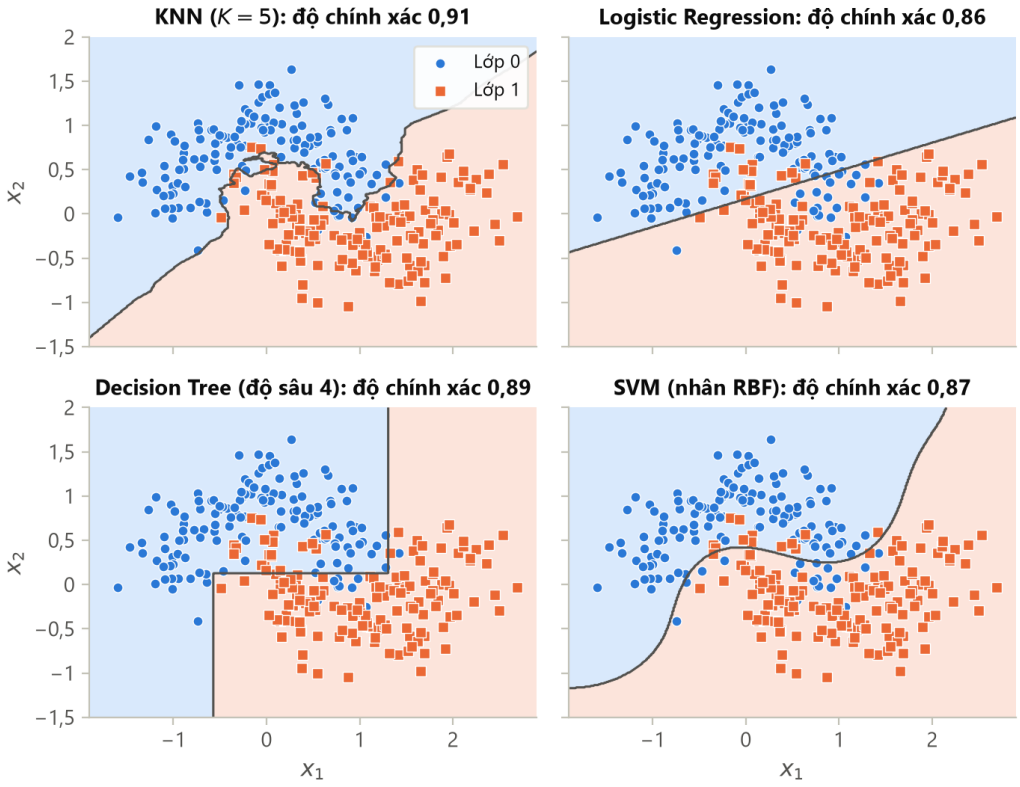
Phân lớp (classification) là bài toán học có giám sát trong đó đầu ra là một **nhãn rời rạc**: thư rác hay không, khối u lành hay ác tính, khách hàng có rời bỏ dịch vụ hay không, chữ số viết tay là 0 hay 9… Nếu chỉ có hai nhãn ta có **phân lớp nhị phân**; nhiều hơn hai nhãn là **phân lớp đa lớp**. Theo kế hoạch giảng dạy, chương này trình bày các thuật toán nền tảng: **KNN, Naive Bayes, Logistic Regression, Decision Tree, SVM**; các phương pháp kết hợp (Random Forest, AdaBoost, XGBoost) được trình bày ở chương kế tiếp.
Sau khi học xong chương này, người học có khả năng:

- Trình bày ý tưởng, lịch sử và giả định của năm họ thuật toán phân lớp kinh điển.
- Tự tính tay một bước dự đoán của KNN (bỏ phiếu đều, có trọng số, và KNN cho hồi quy), Naive Bayes, Logistic Regression và một lần chia nút của cây quyết định.
- Suy ra hàm mất mát logistic từ nguyên lý hợp lý cực đại, tính tay một bước Gradient Descent và giải thích vai trò của tham số chính quy hóa C.
- Cài đặt và so sánh các thuật toán bằng scikit-learn trên dữ liệu thực.
- Lựa chọn thuật toán và bước tiền xử lý phù hợp cho từng loại dữ liệu.

7.1 Bài toán phân lớp

Định nghĩa 7.1 (Bài toán phân lớp)**.** *Cho tập huấn luyện* $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ *với* $\mathbf{x}_i \in \mathbb{R}^d$ *và* $y_i \in \{1, \ldots, K\}$*. Cần học hàm* $f : \mathbb{R}^d \to \{1, \ldots, K\}$ *sao cho* $f(\mathbf{x})$ *dự đoán đúng nhãn của các điểm* ***chưa từng thấy****. Nhiều thuật toán còn ước lượng xác suất* $P(y = k \mid \mathbf{x})$ *và chọn nhãn có xác suất lớn nhất.*

Về mặt hình học, bộ phân lớp chia không gian đặc trưng thành các vùng; ranh giới giữa các vùng gọi là **biên quyết định** (decision boundary). Các thuật toán khác nhau tạo ra biên có hình dạng khác nhau: tuyến tính (Logistic Regression, SVM tuyến tính), các đoạn song song trục (Decision Tree), hay biên phức tạp tùy dữ liệu (KNN, SVM với nhân).



Hình 7.1: Biên quyết định của KNN, Logistic Regression, Decision Tree và SVM (nhân RBF) trên cùng một tập dữ liệu hai chiều (`make_moons`, 300 mẫu; độ chính xác tính trên 30% dữ liệu giữ lại để kiểm tra).

Xuyên suốt chương, ta minh họa bằng bộ dữ liệu **Iris** do Ronald Fisher công bố năm 1936: 150 bông hoa thuộc 3 loài (*setosa*, *versicolor*, *virginica*), mỗi bông có 4 đặc trưng: chiều dài, chiều rộng của đài hoa và cánh hoa (cm).

Listing 7.1: Chuẩn bị dữ liệu Iris

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X, y = iris.data, iris.target
print(X.shape, iris.target_names)   # (150, 4) ['setosa' 'versicolor' '
    virginica']

# stratify=y: giu nguyen ti le cac lop trong tap train va test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

7.2 K láng giềng gần nhất (KNN)

KNN (K-Nearest Neighbors) được Evelyn Fix và Joseph Hodges đề xuất năm 1951 trong một báo cáo cho Không quân Hoa Kỳ; năm 1967, Cover và Hart chứng minh rằng khi số mẫu tiến ra vô cùng, sai số của luật 1-NN không vượt quá **hai lần** sai số Bayes tối ưu. KNN là thuật toán

lười (lazy learning): không học tham số nào, chỉ ghi nhớ dữ liệu và trì hoãn mọi tính toán đến lúc dự đoán.

Định nghĩa 7.2 (Thuật toán KNN)**.** *Để phân lớp điểm mới* $\mathbf{q}$*:*

1. *Tính khoảng cách từ* $\mathbf{q}$ *đến mọi điểm huấn luyện, thường dùng khoảng cách Euclid* $d(\mathbf{q}, \mathbf{x}) = \sqrt{\sum_j (q_j - x_j)^2}$.
2. *Chọn* K *điểm gần nhất.*
3. *Gán nhãn chiếm đa số trong* K *điểm đó (có thể bỏ phiếu có trọng số* $1/d$*).*

Ví dụ 7.1 (Tính tay KNN phân lớp)**.** *Cho 5 mẫu huấn luyện có hai đặc trưng* x_1, x_2 *và nhãn* $y \in \{0, 1\}$ *(Bảng 7.1). Dự đoán nhãn của mẫu mới* $\mathbf{q} = (2, 4)$ *với khoảng cách Euclid* $d = \sqrt{(x_1 - 2)^2 + (x_2 - 4)^2}$.

STT	x_1	x_2	y	Khoảng cách đến $\mathbf{q} = (2,4)$	Thứ hạng
1	7	6	0	$\sqrt{5^2 + 2^2} = \sqrt{29} \approx 5{,}385$	4
2	2	4	1	$\sqrt{0^2 + 0^2} = 0$	1
3	1	5	1	$\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1{,}414$	2
4	3	9	1	$\sqrt{1^2 + 5^2} = \sqrt{26} \approx 5{,}099$	3
5	8	7	0	$\sqrt{6^2 + 3^2} = \sqrt{45} \approx 6{,}708$	5

Bảng 7.1: Dữ liệu huấn luyện và khoảng cách đến điểm cần dự đoán

Với $K = 3$: ba mẫu gần nhất là 2, 3, 4, đều có nhãn $1 \Rightarrow \hat{y} = 1$. Với $K = 4$: thêm mẫu 1 (nhãn 0), kết quả bỏ phiếu $3 : 1$, vẫn dự đoán $\hat{y} = 1$.

7.2.1 KNN có trọng số

Bỏ phiếu đều coi láng giềng gần nhất và láng giềng thứ K như nhau, dù một điểm có thể rất gần còn điểm kia khá xa. Dudani (1976) đề xuất **bỏ phiếu có trọng số**: láng giềng càng gần, lá phiếu càng nặng. Trọng số phổ biến nhất là nghịch đảo khoảng cách:

$$w_i = \frac{1}{d(\mathbf{q}, \mathbf{x}_i) + \varepsilon}, \qquad \hat{y} = \arg\max_{c} \sum_{i \in N_K(\mathbf{q}),\ y_i = c} w_i, \tag{7.1}$$

trong đó ε là một số rất nhỏ (ví dụ 10^{-5}) để tránh chia cho 0 khi $\mathbf{q}$ trùng với một điểm huấn luyện.

Ví dụ 7.2 (Trọng số khi điểm cần dự đoán trùng một mẫu)**.** *Tiếp ví dụ trên với* $K = 4$ *và* $\varepsilon = 10^{-5}$*:*

$$w_2 = \frac{1}{0 + 10^{-5}} = 100\,000, \qquad w_3 = \frac{1}{1{,}414} \approx 0{,}707,$$
$$w_4 = \frac{1}{5{,}099} \approx 0{,}196, \qquad w_1 = \frac{1}{5{,}385} \approx 0{,}186.$$

Tổng trọng số của lớp 1 là $100\,000 + 0{,}707 + 0{,}196 \approx 100\,000{,}9$*, của lớp 0 là* $0{,}186$*. Chuẩn hóa về tỉ lệ: lớp 0 chiếm* $\frac{0{,}186}{100\,001{,}09} \approx 1{,}86 \times 10^{-6} \approx 0$*, lớp 1 chiếm* ≈ 1*. Mẫu trùng khớp gần như quyết định toàn bộ kết quả. Chú ý* $\frac{1}{0 + 10^{-5}} = 10^5$ *chứ không phải 1 — một nhầm lẫn hay gặp khi tính tay. Trong scikit-learn, nếu có láng giềng cách đúng 0 thì* `weights="distance"` *chỉ tính các láng giềng đó.*

Ví dụ 7.3 (Trọng số phá thế hòa phiếu)**.** *Cũng với dữ liệu ở Bảng 7.1, xét* $\mathbf{q} = (5, 6)$ *và* $K = 4$*. Khoảng cách đến các mẫu 1, 5, 2, 4 lần lượt là* 2*;* $\sqrt{10} \approx 3{,}162$*;* $\sqrt{13} \approx 3{,}606$*;* $\sqrt{13} \approx 3{,}606$ *(mẫu 3 xa nhất,* $\sqrt{17} \approx 4{,}123$*). Bỏ phiếu đều cho kết quả* ***hòa*** $2 : 2$*: mẫu 1, 5 nhãn 0; mẫu 2, 4 nhãn 1. Với trọng số* $1/d$*:*

$$W_0 = \frac{1}{2} + \frac{1}{3{,}162} \approx 0{,}816, \qquad W_1 = \frac{1}{3{,}606} + \frac{1}{3{,}606} \approx 0{,}555 \;\Rightarrow\; \hat{y} = 0.$$

Khi hòa phiếu, scikit-learn với `weights="uniform"` *chọn lớp có nhãn nhỏ hơn — một quy ước tùy ý; trọng số khoảng cách cho lời giải có cơ sở hơn. Đây là lý do khi* K *chẵn (dễ hòa phiếu) nên dùng KNN có trọng số.*

7.2.2 KNN cho bài toán hồi quy

KNN dùng được cho cả hồi quy: thay vì bỏ phiếu, ta lấy **trung bình** (hoặc trung bình có trọng số) **giá trị mục tiêu** của K láng giềng:

$$\hat{y} = \frac{1}{K} \sum_{i \in N_K(\mathbf{q})} y_i \qquad \text{hoặc} \qquad \hat{y} = \frac{\sum_{i \in N_K(\mathbf{q})} w_i\, y_i}{\sum_{i \in N_K(\mathbf{q})} w_i}, \quad w_i = \frac{1}{d(\mathbf{q}, \mathbf{x}_i)}. \tag{7.2}$$

Ví dụ 7.4 (Tính tay KNN hồi quy với $K = 4$)**.** *Cùng 5 mẫu như Bảng 7.1 nhưng* y *là số thực. Dự đoán cho* $\mathbf{q} = (3, 4)$*.*

STT	x_1	x_2	y	$d(\mathbf{q}, \mathbf{x}_i)$	$w_i = 1/d$
1	7	6	7,3	$\sqrt{4^2 + 2^2} \approx 4{,}472$	0,224
2	2	4	5,4	$\sqrt{1^2 + 0^2} = 1$	1
3	1	5	4,2	$\sqrt{2^2 + 1^2} \approx 2{,}236$	0,447
4	3	9	5,1	$\sqrt{0^2 + 5^2} = 5$	0,2
5	8	7	9,6	$\sqrt{5^2 + 3^2} \approx 5{,}831$	(không thuộc $K = 4$)

Bảng 7.2: KNN hồi quy: khoảng cách và trọng số đến $\mathbf{q} = (3, 4)$

Bốn láng giềng gần nhất là mẫu 2, 3, 1, 4.

- Trung bình đều: $\hat{y} = \dfrac{5{,}4 + 4{,}2 + 7{,}3 + 5{,}1}{4} = \dfrac{22}{4} = 5{,}5.$
- Trung bình có trọng số: $\hat{y} = \dfrac{5{,}4 \times 1 + 4{,}2 \times 0{,}447 + 7{,}3 \times 0{,}224 + 5{,}1 \times 0{,}2}{1 + 0{,}447 + 0{,}224 + 0{,}2} \approx 5{,}31.$

Lưu ý: phép trung bình lấy trên **giá trị mục tiêu** y_i của các láng giềng, không phải trên khoảng cách. Đây cũng chính là cơ chế của KNN Imputer ở mục 5.4: cột cần điền đóng vai trò y.

Lưu ý thực hành.

- KNN dựa hoàn toàn vào khoảng cách nên **bắt buộc chuẩn hóa dữ liệu** và **mã hóa One Hot** cho biến định danh (Chương 5).
- K nhỏ: biên quyết định gồ ghề, nhạy với nhiễu; K lớn: biên mượt nhưng có thể thiếu khớp. Với bài toán hai lớp nên chọn K lẻ để tránh hòa phiếu.

– Chi phí dự đoán lớn với dữ liệu nhiều mẫu; chất lượng giảm mạnh khi số chiều cao (*lời nguyền số chiều*, xem chương Giảm chiều dữ liệu).

Listing 7.2: KNN với scikit-learn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier

knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
knn.fit(X_train, y_train)
print("Accuracy:", knn.score(X_test, y_test))

# Cac vi du tinh tay o tren
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

X5 = np.array([[7, 6], [2, 4], [1, 5], [3, 9], [8, 7]])
y5 = np.array([0, 1, 1, 1, 0])
print(KNeighborsClassifier(3).fit(X5, y5).predict([[2, 4]]))          # [1]
knn_w = KNeighborsClassifier(4, weights="distance").fit(X5, y5)
print(knn_w.predict_proba([[5, 6]]).round(4))                         #
    [[0.5954 0.4046]]

y_reg = np.array([7.3, 5.4, 4.2, 5.1, 9.6])
for w in ["uniform", "distance"]:
    reg = KNeighborsRegressor(4, weights=w).fit(X5, y_reg)
    print(w, reg.predict([[3, 4]]).round(4))                          # 5.5 ;
    5.3082
```

7.3 Naive Bayes

Định lý Bayes do mục sư Thomas Bayes đề xuất và được công bố sau khi ông mất (1763), sau đó Pierre-Simon Laplace phát triển độc lập và tổng quát hóa. Năm 1961, Maron dùng mô hình Bayes "ngây thơ" để phân loại tài liệu tự động; đến cuối thập niên 1990, Naive Bayes trở thành phương pháp lọc thư rác kinh điển.

Định nghĩa 7.3 (Bộ phân lớp Naive Bayes)**.** *Theo định lý Bayes:*

$$P(y \mid \mathbf{x}) = \frac{P(y)\,P(\mathbf{x} \mid y)}{P(\mathbf{x})} \propto P(y)\,P(\mathbf{x} \mid y). \tag{7.3}$$

Giả định "ngây thơ" (naive): các đặc trưng ***độc lập có điều kiện*** *khi biết lớp, nên* $P(\mathbf{x} \mid y) = \prod_{j=1}^{d} P(x_j \mid y)$*. Nhãn dự đoán là*

$$\hat{y} = \arg\max_{k}\; P(y=k) \prod_{j=1}^{d} P(x_j \mid y=k). \tag{7.4}$$

Ví dụ 7.5 (Lọc thư rác)**.** *Một hộp thư có 6 email: 3 spam, 3 không spam (ham). Từ "free" xuất hiện trong 2/3 email spam và 1/3 email ham; từ "meeting" xuất hiện trong 1/3 email spam và 2/3 email ham. Phân lớp một email **có** "free" và **không có** "meeting":*

$$\begin{aligned} \textit{Spam}: &\quad P(\textit{spam}) \cdot P(\textit{free} \mid \textit{spam}) \cdot P(\neg\textit{meeting} \mid \textit{spam}) = \tfrac{1}{2} \cdot \tfrac{2}{3} \cdot \tfrac{2}{3} = 0{,}2222, \\ \textit{Ham}: &\quad P(\textit{ham}) \cdot P(\textit{free} \mid \textit{ham}) \cdot P(\neg\textit{meeting} \mid \textit{ham}) = \tfrac{1}{2} \cdot \tfrac{1}{3} \cdot \tfrac{1}{3} = 0{,}0556. \end{aligned}$$

Chuẩn hóa: $P(\textit{spam} \mid \mathbf{x}) = 0{,}2222/(0{,}2222 + 0{,}0556) = 0{,}8$. *Email được dự đoán là **spam** với xác suất 80%.*

Các biến thể. `GaussianNB` (đặc trưng liên tục, giả định phân phối chuẩn trong mỗi lớp), `MultinomialNB` (đặc trưng đếm — rất hợp với ma trận Count/TF-IDF ở Chương 5), `BernoulliNB` (đặc trưng nhị phân có/không). Để tránh xác suất bằng 0 khi một từ chưa từng xuất hiện trong một lớp, dùng **làm trơn Laplace**: $P(x_j \mid y) = \frac{\text{đếm} + \alpha}{\text{tổng} + \alpha \cdot |V|}$ (tham số `alpha`).

Listing 7.3: Naive Bayes cho phân loại văn bản kết hợp TF-IDF

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.pipeline import make_pipeline

texts = ["free money now", "win a free prize", "free offer today",
         "meeting at noon", "project meeting notes", "lunch meeting free"]
labels = ["spam", "spam", "spam", "ham", "ham", "ham"]

spam_filter = make_pipeline(TfidfVectorizer(), MultinomialNB(alpha=1.0))
spam_filter.fit(texts, labels)
print(spam_filter.predict(["free prize money", "notes from the meeting"]))
print(spam_filter.predict_proba(["free prize money"]).round(3))

# Du lieu so lien tuc: GaussianNB
print(GaussianNB().fit(X_train, y_train).score(X_test, y_test))
```

Dù giả định độc lập hiếm khi đúng, Naive Bayes thường cho kết quả tốt bất ngờ với văn bản, huấn luyện cực nhanh và cần ít dữ liệu. Nhược điểm: xác suất ước lượng thường bị "quá tự tin" (gần 0 hoặc 1).

7.4 Hồi quy logistic (Logistic Regression)

Hàm logistic được nhà toán học Bỉ Pierre-François Verhulst dùng từ thập niên 1830–1840 để mô tả tăng trưởng dân số. Năm 1958, David Cox đề xuất mô hình hồi quy logistic cho biến đầu ra nhị phân. Dù tên gọi có chữ "hồi quy", đây là **thuật toán phân lớp**.

Định nghĩa 7.4 (Hồi quy logistic nhị phân)**.** *Mô hình ước lượng xác suất thuộc lớp dương:*

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{7.5}$$

và dự đoán $\hat{y} = 1$ *khi xác suất* $\geq 0{,}5$*, tức khi* $\mathbf{w}^\top \mathbf{x} + b \geq 0$*. Do đó biên quyết định là một **siêu phẳng**.*

z	-2	0	1	2
$\sigma(z)$	0.1192	0.5	0.7311	0.8808

Bảng 7.3: Một số giá trị của hàm sigmoid

Hàm mất mát. Tham số được tìm bằng cách cực tiểu **entropy chéo** (log loss), tương đương cực đại hợp lý:

$$J(\mathbf{w}, b) = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \ln \hat{p}_i + (1-y_i)\ln(1-\hat{p}_i)\Big], \qquad \hat{p}_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b). \tag{7.6}$$

Hàm này lồi, không có nghiệm đóng, nên được giải bằng Gradient Descent hoặc các phương pháp tối ưu bậc hai (thuật toán `lbfgs` mặc định của scikit-learn). Gradient có dạng rất gọn: $\nabla_{\mathbf{w}} J = \frac{1}{n}\sum_i (\hat{p}_i - y_i)\,\mathbf{x}_i$ — giống hệt hồi quy tuyến tính, chỉ thay $\hat{y}$ bằng $\hat{p}$.

7.4.1 Suy ra hàm mất mát từ nguyên lý hợp lý cực đại

Vì sao lại là entropy chéo mà không phải bình phương sai số như hồi quy tuyến tính? Câu trả lời đến từ **ước lượng hợp lý cực đại** (Maximum Likelihood Estimation – MLE) của Fisher (Raschka và cộng sự, 2022, Chương 3). Với mỗi mẫu, mô hình cho xác suất $\hat{p}_i = P(y_i = 1 \mid \mathbf{x}_i)$, nên xác suất quan sát được đúng nhãn $y_i \in \{0, 1\}$ viết gọn thành $\hat{p}_i^{\,y_i}(1-\hat{p}_i)^{1-y_i}$ (bằng $\hat{p}_i$ khi $y_i = 1$ và bằng $1-\hat{p}_i$ khi $y_i = 0$). Giả sử các mẫu độc lập, **hàm hợp lý** của toàn bộ dữ liệu là

$$\mathcal{L}(\mathbf{w}, b) = \prod_{i=1}^{n} \hat{p}_i^{\,y_i}\,(1-\hat{p}_i)^{1-y_i}. \tag{7.7}$$

Cực đại $\mathcal{L}$ tương đương cực đại $\ln \mathcal{L}$ (logarit đơn điệu tăng và biến tích thành tổng, tránh tràn số khi nhân nhiều xác suất nhỏ), và tương đương **cực tiểu** $-\frac{1}{n}\ln \mathcal{L}$ — chính là hàm J ở trên. Đạo hàm theo w_j dùng tính chất $\sigma'(z) = \sigma(z)(1-\sigma(z))$:

$$\frac{\partial J}{\partial w_j} = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i)\,x_{ij}, \qquad \frac{\partial J}{\partial b} = \frac{1}{n}\sum_{i=1}^{n}(\hat{p}_i - y_i). \tag{7.8}$$

Ví dụ 7.6 (Một bước Gradient Descent cho hồi quy logistic)**.** *Bốn mẫu một chiều $x = (1, 2, 3, 4)$ với nhãn $y = (0, 0, 1, 1)$; khởi tạo $w = 0$, $b = 0$, tốc độ học $\eta = 1$.*

- *Lan truyền xuôi: $z_i = 0$ nên $\hat{p}_i = \sigma(0) = 0{,}5$ với mọi i; mất mát $J = -\ln 0{,}5 = \ln 2 \approx 0{,}6931$.*
- *Gradient: $\frac{\partial J}{\partial w} = \frac{1}{4}\big[0{,}5\cdot 1 + 0{,}5\cdot 2 + (-0{,}5)\cdot 3 + (-0{,}5)\cdot 4\big] = \frac{-2}{4} = -0{,}5$, còn $\frac{\partial J}{\partial b} = \frac{1}{4}(0{,}5 + 0{,}5 - 0{,}5 - 0{,}5) = 0$.*
- *Cập nhật: $w \leftarrow 0 - 1\times(-0{,}5) = 0{,}5$, $b \leftarrow 0$.*
- *Sau một bước: $\hat{p} = \sigma(0{,}5x) = (0{,}6225;\ 0{,}7311;\ 0{,}8176;\ 0{,}8808)$ và J giảm còn $0{,}6539$.*

Xác suất của hai mẫu lớp 1 đã tăng như mong muốn; xác suất của hai mẫu lớp 0 cũng tăng vì b chưa kịp điều chỉnh — các bước tiếp theo sẽ đẩy b về giá trị âm để tách hai lớp.

Điểm đáng chú ý: gradient của hồi quy logistic có **cùng dạng** với gradient của hồi quy tuyến tính (Chương 6), chỉ khác ở chỗ $\hat{p}_i$ đi qua hàm sigmoid. Raschka và cộng sự (2022) tận dụng điều này để biến bản cài đặt Gradient Descent của hồi quy tuyến tính thành hồi quy logistic chỉ bằng cách thay hàm kích hoạt và hàm mất mát.

7.4.2 Ví dụ: mini-batch Gradient Descent cho phân lớp

Ví dụ 7.7 (Mini-batch Gradient Descent cho hồi quy logistic)**.** *Dùng lại đặc trưng, cách chia lô và trọng số khởi tạo của ví dụ mini-batch ở mục 6.4.2 (Bảng 6.4), nhưng với nhãn lớp* $y = (0,0,0,1,1)$*: ba mẫu đầu thuộc lớp 0, hai mẫu cuối thuộc lớp 1.* `batch_size` $= 3$*;* $b = 0{,}3$*;* $w_1 = 0{,}2$*;* $w_2 = 0{,}1$*;* $\alpha = 0{,}1$.

So với hồi quy tuyến tính chỉ có hai thay đổi:

- Dự đoán đi qua hàm sigmoid: $z = b + w_1x_1 + w_2x_2$ và $\hat{y} = \sigma(z) = \dfrac{1}{1+e^{-z}}$.
- Mất mát là entropy chéo nên gradient **không có hệ số 2**: $\dfrac{\partial L}{\partial w_j} = \dfrac{1}{n}\sum_i(\hat{y}_i - y_i)\,x_{ij}$, dạng ma trận $\nabla L = \dfrac{1}{B}\mathbf{X}_B^\top(\hat{\mathbf{y}} - \mathbf{y}_B)$.

Lô 1. $z = (0{,}8;\ 0{,}8;\ 0{,}6)$ — trùng với $\hat{y}$ của ví dụ hồi quy — nên $\hat{y} = \sigma(z) = (0{,}6900;\ 0{,}6900;\ 0{,}6457)$. Cả ba mẫu có nhãn 0, nên $\hat{y} - y = \hat{y}$:

$$\begin{aligned}\frac{\partial L}{\partial b} &= \tfrac{1}{3}(0{,}69 + 0{,}69 + 0{,}6457) = 0{,}6752,\\ \frac{\partial L}{\partial w_1} &= \tfrac{1}{3}(1\cdot 0{,}69 + 2\cdot 0{,}69 + 1\cdot 0{,}6457) = 0{,}9052,\\ \frac{\partial L}{\partial w_2} &= \tfrac{1}{3}(3\cdot 0{,}69 + 1\cdot 0{,}69 + 1\cdot 0{,}6457) = 1{,}1352.\end{aligned}$$

Cập nhật: $b = 0{,}3 - 0{,}0675 = 0{,}2325$; $w_1 = 0{,}2 - 0{,}0905 = 0{,}1095$; $w_2 = 0{,}1 - 0{,}1135 = -0{,}0135$. Lưu ý $\sigma(0{,}6) = 0{,}6457$; nếu làm tròn xuống 0,64 thì gradient thành 0,673; 0,903; 1,133 — lệch ở chữ số thứ ba.

Lô 2 ($n = 2$, cả hai mẫu có nhãn 1):

$$\begin{aligned} z_4 &= 0{,}2325 + 0{,}1095\cdot 7 - 0{,}0135\cdot 8 = 0{,}8907, \quad \hat{y}_4 = 0{,}7090, \quad \hat{y}_4 - y_4 = -0{,}2910,\\ z_5 &= 0{,}2325 + 0{,}1095\cdot 6 - 0{,}0135\cdot 4 = 0{,}8353, \quad \hat{y}_5 = 0{,}6975, \quad \hat{y}_5 - y_5 = -0{,}3025.\end{aligned}$$

$$\begin{aligned}\frac{\partial L}{\partial b} &= \tfrac{1}{2}(-0{,}2910 - 0{,}3025) = -0{,}2967,\\ \frac{\partial L}{\partial w_1} &= \tfrac{1}{2}\big[7\cdot(-0{,}2910) + 6\cdot(-0{,}3025)\big] = -1{,}9260,\\ \frac{\partial L}{\partial w_2} &= \tfrac{1}{2}\big[8\cdot(-0{,}2910) + 4\cdot(-0{,}3025)\big] = -1{,}7689.\end{aligned}$$

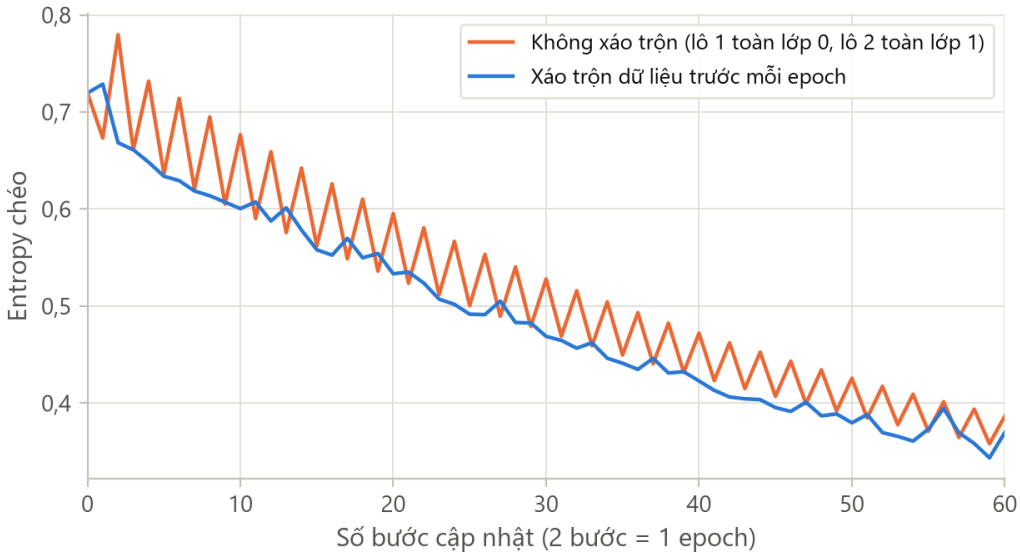
Cập nhật: $b = 0{,}2325 + 0{,}0297 = 0{,}2622$; $w_1 = 0{,}1095 + 0{,}1926 = 0{,}3021$; $w_2 = -0{,}0135 + 0{,}1769 = 0{,}1634$.

Gradient bị chặn nên không phân kỳ. Vì $\sigma(z)$ luôn nằm trong khoảng $(0;\ 1)$, sai lệch $\hat{y} - y$ luôn nằm trong $(-1;\ 1)$. Gradient vì vậy không thể bùng nổ như ở ví dụ hồi quy, dù đặc trưng lớn tới 7 và 8: ở lô 2, w_1 chỉ thay đổi 0,19 thay vì 3,93.

Lô chỉ chứa một lớp: vì sao phải xáo trộn dữ liệu. Entropy chéo trên cả 5 mẫu là 0,7196 trước khi học, giảm còn 0,6732 sau lô 1, nhưng **tăng** lên 0,7796 sau lô 2. Lô 1 toàn lớp 0 nên kéo mọi xác suất xuống; lô 2 toàn lớp 1 nên đẩy mọi xác suất lên, kể cả xác suất lớp 1 của ba mẫu lớp 0 (từ khoảng 0,69 lên 0,74). Gradient của một lô chỉ là ước lượng tốt cho gradient của toàn bộ dữ liệu khi lô đó **đại diện** cho dữ liệu. Vì vậy trong thực hành luôn **xáo trộn** dữ liệu trước mỗi epoch; `SGDClassifier` của scikit-learn và hàm `fit` của Keras đều làm điều này theo mặc định.

Epoch	0	1	2	5	10	50	200
Không xáo trộn	0,7196	0,7796	0,7319	0,6767	0,5952	0,2789	0,0849
Xáo trộn mỗi epoch	0,7196	0,6683	0,6483	0,6005	0,5333	0,2492	0,0771

Bảng 7.4: Entropy chéo trên toàn bộ 5 mẫu theo epoch ($\alpha = 0{,}1$). Sau 200 epoch, cả hai cách đều phân lớp đúng 5/5 mẫu, nhưng cách xáo trộn giảm đều ngay từ đầu



Hình 7.2: Entropy chéo trên toàn bộ dữ liệu sau từng bước cập nhật (mỗi epoch gồm 2 bước). Không xáo trộn, đường mất mát dao động răng cưa vì hai lô kéo trọng số về hai phía ngược nhau.

Listing 7.4: Mini-batch Gradient Descent cho hồi quy logistic, có và không xáo trộn

```
import numpy as np

X = np.array([[1, 1, 3], [1, 2, 1], [1, 1, 1], [1, 7, 8], [1, 6, 4]], dtype=
    float)
y = np.array([0, 0, 0, 1, 1], dtype=float)

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def cross_entropy(w):
    p = sigmoid(X @ w)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))
```

```
12
13 def minibatch_logreg(w, lr=0.1, batch_size=3, epochs=1, shuffle=False, seed=0):
14     rng = np.random.default_rng(seed)
15     for epoch in range(epochs):
16         idx = rng.permutation(len(y)) if shuffle else np.arange(len(y))
17         for start in range(0, len(y), batch_size):
18             b = idx[start:start + batch_size]
19             p = sigmoid(X[b] @ w)                    # xac suat thuoc lop 1
20             grad = X[b].T @ (p - y[b]) / len(b)       # entropy cheo: khong co
    he so 2
21             w = w - lr * grad
22     return w
23
24 w0 = np.array([0.3, 0.2, 0.1])                       # b, w1, w2
25 w = minibatch_logreg(w0)
26 print(w.round(4), round(cross_entropy(w), 4))       # [0.2622 0.3021 0.1634]
    0.7796
27
28 for shuffle in (False, True):
29     w = minibatch_logreg(w0, epochs=200, shuffle=shuffle)
30     print(shuffle, round(cross_entropy(w), 4), (sigmoid(X @ w) >= 0.5).astype(
    int))
31 # False 0.0849 [0 0 0 1 1]
32 # True 0.0771 [0 0 0 1 1]
```

Công thức gradient $\frac{1}{B}\mathbf{X}_B^\top(\hat{\mathbf{y}} - \mathbf{y})$ ở trên được suy ra bằng tay. Lab 8 (Mục 12.8.1, Chương 12) kiểm chứng lại toàn bộ ví dụ này bằng cơ chế tự động tính đạo hàm của PyTorch. Ở đó chỉ cần khai báo hàm mất mát entropy chéo, máy tự tìm ra đúng các gradient $(0{,}6752;\ 0{,}9052;\ 1{,}1352)$ và $(-0{,}2967;\ -1{,}9260;\ -1{,}7689)$ đã tính ở trên.

7.4.3 Chính quy hóa và tham số C

Khi dữ liệu tách được tuyến tính, cực đại hợp lý đẩy $\|\mathbf{w}\| \to \infty$ (sigmoid tiến tới hàm bước để xác suất tiến tới đúng 0 và 1), khiến mô hình quá tự tin và quá khớp. scikit-learn mặc định thêm khoản phạt L2, với C là **nghịch đảo** của độ mạnh chính quy hóa:

$$\min_{\mathbf{w},b}\ \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\Big[-y_i\ln\hat{p}_i - (1-y_i)\ln(1-\hat{p}_i)\Big]. \tag{7.9}$$

C	$\|\mathbf{w}\|_2$	**Accuracy train**	**Accuracy test**
0,001	0,287	0,899	0,881
0,01	0,813	0,953	0,951
0,1	1,787	0,988	0,958
1	3,594	0,991	0,958
100	23,754	0,998	0,951

Bảng 7.5: Ảnh hưởng của C với hồi quy logistic trên `load_breast_cancer` (30 đặc trưng đã chuẩn hóa; chia 75/25 phân tầng, `random_state=0`)

Bảng 7.5 cho thấy đúng bức tranh bias – variance (Chương 8): C quá nhỏ làm hệ số bị co mạnh và mô hình thiếu khớp (độ chính xác thấp trên cả hai tập); C quá lớn làm chuẩn của $\mathbf{w}$ phình to gấp gần 7 lần, độ chính xác train tăng nhưng test giảm — dấu hiệu quá khớp. Giá trị C tốt được chọn bằng kiểm định chéo (`LogisticRegressionCV` hoặc `GridSearchCV`). Muốn phạt L1 để có hệ số thưa (lựa chọn đặc trưng, Chương 5), dùng `l1_ratio=1.0` với `solver="saga"`.

Listing 7.5: Đường chính quy hóa của hồi quy logistic

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

Xb, yb = load_breast_cancer(return_X_y=True)
Xb_tr, Xb_te, yb_tr, yb_te = train_test_split(Xb, yb, test_size=0.25,
                                              stratify=yb, random_state=0)
for C in [0.001, 0.01, 0.1, 1, 100]:
    m = make_pipeline(StandardScaler(), LogisticRegression(C=C, max_iter=5000))
    m.fit(Xb_tr, yb_tr)
    print(C, np.linalg.norm(m[-1].coef_).round(3),
          round(m.score(Xb_tr, yb_tr), 3), round(m.score(Xb_te, yb_te), 3))
```

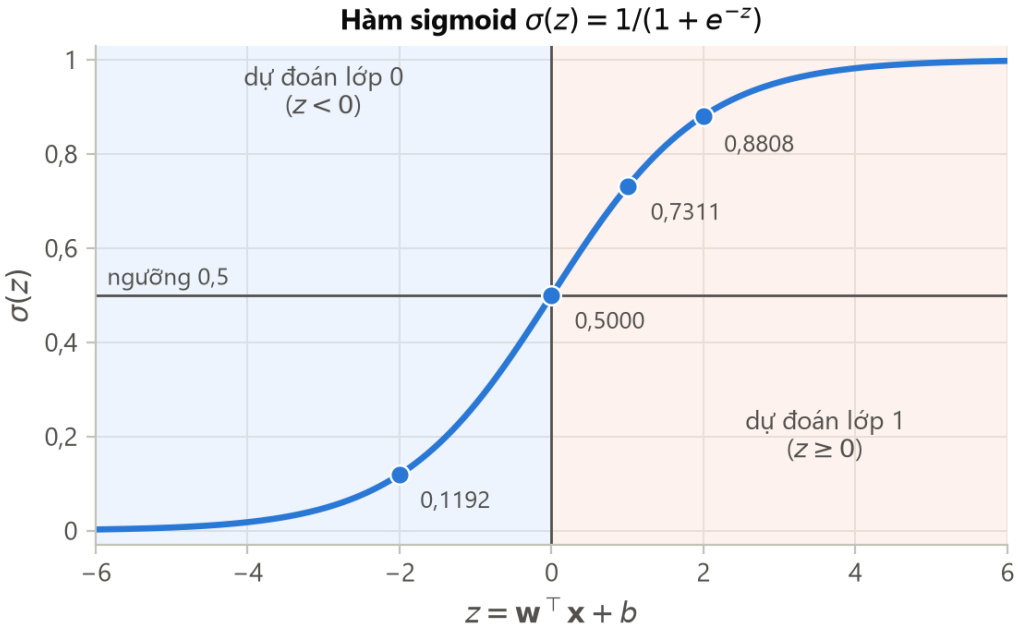
Diễn giải. $\ln \frac{p}{1-p} = \mathbf{w}^\top \mathbf{x} + b$: mỗi khi x_j tăng 1 đơn vị, *odds* (tỉ số $p/(1-p)$) được nhân với e^{w_j}. Đây là lý do hồi quy logistic được ưa chuộng trong y khoa và tài chính, nơi cần giải thích mô hình.

Đa lớp. Với K lớp, dùng hàm **softmax**: $P(y = k \mid \mathbf{x}) = \frac{e^{z_k}}{\sum_j e^{z_j}}$ với $z_k = \mathbf{w}_k^\top \mathbf{x} + b_k$. Tham số `C` trong scikit-learn là nghịch đảo của mức chính quy hóa L2 (giống Ridge): C nhỏ $\Rightarrow$ chính quy hóa mạnh.

Listing 7.6: Logistic Regression với scikit-learn

```
from sklearn.linear_model import LogisticRegression

logreg = make_pipeline(StandardScaler(), LogisticRegression(C=1.0, max_iter
    =1000))
logreg.fit(X_train, y_train)
print("Accuracy:", logreg.score(X_test, y_test))
print(logreg.predict_proba(X_test[:3]).round(3))   # xac suat 3 lop
print(logreg[-1].coef_.round(2))                   # he so cho tung lop
```



Hình 7.3: Đồ thị hàm sigmoid $\sigma(z)$ và ngưỡng quyết định 0,5.

7.5 Cây quyết định (Decision Tree)

Cây quyết định mô phỏng cách con người ra quyết định bằng chuỗi câu hỏi "nếu – thì". Hai dòng thuật toán kinh điển: **CART** (Classification and Regression Trees) của Breiman, Friedman, Olshen và Stone (1984), và **ID3** (1986) cùng hậu duệ C4.5 (1993) của Ross Quinlan. scikit-learn cài đặt phiên bản tối ưu của CART.

7.5.1 Độ đo độ hỗn tạp

Tại mỗi nút có tỉ lệ các lớp $p_1, \ldots, p_K$, độ "hỗn tạp" (impurity) được đo bằng:

$$\text{Gini (CART)}: \quad G = 1 - \sum_{k=1}^{K} p_k^2, \tag{7.10}$$

$$\text{Entropy (ID3/C4.5)}: \quad H = -\sum_{k=1}^{K} p_k \log_2 p_k. \tag{7.11}$$

Nút "thuần" (chỉ một lớp) có $G = H = 0$; với hai lớp cân bằng, $G = 0{,}5$ và $H = 1$. Thuật toán chọn câu hỏi chia nút làm **giảm độ hỗn tạp nhiều nhất**, tính theo trung bình có trọng số của các nút con. Với entropy, mức giảm này gọi là **độ lợi thông tin** (Information Gain):

$$\text{IG} = H(\text{cha}) - \sum_{c \in \text{con}} \frac{n_c}{n} H(c). \tag{7.12}$$

Ví dụ 7.8 (Tính một lần chia nút)**.** *Nút cha có 10 mẫu: 6 "Có", 4 "Không". Một câu hỏi chia thành nút trái 4 mẫu (4 Có, 0 Không) và nút phải 6 mẫu (2 Có, 4 Không).*

– *Gini cha:* $1 - (0{,}6^2 + 0{,}4^2) = 0{,}48$.

– *Gini trái:* 0*; Gini phải:* $1 - \left((2/6)^2 + (4/6)^2\right) = 0{,}4444$.

– *Gini sau chia:* $0{,}4 \times 0 + 0{,}6 \times 0{,}4444 = 0{,}2667 \Rightarrow$ *giảm* $0{,}2133$.

– *Entropy cha:* $0{,}9710$*; entropy phải:* $0{,}9183$*;* $\text{IG} = 0{,}9710 - 0{,}6 \times 0{,}9183 = 0{,}4200$.

Thuật toán thử mọi đặc trưng và mọi ngưỡng, rồi chọn phép chia có mức giảm lớn nhất.

Kiểm soát quá khớp. Cây phát triển hết cỡ sẽ có lá thuần, khớp hoàn hảo dữ liệu huấn luyện nhưng tổng quát kém. Các tham số giới hạn: `max_depth`, `min_samples_split`, `min_samples_leaf`, hoặc cắt tỉa sau bằng `ccp_alpha`.

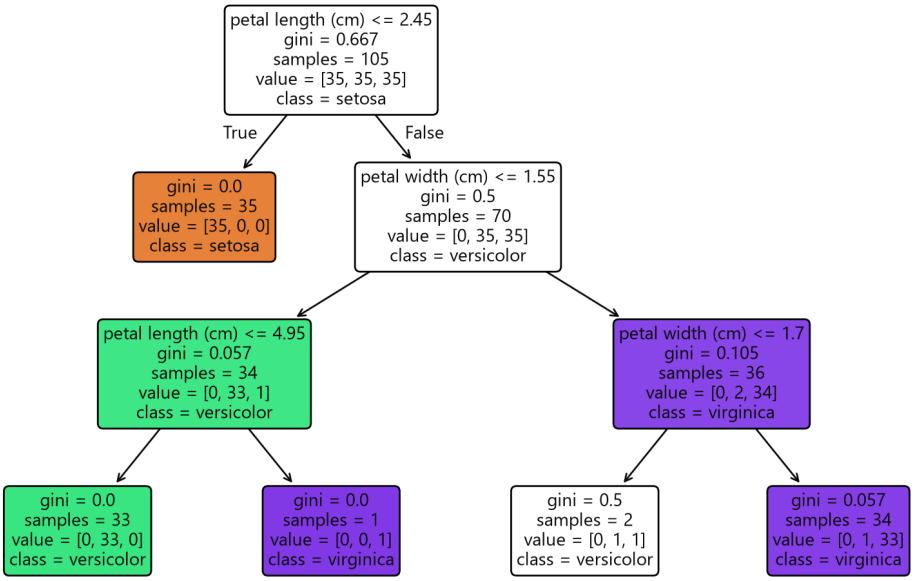
Ưu điểm. Dễ diễn giải, vẽ được; không cần chuẩn hóa dữ liệu; xử lý được quan hệ phi tuyến và tương tác. **Nhược điểm:** không ổn định (đổi dữ liệu một chút, cây thay đổi nhiều), dễ quá khớp — được khắc phục bởi Random Forest ở chương sau.

Listing 7.7: Cây quyết định và trực quan hóa

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
import matplotlib.pyplot as plt

tree = DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
tree.fit(X_train, y_train)
print("Accuracy:", tree.score(X_test, y_test))
print(export_text(tree, feature_names=iris.feature_names))

plt.figure(figsize=(12, 6))
plot_tree(tree, feature_names=iris.feature_names,
          class_names=iris.target_names, filled=True)
plt.show()
```



Hình 7.4: Cây quyết định độ sâu 3 trên dữ liệu Iris (kết quả của `plot_tree`).

7.6 Máy vector hỗ trợ (SVM)

Ý tưởng siêu phẳng lề cực đại được Vladimir Vapnik và Alexey Chervonenkis phát triển tại Liên Xô từ thập niên 1960. Năm 1992, Boser, Guyon và Vapnik kết hợp với **thủ thuật nhân** (kernel trick) để tạo biên phi tuyến; năm 1995, Cortes và Vapnik đề xuất phiên bản **lề mềm** (soft margin) dùng phổ biến đến ngày nay.

7.6.1 Lề cực đại

Với dữ liệu hai lớp $y_i \in \{-1, +1\}$ tách được tuyến tính, có vô số siêu phẳng $\mathbf{w}^\top \mathbf{x} + b = 0$ phân tách hai lớp. SVM chọn siêu phẳng có **lề** (khoảng cách đến điểm gần nhất của mỗi lớp) lớn nhất. Độ rộng lề bằng $2/\|\mathbf{w}\|$, nên bài toán là:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1,\ \forall i. \tag{7.13}$$

Các điểm nằm đúng trên biên của lề gọi là **vector hỗ trợ** (support vectors); chỉ chúng quyết định siêu phẳng — bỏ các điểm khác đi, kết quả không đổi.

7.6.2 Lề mềm và tham số C

Dữ liệu thực thường không tách được hoàn toàn. Lề mềm cho phép vi phạm với biến bù $\xi_i \geq 0$:

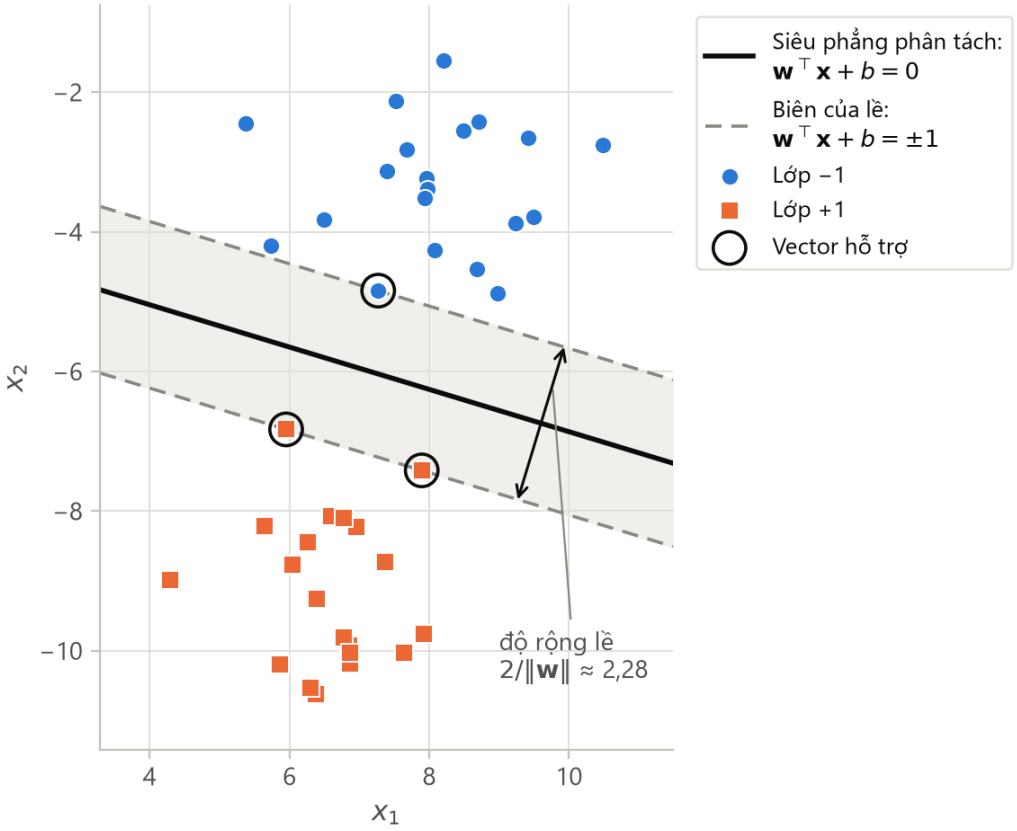
$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \quad \text{với} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i. \tag{7.14}$$

C lớn: phạt vi phạm nặng, lề hẹp, dễ quá khớp; C nhỏ: lề rộng, chấp nhận sai một số điểm, tổng quát tốt hơn.

7.6.3 Thủ thuật nhân

Ý tưởng: ánh xạ dữ liệu sang không gian nhiều chiều hơn, nơi chúng tách được tuyến tính. Vì bài toán đối ngẫu của SVM chỉ cần tích vô hướng giữa các điểm, ta thay tích vô hướng bằng một **hàm nhân** $k(\mathbf{x}, \mathbf{z})$ mà không cần tính ánh xạ tường minh:

- Tuyến tính: $k(\mathbf{x}, \mathbf{z}) = \mathbf{x}^\top \mathbf{z}$.
- Đa thức: $k(\mathbf{x}, \mathbf{z}) = (\gamma\, \mathbf{x}^\top \mathbf{z} + r)^p$.
- RBF (Gauss): $k(\mathbf{x}, \mathbf{z}) = \exp(-\gamma\|\mathbf{x} - \mathbf{z}\|^2)$ — mặc định trong scikit-learn; γ lớn cho biên càng "uốn lượn".



Hình 7.5: SVM tuyến tính: siêu phẳng phân tách, lề cực đại và các vector hỗ trợ (khoanh tròn).

Listing 7.8: SVM với tìm kiếm tham số C và gamma

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

svm = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
grid = GridSearchCV(svm, {"svc__C": [0.1, 1, 10, 100],
                          "svc__gamma": [0.01, 0.1, 1]}, cv=5)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.score(X_test, y_test))
print("So vector ho tro:", grid.best_estimator_[-1].n_support_)
```

SVM cần **chuẩn hóa dữ liệu** (vì dựa trên khoảng cách và tích vô hướng), hoạt động tốt trên dữ liệu số chiều cao như văn bản, nhưng huấn luyện chậm khi số mẫu lớn (độ phức tạp cỡ $O(n^2)$ đến $O(n^3)$) và không trực tiếp cho xác suất.

7.7 So sánh các thuật toán

Listing 7.9: So sánh năm thuật toán bằng kiểm định chéo

```
from sklearn.model_selection import cross_val_score

X, y = load_iris(return_X_y=True)      # tro lai bo du lieu Iris
models = {
```

```
    "KNN":          make_pipeline(StandardScaler(), KNeighborsClassifier(5)),
    "NaiveBayes":   GaussianNB(),
    "Logistic":     make_pipeline(StandardScaler(), LogisticRegression(max_iter
    =1000)),
    "DecisionTree": DecisionTreeClassifier(max_depth=3, random_state=42),
    "SVM":          make_pipeline(StandardScaler(), SVC()),
}
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=5)
    print(f"{name:12s} {s.mean():.3f} +/- {s.std():.3f}")
```

Thuật toán	Biên quyết định	Cần chuẩn hóa	Ưu điểm	Nhược điểm
KNN	Linh hoạt	Có	Đơn giản, không cần huấn luyện	Chậm khi dự đoán, kém ở số chiều cao
Naive Bayes	Tuyến tính/bậc 2	Không	Rất nhanh, tốt với văn bản	Giả định độc lập, xác suất quá tự tin
Logistic Regression	Tuyến tính	Có	Cho xác suất, dễ diễn giải	Không nắm bắt phi tuyến nếu không tạo đặc trưng
Decision Tree	Song song trục	Không	Dễ giải thích, xử lý dữ liệu hỗn hợp	Dễ quá khớp, không ổn định
SVM	Tuyến tính hoặc theo nhân	Có	Mạnh ở số chiều cao, lề tối ưu	Chậm với dữ liệu lớn, khó chọn tham số

Bảng 7.6: So sánh các thuật toán phân lớp cơ bản

Các chỉ số đánh giá như ma trận nhầm lẫn, precision, recall, F1 và ROC-AUC được trình bày ở chương Đánh giá mô hình.

7.8 Tóm tắt chương

– KNN gán nhãn theo đa số của K láng giềng gần nhất; bỏ phiếu có trọng số $1/(d+\varepsilon)$ giúp phá thế hòa; với hồi quy, KNN lấy trung bình (có trọng số) giá trị mục tiêu của láng giềng. Cần chuẩn hóa dữ liệu và chọn K phù hợp.

– Naive Bayes áp dụng định lý Bayes với giả định độc lập có điều kiện; rất hiệu quả cho văn bản.

– Logistic Regression ước lượng xác suất qua hàm sigmoid/softmax, tối ưu entropy chéo; biên tuyến tính, dễ diễn giải. Khi huấn luyện bằng mini-batch Gradient Descent, gradient là $\frac{1}{B}\mathbf{X}_B^\top(\hat{\mathbf{y}}-\mathbf{y}_B)$, bị chặn nhờ hàm sigmoid; phải xáo trộn dữ liệu trước mỗi epoch để mỗi lô đại diện cho dữ liệu.

– Cây quyết định chia nút theo Gini hoặc Entropy; dễ giải thích nhưng dễ quá khớp.

– SVM tìm siêu phẳng lề cực đại; thủ thuật nhân cho phép biên phi tuyến.

7.9 Bài tập

1. Cho 6 điểm $P_1(1,1)$, $P_2(2,1)$, $P_3(1,2)$ thuộc lớp A và $P_4(5,5)$, $P_5(6,5)$, $P_6(5,6)$ thuộc lớp B. (a) Phân lớp $\mathbf{q} = (2{,}5;\ 2{,}5)$ với $K = 3$ (đáp số: A). (b) Phân lớp $\mathbf{q} = (4,4)$ với $K = 1, 3, 5$. Khi $K = 5$, kết quả có thay đổi nếu dùng bỏ phiếu có trọng số $1/d$ không?

2. Với dữ liệu Bảng 7.2, dự đoán y tại $\mathbf{q} = (6,6)$ bằng KNN hồi quy với $K = 2$ và $K = 3$, theo cả trung bình đều và trung bình có trọng số. Kiểm tra bằng `KNeighborsRegressor`.

3. Trong ví dụ lọc thư rác, tính lại xác suất spam khi email có cả "free" và "meeting".

4. Một nút có 8 mẫu (5 A, 3 B) được chia thành hai nút (4 A, 0 B) và (1 A, 3 B). Tính Gini, Entropy trước và sau chia và độ lợi thông tin.

5. Chứng minh $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ và suy ra công thức gradient của hàm log loss.

6. Với ví dụ mini-batch phân lớp (Bảng 6.4), tính tay epoch thứ hai khi không xáo trộn, bắt đầu từ $b = 0{,}2622$; $w_1 = 0{,}3021$; $w_2 = 0{,}1634$. Sau đó đổi thứ tự lô thành $\{1, 4, 2\}$ và $\{3, 5\}$ (mỗi lô có cả hai lớp) rồi tính lại epoch đầu tiên. Entropy chéo có còn tăng sau lô thứ hai không?

7. Trên bộ dữ liệu `load_breast_cancer`, so sánh năm thuật toán có và không có `StandardScaler`. Thuật toán nào nhạy với chuẩn hóa nhất? Giải thích.

8. Dùng `TfidfVectorizer` kết hợp `MultinomialNB` và `LinearSVC` trên bộ dữ liệu 20 Newsgroups (hàm `fetch_20newsgroups`, chọn 4 chủ đề). So sánh độ chính xác và thời gian huấn luyện.

Tài liệu tham khảo của chương

1. Bayes, T. (1763). An Essay towards Solving a Problem in the Doctrine of Chances. *Philosophical Transactions of the Royal Society of London*, 53, 370–418.

2. Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. *Proceedings of COLT '92*, 144–152.

3. Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth.

4. Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297.

5. Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.

6. Cox, D. R. (1958). The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society, Series B*, 20(2), 215–242.

7. Dudani, S. A. (1976). The Distance-Weighted k-Nearest-Neighbor Rule. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-6(4), 325–327.

8. Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.

9. Fix, E., & Hodges, J. L. (1951). *Discriminatory Analysis. Nonparametric Discrimination: Consistency Properties*. USAF School of Aviation Medicine, Randolph Field, Texas.

10. Liu, Y. (H.) (2019). *Python Machine Learning By Example* (2nd ed.), Chương 4–5 (Naive Bayes, SVM) và Chương 7 (Logistic Regression). Packt Publishing.

11. Maron, M. E. (1961). Automatic Indexing: An Experimental Inquiry. *Journal of the ACM*, 8(3), 404–417.

12. Quinlan, J. R. (1986). Induction of decision trees. *Machine Learning*, 1(1), 81–106.

13. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*, Chương 3: A Tour of Machine Learning Classifiers Using Scikit-Learn. Packt Publishing.

Chương 8: Đánh giá và lựa chọn mô hình (Validation)

Hai chương trước đã giới thiệu nhiều mô hình hồi quy và phân lớp, mỗi mô hình lại có nhiều siêu tham số. Câu hỏi trung tâm của chương này là: **làm sao biết một mô hình tốt đến đâu trên dữ liệu nó chưa từng thấy**, và **làm sao chọn được mô hình và siêu tham số tốt nhất** mà không tự lừa mình. Đánh giá sai là nguyên nhân phổ biến nhất khiến mô hình đạt kết quả rất cao trong phòng thí nghiệm nhưng thất bại khi triển khai. Chương này được đặt ngay sau hồi quy và phân lớp vì từ đây trở đi, mọi mô hình của giáo trình, từ học kết hợp đến phân cụm, đều được đánh giá và tinh chỉnh theo quy trình xây dựng ở chương này.
Sau khi học xong chương này, người học có khả năng:

- Giải thích lỗi tổng quát hóa, quá khớp, dưới khớp và đánh đổi bias – variance.
- Thiết kế đúng việc chia dữ liệu: train / validation / test, k-fold, stratified, time series split.
- Tính tay và diễn giải các độ đo phân lớp (ma trận nhầm lẫn, precision, recall, F1, ROC – AUC) và hồi quy (MAE, MSE, RMSE, R^2).
- Tìm siêu tham số bằng `GridSearchCV`, `RandomizedSearchCV`, phương pháp chia đôi liên tiếp (successive halving), và tránh rò rỉ dữ liệu.

8.1 Lỗi tổng quát hóa

Mục tiêu của học máy không phải là khớp dữ liệu huấn luyện mà là **tổng quát hóa** (generalization): dự đoán tốt trên dữ liệu mới lấy từ cùng phân phối. Gọi $\mathcal{D}$ là phân phối dữ liệu và L là hàm mất mát; **lỗi tổng quát hóa** (rủi ro thực) của mô hình f là

$$R(f) = \mathbb{E}_{(\mathbf{x},y)\sim\mathcal{D}}\big[L(y, f(\mathbf{x}))\big]. \tag{8.1}$$

Ta không biết $\mathcal{D}$, nên chỉ có thể **ước lượng** $R(f)$ bằng trung bình mất mát trên một tập dữ liệu **độc lập** với quá trình huấn luyện. Lỗi trên tập huấn luyện luôn là ước lượng **quá lạc quan**.

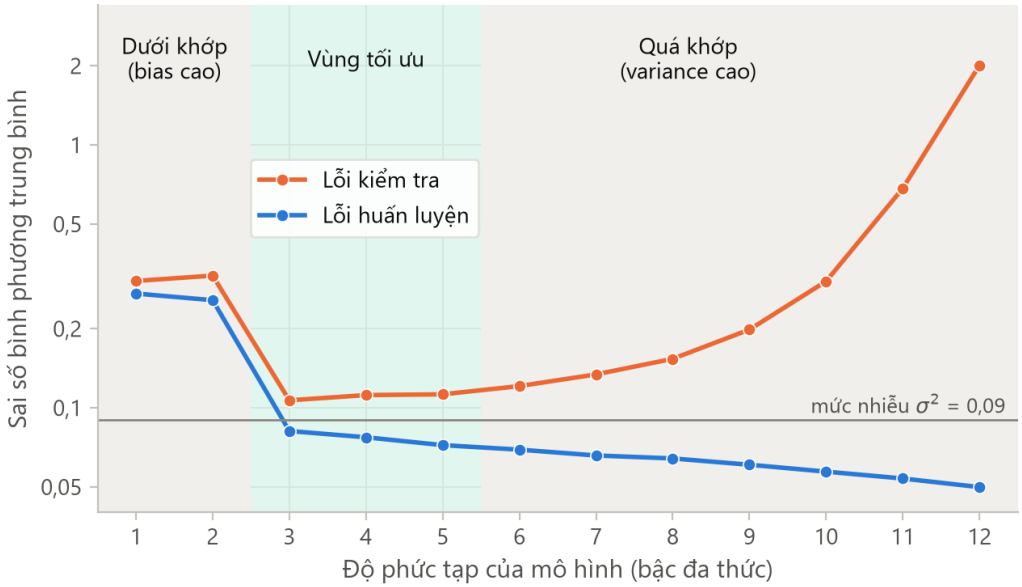
Tình trạng	Lỗi train	Lỗi test	Cách khắc phục
Dưới khớp (underfitting)	Cao	Cao	Mô hình phức tạp hơn, thêm đặc trưng, giảm điều chuẩn
Vừa khớp	Thấp	Thấp	—
Quá khớp (overfitting)	Rất thấp	Cao	Thêm dữ liệu, điều chuẩn, giảm độ phức tạp, ensemble

Bảng 8.1: Chẩn đoán qua lỗi huấn luyện và lỗi kiểm tra

Đối với mất mát bình phương, sai số kỳ vọng tại một điểm phân tích được thành (Geman và cộng sự, 1992):

$$\mathbb{E}\big[(y - \hat{f}(\mathbf{x}))^2\big] = \underbrace{\big(\mathbb{E}[\hat{f}(\mathbf{x})] - f(\mathbf{x})\big)^2}_{\text{Bias}^2} + \underbrace{\text{Var}[\hat{f}(\mathbf{x})]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{nhiễu}}. \tag{8.2}$$

Mô hình đơn giản có bias cao, mô hình phức tạp có variance cao; điểm tối ưu nằm ở giữa.



Hình 8.1: Lỗi huấn luyện và lỗi kiểm tra theo độ phức tạp của mô hình: vùng dưới khớp, vùng tối ưu và vùng quá khớp. Hồi quy đa thức trên 30 điểm mô phỏng ($y = \sin$ cộng nhiễu), lấy trung vị của 300 lần lặp.

8.2 Chiến lược chia dữ liệu

8.2.1 Hold-out: train / validation / test

- **Tập huấn luyện** (train): học tham số mô hình.
- **Tập kiểm định** (validation): so sánh mô hình, chọn siêu tham số.
- **Tập kiểm tra** (test): đánh giá **một lần duy nhất** ở cuối. Nếu dùng tập test để chọn mô hình, nó trở thành tập validation và ước lượng không còn khách quan.

Tỉ lệ thường dùng là 60/20/20 hoặc 70/15/15; với dữ liệu rất lớn, tập test chỉ cần 1%. Với bài toán phân lớp, dùng `stratify=y` để giữ tỉ lệ các lớp.

Listing 8.1: Chia dữ liệu ba phần

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

X, y = load_breast_cancer(return_X_y=True)
X_tmp, X_test, y_tmp, y_test = train_test_split(
```

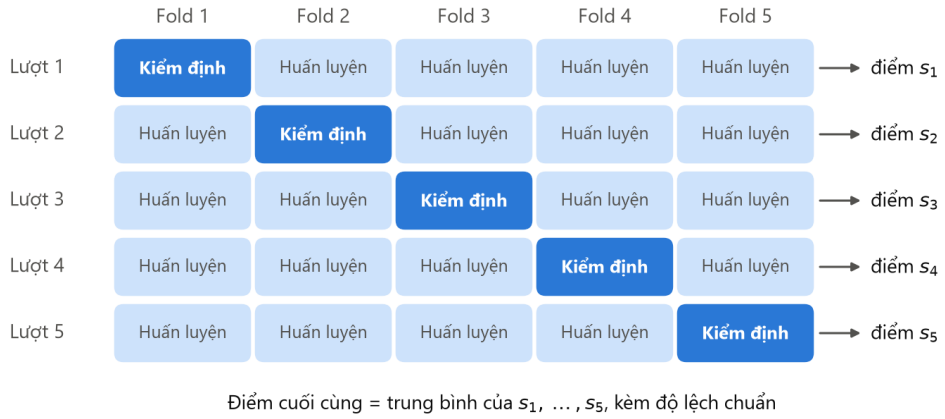
```
    X, y, test_size=0.2, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_tmp, y_tmp, test_size=0.25, stratify=y_tmp, random_state=42)  # 0.25*0.8 = 0.2
print(len(X_train), len(X_val), len(X_test))   # 341 114 114
```

8.2.2 Kiểm định chéo k-fold

Hold-out lãng phí dữ liệu và kết quả phụ thuộc vào cách chia ngẫu nhiên. **Kiểm định chéo** (cross-validation), được Stone (1974) và Geisser (1975) hình thức hóa, khắc phục điều này:

1. Chia dữ liệu thành k phần (fold) gần bằng nhau.
2. Lần lượt dùng mỗi phần làm tập validation, $k-1$ phần còn lại làm tập huấn luyện.
3. Điểm cuối cùng là trung bình (và độ lệch chuẩn) của k lần đánh giá.

Mỗi mẫu được dùng để kiểm định đúng một lần. $k = 5$ hoặc 10 là lựa chọn phổ biến (Kohavi, 1995). Trường hợp $k = n$ gọi là **Leave-One-Out** (LOOCV): ít bias nhưng tốn kém và có variance cao.



Hình 8.2: Sơ đồ kiểm định chéo 5-fold: mỗi lượt một fold (tô đậm) làm tập kiểm định.

Bộ chia	**Khi nào dùng**
`KFold`	Hồi quy, dữ liệu độc lập cùng phân phối
`StratifiedKFold`	Phân lớp (mặc định của `cross_val_score` khi mô hình là bộ phân lớp), đặc biệt khi lớp mất cân bằng
`GroupKFold`	Nhiều mẫu từ cùng một đối tượng (bệnh nhân, khách hàng): mọi mẫu của một nhóm phải nằm cùng phía
`TimeSeriesSplit`	Chuỗi thời gian: luôn huấn luyện trên quá khứ, kiểm định trên tương lai (Chương 6)

Bảng 8.2: Các bộ chia dữ liệu trong scikit-learn

Listing 8.2: Kiểm định chéo với cross_val_score và cross_validate

```
from sklearn.model_selection import cross_val_score, cross_validate,
    StratifiedKFold
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_val_score(model, X_tmp, y_tmp, cv=cv, scoring="accuracy")
print(scores.round(4), scores.mean().round(4), scores.std().round(4))

res = cross_validate(model, X_tmp, y_tmp, cv=cv,
                     scoring=["accuracy", "f1", "roc_auc"], return_train_score=
    True)
print({k: v.mean().round(4) for k, v in res.items() if "test" in k})
```

Quan trọng: bộ tiền xử lý (scaler, encoder, PCA, …) phải nằm **trong pipeline** để được `fit` lại trên từng fold huấn luyện. Nếu chuẩn hóa toàn bộ dữ liệu trước rồi mới kiểm định chéo, thông tin của fold kiểm định đã rò rỉ vào quá trình huấn luyện (xem Chương 5).

8.3 Độ đo cho bài toán phân lớp

8.3.1 Ma trận nhầm lẫn

Với bài toán hai lớp (lớp **dương** là lớp ta quan tâm: có bệnh, thư rác, gian lận …), mọi dự đoán rơi vào một trong bốn ô:

	Dự đoán dương	**Dự đoán âm**
Thực tế dương	TP (True Positive)	FN (False Negative)
Thực tế âm	FP (False Positive)	TN (True Negative)

Bảng 8.3: Ma trận nhầm lẫn (confusion matrix)

Độ đo	**Công thức**	**Ý nghĩa**
Accuracy	$\frac{TP+TN}{TP+TN+FP+FN}$	Tỉ lệ dự đoán đúng
Precision	$\frac{TP}{TP+FP}$	Trong các mẫu báo dương, bao nhiêu là dương thật
Recall (Sensitivity)	$\frac{TP}{TP+FN}$	Trong các mẫu dương thật, tìm ra được bao nhiêu
Specificity	$\frac{TN}{TN+FP}$	Trong các mẫu âm thật, nhận đúng bao nhiêu
F1-score	$\frac{2 \cdot P \cdot R}{P+R}$	Trung bình điều hòa của precision và recall

Bảng 8.4: Các độ đo phân lớp

Ví dụ 8.1 (Tính tay các độ đo)**.** *Một mô hình sàng lọc bệnh được thử trên 100 người: 50 người có bệnh, 50 người không. Kết quả:* $TP = 40$*,* $FN = 10$*,* $FP = 5$*,* $TN = 45$*.*

$$\textit{Accuracy} = \tfrac{40+45}{100} = 0{,}85, \qquad \textit{Precision} = \tfrac{40}{45} = 0{,}8889,$$
$$\textit{Recall} = \tfrac{40}{50} = 0{,}80, \qquad \textit{Specificity} = \tfrac{45}{50} = 0{,}90,$$
$$F_1 = \tfrac{2\times 0{,}8889\times 0{,}80}{0{,}8889+0{,}80} = 0{,}8421.$$

Mô hình bỏ sót 10 trong 50 người bệnh (recall 80%) — với bài toán y tế, đây là điều cần cải thiện dù accuracy khá cao.

Nghịch lý accuracy. Với dữ liệu mất cân bằng (ví dụ 1% giao dịch gian lận), mô hình luôn đoán "không gian lận" đạt accuracy 99% nhưng recall bằng 0. Khi đó phải dùng precision, recall, F1 hoặc AUC. Việc ưu tiên precision hay recall tùy vào **chi phí** của từng loại lỗi: lọc thư rác ưu tiên precision (không muốn chặn nhầm thư quan trọng), sàng lọc ung thư ưu tiên recall (không muốn bỏ sót).

Bài toán nhiều lớp. Tính độ đo cho từng lớp (một lớp là dương, còn lại là âm) rồi lấy trung bình: **macro** (trung bình cộng, mỗi lớp như nhau), **weighted** (trọng số theo số mẫu), **micro** (cộng dồn TP, FP, FN của mọi lớp).

8.3.2 Đường ROC và AUC

Hầu hết bộ phân lớp đưa ra **điểm số** hoặc xác suất (`predict_proba`), sau đó so với **ngưỡng** (mặc định 0,5). Thay đổi ngưỡng sẽ đổi precision lấy recall. Đường **ROC** (Receiver Operating Characteristic) — bắt nguồn từ lý thuyết phát hiện tín hiệu radar trong Thế chiến II — vẽ $TPR =$ Recall theo $FPR = 1 -$ Specificity khi ngưỡng chạy từ cao đến thấp. **AUC** là diện tích dưới đường ROC: bằng 1 với bộ phân lớp hoàn hảo, 0,5 với đoán ngẫu nhiên. AUC có một diễn giải xác suất đẹp (Hanley & McNeil, 1982): *AUC là xác suất một mẫu dương ngẫu nhiên được chấm điểm cao hơn một mẫu âm ngẫu nhiên.*

Ví dụ 8.2 (Tính AUC bằng cách đếm cặp)**.** *Bốn mẫu có nhãn* $y = (0, 0, 1, 1)$ *và điểm số* $s = (0{,}1;\ 0{,}4;\ 0{,}35;\ 0{,}8)$*. Có* $2 \times 2 = 4$ *cặp (dương, âm):*

- $(0{,}35$ *vs* $0{,}1)$*: đúng thứ tự;* $(0{,}35$ *vs* $0{,}4)$*:* ***sai*** *thứ tự;*
- $(0{,}8$ *vs* $0{,}1)$*: đúng;* $(0{,}8$ *vs* $0{,}4)$*: đúng.*

$AUC = 3/4 = 0{,}75$*.*

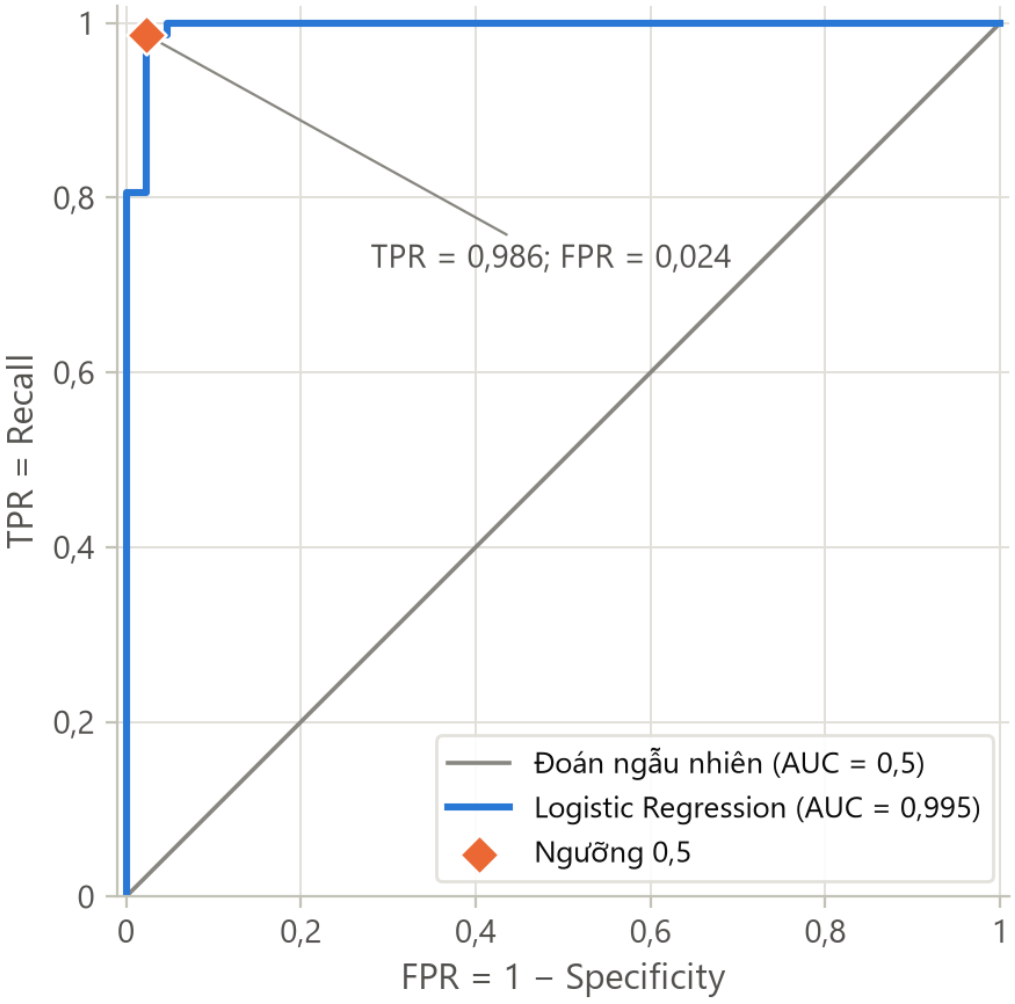
Listing 8.3: Ma trận nhầm lẫn, báo cáo phân lớp và ROC

```
from sklearn.metrics import (confusion_matrix, classification_report,
                             roc_auc_score, RocCurveDisplay,
    ConfusionMatrixDisplay)

print(roc_auc_score([0, 0, 1, 1], [0.1, 0.4, 0.35, 0.8]))   # 0.75

model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_score = model.predict_proba(X_test)[:, 1]
```

```
9
10 print(confusion_matrix(y_test, y_pred))
11 print(classification_report(y_test, y_pred, digits=4))
12 print("AUC:", roc_auc_score(y_test, y_score))
13
14 ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
15 RocCurveDisplay.from_predictions(y_test, y_score)
```



Hình 8.3: Đường ROC của mô hình Logistic Regression trên tập ung thư vú so với đường chéo của bộ đoán ngẫu nhiên.

Với dữ liệu mất cân bằng nặng, đường **Precision – Recall** và diện tích dưới nó (Average Precision) thường phản ánh chất lượng mô hình rõ hơn ROC (Saito & Rehmsmeier, 2015).

8.4 Độ đo cho bài toán hồi quy

Độ đo	Công thức	Ghi chú
MAE	$\frac{1}{n}\sum \lvert y_i - \hat{y}_i \rvert$	Cùng đơn vị với y, ít nhạy với ngoại lai
MSE	$\frac{1}{n}\sum (y_i - \hat{y}_i)^2$	Phạt nặng sai số lớn
RMSE	$\sqrt{\text{MSE}}$	Cùng đơn vị với y
R^2	$1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$	Tỉ lệ phương sai được giải thích; có thể âm
MAPE	$\frac{1}{n}\sum \left\lvert \frac{y_i - \hat{y}_i}{y_i} \right\rvert$	Sai số tương đối; không dùng khi $y_i \approx 0$

Bảng 8.5: Các độ đo hồi quy

Ví dụ 8.3 (Tính tay độ đo hồi quy)**.** $y = (3;\ -0{,}5;\ 2;\ 7)$, $\hat{y} = (2{,}5;\ 0;\ 2;\ 8)$. *Sai số:* $(0{,}5;\ -0{,}5;\ 0;\ -1)$.

$$\textit{MAE} = \frac{0{,}5+0{,}5+0+1}{4} = 0{,}5, \quad \textit{MSE} = \frac{0{,}25+0{,}25+0+1}{4} = 0{,}375, \quad \textit{RMSE} = 0{,}6124.$$

$\bar{y} = 2{,}875$, $\sum (y_i - \bar{y})^2 = 29{,}1875$, *nên* $R^2 = 1 - 1{,}5/29{,}1875 = 0{,}9486$.

Listing 8.4: Độ đo hồi quy trong scikit-learn

```
from sklearn.metrics import (mean_absolute_error, mean_squared_error,
                             root_mean_squared_error, r2_score)
y_true, y_hat = [3, -0.5, 2, 7], [2.5, 0.0, 2, 8]
print(mean_absolute_error(y_true, y_hat))       # 0.5
print(mean_squared_error(y_true, y_hat))        # 0.375
print(root_mean_squared_error(y_true, y_hat))   # 0.6124
print(r2_score(y_true, y_hat))                  # 0.9486
```

Lưu ý: trong `cross_val_score`, scikit-learn quy ước "điểm càng lớn càng tốt", nên các độ đo sai số có dạng âm như `scoring="neg_root_mean_squared_error"`.

8.5 Tìm siêu tham số

Tham số (trọng số hồi quy, ngưỡng chia của cây) được học từ dữ liệu; **siêu tham số** (`alpha` của Ridge, `n_neighbors` của KNN, `max_depth` của cây) phải được chọn trước khi huấn luyện. Ta chọn siêu tham số bằng cách tối ưu điểm kiểm định chéo.

- **Grid search**: thử mọi tổ hợp trên lưới giá trị — đầy đủ nhưng chi phí tăng theo cấp số nhân với số siêu tham số.
- **Random search**: lấy mẫu ngẫu nhiên N tổ hợp từ các phân phối. Bergstra và Bengio (2012) chỉ ra rằng với cùng ngân sách, random search thường tốt hơn vì chỉ vài siêu tham số thực sự quan trọng.

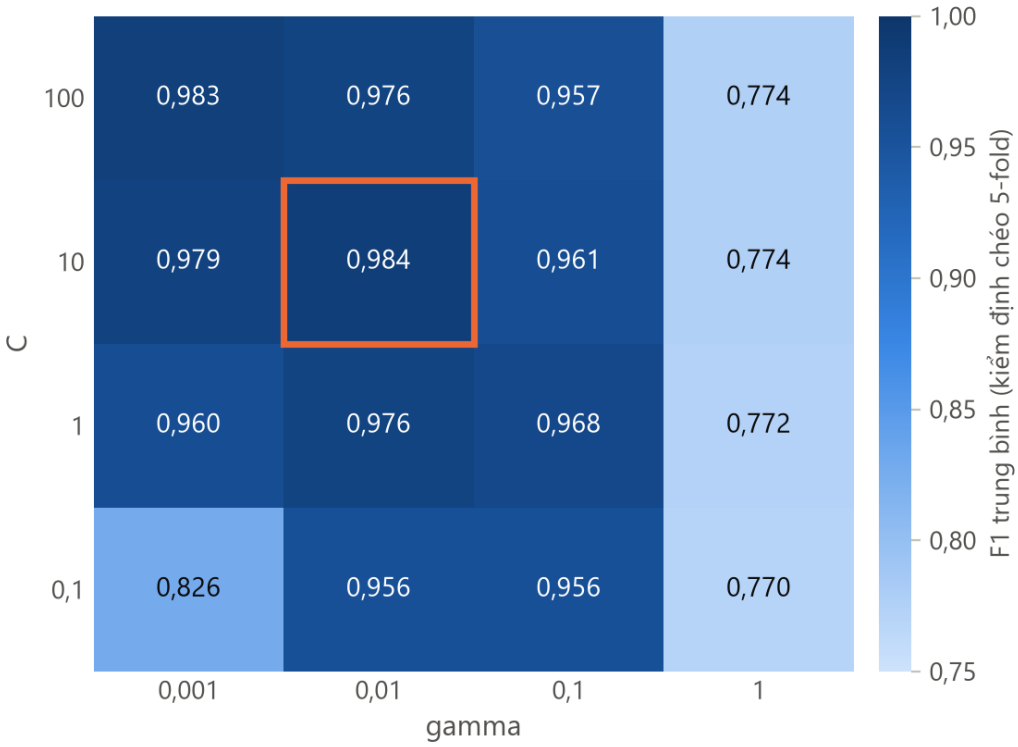
– **Tối ưu Bayes** (Optuna, scikit-optimize): dùng kết quả các lần thử trước để chọn điểm thử tiếp theo.

Listing 8.5: GridSearchCV và RandomizedSearchCV

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.svm import SVC
from scipy.stats import loguniform

pipe = make_pipeline(StandardScaler(), SVC())
param_grid = {"svc__C": [0.1, 1, 10, 100],
              "svc__gamma": [0.001, 0.01, 0.1, 1],
              "svc__kernel": ["rbf"]}
grid = GridSearchCV(pipe, param_grid, cv=5, scoring="f1", n_jobs=-1)
grid.fit(X_tmp, y_tmp)                        # 16 to hop x 5 fold = 80 lan fit
print(grid.best_params_, round(grid.best_score_, 4))
print("Test F1:", round(grid.score(X_test, y_test), 4))   # danh gia 1 lan cuoi

rand = RandomizedSearchCV(pipe, {"svc__C": loguniform(1e-2, 1e3),
                                 "svc__gamma": loguniform(1e-4, 1e0)},
                          n_iter=30, cv=5, scoring="f1", random_state=0, n_jobs
    =-1)
rand.fit(X_tmp, y_tmp)
print(rand.best_params_)
```



Hình 8.4: Bản đồ nhiệt điểm F1 kiểm định chéo theo `C` và `gamma` của SVM. Ô viền cam là tổ hợp tốt nhất ($C = 10$, gamma = 0,01).

Chia đôi liên tiếp (successive halving). Grid search và random search dành cùng một lượng tài nguyên cho mọi ứng viên, kể cả những ứng viên tồi lộ rõ ngay từ đầu. Phương pháp **chia đôi liên tiếp** (Jamieson & Talwalkar, 2016; nền tảng của Hyperband – Li và cộng sự, 2018) phân bổ tài nguyên thích nghi, giống một giải đấu loại trực tiếp (Raschka và cộng sự, 2022, Chương 6):

1. Đánh giá **rất nhiều** ứng viên với tài nguyên **nhỏ**, chẳng hạn chỉ dùng một ít mẫu huấn luyện.
2. Giữ lại $1/\eta$ ứng viên tốt nhất (`factor`, thường $\eta = 3$), nhân tài nguyên lên η lần.
3. Lặp lại đến khi còn một ứng viên hoặc dùng hết tài nguyên.

Với 81 ứng viên và $\eta = 3$: các vòng có $81 \to 27 \to 9 \to 3 \to 1$ ứng viên, với tài nguyên $r \to 3r \to 9r \to 27r \to 81r$. Tổng chi phí là $5 \times 81r = 405r$, so với $81 \times 81r = 6561r$ nếu đánh giá mọi ứng viên với tài nguyên đầy đủ — rẻ hơn khoảng 16 lần. Giả định ngầm của phương pháp là thứ hạng của các ứng viên khi có ít tài nguyên phản ánh đúng thứ hạng khi có đủ tài nguyên; điều này không phải lúc nào cũng đúng, nhất là khi tài nguyên ban đầu quá nhỏ.

Listing 8.6: Tìm siêu tham số bằng HalvingRandomSearchCV

```
from sklearn.experimental import enable_halving_search_cv  # noqa: F401 (tinh
    nang thu nghiem)
from sklearn.model_selection import HalvingRandomSearchCV

halving = HalvingRandomSearchCV(pipe, {"svc__C": loguniform(1e-2, 1e3),
                                       "svc__gamma": loguniform(1e-4, 1e0)},
                                factor=3, resource="n_samples", scoring="f1",
                                random_state=0, n_jobs=-1)
halving.fit(X_tmp, y_tmp)
print(halving.n_candidates_)    # [22, 8, 3]   so ung vien moi vong
print(halving.n_resources_)     # [20, 60, 180] so mau huan luyen moi vong
print(halving.best_params_, round(halving.score(X_test, y_test), 4))   # F1
    test ~0.986
```

Kiểm định chéo lồng nhau. `best_score_` của GridSearchCV là ước lượng **lạc quan**, vì chính nó đã được dùng để chọn siêu tham số (Cawley & Talbot, 2010). Để ước lượng khách quan khi không có tập test riêng, dùng **nested CV**: vòng ngoài đánh giá, vòng trong tìm siêu tham số.

Listing 8.7: Nested cross-validation

```
outer = cross_val_score(GridSearchCV(pipe, param_grid, cv=3),
                        X, y, cv=5, scoring="f1")
print(outer.mean().round(4), outer.std().round(4))
```

8.5.1 Đường cong học và đường cong kiểm định

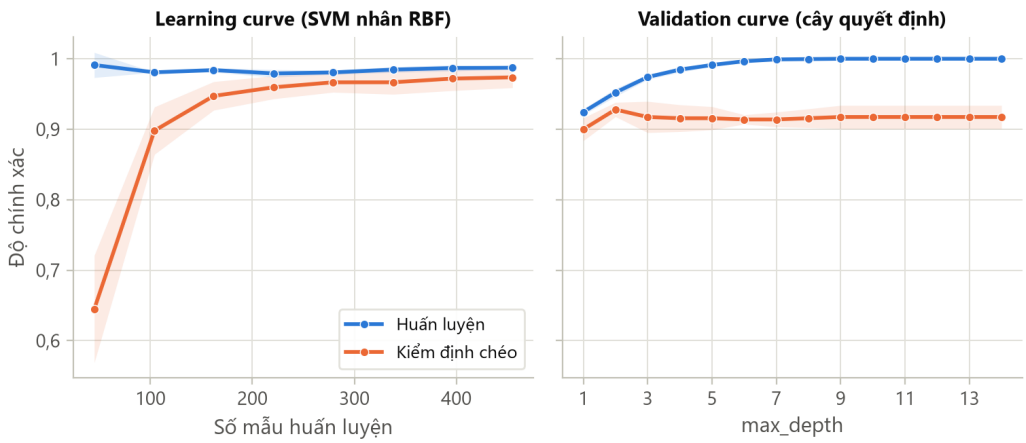
– **Learning curve**: điểm train và validation theo **kích thước tập huấn luyện**. Hai đường cách xa nhau là dấu hiệu quá khớp, thêm dữ liệu sẽ có ích; hai đường hội tụ ở mức thấp là dưới khớp.

– **Validation curve**: điểm theo **một siêu tham số**, cho thấy vùng dưới khớp – quá khớp.

Listing 8.8: Learning curve và validation curve

```
import numpy as np
from sklearn.model_selection import LearningCurveDisplay,
    ValidationCurveDisplay
from sklearn.tree import DecisionTreeClassifier

LearningCurveDisplay.from_estimator(pipe, X, y, cv=5,
                                    train_sizes=np.linspace(0.1, 1.0, 8))
ValidationCurveDisplay.from_estimator(DecisionTreeClassifier(random_state=0), X
    , y,
                                      param_name="max_depth",
                                      param_range=range(1, 15), cv=5)
```



Hình 8.5: Trái: learning curve; phải: validation curve theo `max_depth` của cây quyết định. Vùng tô là ± 1 độ lệch chuẩn giữa các fold.

8.6 Những sai lầm thường gặp

1. **Rò rỉ dữ liệu**: tiền xử lý, chọn đặc trưng hoặc cân bằng lớp (SMOTE) trên toàn bộ dữ liệu trước khi chia. Cách tránh: mọi bước đặt trong `Pipeline`.
2. **Đặc trưng chứa thông tin tương lai**: ví dụ dùng "ngày xuất viện" để dự đoán "có nhập viện không".
3. **Xáo trộn chuỗi thời gian** khi kiểm định chéo.
4. **Trùng lặp mẫu** giữa train và test (cùng một bệnh nhân, ảnh gần giống nhau).
5. **Nhìn tập test nhiều lần** để chỉnh mô hình.
6. **Chỉ báo accuracy** với dữ liệu mất cân bằng; không báo độ lệch chuẩn giữa các fold.

8.7 Tóm tắt chương

– Mục tiêu là lỗi tổng quát hóa; lỗi huấn luyện luôn lạc quan. Cân bằng bias và variance.

– Chia train / validation / test; tập test chỉ dùng một lần. Kiểm định chéo k-fold cho ước lượng ổn định hơn; chọn bộ chia phù hợp (stratified, group, time series).

– Phân lớp: ma trận nhầm lẫn, precision, recall, F1, ROC – AUC; hồi quy: MAE, MSE, RMSE, R^2.

– Chia đôi liên tiếp (successive halving) loại dần ứng viên kém khi tài nguyên còn nhỏ, rẻ hơn grid search nhiều lần.

– Tìm siêu tham số bằng Grid/Random search trên pipeline; dùng nested CV để có ước lượng khách quan.

8.8 Bài tập

1. Một bộ lọc thư rác trên 1000 email (100 thư rác) cho $TP = 80$, $FP = 30$. Lập ma trận nhầm lẫn và tính accuracy, precision, recall, F1, specificity.

2. Tính AUC bằng cách đếm cặp cho $y = (1, 0, 1, 0, 1)$, $s = (0{,}9;\ 0{,}8;\ 0{,}7;\ 0{,}3;\ 0{,}2)$, rồi kiểm tra bằng `roc_auc_score`.

3. Chứng minh rằng R^2 của mô hình luôn dự đoán $\bar{y}$ bằng 0. Khi nào $R^2 < 0$?

4. So sánh điểm accuracy của KNN ($K = 5$) trên tập Iris khi đánh giá bằng hold-out với 10 seed khác nhau và bằng 10-fold CV. Nhận xét về độ biến động.

5. Chứng minh bằng thực nghiệm hiện tượng rò rỉ dữ liệu: sinh dữ liệu ngẫu nhiên X (100 mẫu, 10 000 cột) và nhãn ngẫu nhiên; chọn 20 cột tương quan cao nhất với y *trên toàn bộ dữ liệu* rồi mới kiểm định chéo. So sánh với việc đặt `SelectKBest` trong pipeline.

6. Trên Ames Housing, dùng `GridSearchCV` để chọn `alpha` cho Ridge và Lasso (thang log), báo cáo RMSE kiểm định chéo và RMSE trên tập test.

Tài liệu tham khảo của chương

1. Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13, 281–305.

2. Cawley, G. C., & Talbot, N. L. C. (2010). On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation. *Journal of Machine Learning Research*, 11, 2079–2107.

3. Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8), 861–874.

4. Geisser, S. (1975). The Predictive Sample Reuse Method with Applications. *Journal of the American Statistical Association*, 70(350), 320–328.

5. Geman, S., Bienenstock, E., & Doursat, R. (1992). Neural Networks and the Bias/Variance Dilemma. *Neural Computation*, 4(1), 1–58.

6. Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1), 29–36.

7. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning* (2nd ed.), Chapter 7. Springer.

8. Jamieson, K., & Talwalkar, A. (2016). Non-stochastic Best Arm Identification and Hyperparameter Optimization. *Proceedings of AISTATS 2016*, 240–248.

9. Kohavi, R. (1995). A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection. *Proceedings of IJCAI '95*, 1137–1143.

10. Li, L., Jamieson, K., DeSalvo, G., Rostamizadeh, A., & Talwalkar, A. (2018). Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization. *Journal of Machine Learning Research*, 18(185), 1–52.

11. Liu, Y. (H.) (2019). *Python Machine Learning By Example* (2nd ed.), Chương 10: Machine Learning Best Practices. Packt Publishing.

12. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*, Chương 6: Learning Best Practices for Model Evaluation and Hyperparameter Tuning. Packt Publishing.

13. Saito, T., & Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLoS ONE*, 10(3), e0118432.

14. Sarkar, D., Bali, R., & Sharma, T. (2018). *Practical Machine Learning with Python*, Chương 5: Building, Tuning, and Deploying Models. Apress.

15. Stone, M. (1974). Cross-Validatory Choice and Assessment of Statistical Predictions. *Journal of the Royal Statistical Society: Series B*, 36(2), 111–147.

Chương 9: Học kết hợp (Ensemble Learning)

Một cây quyết định đơn lẻ dễ quá khớp và không ổn định; một mô hình tuyến tính có thể quá đơn giản. **Học kết hợp** (ensemble learning) xây dựng **nhiều mô hình** rồi gộp dự đoán của chúng, thường cho kết quả tốt hơn bất kỳ mô hình thành phần nào. Trong kế hoạch giảng dạy, các thuật toán **Random Forest, AdaBoost, XGBoost** thuộc nhóm phân lớp; chúng cũng dùng được cho hồi quy. Trên dữ liệu dạng bảng (tabular), các phương pháp boosting hiện vẫn là lựa chọn hàng đầu trong nhiều cuộc thi khoa học dữ liệu.
Sau khi học xong chương này, người học có khả năng:

- Giải thích vì sao kết hợp nhiều mô hình yếu lại cho mô hình mạnh (định lý bồi thẩm đoàn Condorcet, phân rã độ chệch – phương sai).
- Phân biệt hai chiến lược chính: **bagging** (song song, giảm phương sai) và **boosting** (tuần tự, giảm độ chệch).
- Tự tính một vòng cập nhật trọng số của AdaBoost.
- Cài đặt và tinh chỉnh Random Forest, AdaBoost, Gradient Boosting, XGBoost; đọc mức độ quan trọng của đặc trưng.

9.1 Trí tuệ đám đông

Năm 1785, Hầu tước de Condorcet phát biểu **định lý bồi thẩm đoàn**: nếu mỗi thành viên độc lập đưa ra quyết định đúng với xác suất $p > 0{,}5$, thì xác suất quyết định theo đa số là đúng sẽ tăng dần về 1 khi số thành viên tăng. Năm 1907, Francis Galton quan sát 787 người dự đoán cân nặng một con bò tại hội chợ: giá trị *trung vị* các dự đoán chỉ lệch khoảng 1% so với cân nặng thật — chính xác hơn hầu hết từng cá nhân.

Ví dụ 9.1. *Ba bộ phân lớp **độc lập**, mỗi bộ đúng với xác suất* $0{,}7$. *Bỏ phiếu đa số đúng khi có ít nhất 2 trong 3 bộ đúng:*

$$P(\text{đa số đúng}) = 0{,}7^3 + 3 \times 0{,}7^2 \times 0{,}3 = 0{,}343 + 0{,}441 = 0{,}784 > 0{,}7.$$

Điều kiện then chốt là các mô hình phải **đủ tốt** (tốt hơn đoán ngẫu nhiên) và **đa dạng** (sai ở những chỗ khác nhau). Nếu cả ba mô hình giống hệt nhau, bỏ phiếu chẳng cải thiện gì. Mọi phương pháp ensemble đều là các cách khác nhau để tạo ra sự đa dạng.

9.1.1 Độ chệch và phương sai

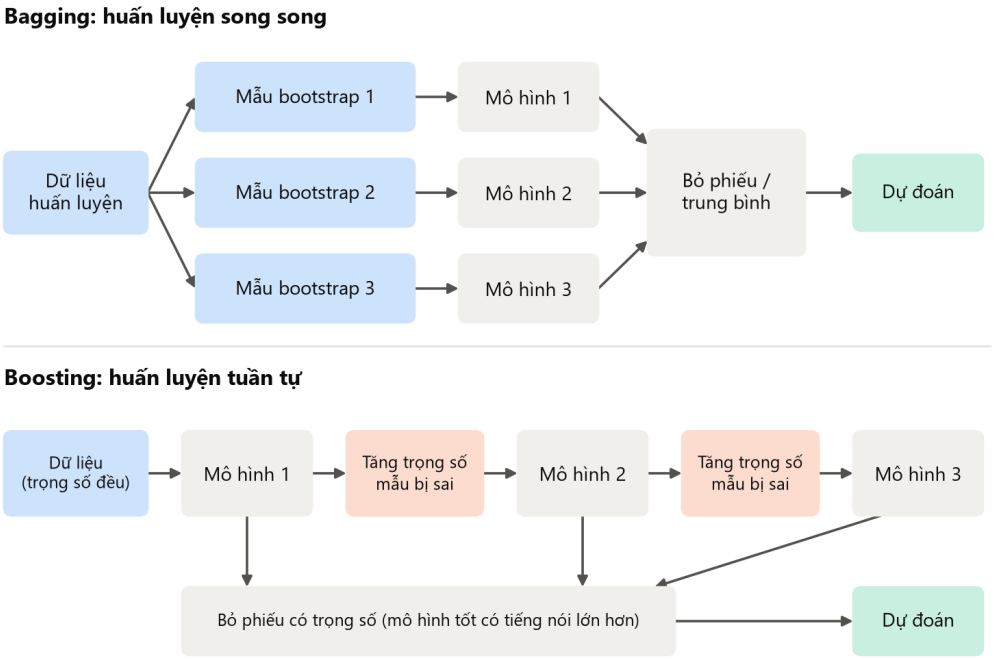
Nhắc lại phân rã độ chệch – phương sai ở Chương 8 (phương trình (8.2), Hình 8.1): sai số kỳ vọng của một mô hình trên dữ liệu mới bằng

$$\text{Sai số} = \text{Độ chệch}^2 + \text{Phương sai} + \text{Nhiễu không thể giảm}. \tag{9.1}$$

Độ chệch (bias) cao: mô hình quá đơn giản, thiếu khớp. **Phương sai** (variance) cao: mô hình thay đổi mạnh theo dữ liệu huấn luyện, quá khớp. Học kết hợp tác động trực tiếp lên hai thành phần này: bagging giảm phương sai, boosting giảm độ chệch. Lấy trung bình B mô hình có phương sai σ^2, tương quan đôi một ρ, phương sai của trung bình là

$$\rho\,\sigma^2 + \frac{1-\rho}{B}\,\sigma^2, \tag{9.2}$$

nên **giảm tương quan** ρ giữa các mô hình là chìa khóa — đây chính là ý tưởng của Random Forest.



Hình 9.1: So sánh hai chiến lược: bagging huấn luyện song song trên các mẫu bootstrap rồi bỏ phiếu; boosting huấn luyện tuần tự, mỗi mô hình tập trung vào lỗi của mô hình trước.

9.2 Bagging và Random Forest

9.2.1 Bagging

Bagging (Bootstrap AGGregatING) do Leo Breiman đề xuất năm 1996:

1. Tạo B **mẫu bootstrap**: mỗi mẫu gồm n điểm được rút **có hoàn lại** từ n điểm huấn luyện (kỹ thuật bootstrap của Efron, 1979).

2. Huấn luyện một mô hình (thường là cây quyết định sâu) trên mỗi mẫu.

3. Dự đoán bằng bỏ phiếu đa số (phân lớp) hoặc lấy trung bình (hồi quy).

Xác suất một điểm **không** được chọn vào một mẫu bootstrap là $(1-1/n)^n \approx e^{-1} \approx 0{,}368$. Như vậy mỗi cây chỉ thấy khoảng 63,2% dữ liệu; 36,8% còn lại gọi là **dữ liệu ngoài túi** (out-of-bag, OOB) và được dùng để ước lượng sai số miễn phí, không cần tập kiểm tra riêng.

9.2.2 Random Forest

Năm 2001, Breiman đề xuất **Random Forest**: bagging cây quyết định, và thêm một nguồn ngẫu nhiên — **tại mỗi nút**, chỉ xét một tập con ngẫu nhiên gồm m đặc trưng để chọn phép chia (thường $m = \sqrt{d}$ cho phân lớp). Nhờ vậy, dù có một đặc trưng rất mạnh, không phải cây nào cũng dùng nó ở gốc; các cây ít tương quan hơn và phương sai của rừng giảm mạnh.

Tham số	**Ý nghĩa**
`n_estimators`	Số cây; càng nhiều càng ổn định (không gây quá khớp), chỉ tốn thời gian.
`max_features`	Số đặc trưng xét ở mỗi nút (`"sqrt"`, `"log2"`, số thực); nhỏ ⇒ cây đa dạng hơn.
`max_depth`, `min_samples_leaf`	Giới hạn độ sâu và kích thước lá của từng cây.
`oob_score`	Tính độ chính xác trên dữ liệu ngoài túi.
`n_jobs=-1`	Huấn luyện song song trên mọi nhân CPU.

Bảng 9.1: Các tham số chính của `RandomForestClassifier`

Mức độ quan trọng của đặc trưng. Random Forest cung cấp `feature_importances_`: tổng mức giảm Gini do mỗi đặc trưng đóng góp, lấy trung bình qua các cây. Chỉ số này thiên vị đặc trưng có nhiều giá trị; **permutation importance** (xáo trộn một cột và đo mức giảm độ chính xác trên tập kiểm tra) đáng tin cậy hơn.

Listing 9.1: Random Forest trên dữ liệu ung thư vú Wisconsin

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
import pandas as pd

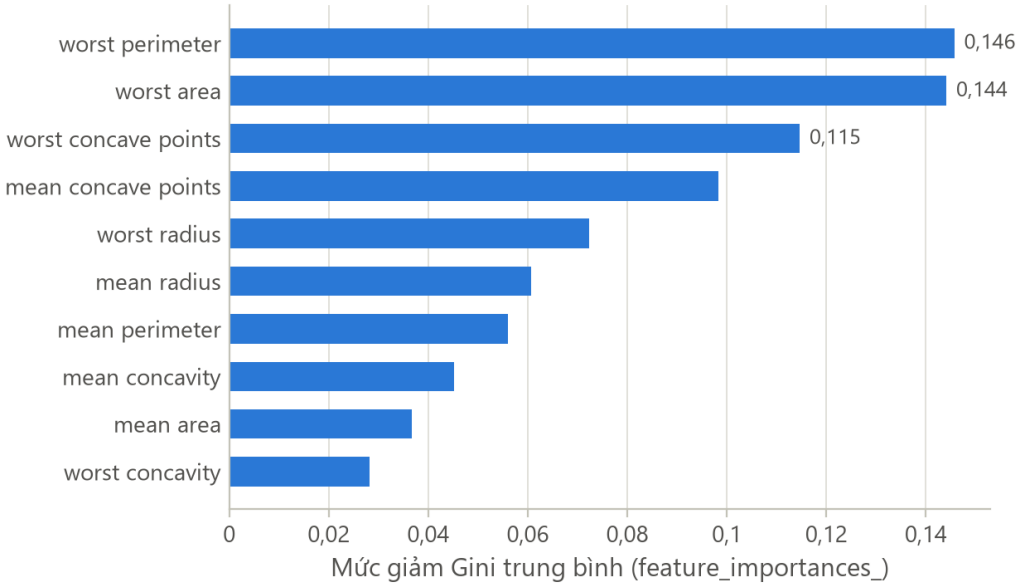
data = load_breast_cancer()
X, y = data.data, data.target              # 569 mau, 30 dac trung
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

rf = RandomForestClassifier(n_estimators=300, max_features="sqrt",
                            oob_score=True, n_jobs=-1, random_state=42)
rf.fit(X_train, y_train)
print("OOB score :", rf.oob_score_)
print("Test score:", rf.score(X_test, y_test))

imp = pd.Series(rf.feature_importances_, index=data.feature_names)
print(imp.sort_values(ascending=False).head(5))

```

```
perm = permutation_importance(rf, X_test, y_test, n_repeats=10, random_state=0)
print(pd.Series(perm.importances_mean, index=data.feature_names)
        .sort_values(ascending=False).head(5))
```



Hình 9.2: Mức độ quan trọng của 10 đặc trưng hàng đầu theo Random Forest trên dữ liệu ung thư vú.

9.3 AdaBoost

Năm 1988, Michael Kearns và Leslie Valiant đặt câu hỏi: liệu có thể "khuếch đại" một tập các bộ học **yếu** (chỉ tốt hơn đoán ngẫu nhiên một chút) thành một bộ học **mạnh**? Robert Schapire trả lời khẳng định năm 1990, và năm 1997 Yoav Freund cùng Schapire công bố **AdaBoost** (Adaptive Boosting) — công trình được trao giải Gödel năm 2003.

Định nghĩa 9.1 (Thuật toán AdaBoost nhị phân, $y_i \in \{-1, +1\}$)**.** *1. Khởi tạo trọng số các mẫu* $w_i = 1/n$.

2. *Với* $t = 1, \dots, T$:
   - (a) *Huấn luyện bộ học yếu* h_t *(thường là* ***gốc cây*** *– decision stump, cây độ sâu 1) trên dữ liệu có trọng số* w.
   - (b) *Tính sai số có trọng số* $\varepsilon_t = \sum_{i:\, h_t(\mathbf{x}_i) \neq y_i} w_i$.
   - (c) *Tính trọng số của bộ học:* $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$.
   - (d) *Cập nhật:* $w_i \leftarrow w_i \cdot e^{-\alpha_t y_i h_t(\mathbf{x}_i)}$*, rồi chuẩn hóa để* $\sum_i w_i = 1$.
3. *Dự đoán:* $H(\mathbf{x}) = \operatorname{sign}\Big(\sum_{t=1}^{T} \alpha_t h_t(\mathbf{x}) \Big)$.

Mẫu bị đoán sai được nhân với $e^{\alpha_t} > 1$ (tăng trọng số), mẫu đoán đúng nhân với $e^{-\alpha_t} < 1$ (giảm trọng số). Bộ học kế tiếp vì thế phải "chú ý" nhiều hơn đến các mẫu khó. Bộ học có sai số nhỏ có α_t lớn, tức tiếng nói lớn hơn trong lần bỏ phiếu cuối.

Ví dụ 9.2 (Một vòng AdaBoost)**.** *Có 10 mẫu, trọng số ban đầu* $0{,}1$*. Bộ học yếu đầu tiên đoán sai 3 mẫu:*

- $\varepsilon_1 = 3 \times 0{,}1 = 0{,}3$*;* $\alpha_1 = \frac{1}{2}\ln\frac{0{,}7}{0{,}3} = 0{,}4236$*.*
- *Mẫu sai:* $0{,}1 \times e^{0{,}4236} = 0{,}1 \times 1{,}5275 = 0{,}15275$*.*
- *Mẫu đúng:* $0{,}1 \times e^{-0{,}4236} = 0{,}1 \times 0{,}6547 = 0{,}06547$*.*
- *Hằng số chuẩn hóa:* $Z = 3 \times 0{,}15275 + 7 \times 0{,}06547 = 0{,}9165$*.*
- *Trọng số mới: mẫu sai* $0{,}15275/0{,}9165 = 0{,}1667$*; mẫu đúng* $0{,}06547/0{,}9165 = 0{,}0714$*.*

Tổng trọng số của các mẫu sai bây giờ là $3 \times 0{,}1667 = 0{,}5$*: đối với trọng số mới, bộ học vừa rồi chỉ còn ngang đoán ngẫu nhiên, buộc bộ học tiếp theo phải khác nó.*

Listing 9.2: AdaBoost với gốc cây

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1),   # decision stump
    n_estimators=200, learning_rate=0.5, random_state=42)
ada.fit(X_train, y_train)
print("Test score:", ada.score(X_test, y_test))
print(ada.estimator_weights_[:5].round(3))    # cac alpha_t dau tien
```

AdaBoost nhạy với **nhiễu nhãn và ngoại lai**: các mẫu bị gán nhãn sai liên tục được tăng trọng số và kéo mô hình lệch đi.

9.4 Gradient Boosting và XGBoost

9.4.1 Gradient Boosting

Năm 2001, Jerome Friedman tổng quát hóa boosting thành **Gradient Boosting**: xây dựng mô hình cộng $F_M(\mathbf{x}) = F_0 + \sum_{m=1}^{M} \eta\, h_m(\mathbf{x})$, trong đó mỗi cây mới h_m được huấn luyện để khớp với **gradient âm của hàm mất mát** (gọi là *giả phần dư*) tại mô hình hiện tại:

$$r_{im} = -\left[\frac{\partial L(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}. \tag{9.3}$$

Với mất mát bình phương $L = \frac{1}{2}(y-F)^2$, giả phần dư chính là **phần dư** $y_i - F_{m-1}(\mathbf{x}_i)$: mỗi cây mới học cách sửa sai số còn lại của các cây trước. Đây là Gradient Descent trong *không gian hàm*. Tốc độ học η (`learning_rate`, thường 0,01–0,1) càng nhỏ càng cần nhiều cây nhưng tổng quát càng tốt.

Ví dụ 9.3 (Ý tưởng phần dư)**.** *Cần dự đoán giá ba căn nhà (nghìn USD): 150, 200, 250. Mô hình khởi đầu* F_0 *= trung bình = 200, phần dư là* $(-50, 0, +50)$*. Cây thứ nhất học phần dư này, giả sử dự đoán được* $(-40, 0, +40)$*; với* $\eta = 0{,}5$*, mô hình mới là* $200 + 0{,}5 \times (-40, 0, 40) = (180, 200, 220)$*, phần dư còn* $(-30, 0, 30)$*. Cây thứ hai tiếp tục học phần dư mới, cứ thế sai số giảm dần.*

9.4.2 XGBoost

XGBoost (eXtreme Gradient Boosting) do Tianqi Chen và Carlos Guestrin công bố năm 2016, trở nên nổi tiếng nhờ thống trị các cuộc thi Kaggle. So với Gradient Boosting gốc, XGBoost:

- Thêm **chính quy hóa** vào hàm mục tiêu: $\mathcal{L} = \sum_i L(y_i, \hat{y}_i) + \sum_m \left(\gamma T_m + \frac{1}{2}\lambda \|\mathbf{w}_m\|^2\right)$, với T_m là số lá và $\mathbf{w}_m$ là giá trị các lá của cây m.
- Dùng **khai triển Taylor bậc hai** (gradient g_i và hessian h_i), cho công thức tối ưu tường minh của giá trị lá $w_j^* = -\frac{\sum_{i \in j} g_i}{\sum_{i \in j} h_i + \lambda}$ và độ lợi khi chia nút:

$$\text{Gain} = \frac{1}{2}\left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma. \tag{9.4}$$

- Tự xử lý **giá trị thiếu** (học hướng đi mặc định ở mỗi nút), lấy mẫu con theo hàng và theo cột, tính toán song song và tối ưu bộ nhớ đệm.

Các thư viện cùng họ: **LightGBM** (Microsoft, 2017) và **CatBoost** (Yandex, 2017, xử lý biến phân loại trực tiếp). scikit-learn có `HistGradientBoostingClassifier` với ý tưởng tương tự LightGBM.

Listing 9.3: Gradient Boosting và XGBoost

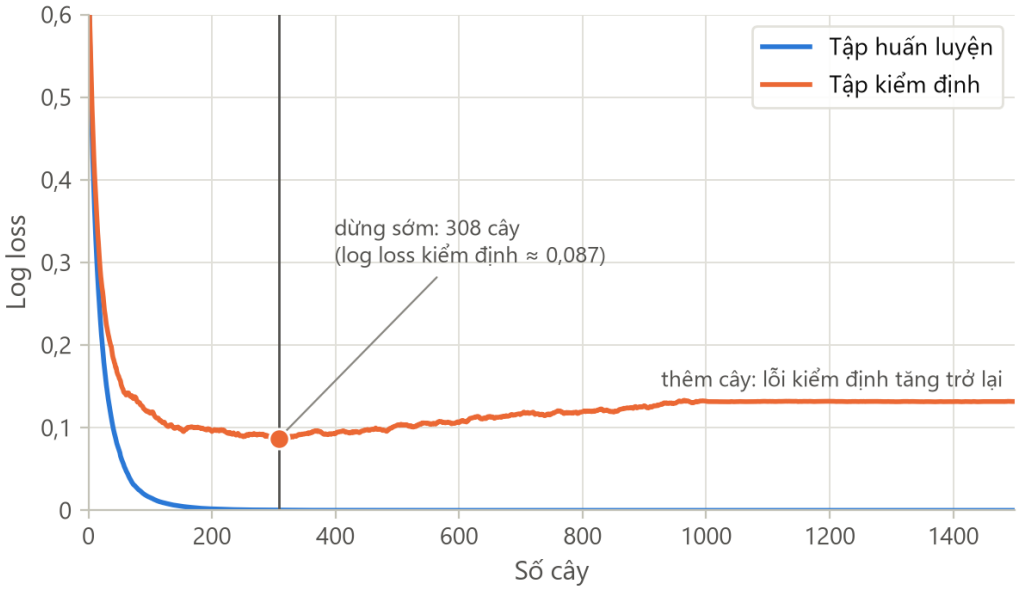
```
from sklearn.ensemble import GradientBoostingClassifier,
    HistGradientBoostingClassifier

gb = GradientBoostingClassifier(n_estimators=300, learning_rate=0.05,
                                max_depth=3, subsample=0.8, random_state=42)
gb.fit(X_train, y_train)
print("GradientBoosting:", gb.score(X_test, y_test))

hgb = HistGradientBoostingClassifier(max_iter=300, learning_rate=0.05)
print("HistGB:", hgb.fit(X_train, y_train).score(X_test, y_test))

# pip install xgboost
from xgboost import XGBClassifier
xgb = XGBClassifier(n_estimators=500, learning_rate=0.05, max_depth=4,
                    subsample=0.8, colsample_bytree=0.8,
                    reg_lambda=1.0, eval_metric="logloss",
                    early_stopping_rounds=30)
xgb.fit(X_train, y_train, eval_set=[(X_test, y_test)], verbose=False)
print("XGBoost:", xgb.score(X_test, y_test), "so cay:", xgb.best_iteration)
```

> **Lưu ý:** Ví dụ trên dùng tập kiểm tra để dừng sớm cho gọn. Trong thực tế cần tách thêm một **tập kiểm định** (validation) riêng; dùng tập kiểm tra để chọn số cây là một dạng rò rỉ dữ liệu (xem chương Đánh giá mô hình).



Hình 9.3: Sai số trên tập huấn luyện và tập kiểm định theo số cây của Gradient Boosting; điểm dừng sớm (early stopping) là nơi sai số kiểm định nhỏ nhất. Dùng cùng siêu tham số với đoạn code trên nhưng tăng lên 1500 cây; tập kiểm định được tách riêng từ tập huấn luyện.

9.5 Voting và Stacking

Ngoài việc dùng nhiều phiên bản của cùng một thuật toán, ta có thể kết hợp các thuật toán **khác loại**:

- **Voting**: bỏ phiếu cứng (đa số nhãn) hoặc mềm (trung bình xác suất) giữa các mô hình như Logistic Regression, SVM, Random Forest.
- **Stacking** (Wolpert, 1992): dùng dự đoán của các mô hình tầng 1 làm đầu vào cho một **mô hình tầng 2** (meta-learner); các dự đoán này phải lấy từ kiểm định chéo để tránh rò rỉ.

Listing 9.4: Voting và Stacking

```
from sklearn.ensemble import VotingClassifier, StackingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

base = [
    ("lr",  make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))),
    ("svm", make_pipeline(StandardScaler(), SVC(probability=True))),
    ("rf",  RandomForestClassifier(n_estimators=200, random_state=42)),
]
vote = VotingClassifier(base, voting="soft").fit(X_train, y_train)
stack = StackingClassifier(base, final_estimator=LogisticRegression(), cv=5)
stack.fit(X_train, y_train)
print("Voting:", vote.score(X_test, y_test), "Stacking:", stack.score(X_test,
    y_test))
```

9.6 Tổng kết so sánh

Phương pháp	Chiến lược	Giảm chủ yếu	Ưu điểm	Nhược điểm
Bagging / Random Forest	Song song	Phương sai	Ổn định, ít cần tinh chỉnh, có OOB	Mô hình lớn, khó diễn giải
AdaBoost	Tuần tự	Độ chệch	Đơn giản, lý thuyết đẹp	Nhạy nhiễu và ngoại lai
Gradient Boosting / XGBoost	Tuần tự	Độ chệch (và phương sai nhờ chính quy hóa)	Độ chính xác cao nhất trên dữ liệu bảng	Nhiều tham số, dễ quá khớp nếu không dừng sớm
Voting / Stacking	Kết hợp khác loại	Cả hai	Tận dụng thế mạnh từng mô hình	Tốn tài nguyên

Bảng 9.2: So sánh các phương pháp học kết hợp

9.7 Tóm tắt chương

- Ensemble dựa trên "trí tuệ đám đông": các mô hình tốt hơn ngẫu nhiên và đa dạng, khi kết hợp sẽ cho kết quả tốt hơn.
- Bagging huấn luyện trên các mẫu bootstrap, giảm phương sai; Random Forest thêm chọn đặc trưng ngẫu nhiên ở mỗi nút để giảm tương quan giữa các cây.
- AdaBoost tăng trọng số các mẫu bị đoán sai; $\alpha_t = \frac{1}{2} \ln \frac{1-\varepsilon_t}{\varepsilon_t}$.
- Gradient Boosting khớp từng cây với gradient âm của hàm mất mát; XGBoost bổ sung chính quy hóa, đạo hàm bậc hai và nhiều tối ưu kỹ thuật.

9.8 Bài tập

1. Tính xác suất bỏ phiếu đa số đúng với 5 bộ phân lớp độc lập, mỗi bộ đúng với xác suất 0,6. So sánh với trường hợp 1 bộ phân lớp.
2. Mô phỏng bằng Python: rút 1000 mẫu bootstrap từ $n = 100$ điểm và kiểm chứng tỉ lệ điểm ngoài túi xấp xỉ 36,8%.
3. Làm tiếp ví dụ AdaBoost: ở vòng 2, bộ học yếu đoán sai 2 mẫu vốn được đoán đúng ở vòng 1. Tính ε_2, α_2 và trọng số mới.
4. Trên dữ liệu ung thư vú, vẽ độ chính xác của Random Forest theo `n_estimators` $\in \{1, 5, 10, 50, 100, 300\}$ và theo `max_features`. Nhận xét.
5. So sánh Decision Tree, Random Forest, AdaBoost, Gradient Boosting và XGBoost trên Ames Housing (bài toán hồi quy, dùng các lớp `...Regressor`). Mô hình nào có RMSE thấp nhất?

Tài liệu tham khảo của chương

1. Breiman, L. (1996). Bagging Predictors. *Machine Learning*, 24(2), 123–140.
2. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
3. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of KDD '16*, 785–794.
4. Condorcet, M. de (1785). *Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix*. Paris.
5. Efron, B. (1979). Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*, 7(1), 1–26.
6. Freund, Y., & Schapire, R. E. (1997). A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
7. Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*, 29(5), 1189–1232.
8. Galton, F. (1907). Vox Populi. *Nature*, 75, 450–451.
9. Ke, G. et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems 30*.
10. Liu, Y. (H.) (2019). *Python Machine Learning By Example* (2nd ed.), Chương 6: Predicting Online Ad Click-Through with Tree-Based Algorithms. Packt Publishing.
11. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*, Chương 7: Combining Different Models for Ensemble Learning. Packt Publishing.
12. Schapire, R. E. (1990). The Strength of Weak Learnability. *Machine Learning*, 5(2), 197–227.
13. Wolpert, D. H. (1992). Stacked Generalization. *Neural Networks*, 5(2), 241–259.

Phần III

Học không giám sát

Chương 10: Giảm chiều dữ liệu (Dimensionality Reduction)

Dữ liệu thực tế thường có rất nhiều đặc trưng: một ảnh 28×28 có 784 chiều, một văn bản sau khi qua TF-IDF (Chương 5) có hàng chục nghìn chiều, dữ liệu gen có hàng triệu chiều. Số chiều lớn gây ra nhiều khó khăn mà Richard Bellman (1957) gọi chung là **lời nguyền số chiều** (curse of dimensionality). **Giảm chiều dữ liệu** là quá trình biểu diễn dữ liệu bằng ít chiều hơn nhưng giữ lại phần lớn thông tin.

Chương này mở đầu phần **học không giám sát** và đóng vai trò cầu nối với các chương trước. PCA tìm cấu trúc của ma trận $\mathbf{X}$ mà không cần nhãn. Ngược lại, LDA ở cuối chương dùng nhãn lớp giống như một bộ phân lớp (Chương 7). Số thành phần giữ lại được chọn bằng kiểm định chéo (Chương 8). PCA còn được dùng ngay ở Chương 11 để trực quan hóa các cụm trên mặt phẳng.

Theo kế hoạch giảng dạy, chương này tập trung vào hai kỹ thuật nền tảng: **SVD** (phân tích giá trị suy biến) và **PCA** (phân tích thành phần chính).
Sau khi học xong chương này, người học có khả năng:

- Giải thích lời nguyền số chiều và phân biệt lựa chọn đặc trưng với trích xuất đặc trưng.
- Tính tay phương sai, hiệp phương sai, hệ số tương quan và cặp trị riêng – vector riêng của ma trận 2×2.
- Tính tay SVD của ma trận nhỏ và xấp xỉ hạng thấp.
- Tính tay PCA (chuẩn hóa tâm, ma trận hiệp phương sai, trị riêng, chiếu dữ liệu).
- Dùng `PCA`, `TruncatedSVD` của scikit-learn và chọn số thành phần theo phương sai giải thích.
- Tính tay biệt thức tuyến tính Fisher (LDA), phân biệt giảm chiều có giám sát và không giám sát; sử dụng Kernel PCA và t-SNE đúng mục đích.

10.1 Lời nguyền số chiều

Khi số chiều d tăng:

- **Dữ liệu trở nên thưa**: để phủ đều khối lập phương đơn vị với mật độ 10 điểm mỗi trục cần 10^d điểm — 100 điểm cho $d = 2$, nhưng 10^{10} điểm cho $d = 10$.
- **Khoảng cách mất ý nghĩa**: trong không gian nhiều chiều, khoảng cách từ một điểm đến láng giềng gần nhất và xa nhất gần bằng nhau (Beyer và cộng sự, 1999), khiến các thuật toán dựa trên khoảng cách như KNN (Chương 7) và K-Means (Chương 11) kém hiệu quả.

– **Quá khớp**: số tham số tăng theo d, mô hình dễ học thuộc nhiễu khi số mẫu không đủ.

– **Chi phí** lưu trữ và tính toán tăng; không thể trực quan hóa trên 3 chiều.

Hướng tiếp cận	Ý tưởng	Ví dụ
Lựa chọn đặc trưng (feature selection)	Giữ lại một tập con các cột gốc	Lọc theo phương sai, tương quan, Lasso, `SelectKBest`
Trích xuất đặc trưng tuyến tính	Tạo cột mới là tổ hợp tuyến tính các cột gốc	**PCA**, **SVD**, LDA
Trích xuất phi tuyến (manifold learning)	Bảo toàn cấu trúc lân cận cục bộ	Kernel PCA, t-SNE, UMAP, Autoencoder

Bảng 10.1: Các hướng giảm chiều dữ liệu

10.2 Kiến thức nền: phương sai, hiệp phương sai và trị riêng

Cả SVD và PCA đều được xây dựng trên ba khái niệm: phương sai, hiệp phương sai và cặp trị riêng – vector riêng. Mục này ôn lại chúng qua các ví dụ tính tay.

10.2.1 Phương sai

Định nghĩa 10.1 (Phương sai)**.** *Phương sai (variance) đo mức độ phân tán của dữ liệu quanh giá trị trung bình:*

$$\mathrm{Var}(A) = \frac{1}{n}\sum_{i=1}^{n}(a_i - \bar{a})^2. \tag{10.1}$$

Khi ước lượng phương sai của tổng thể từ một mẫu, người ta thường chia cho $n-1$ thay vì n (hiệu chỉnh Bessel) để được ước lượng không chệch. Độ lệch chuẩn σ là căn bậc hai của phương sai.

Ví dụ 10.1 (Tính phương sai)**.** $A = (3, 7, 5)$ *có* $\bar{a} = \frac{3+7+5}{3} = 5$ *và*

$$\mathrm{Var}(A) = \frac{(3-5)^2 + (7-5)^2 + (5-5)^2}{3} = \frac{8}{3} \approx 2{,}67.$$

Nếu chia cho $n-1$ *thì được* $\frac{8}{2} = 4$.

Lưu ý khi lập trình: `np.var` mặc định chia cho n (`ddof=0`), còn `pandas.Series.var` và `np.cov` mặc định chia cho $n-1$ (`ddof=1`). Với mẫu nhỏ, hai quy ước cho kết quả khác nhau đáng kể, nên cần ghi rõ quy ước đang dùng.

10.2.2 Hiệp phương sai và hệ số tương quan

Định nghĩa 10.2 (Hiệp phương sai và hệ số tương quan)**.** *Hiệp phương sai (covariance) đo mức độ* ***cùng biến thiên*** *của hai biến:*

$$\mathrm{Cov}(A, B) = \frac{1}{n}\sum_{i=1}^{n}(a_i - \bar{a})(b_i - \bar{b}). \tag{10.2}$$

$\text{Cov} > 0$*: hai biến có xu hướng cùng tăng;* $\text{Cov} < 0$*: biến này tăng thì biến kia giảm;* $\text{Cov} \approx 0$*: không có quan hệ tuyến tính. Vì độ lớn của hiệp phương sai phụ thuộc đơn vị đo, Karl Pearson (1896) chuẩn hóa nó thành* ***hệ số tương quan***

$$r = \frac{\text{Cov}(A,B)}{\sigma_A\,\sigma_B} \in [-1,1]. \tag{10.3}$$

Ví dụ 10.2 (Tính hiệp phương sai và hệ số tương quan)**.** $A = (2,8,4)$ *và* $B = (6,1,5)$ *có* $\bar{a} = \frac{14}{3} \approx 4{,}67$*,* $\bar{b} = \frac{12}{3} = 4$*:*

$$\begin{aligned}\text{Cov}(A,B) &= \tfrac{1}{3}\big[(2-4{,}67)(6-4) + (8-4{,}67)(1-4) + (4-4{,}67)(5-4)\big] \\ &= \tfrac{1}{3}\,(-5{,}33 - 10 - 0{,}67) = -\tfrac{16}{3} \approx -5{,}33.\end{aligned}$$

Dấu ***âm*** *cho biết khi* A *tăng thì* B *giảm: quan sát thứ hai có* A *lớn nhất nhưng* B *nhỏ nhất. Với* $\sigma_A^2 = \frac{56}{9} \approx 6{,}22$ *và* $\sigma_B^2 = \frac{14}{3} \approx 4{,}67$*:*

$$r = \frac{-5{,}33}{\sqrt{6{,}22 \times 4{,}67}} \approx -0{,}99,$$

tức hai biến có quan hệ tuyến tính nghịch rất mạnh. Khi tính tay cần giữ đúng ***dấu*** *của từng tích* $(a_i - \bar{a})(b_i - \bar{b})$*: bỏ sót dấu âm sẽ đảo ngược hoàn toàn ý nghĩa của kết quả.*

Ma trận hiệp phương sai. Với dữ liệu d chiều, các phương sai và hiệp phương sai được gom thành ma trận đối xứng $\mathbf{C} \in \mathbb{R}^{d\times d}$ với $C_{jk} = \text{Cov}(x_j, x_k)$; đường chéo chính là phương sai của từng biến. Nếu $\mathbf{X}_c$ là dữ liệu đã trừ trung bình từng cột thì $\mathbf{C} = \frac{1}{n-1}\mathbf{X}_c^\top \mathbf{X}_c$ — quy ước chia $n-1$ mà PCA của scikit-learn sử dụng.

10.2.3 Trị riêng và vector riêng

Định nghĩa 10.3 (Trị riêng, vector riêng)**.** *Cho ma trận vuông* $\mathbf{A}$*. Số* λ *và vector* $\mathbf{v} \neq \mathbf{0}$ *thỏa* $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ *được gọi là* ***trị riêng*** *và* ***vector riêng*** *tương ứng: nhân với* $\mathbf{A}$ *chỉ co giãn* $\mathbf{v}$ *theo hệ số* λ *mà không làm đổi phương của nó. Các trị riêng là nghiệm của* ***phương trình đặc trưng*** $\det(\mathbf{A} - \lambda\mathbf{I}) = 0$*.*

Ví dụ 10.3 (Tính tay trị riêng và vector riêng)**.** *Cho* $\mathbf{A} = \begin{pmatrix} 2 & 3 \\ 4 & 1 \end{pmatrix}$.

Bước 1 – phương trình đặc trưng:

$$\det\begin{pmatrix} 2-\lambda & 3 \\ 4 & 1-\lambda \end{pmatrix} = (2-\lambda)(1-\lambda) - 3\cdot 4 = \lambda^2 - 3\lambda - 10 = 0 \;\Leftrightarrow\; \lambda_1 = 5,\ \lambda_2 = -2.$$

Bước 2 – vector riêng ứng với $\lambda_1 = 5$**:** giải $(\mathbf{A} - 5\mathbf{I})\mathbf{v} = \mathbf{0}$,

$$\begin{pmatrix} -3 & 3 \\ 4 & -4 \end{pmatrix}\begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \mathbf{0} \;\Leftrightarrow\; -v_1 + v_2 = 0 \;\Leftrightarrow\; v_1 = v_2.$$

Chọn $v_2 = 1$ thì $v_1 = 1$; chuẩn hóa về độ dài 1: $\mathbf{v}_1 = \frac{1}{\sqrt{1^2+1^2}}(1,1) \approx (0{,}7071;\ 0{,}7071)$.
Bước 3 – vector riêng ứng với $\lambda_2 = -2$**:**

$$\begin{pmatrix} 4 & 3 \\ 4 & 3 \end{pmatrix}\begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \mathbf{0} \;\Leftrightarrow\; 4v_1 + 3v_2 = 0 \;\Leftrightarrow\; v_1 = -\tfrac{3}{4}v_2.$$

Chọn $v_2 = 4$ thì $v_1 = -3$; chuẩn hóa: $\mathbf{v}_2 = \frac{1}{\sqrt{(-3)^2+4^2}}(-3, 4) = (-0{,}6;\ 0{,}8)$.

Kiểm tra nhanh: tổng các trị riêng bằng **vết** (tổng đường chéo) của ma trận, $5 + (-2) = 2 + 1$; tích các trị riêng bằng **định thức**, $5 \times (-2) = 2 \cdot 1 - 3 \cdot 4 = -10$.

Ma trận $\mathbf{A}$ ở ví dụ này **không đối xứng** nên hai vector riêng không vuông góc ($\mathbf{v}_1 \cdot \mathbf{v}_2 \approx 0{,}14 \neq 0$). Ngược lại, với ma trận **đối xứng** như $\mathbf{X}^\top\mathbf{X}$ hay ma trận hiệp phương sai, **định lý phổ** bảo đảm mọi trị riêng là số thực và có thể chọn các vector riêng đôi một vuông góc — đây chính là tính chất mà SVD và PCA khai thác.

Listing 10.1: Phương sai, hiệp phương sai và trị riêng với NumPy

```
import numpy as np

A = np.array([3, 7, 5])
print(np.var(A), np.var(A, ddof=1))        # 2.6667 4.0
a, b = np.array([2, 8, 4]), np.array([6, 1, 5])
print(np.cov(a, b, ddof=0)[0, 1])          # -5.3333
print(np.corrcoef(a, b)[0, 1])             # -0.9897

M = np.array([[2, 3], [4, 1]])
eigval, eigvec = np.linalg.eig(M)   # NumPy moi co the tra ve kieu so phuc (5.+0.j)
print(eigval.real)     # [ 5. -2.]
print(eigvec.real)     # moi cot la mot vector rieng: (0.7071, 0.7071) va (-0.6, 0.8)
```

10.3 Phân tích giá trị suy biến (SVD)

10.3.1 Lịch sử

SVD được Eugenio Beltrami (1873) và Camille Jordan (1874) phát hiện độc lập; Eckart và Young (1936) chứng minh tính chất xấp xỉ hạng thấp tối ưu — nền tảng của mọi ứng dụng nén dữ liệu. Thuật toán tính SVD ổn định số do Golub và Kahan (1965) đề xuất là cái mà NumPy dùng ngày nay. SVD còn nổi tiếng nhờ cuộc thi Netflix Prize (2006–2009), nơi các biến thể phân rã ma trận chiếm ưu thế trong hệ gợi ý.

Định lý 10.1 (Phân tích giá trị suy biến)**.** *Mọi ma trận thực $\mathbf{A} \in \mathbb{R}^{m\times n}$ đều phân tích được thành*

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top, \tag{10.4}$$

*trong đó $\mathbf{U} \in \mathbb{R}^{m\times m}$ và $\mathbf{V} \in \mathbb{R}^{n\times n}$ là các ma trận trực giao (cột là vector suy biến trái/phải), $\mathbf{\Sigma} \in \mathbb{R}^{m\times n}$ là ma trận đường chéo với các **giá trị suy biến** $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. Ngoài ra σ_i^2 là trị riêng của $\mathbf{A}\mathbf{A}^\top$ (và của $\mathbf{A}^\top\mathbf{A}$), cột của $\mathbf{U}$ là vector riêng của $\mathbf{A}\mathbf{A}^\top$, cột của $\mathbf{V}$ là vector riêng của $\mathbf{A}^\top\mathbf{A}$.*

Định lý 10.2 (Eckart–Young)**.** *Giữ lại k giá trị suy biến lớn nhất, $\mathbf{A}_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$ là ma trận hạng k **gần $\mathbf{A}$ nhất** theo chuẩn Frobenius, với sai số $\|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sigma_{k+1}^2 + \cdots + \sigma_r^2}$.*

Ví dụ 10.4 (Tính tay SVD)**.** *Cho* $\mathbf{M} = \begin{pmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{pmatrix}$.

Bước 1. $\mathbf{M}\mathbf{M}^\top = \begin{pmatrix} 17 & 8 \\ 8 & 17 \end{pmatrix}$, phương trình đặc trưng $(17-\lambda)^2 - 64 = 0$ cho $\lambda_1 = 25$, $\lambda_2 = 9$. Vậy $\sigma_1 = 5$, $\sigma_2 = 3$.
Bước 2. Vector riêng của $\mathbf{M}\mathbf{M}^\top$: $\mathbf{u}_1 = \frac{1}{\sqrt{2}}(1,1)^\top$, $\mathbf{u}_2 = \frac{1}{\sqrt{2}}(1,-1)^\top$.
Bước 3. $\mathbf{v}_i = \mathbf{M}^\top \mathbf{u}_i / \sigma_i$:

$$\mathbf{v}_1 = \tfrac{1}{5\sqrt{2}}(5,5,0)^\top = \tfrac{1}{\sqrt{2}}(1,1,0)^\top, \qquad \mathbf{v}_2 = \tfrac{1}{3\sqrt{2}}(1,-1,4)^\top = \tfrac{1}{\sqrt{18}}(1,-1,4)^\top.$$

Vector thứ ba $\mathbf{v}_3 = \frac{1}{3}(2,-2,-1)^\top$ vuông góc với hai vector trên (ứng với giá trị suy biến 0).
Bước 4 – xấp xỉ hạng 1. $\mathbf{M}_1 = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top = 5 \cdot \frac{1}{2}\begin{pmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 2{,}5 & 2{,}5 & 0 \\ 2{,}5 & 2{,}5 & 0 \end{pmatrix}$, sai số $\|\mathbf{M} - \mathbf{M}_1\|_F = \sigma_2 = 3$. Thành phần thứ nhất giữ $\sigma_1^2/(\sigma_1^2+\sigma_2^2) = 25/34 \approx 73{,}5\%$ "năng lượng" của ma trận.

Listing 10.2: SVD với NumPy (ví dụ trên lớp)

```
import numpy as np

M = np.array([[3, 2, 2], [2, 3, -2]])
U, s, Vt = np.linalg.svd(M)
print(s)          # [5. 3.]
print(U)          # cot = vector suy bien trai (co the doi dau)
print(Vt)         # dong = vector suy bien phai

# Tai tao lai M tu 3 thanh phan
Sigma = np.zeros(M.shape); Sigma[:2, :2] = np.diag(s)
print(np.allclose(U @ Sigma @ Vt, M))   # True

# Xap xi hang 1
M1 = s[0] * np.outer(U[:, 0], Vt[0])
print(M1.round(2))                       # [[2.5 2.5 0.] [2.5 2.5 0.]]
print(np.linalg.norm(M - M1))            # 3.0 = sigma_2
```

Lưu ý: vector suy biến chỉ xác định duy nhất đến **dấu**: NumPy có thể trả về $-\mathbf{u}_1, -\mathbf{v}_1$, kết quả vẫn đúng.

10.3.2 Tính SVD thông qua $\mathbf{X}^\top\mathbf{X}$

Định lý trên gợi ý cách tính SVD bằng tay cho ma trận có nhiều dòng: (1) lập $\mathbf{X}^\top\mathbf{X}$ — ma trận vuông, đối xứng, kích thước bằng số cột; (2) tìm các trị riêng λ_i và vector riêng đơn vị $\mathbf{v}_i$ của nó; (3) giá trị suy biến $\sigma_i = \sqrt{\lambda_i}$; (4) vector suy biến trái $\mathbf{u}_i = \mathbf{X}\mathbf{v}_i/\sigma_i$.

Ví dụ 10.5 (SVD của ma trận 5×2)**.** *Cho* $\mathbf{X} = \begin{pmatrix} 3 & 7 \\ 1 & 5 \\ 6 & 4 \\ 8 & 1 \\ 2 & 3 \end{pmatrix}$.

Bước 1 – lập $\mathbf{X}^\top\mathbf{X}$:

$$\mathbf{X}^\top\mathbf{X} = \begin{pmatrix} 3 & 1 & 6 & 8 & 2 \\ 7 & 5 & 4 & 1 & 3 \end{pmatrix} \begin{pmatrix} 3 & 7 \\ 1 & 5 \\ 6 & 4 \\ 8 & 1 \\ 2 & 3 \end{pmatrix} = \begin{pmatrix} 114 & 64 \\ 64 & 100 \end{pmatrix},$$

trong đó $114 = 3^2 + 1^2 + 6^2 + 8^2 + 2^2$ và $64 = 3 \cdot 7 + 1 \cdot 5 + 6 \cdot 4 + 8 \cdot 1 + 2 \cdot 3$.
Bước 2 – trị riêng:

$$(114 - \lambda)(100 - \lambda) - 64^2 = \lambda^2 - 214\lambda + 7304 = 0 \;\Leftrightarrow\; \lambda_1 \approx 171{,}38,\ \lambda_2 \approx 42{,}62.$$

Bước 3 – vector riêng. Với λ_1: $(114 - 171{,}38)\,v_1 + 64\,v_2 = 0 \Leftrightarrow v_1 = 1{,}1153\,v_2$. Chọn $v_2 = 1$ được $(1{,}1153;\ 1)$, chuẩn hóa cho $\mathbf{v}_1 = (1{,}1153;\ 1)/1{,}4980 \approx (0{,}7446;\ 0{,}6676)$. Với λ_2: $(114 - 42{,}62)\,v_1 + 64\,v_2 = 0 \Leftrightarrow v_1 = -0{,}8966\,v_2$. Chọn $v_2 = 1$ được $(-0{,}8966;\ 1)$, chuẩn hóa cho $\mathbf{v}_2 \approx (-0{,}6676;\ 0{,}7446)$. Hai vector vuông góc: $0{,}7446 \times (-0{,}6676) + 0{,}6676 \times 0{,}7446 = 0$.
Bước 4 – giá trị và vector suy biến: $\sigma_1 = \sqrt{171{,}38} \approx 13{,}09$, $\sigma_2 = \sqrt{42{,}62} \approx 6{,}53$, và $\mathbf{u}_1 = \mathbf{X}\mathbf{v}_1/\sigma_1 \approx (0{,}528;\ 0{,}312;\ 0{,}545;\ 0{,}506;\ 0{,}267)$.

Sai lầm thường gặp khi chuẩn hóa vector riêng. Từ $v_1 = -0{,}8966\,v_2$, nếu chọn $v_2 = 1$ thì vector là $(v_1, v_2) = (-0{,}8966;\ 1)$ — **không phải** $(1;\ -0{,}8966)$. Đảo chỗ hai thành phần cho ra một vector khác hẳn, không còn là vector riêng. Cách tự kiểm tra đơn giản nhất là nhân lại: $\mathbf{X}^\top\mathbf{X}\,\mathbf{v}$ phải bằng $\lambda\mathbf{v}$, và vector sau khi chuẩn hóa phải có độ dài đúng bằng 1.

Listing 10.3: Kiểm tra cách tính SVD qua trị riêng

```
X = np.array([[3, 7], [1, 5], [6, 4], [8, 1], [2, 3]])
lam, V = np.linalg.eigh(X.T @ X)     # tri rieng tang dan: [ 42.6183 171.3817]
print(np.sqrt(lam[::-1]))            # [13.0913  6.5283]
U, s, Vt = np.linalg.svd(X, full_matrices=False)
print(s)                             # [13.0913  6.5283] -- trung khop
print(Vt)                            # moi dong la mot v_i (co the doi dau)
```

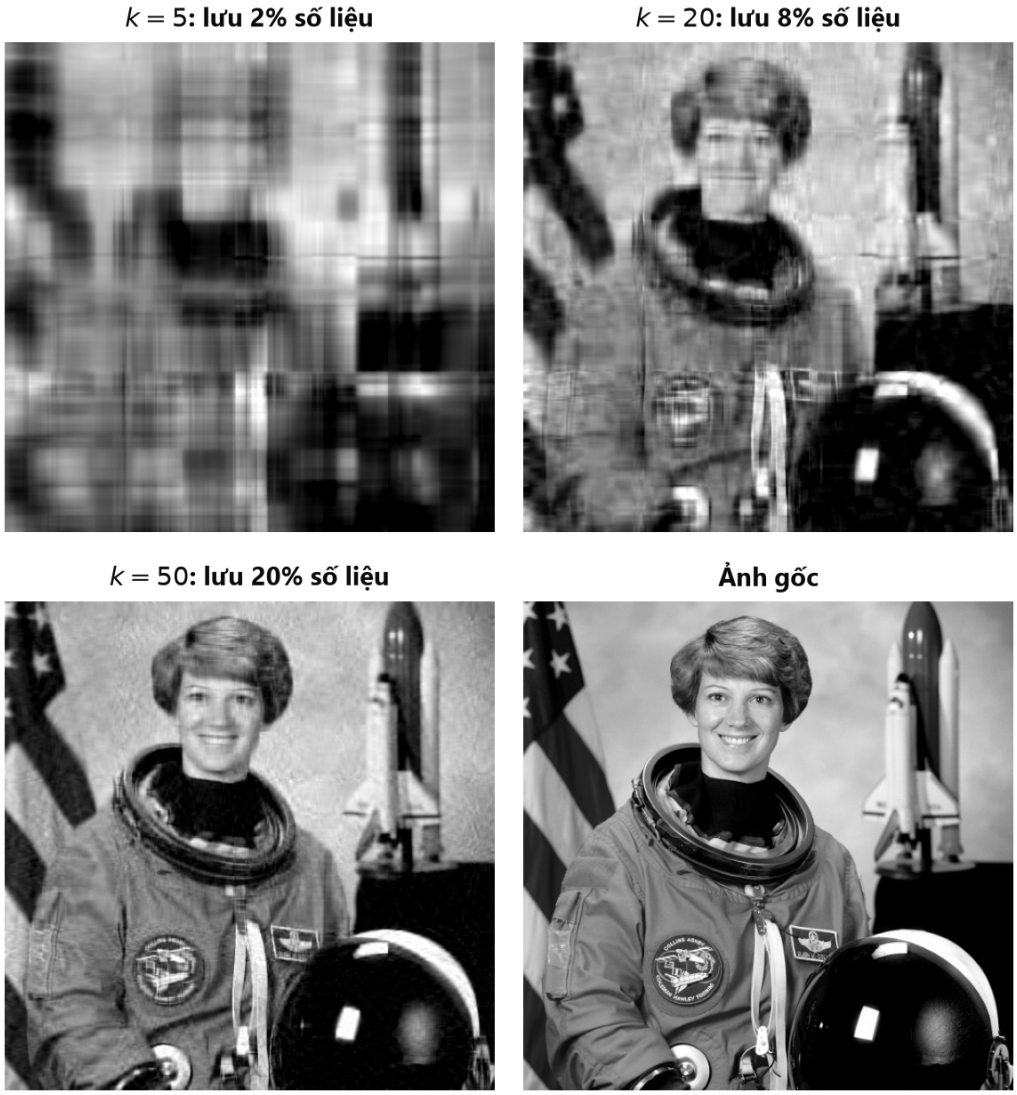
10.3.3 Ứng dụng: nén ảnh và TruncatedSVD

Một ảnh xám là ma trận $m \times n$. Lưu $\mathbf{A}_k$ chỉ cần $k(m+n+1)$ số thay vì mn. Với ảnh 512×512 và $k = 50$: $50 \times 1025 = 51\,250$ số so với $262\,144$ — tiết kiệm khoảng 80%.

Listing 10.4: Nén ảnh bằng SVD

```
import matplotlib.pyplot as plt
from skimage import data, color

img = color.rgb2gray(data.astronaut())       # anh xam 512 x 512
U, s, Vt = np.linalg.svd(img, full_matrices=False)
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, k in zip(axes, [5, 20, 50, 512]):
    ax.imshow(U[:, :k] @ np.diag(s[:k]) @ Vt[:k], cmap="gray")
    ax.set_title(f"k = {k}"); ax.axis("off")
plt.show()
```



Hình 10.1: Ảnh tái tạo từ $k = 5, 20, 50$ giá trị suy biến lớn nhất so với ảnh gốc (ảnh mẫu `astronaut` của scikit-image, 512×512 điểm ảnh).

Với ma trận thưa như kết quả TF-IDF, scikit-learn cung cấp `TruncatedSVD`: tính trực tiếp k thành phần đầu mà không cần trừ trung bình (trừ trung bình sẽ phá vỡ tính thưa). Áp dụng lên ma trận văn bản – từ, kỹ thuật này được gọi là **Phân tích ngữ nghĩa tiềm ẩn** (Latent Semantic Analysis – LSA, Deerwester và cộng sự, 1990): mỗi thành phần tương ứng một "chủ đề".

Listing 10.5: LSA: TF-IDF + TruncatedSVD

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD

docs = ["python machine learning", "python data analysis",
        "deep learning neural network", "neural network training",
        "data analysis with pandas"]
X = TfidfVectorizer().fit_transform(docs)          # ma tran thua 5 x V
lsa = TruncatedSVD(n_components=2, random_state=0)
Z = lsa.fit_transform(X)                           # 5 x 2
print(lsa.explained_variance_ratio_)
print(Z.round(3))   # van ban cung chu de nam gan nhau
```

10.4 Phân tích thành phần chính (PCA)

10.4.1 Lịch sử và ý tưởng

PCA do Karl Pearson phát minh năm 1901 dưới dạng bài toán "tìm đường thẳng và mặt phẳng khớp nhất với hệ điểm", và được Harold Hotelling (1933) phát triển độc lập, đặt tên "thành phần chính". Trong xử lý tín hiệu, PCA còn được gọi là biến đổi Karhunen–Loève.

Định nghĩa 10.4 (Thành phần chính)**.** *Cho dữ liệu đã chuẩn hóa tâm* $\mathbf{X}_c$ *(*$n \times d$*).* ***Thành phần chính thứ nhất*** *là vector đơn vị* $\mathbf{w}_1$ *sao cho phương sai của dữ liệu chiếu lên* $\mathbf{w}_1$ *là* ***lớn nhất****:*

$$\mathbf{w}_1 = \arg\max_{\|\mathbf{w}\|=1} \mathbf{w}^\top \mathbf{C}\mathbf{w}, \qquad \mathbf{C} = \tfrac{1}{n-1}\mathbf{X}_c^\top \mathbf{X}_c. \tag{10.5}$$

Các thành phần tiếp theo cũng cực đại phương sai nhưng phải ***vuông góc*** *với các thành phần trước.*

Giải bài toán tối ưu bằng nhân tử Lagrange cho $\mathbf{C}\mathbf{w} = \lambda\mathbf{w}$: các thành phần chính là **vector riêng** của ma trận hiệp phương sai, và phương sai theo thành phần thứ j chính là **trị riêng** λ_j. Tương đương, cực đại phương sai cũng là cực tiểu sai số tái tạo (khoảng cách bình phương từ điểm đến không gian con) — đúng cách nhìn ban đầu của Pearson.

Quan hệ PCA – SVD. Nếu $\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ thì $\mathbf{C} = \mathbf{V}\frac{\mathbf{\Sigma}^2}{n-1}\mathbf{V}^\top$: thành phần chính là cột của $\mathbf{V}$, trị riêng $\lambda_j = \sigma_j^2/(n-1)$. scikit-learn tính PCA bằng SVD vì ổn định số hơn việc lập ma trận hiệp phương sai.

Các bước PCA:

1. Chuẩn hóa tâm (trừ trung bình mỗi cột); nếu các cột khác đơn vị đo thì chuẩn hóa thêm bằng `StandardScaler`.
2. Tính ma trận hiệp phương sai $\mathbf{C}$.
3. Tìm trị riêng, vector riêng; sắp xếp trị riêng giảm dần.
4. Chọn k vector riêng đầu làm ma trận $\mathbf{W}_k$ ($d \times k$).
5. Chiếu dữ liệu: $\mathbf{Z} = \mathbf{X}_c\mathbf{W}_k$ ($n \times k$).

Ví dụ 10.6 (Tính tay PCA)**.** *Giảm 5 điểm hai chiều trong Bảng 10.2 xuống còn một chiều.*

x_1	3	5	0	1	2
x_2	6	4	2	0	3

Bảng 10.2: Dữ liệu ví dụ PCA

Bước 1 – chuẩn hóa tâm. $\bar{x}_1 = \frac{3+5+0+1+2}{5} = 2{,}2$ và $\bar{x}_2 = \frac{6+4+2+0+3}{5} = 3$. Trừ trung bình từng cột, $x'_{ij} = x_{ij} - \bar{x}_j$:

$$\mathbf{X}_c = \begin{pmatrix} 0{,}8 & 3 \\ 2{,}8 & 1 \\ -2{,}2 & -1 \\ -1{,}2 & -3 \\ -0{,}2 & 0 \end{pmatrix}.$$

Bước 2 – ma trận hiệp phương sai:

$$\mathbf{X}_c^\top \mathbf{X}_c = \begin{pmatrix} 14{,}8 & 11 \\ 11 & 20 \end{pmatrix}, \qquad \mathbf{C} = \frac{1}{n-1}\mathbf{X}_c^\top \mathbf{X}_c = \frac{1}{4}\begin{pmatrix} 14{,}8 & 11 \\ 11 & 20 \end{pmatrix} = \begin{pmatrix} 3{,}7 & 2{,}75 \\ 2{,}75 & 5 \end{pmatrix}.$$

Hiệp phương sai dương $(2{,}75)$ cho thấy hai biến có xu hướng tăng cùng nhau.
Bước 3 – trị riêng:

$$(3{,}7-\lambda)(5-\lambda) - 2{,}75^2 = \lambda^2 - 8{,}7\lambda + 10{,}9375 = 0 \;\Leftrightarrow\; \lambda_1 \approx 7{,}1758,\ \lambda_2 \approx 1{,}5242.$$

Bước 4 – vector riêng. Với $\lambda_1 = 7{,}1758$:

$$\begin{pmatrix} -3{,}4758 & 2{,}75 \\ 2{,}75 & -2{,}1758 \end{pmatrix}\begin{pmatrix} v_1 \\ v_2 \end{pmatrix} = \mathbf{0} \;\Leftrightarrow\; v_1 = 0{,}7912\, v_2.$$

Chọn $v_2 = 1$ được $(v_1, v_2) = (0{,}7912;\ 1)$; chuẩn hóa: $\mathbf{w}_1 = \dfrac{(0{,}7912;\ 1)}{\sqrt{0{,}7912^2 + 1^2}} = \dfrac{(0{,}7912;\ 1)}{1{,}2751} \approx (0{,}6205;\ 0{,}7842)$.

Với $\lambda_2 = 1{,}5242$: $2{,}1758\, v_1 + 2{,}75\, v_2 = 0 \Leftrightarrow v_1 = -1{,}2639\, v_2$; chọn $v_2 = 1$ được $(-1{,}2639;\ 1)$, chuẩn hóa cho $\mathbf{w}_2 \approx (-0{,}7842;\ 0{,}6205)$.

Kiểm tra: $\mathbf{C}\mathbf{w}_1 = (3{,}7 \times 0{,}6205 + 2{,}75 \times 0{,}7842;\ \ 2{,}75 \times 0{,}6205 + 5 \times 0{,}7842) \approx (4{,}452;\ 5{,}627) \approx 7{,}1758 \times (0{,}6205;\ 0{,}7842)$.
Bước 5 – tỉ lệ phương sai giải thích: $\frac{7{,}1758}{7{,}1758+1{,}5242} = \frac{7{,}1758}{8{,}7} \approx 82{,}5\%$; thành phần thứ nhất giữ khoảng 82,5% thông tin.
Bước 6 – chiếu dữ liệu $\mathbf{z} = \mathbf{X}_c\,\mathbf{w}_1$. Chẳng hạn với điểm đầu tiên: $z_1 = 0{,}8 \times 0{,}6205 + 3 \times 0{,}7842 \approx 2{,}849$. Toàn bộ:

$$\mathbf{z} \approx (2{,}849;\ 2{,}522;\ -2{,}149;\ -3{,}097;\ -0{,}124).$$

Mỗi điểm hai chiều giờ chỉ còn một con số; có thể tái tạo gần đúng điểm gốc bằng $\hat{\mathbf{x}} = \bar{\mathbf{x}} + z\,\mathbf{w}_1$.

Sai lầm thường gặp. (1) Từ $v_1 = 0{,}7912\, v_2$ mà viết vector thành $(1;\ 0{,}7912)$ là **đảo nhầm thành phần**: vector chuẩn hóa thu được, $(0{,}7842;\ 0{,}6205)$, không còn là vector riêng của $\mathbf{C}$. Hãy luôn kiểm tra lại $\mathbf{C}\mathbf{w} = \lambda\mathbf{w}$ như ở bước 4. (2) Quên chuẩn hóa tâm trước khi lập ma trận hiệp phương sai: khi đó ta đang tính SVD của dữ liệu gốc (như ví dụ 5×2 ở mục SVD) chứ không phải PCA, và thành phần chính đầu tiên bị kéo lệch về phía vector trung bình của dữ liệu.

Listing 10.6: Kiểm tra ví dụ bằng NumPy và scikit-learn

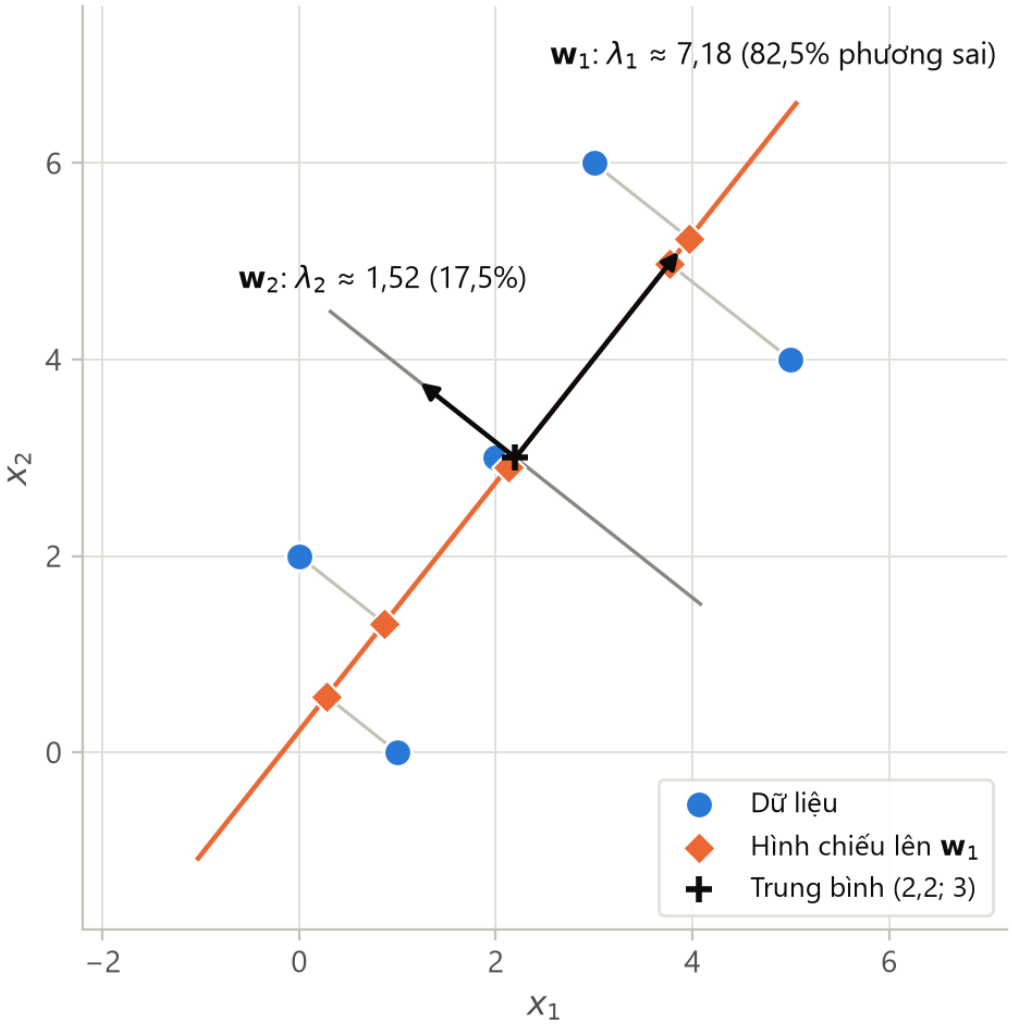
```
import numpy as np
from sklearn.decomposition import PCA

X = np.array([[3, 6], [5, 4], [0, 2], [1, 0], [2, 3]])

```

```
# Cach 1: tu cai dat
Xc = X - X.mean(axis=0)
C = np.cov(Xc, rowvar=False)                # chia n - 1: [[3.7 2.75] [2.75
    5.]]
eigval, eigvec = np.linalg.eigh(C)          # tri rieng tang dan
order = np.argsort(eigval)[::-1]
eigval, eigvec = eigval[order], eigvec[:, order]
print(eigval)                               # [7.1758 1.5242]
print(Xc @ eigvec[:, 0])                    # toa do tren PC1 (co the doi dau)

# Cach 2: scikit-learn
pca = PCA(n_components=1).fit(X)
print(pca.components_)                      # [[0.6205 0.7842]]
print(pca.explained_variance_ratio_)        # [0.8248]
Z = pca.transform(X)                        # [2.8491 2.5216 -2.1493 -3.0973
    -0.1241]
X_rec = pca.inverse_transform(Z)            # tai tao (nam tren duong PC1)
```



Hình 10.2: Dữ liệu ví dụ với hai trục thành phần chính $\mathbf{w}_1$, $\mathbf{w}_2$ và hình chiếu của các điểm lên $\mathbf{w}_1$.

10.4.2 Chọn số thành phần

- **Ngưỡng phương sai tích lũy**: chọn k nhỏ nhất để $\sum_{j\le k}\lambda_j / \sum_j \lambda_j \ge 90\%$ (hoặc 95%). Trong scikit-learn chỉ cần `PCA(n_components=0.95)`.
- **Biểu đồ scree** (Cattell, 1966): vẽ trị riêng theo thứ tự, chọn tại điểm "gãy".
- **Tiêu chí Kaiser**: với dữ liệu đã chuẩn hóa, giữ các thành phần có $\lambda_j > 1$.
- **Theo mục tiêu**: $k = 2$ hoặc 3 để trực quan hóa; với bài toán dự đoán, coi k là siêu tham số và chọn bằng kiểm định chéo (Chương 8).

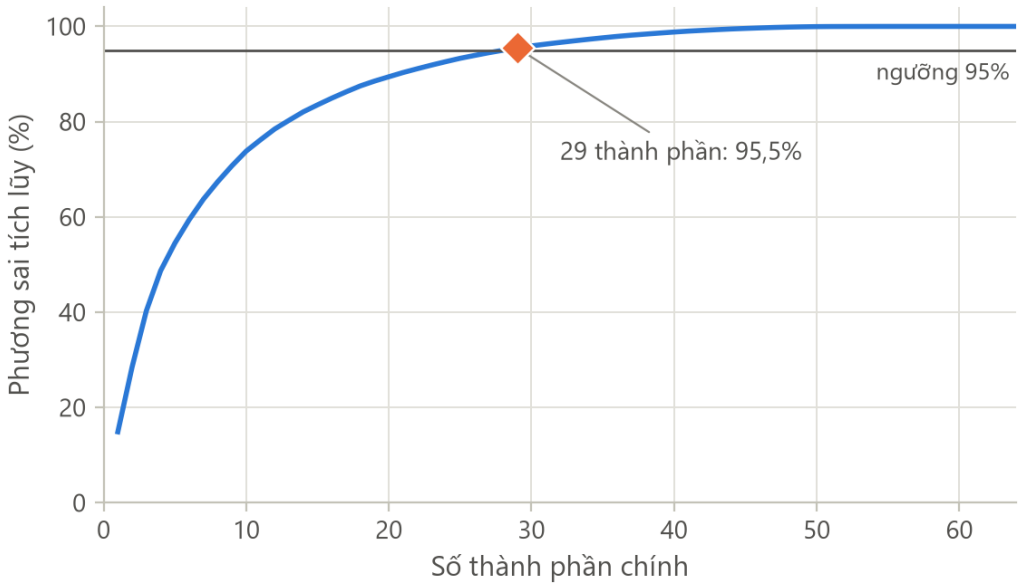
Listing 10.7: PCA trên tập chữ số viết tay

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt

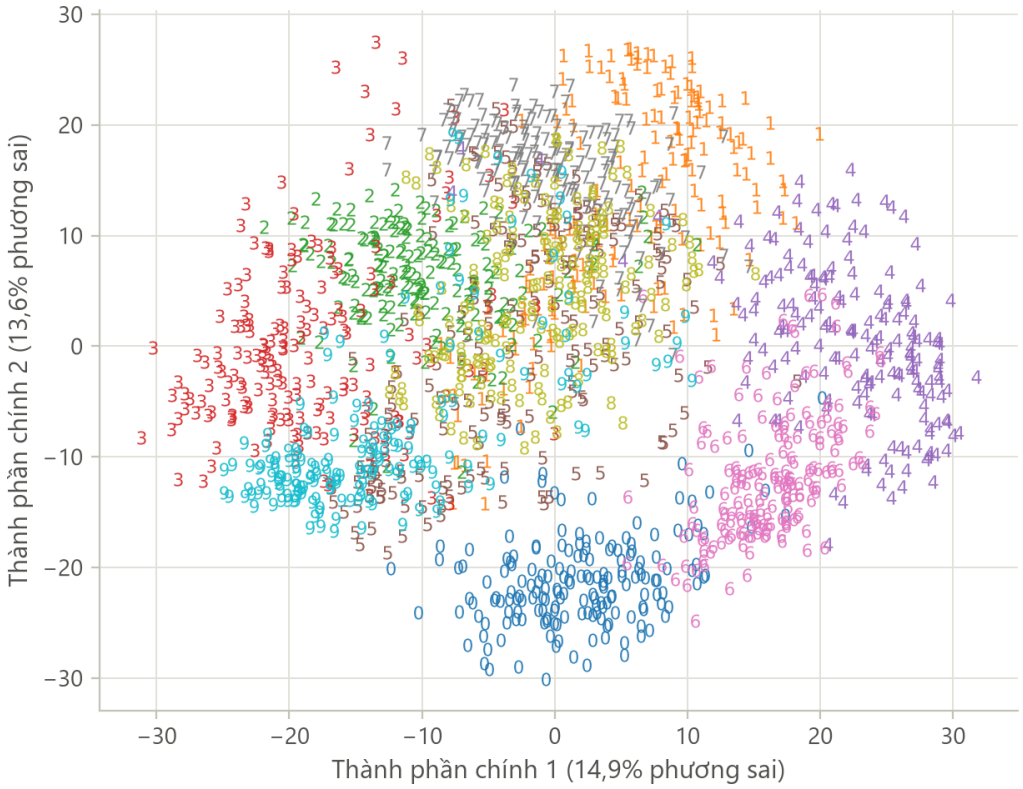
digits = load_digits()                  # 1797 anh 8x8 = 64 chieu
pca = PCA().fit(digits.data)
cum = np.cumsum(pca.explained_variance_ratio_)
print("So thanh phan giu 95% phuong sai:", np.argmax(cum >= 0.95) + 1)   # 29

plt.plot(range(1, 65), cum, marker=".")
plt.axhline(0.95, linestyle="--"); plt.xlabel("So thanh phan")
plt.ylabel("Phuong sai tich luy"); plt.show()

Z = PCA(n_components=2).fit_transform(digits.data)
plt.scatter(Z[:, 0], Z[:, 1], c=digits.target, cmap="tab10", s=8)
plt.colorbar(); plt.show()
```



Hình 10.3: Phương sai tích lũy theo số thành phần chính trên tập `digits`: khoảng 29 thành phần giữ 95% phương sai.



Hình 10.4: Tập `digits` chiếu xuống 2 thành phần chính đầu tiên, tô màu theo chữ số.

10.4.3 PCA trong pipeline học máy

PCA là một bộ biến đổi (transformer) như mọi bộ tiền xử lý khác: phải `fit` trên tập huấn luyện rồi mới `transform` tập kiểm tra, để tránh rò rỉ dữ liệu.

Listing 10.8: StandardScaler + PCA + Logistic Regression

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

X_tr, X_te, y_tr, y_te = train_test_split(digits.data, digits.target,
                                          test_size=0.25, random_state=0)
for k in [5, 10, 20, 40]:
    model = make_pipeline(StandardScaler(), PCA(n_components=k),
                          LogisticRegression(max_iter=2000))
    model.fit(X_tr, y_tr)
    print(k, round(model.score(X_te, y_te), 4))
```

Hạn chế của PCA.

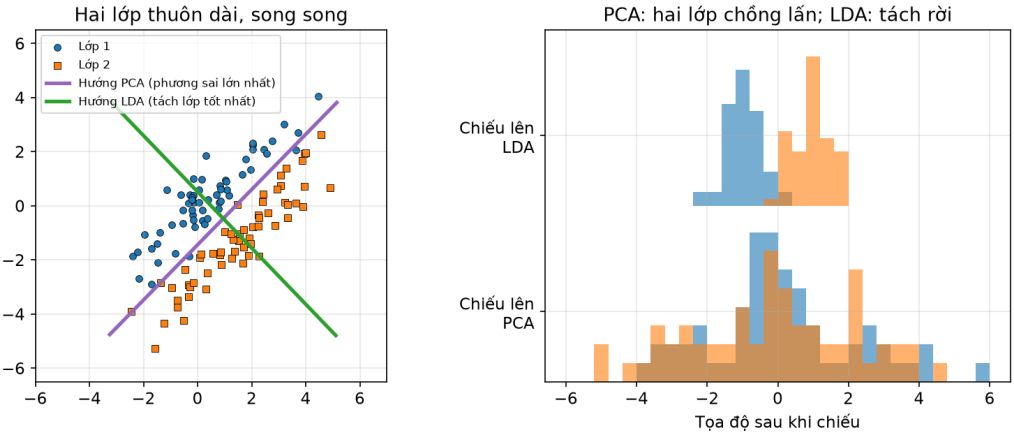
- Chỉ bắt được quan hệ **tuyến tính**; với dữ liệu nằm trên đa tạp cong (ví dụ "cuộn Thụy Sĩ") cần Kernel PCA, t-SNE (van der Maaten & Hinton, 2008) hoặc UMAP.
- **Nhạy với thang đo** và ngoại lai (phương sai bị chi phối bởi cột có giá trị lớn).

– Thành phần mới là tổ hợp của mọi cột gốc nên **khó diễn giải**.

– PCA **không dùng nhãn**: hướng có phương sai lớn chưa chắc là hướng phân biệt lớp tốt nhất — khi có nhãn, có thể dùng LDA (Fisher, 1936).

10.5 Phân tích biệt thức tuyến tính (LDA)

PCA tìm hướng có **phương sai lớn nhất** mà không nhìn nhãn, nên hướng đó chưa chắc giúp phân biệt các lớp. **Phân tích biệt thức tuyến tính** (Linear Discriminant Analysis – LDA) là phương pháp **có giám sát**: nó tìm hướng chiếu sao cho các lớp **tách xa nhau nhất**. Ý tưởng do R. A. Fisher (1936) đề xuất trong chính bài báo giới thiệu bộ dữ liệu hoa Iris, và được C. R. Rao (1948) mở rộng cho nhiều lớp.



Hình 10.5: PCA chọn hướng có phương sai lớn nhất (có thể làm hai lớp chồng lên nhau khi chiếu), còn LDA chọn hướng tách hai lớp tốt nhất.

Định nghĩa 10.5 (Tiêu chí Fisher)**.** *Với C lớp, gọi $\mathbf{m}_c$ là vector trung bình của lớp c (có n_c mẫu) và $\mathbf{m}$ là trung bình chung. Định nghĩa **ma trận tán xạ trong lớp** và **ma trận tán xạ giữa các lớp**:*

$$\mathbf{S}_W = \sum_{c=1}^{C} \sum_{\mathbf{x} \in \text{lớp } c} (\mathbf{x} - \mathbf{m}_c)(\mathbf{x} - \mathbf{m}_c)^\top, \qquad \mathbf{S}_B = \sum_{c=1}^{C} n_c \, (\mathbf{m}_c - \mathbf{m})(\mathbf{m}_c - \mathbf{m})^\top. \tag{10.6}$$

LDA tìm hướng $\mathbf{w}$ cực đại tỉ số giữa độ tách các lớp và độ phân tán bên trong lớp:

$$J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \, \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \, \mathbf{w}}. \tag{10.7}$$

Lấy đạo hàm và cho bằng 0 dẫn đến bài toán trị riêng tổng quát $\mathbf{S}_W^{-1}\mathbf{S}_B \, \mathbf{w} = \lambda \mathbf{w}$, với $J(\mathbf{w}) = \lambda$. Vì $\mathbf{S}_B$ là tổng của C ma trận hạng 1 bị ràng buộc bởi trung bình chung, hạng của nó không quá $C - 1$: **LDA giảm được tối đa xuống $C - 1$ chiều** (2 chiều với 3 lớp). Với hai lớp, nghiệm có dạng tường minh $\mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$.

Các bước của LDA (Raschka và cộng sự, 2022, Chương 5):

1. Chuẩn hóa d đặc trưng.

2. Tính vector trung bình d chiều của từng lớp.
3. Lập ma trận $\mathbf{S}_B$ và $\mathbf{S}_W$.
4. Tìm trị riêng và vector riêng của $\mathbf{S}_W^{-1}\mathbf{S}_B$.
5. Sắp xếp trị riêng giảm dần; chọn k vector riêng đầu làm ma trận chiếu $\mathbf{W}$ kích thước $d \times k$.
6. Chiếu dữ liệu: $\mathbf{X}' = \mathbf{X}\mathbf{W}$.

Ví dụ 10.7 (Tính tay LDA hai lớp)**.** *Lớp 1:* $(4,2)$, $(2,4)$, $(2,3)$, $(3,6)$, $(4,4)$. *Lớp 2:* $(9,10)$, $(6,8)$, $(9,5)$, $(8,7)$, $(10,8)$.

Bước 1 – trung bình lớp: $\mathbf{m}_1 = (3;\ 3{,}8)$, $\mathbf{m}_2 = (8{,}4;\ 7{,}6)$.
Bước 2 – tán xạ trong từng lớp. Độ lệch của lớp 1 so với $\mathbf{m}_1$ là $(1;\ -1{,}8)$, $(-1;\ 0{,}2)$, $(-1;\ -0{,}8)$, $(0;\ 2{,}2)$, $(1;\ 0{,}2)$, nên

$$\mathbf{S}_1 = \begin{pmatrix} 4 & -1 \\ -1 & 8{,}8 \end{pmatrix}, \qquad \mathbf{S}_2 = \begin{pmatrix} 9{,}2 & -0{,}2 \\ -0{,}2 & 13{,}2 \end{pmatrix}, \qquad \mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2 = \begin{pmatrix} 13{,}2 & -1{,}2 \\ -1{,}2 & 22 \end{pmatrix}.$$

Bước 3 – hướng chiếu. $\det \mathbf{S}_W = 13{,}2 \times 22 - 1{,}2^2 = 288{,}96$, nên

$$\mathbf{S}_W^{-1} = \frac{1}{288{,}96}\begin{pmatrix} 22 & 1{,}2 \\ 1{,}2 & 13{,}2 \end{pmatrix}, \qquad \mathbf{m}_1 - \mathbf{m}_2 = \begin{pmatrix} -5{,}4 \\ -3{,}8 \end{pmatrix},$$

$$\mathbf{w} = \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2) = \frac{1}{288{,}96}\begin{pmatrix} 22 \cdot (-5{,}4) + 1{,}2 \cdot (-3{,}8) \\ 1{,}2 \cdot (-5{,}4) + 13{,}2 \cdot (-3{,}8) \end{pmatrix} \approx \begin{pmatrix} -0{,}4269 \\ -0{,}1960 \end{pmatrix}.$$

Chuẩn hóa về độ dài 1 và đổi dấu cho dễ đọc: $\mathbf{u} \approx (0{,}9088;\ 0{,}4173)$.
Bước 4 – chiếu dữ liệu $z = \mathbf{u}^\top \mathbf{x}$: lớp 1 cho $(4{,}47;\ 3{,}49;\ 3{,}07;\ 5{,}23;\ 5{,}30)$, lớp 2 cho $(12{,}35;\ 8{,}79;\ 10{,}27;\ 10{,}19;\ 12{,}43)$. Trên trục mới, mọi điểm lớp 1 đều nhỏ hơn 5,31 và mọi điểm lớp 2 đều lớn hơn 8,79: hai lớp **tách rời hoàn toàn** chỉ bằng một chiều. Giá trị tiêu chí Fisher tại nghiệm bằng $J = \lambda \approx 3{,}05$, cũng là trị riêng khác 0 duy nhất của $\mathbf{S}_W^{-1}\mathbf{S}_B$ (với $\mathbf{S}_B = (\mathbf{m}_1 - \mathbf{m}_2)(\mathbf{m}_1 - \mathbf{m}_2)^\top$ trong trường hợp hai lớp).

PCA và LDA trên bộ dữ liệu Wine. Với 13 đặc trưng đã chuẩn hóa và 3 lớp (Chương 5), LDA cho 2 trục biệt thức với tỉ lệ "khả năng phân biệt" 66,2% và 33,8%. Hồi quy logistic trên 2 trục LDA đạt độ chính xác kiểm tra **1,000**; trên 2 thành phần chính của PCA (giữ 55,4% phương sai), độ chính xác chỉ là 0,926. Kết quả này minh họa ưu thế của giảm chiều có giám sát khi mục tiêu là phân lớp. Đổi lại, vì LDA dùng nhãn, **nguy cơ quá khớp và rò rỉ dữ liệu cao hơn**: LDA phải được `fit` trên tập huấn luyện bên trong pipeline. LDA còn giả định mỗi lớp có phân phối chuẩn với cùng ma trận hiệp phương sai; trên thực tế phương pháp vẫn hoạt động khá tốt khi giả định bị vi phạm vừa phải.

Listing 10.9: LDA tính tay và LDA với scikit-learn

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

X1 = np.array([[4, 2], [2, 4], [2, 3], [3, 6], [4, 4]], dtype=float)
X2 = np.array([[9, 10], [6, 8], [9, 5], [8, 7], [10, 8]], dtype=float)
m1, m2 = X1.mean(axis=0), X2.mean(axis=0)
S_W = (X1 - m1).T @ (X1 - m1) + (X2 - m2).T @ (X2 - m2)
```

```
w = np.linalg.solve(S_W, m1 - m2)             # = S_W^{-1} (m1 - m2)
u = -w / np.linalg.norm(w)
print(S_W)                                    # [[13.2 -1.2] [-1.2 22. ]]
print(u.round(4))                             # [0.9088 0.4173]
print((X1 @ u).round(2), (X2 @ u).round(2))

# Tren bo du lieu Wine: LDA trong pipeline, fit chi tren tap huan luyen
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

X, y = load_wine(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y,
    random_state=0)
for name, reducer in [("PCA", PCA(n_components=2)), ("LDA", LDA(n_components=2)
    )]:
    model = make_pipeline(StandardScaler(), reducer, LogisticRegression())
    model.fit(X_tr, y_tr)
    print(name, round(model.score(X_te, y_te), 3))      # PCA 0.926 ; LDA 1.0
```

Lưu ý về tên gọi: trong xử lý ngôn ngữ tự nhiên, "LDA" còn là viết tắt của *Latent Dirichlet Allocation* (Blei, Ng & Jordan, 2003), một mô hình chủ đề hoàn toàn khác (Raschka và cộng sự, 2022, Chương 8; Liu, 2019, Chương 3). Trong scikit-learn, mô hình đó là `LatentDirichletAllocation`.

10.6 Giảm chiều phi tuyến: Kernel PCA và t-SNE

PCA và LDA đều là phép chiếu **tuyến tính**. Khi dữ liệu nằm trên một **đa tạp cong** — hai vòng tròn lồng nhau, hai nửa mặt trăng đan vào nhau, "cuộn Thụy Sĩ" — không phép chiếu tuyến tính nào "trải phẳng" được cấu trúc đó (Raschka và cộng sự, 2022, Chương 5).

10.6.1 Kernel PCA

Schölkopf, Smola và Müller (1998) áp dụng **thủ thuật nhân** — cùng ý tưởng với SVM ở Chương 7 — cho PCA: ánh xạ dữ liệu lên một không gian đặc trưng nhiều chiều qua hàm ϕ, rồi thực hiện PCA ở đó. Phép ánh xạ không cần tính tường minh, vì mọi phép toán chỉ cần tích vô hướng $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) = K(\mathbf{x}_i, \mathbf{x}_j)$, chẳng hạn nhân RBF $K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$. Trị riêng và vector riêng được tính trên **ma trận nhân** $n \times n$ (đã chuẩn hóa tâm) thay vì ma trận hiệp phương sai $d \times d$, nên Kernel PCA tốn kém với n lớn.

Listing 10.10: Kernel PCA "trải phẳng" hai nửa mặt trăng

```
from sklearn.datasets import make_moons
from sklearn.decomposition import KernelPCA

X_m, y_m = make_moons(n_samples=200, noise=0.05, random_state=123)
Z_lin = PCA(n_components=2).fit_transform(X_m)          # hai lop van dan vao
    nhau
```

```
kpca = KernelPCA(n_components=2, kernel="rbf", gamma=15)
Z_rbf = kpca.fit_transform(X_m)                          # hai lop tach duoc
    tuyen tinh
```

10.6.2 t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding; van der Maaten & Hinton, 2008), phát triển từ SNE của Hinton và Roweis (2002), là kỹ thuật phổ biến nhất để **trực quan hóa** dữ liệu nhiều chiều. Mục tiêu của nó không phải giữ phương sai mà là **giữ quan hệ láng giềng**: các điểm gần nhau trong không gian gốc phải gần nhau trên mặt phẳng.

1. Trong không gian gốc, độ tương tự giữa hai điểm được đo bằng xác suất p_{ij}, tính từ phân phối Gauss quanh mỗi điểm. Độ rộng σ_i của từng phân phối được chọn sao cho **độ bối rối** (perplexity, thường từ 5 đến 50) bằng giá trị người dùng đặt — có thể hiểu là "số láng giềng hiệu dụng".

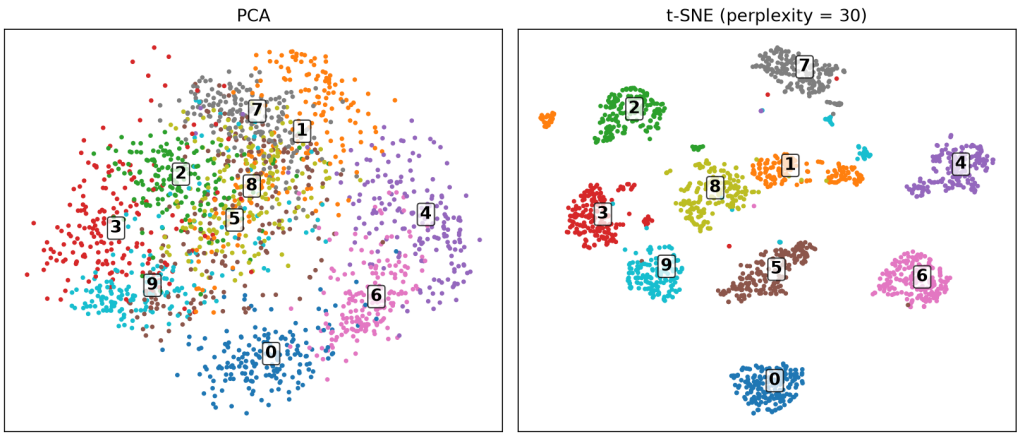
2. Trong không gian hai chiều, độ tương tự q_{ij} dùng phân phối **Student-t một bậc tự do**: $q_{ij} \propto (1 + \|\mathbf{y}_i - \mathbf{y}_j\|^2)^{-1}$. Đuôi dày của phân phối t cho phép các điểm không cùng nhóm nằm xa nhau, khắc phục "hiện tượng chen chúc" (crowding problem) của SNE.

3. Tìm tọa độ $\mathbf{y}_i$ cực tiểu độ phân kỳ Kullback–Leibler $\mathrm{KL}(P \,\|\, Q) = \sum_{i \neq j} p_{ij} \ln \frac{p_{ij}}{q_{ij}}$ bằng Gradient Descent.

Listing 10.11: So sánh PCA và t-SNE trên tập chữ số viết tay

```
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

digits = load_digits()
Z_pca = PCA(n_components=2).fit_transform(digits.data)
Z_tsne = TSNE(n_components=2, perplexity=30, init="pca",
              random_state=123).fit_transform(digits.data)
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
for a, Z, title in [(ax[0], Z_pca, "PCA"), (ax[1], Z_tsne, "t-SNE")]:
    a.scatter(Z[:, 0], Z[:, 1], c=digits.target, cmap="tab10", s=6)
    a.set_title(title)
plt.show()
```



Hình 10.6: Tập `digits` chiếu xuống hai chiều: PCA (trái) để các chữ số chồng lấn, t-SNE (phải) tách thành 10 cụm rõ rệt.

Đọc kết quả t-SNE đúng cách. t-SNE rất dễ bị diễn giải sai (Wattenberg, Viégas & Johnson, 2016):

- **Kích thước cụm** và **khoảng cách giữa các cụm** trên hình không mang ý nghĩa định lượng; chỉ quan hệ láng giềng cục bộ là đáng tin.
- Kết quả thay đổi theo perplexity và `random_state`; nên thử vài giá trị trước khi kết luận.
- t-SNE **không có phương thức `transform`** cho dữ liệu mới, nên chỉ dùng để **trực quan hóa**, không dùng làm bước tiền xử lý cho mô hình dự đoán.
- Chi phí tính toán lớn; với dữ liệu nhiều chiều, nên giảm xuống khoảng 50 chiều bằng PCA trước. UMAP (McInnes, Healy & Melville, 2018) là lựa chọn thay thế nhanh hơn và có `transform`.

Phương pháp	Dùng nhãn	Tuyến tính	`transform`	Mục đích chính
PCA	Không	Có	Có	Nén, khử nhiễu, tiền xử lý
LDA	Có	Có	Có	Giảm chiều cho phân lớp (tối đa $C-1$ chiều)
Kernel PCA	Không	Không	Có	Cấu trúc phi tuyến, n vừa phải
t-SNE	Không	Không	Không	Trực quan hóa 2–3 chiều

Bảng 10.3: So sánh các phương pháp giảm chiều

10.7 Tóm tắt chương

- Số chiều lớn làm dữ liệu thưa, khoảng cách mất ý nghĩa và tăng nguy cơ quá khớp.
- Phương sai, hiệp phương sai (chú ý dấu và quy ước chia n hay $n-1$) và cặp trị riêng – vector riêng (kiểm tra bằng vết, định thức và phép nhân $\mathbf{Av} = \lambda\mathbf{v}$) là nền tảng toán học của SVD và PCA.

- SVD: $\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top$; giữ k giá trị suy biến lớn nhất cho xấp xỉ hạng k tối ưu (Eckart–Young). Dùng cho nén ảnh, LSA, hệ gợi ý.
- PCA tìm các hướng trực giao có phương sai lớn nhất: vector riêng của ma trận hiệp phương sai, tính thực tế bằng SVD của dữ liệu đã chuẩn hóa tâm.
- Chọn số thành phần theo phương sai tích lũy, biểu đồ scree hoặc kiểm định chéo; luôn đặt PCA trong pipeline.
- LDA (Fisher, 1936) là giảm chiều có giám sát: cực đại $J(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{S}_B \mathbf{w}}{\mathbf{w}^\top \mathbf{S}_W \mathbf{w}}$, giải bằng trị riêng của $\mathbf{S}_W^{-1}\mathbf{S}_B$, tối đa $C-1$ chiều; với hai lớp $\mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$.
- Kernel PCA dùng thủ thuật nhân cho cấu trúc phi tuyến; t-SNE giữ quan hệ láng giềng để trực quan hóa, không dùng làm bước tiền xử lý và không diễn giải khoảng cách giữa các cụm.

10.8 Bài tập

1. Tính phương sai (chia n và chia $n-1$) của $(4, 8, 6, 2)$. Tính hiệp phương sai và hệ số tương quan của $A = (1, 2, 3, 4)$ và $B = (8, 6, 5, 1)$; giải thích ý nghĩa của dấu. Kiểm tra bằng `np.var`, `np.cov`, `np.corrcoef`.
2. Tìm trị riêng và vector riêng đơn vị của $\begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$ (đáp số: $\lambda = 5$ và $\lambda = 2$). Kiểm tra kết quả bằng vết, định thức và `np.linalg.eig`.
3. Tiếp ví dụ SVD của ma trận 5×2: tính $\mathbf{u}_2 = \mathbf{X}\mathbf{v}_2/\sigma_2$, kiểm tra $\mathbf{u}_1 \perp \mathbf{u}_2$ và tái tạo $\mathbf{X} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^\top + \sigma_2 \mathbf{u}_2 \mathbf{v}_2^\top$.
4. Làm lại toàn bộ các bước PCA cho 10 điểm của Smith (2002):

$$x_1 = (2{,}5;\ 0{,}5;\ 2{,}2;\ 1{,}9;\ 3{,}1;\ 2{,}3;\ 2{,}0;\ 1{,}0;\ 1{,}5;\ 1{,}1),$$
$$x_2 = (2{,}4;\ 0{,}7;\ 2{,}9;\ 2{,}2;\ 3{,}0;\ 2{,}7;\ 1{,}6;\ 1{,}1;\ 1{,}6;\ 0{,}9).$$

   (Đáp số: $\lambda_1 \approx 1{,}2840$, $\mathbf{w}_1 \approx (0{,}6779;\ 0{,}7352)$, thành phần thứ nhất giữ khoảng 96,3% phương sai.)
5. Tính tay SVD của $\mathbf{A} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \\ 0 & 0 \end{pmatrix}$ và $\mathbf{B} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$. Ma trận nào có hạng 1?
6. Với dữ liệu ở Bảng 10.2, tái tạo các điểm từ 1 thành phần chính và tính tổng bình phương sai số tái tạo. So sánh với $(n-1)\lambda_2$.
7. Chứng minh rằng các tọa độ chiếu $\mathbf{z}$ trên thành phần chính thứ nhất có trung bình 0 và phương sai λ_1.
8. Trên tập `load_breast_cancer`, so sánh tỉ lệ phương sai giải thích của PCA khi có và không có `StandardScaler`. Giải thích.
9. Nén một bức ảnh bằng SVD với $k = 10, 30, 100$; tính tỉ lệ nén và sai số Frobenius tương đối.

10. Tính tay LDA cho hai lớp: lớp 1 gồm $(1,2)$, $(2,3)$, $(3,3)$; lớp 2 gồm $(6,5)$, $(7,8)$, $(8,6)$. Tìm $\mathbf{S}_W$, hướng $\mathbf{w} \propto \mathbf{S}_W^{-1}(\mathbf{m}_1 - \mathbf{m}_2)$ và tọa độ chiếu; kiểm tra bằng `LinearDiscriminantAnalysis`.

11. Trên bộ dữ liệu Iris, vẽ dữ liệu chiếu xuống 2 chiều bằng PCA và bằng LDA cạnh nhau. Vì sao LDA chỉ cho tối đa 2 trục? Phương pháp nào tách loài *versicolor* và *virginica* tốt hơn?

12. Chạy t-SNE trên tập `digits` với perplexity $\in \{2, 5, 30, 100\}$ và hai giá trị `random_state`. Mô tả những gì thay đổi và những gì giữ nguyên. Vì sao không nên đưa đầu ra t-SNE vào một bộ phân lớp?

13. Với tập Ames Housing đã tiền xử lý ở Chương 5, đặt PCA vào pipeline Ridge. Dùng `GridSearchCV` để chọn `n_components`.

Tài liệu tham khảo của chương

1. Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

2. Beyer, K., Goldstein, J., Ramakrishnan, R., & Shaft, U. (1999). When Is "Nearest Neighbor" Meaningful? *Proceedings of ICDT '99*, 217–235.

3. Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3, 993–1022.

4. Deerwester, S., Dumais, S. T., Furnas, G. W., Landauer, T. K., & Harshman, R. (1990). Indexing by Latent Semantic Analysis. *Journal of the American Society for Information Science*, 41(6), 391–407.

5. Eckart, C., & Young, G. (1936). The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3), 211–218.

6. Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.

7. Golub, G., & Kahan, W. (1965). Calculating the Singular Values and Pseudo-Inverse of a Matrix. *SIAM Journal on Numerical Analysis*, 2(2), 205–224.

8. Hinton, G. E., & Roweis, S. T. (2002). Stochastic Neighbor Embedding. *Advances in Neural Information Processing Systems 15*, 833–840.

9. Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24(6), 417–441.

10. Jolliffe, I. T. (2002). *Principal Component Analysis* (2nd ed.). Springer.

11. Liu, Y. (H.) (2019). *Python Machine Learning By Example* (2nd ed.), Chương 3: Mining the 20 Newsgroups Dataset with Clustering and Topic Modeling Algorithms. Packt Publishing.

12. McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction. *arXiv:1802.03426*.

13. Pearson, K. (1896). Mathematical Contributions to the Theory of Evolution. III. Regression, Heredity, and Panmixia. *Philosophical Transactions of the Royal Society of London, Series A*, 187, 253–318.

14. Pearson, K. (1901). On Lines and Planes of Closest Fit to Systems of Points in Space. *Philosophical Magazine*, 2(11), 559–572.

15. Rao, C. R. (1948). The Utilization of Multiple Measurements in Problems of Biological Classification. *Journal of the Royal Statistical Society, Series B*, 10(2), 159–203.

16. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*, Chương 5: Compressing Data via Dimensionality Reduction. Packt Publishing.

17. Schölkopf, B., Smola, A., & Müller, K.-R. (1998). Nonlinear Component Analysis as a Kernel Eigenvalue Problem. *Neural Computation*, 10(5), 1299–1319.

18. Smith, L. I. (2002). *A tutorial on Principal Components Analysis*. University of Otago.

19. Strang, G. (2016). *Introduction to Linear Algebra* (5th ed.). Wellesley-Cambridge Press.

20. van der Maaten, L., & Hinton, G. (2008). Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 9, 2579–2605.

21. Wattenberg, M., Viégas, F., & Johnson, I. (2016). How to Use t-SNE Effectively. *Distill*, 1(10).

Chương 11: Phân cụm (Clustering)

Phần lớn các chương trước là **học có giám sát**: dữ liệu có sẵn nhãn y. Chương 10 đã giới thiệu PCA, một phương pháp tìm các hướng biến thiên chính của dữ liệu mà không cần nhãn. Chương này đi tiếp một bước: tìm các **nhóm** trong dữ liệu không nhãn. Trong thực tế, phần lớn dữ liệu **không có nhãn** — gán nhãn tốn kém, hoặc ta chưa biết dữ liệu có những nhóm nào. **Phân cụm** (clustering) là bài toán **học không giám sát** (unsupervised learning): tự động chia dữ liệu thành các nhóm (cụm) sao cho các điểm trong cùng cụm *giống nhau* và các điểm ở cụm khác nhau thì *khác nhau*. Ứng dụng tiêu biểu: phân khúc khách hàng, gom nhóm tài liệu, nén ảnh (lượng tử hóa màu), phát hiện bất thường.

Theo kế hoạch giảng dạy, chương này trình bày bốn thuật toán, xếp theo cách định nghĩa cụm: hai thuật toán dựa trên **tâm cụm** là **K-Means** và **Mean-shift**; **Hierarchical clustering** xây cây phân cấp các cụm; **DBSCAN** tìm cụm theo mật độ.
Sau khi học xong chương này, người học có khả năng:

- Phân biệt học có giám sát và không giám sát; nêu các họ thuật toán phân cụm chính.
- Tự chạy tay thuật toán K-Means và DBSCAN trên dữ liệu nhỏ.
- Phân biệt phân cụm cứng và phân cụm mềm; tính tay một vòng lặp của Fuzzy C-Means.
- Đọc và cắt cây phân cấp (dendrogram).
- Chọn số cụm và đánh giá chất lượng phân cụm bằng phương pháp khuỷu tay và hệ số Silhouette.

11.1 Bài toán phân cụm

Định nghĩa 11.1 (Phân cụm)**.** *Cho tập dữ liệu $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathbb{R}^d$* ***không có nhãn****. Phân cụm là tìm một phép gán $c : \{1, \ldots, n\} \to \{1, \ldots, K\}$ (hoặc thêm nhãn "nhiễu") sao cho độ tương tự trong cụm cao và độ tương tự giữa các cụm thấp.*

"Giống nhau" được đo bằng **khoảng cách**, nên mọi lưu ý ở Chương 5 đều áp dụng: phải **chuẩn hóa thang đo** và **mã hóa One Hot** các biến định danh; nếu không, đặc trưng có thang đo lớn sẽ quyết định toàn bộ kết quả phân cụm.

Họ thuật toán	Ý tưởng	Đại diện
Phân hoạch	Chia thành K cụm, tối ưu một hàm mục tiêu	K-Means, K-Medoids
Phân cấp	Xây dựng cây gộp/tách cụm lồng nhau	Agglomerative (single, complete, average, Ward)
Dựa trên mật độ	Cụm là vùng dày đặc, ngăn cách bởi vùng thưa	DBSCAN, OPTICS, Mean-shift
Dựa trên mô hình	Dữ liệu sinh ra từ hỗn hợp phân phối	Gaussian Mixture Model

Bảng 11.1: Các họ thuật toán phân cụm

11.2 Phân cụm dựa trên tâm: K-Means và Mean-shift

11.2.1 Thuật toán K-Means

Thuật toán K-Means được Stuart Lloyd tại Bell Labs đề xuất năm 1957 cho bài toán lượng tử hóa tín hiệu (công bố chính thức năm 1982); tên gọi "k-means" do James MacQueen đặt năm 1967. Đến nay đây vẫn là thuật toán phân cụm được dùng nhiều nhất nhờ đơn giản và nhanh.

Định nghĩa 11.2 (Hàm mục tiêu K-Means)**.** *Cho số cụm K, tìm các tâm cụm $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ và phép gán c để cực tiểu* ***tổng bình phương khoảng cách trong cụm*** *(Within-Cluster Sum of Squares – WCSS, thuộc tính `inertia_` trong scikit-learn):*

$$J = \sum_{i=1}^{n} \left\| \mathbf{x}_i - \boldsymbol{\mu}_{c(i)} \right\|^2. \tag{11.1}$$

Thuật toán Lloyd:

1. **Khởi tạo** K tâm cụm (ngẫu nhiên, chọn điểm dữ liệu, hoặc K-Means++).
2. **Bước gán**: gán mỗi điểm vào cụm có tâm gần nhất.
3. **Bước cập nhật**: tâm cụm mới = trung bình cộng các điểm thuộc cụm.
4. Lặp lại bước 2–3 đến khi phép gán không đổi.

Mỗi bước đều không làm tăng J, nên thuật toán luôn hội tụ — nhưng chỉ đến **cực tiểu địa phương**, phụ thuộc vào khởi tạo. Arthur và Vassilvitskii (2007) đề xuất **K-Means++**: chọn tâm đầu tiên ngẫu nhiên, các tâm tiếp theo được chọn với xác suất tỉ lệ với bình phương khoảng cách đến tâm gần nhất đã chọn, giúp các tâm tản ra; đây là khởi tạo mặc định của scikit-learn. Tham số `n_init` chạy nhiều lần khởi tạo và giữ kết quả có J nhỏ nhất.

Ví dụ 11.1 (Chạy tay K-Means)**.** *Cho 5 điểm $P_1(22,10)$, $P_2(25,15)$, $P_3(45,50)$, $P_4(50,55)$, $P_5(23,12)$, $K=2$, khởi tạo tâm $\boldsymbol{\mu}_1 = P_1 = (22,10)$ và $\boldsymbol{\mu}_2 = P_4 = (50,55)$.*

Vòng 1 – bước gán:

Điểm	**Tọa độ**	$d(\cdot, \boldsymbol{\mu}_1)$	$d(\cdot, \boldsymbol{\mu}_2)$	**Cụm**
P_1	(22, 10)	0.00	53.00	1
P_2	(25, 15)	5.83	47.17	1
P_3	(45, 50)	46.14	7.07	2
P_4	(50, 55)	53.00	0.00	2
P_5	(23, 12)	2.24	50.77	1

Bảng 11.2: Khoảng cách Euclid đến hai tâm khởi tạo

Vòng 1 – bước cập nhật:

$$\boldsymbol{\mu}_1 = \tfrac{1}{3}\big[(22,10) + (25,15) + (23,12)\big] = (23{,}33;\ 12{,}33),$$
$$\boldsymbol{\mu}_2 = \tfrac{1}{2}\big[(45,50) + (50,55)\big] = (47{,}5;\ 52{,}5).$$

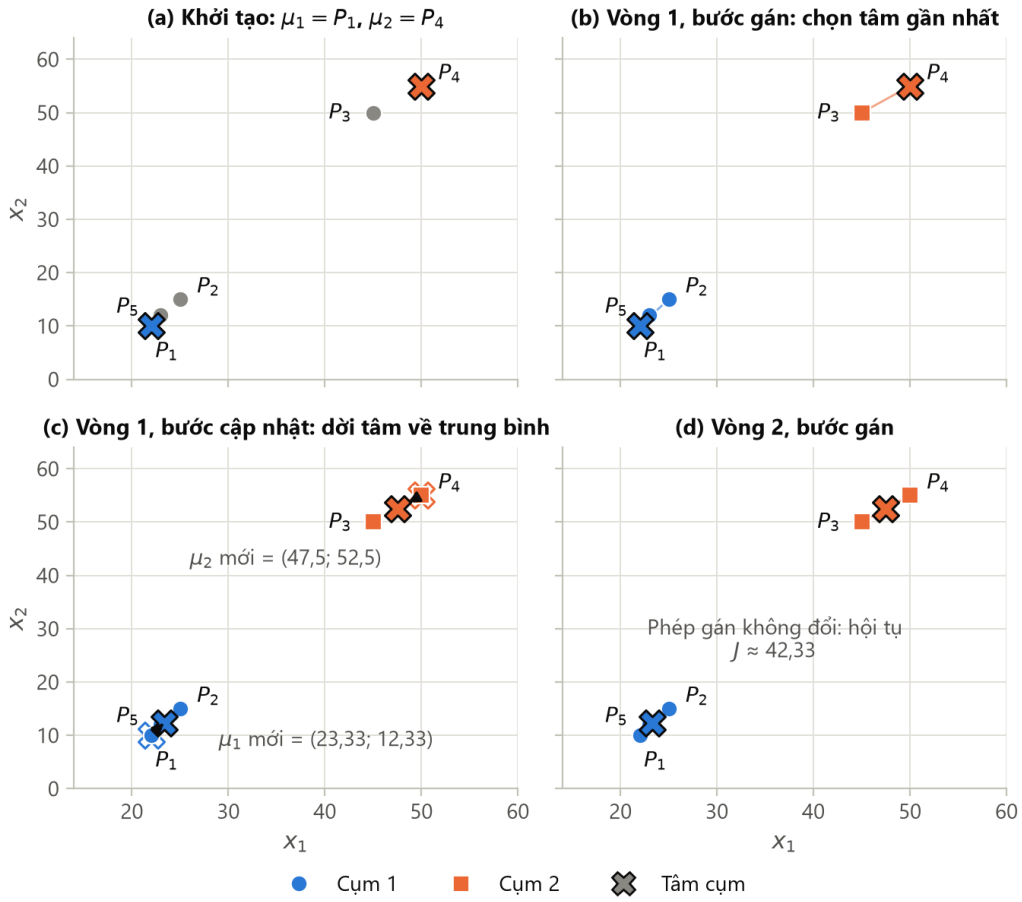
Vòng 2: khoảng cách mới từ $P_1, \ldots, P_5$ đến $\boldsymbol{\mu}_1$ là $(2{,}69;\ 3{,}14;\ 43{,}45;\ 50{,}31;\ 0{,}47)$ và đến $\boldsymbol{\mu}_2$ là $(49{,}56;\ 43{,}73;\ 3{,}54;\ 3{,}54;\ 47{,}33)$. Phép gán không đổi $\{P_1, P_2, P_5\}, \{P_3, P_4\} \Rightarrow$ **hội tụ**. Giá trị hàm mục tiêu $J = 42{,}33$.

Listing 11.1: K-Means với scikit-learn (ví dụ trên lớp)

```
import numpy as np
from sklearn.cluster import KMeans

X = np.array([[22, 10], [25, 15], [45, 50], [50, 55], [23, 12]])

kmeans = KMeans(n_clusters=2, init=X[[0, 3]], n_init=1)
kmeans.fit(X)
print(kmeans.cluster_centers_)   # [[23.33 12.33] [47.5 52.5]]
print(kmeans.labels_)            # [0 0 1 1 0]
print(kmeans.inertia_)           # 42.33
print(kmeans.predict([[30, 20]]))  # gan diem moi vao cum gan nhat
```



Hình 11.1: Các vòng lặp của K-Means trên 5 điểm: khởi tạo tâm, bước gán, bước cập nhật và hội tụ.

11.2.2 Chọn số cụm K

Phương pháp khuỷu tay (elbow). Vẽ J (inertia) theo K. J luôn giảm khi K tăng (bằng 0 khi $K = n$), nhưng đến một điểm thì giảm chậm lại rõ rệt — "khuỷu tay" của đường cong là ứng viên tốt cho K.

Hệ số Silhouette (Rousseeuw, 1987). Với mỗi điểm i: $a(i)$ là khoảng cách trung bình đến các điểm **cùng cụm**, $b(i)$ là khoảng cách trung bình nhỏ nhất đến các điểm của **một cụm khác**:

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]. \tag{11.2}$$

$s(i)$ gần 1: điểm nằm đúng cụm và xa cụm khác; gần 0: nằm ở ranh giới; âm: có lẽ bị gán sai cụm. Silhouette trung bình càng cao, phân cụm càng tốt.

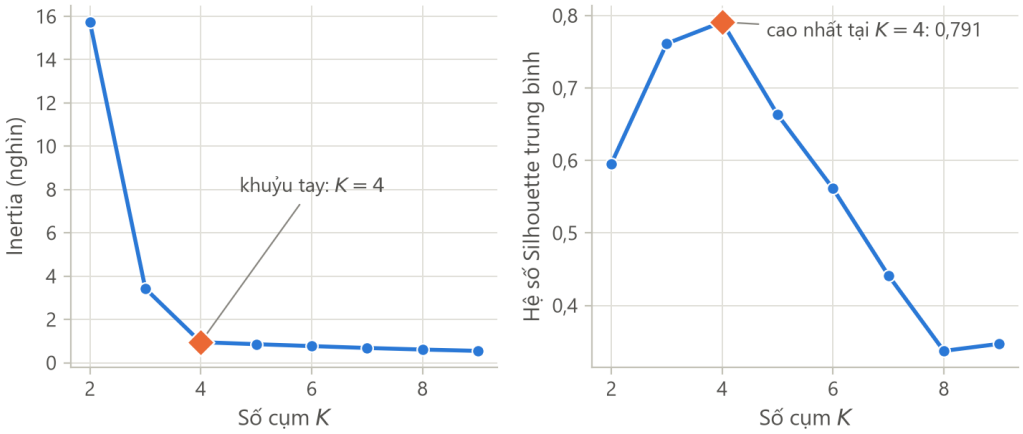
Listing 11.2: Chọn K bằng elbow và silhouette

```
from sklearn.datasets import make_blobs
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.0, random_state=42)
```

```
ks, inertias, sils = range(2, 10), [], []
for k in ks:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertias.append(km.inertia_)
    sils.append(silhouette_score(X, km.labels_))

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ax[0].plot(ks, inertias, "o-"); ax[0].set_title("Elbow (inertia)")
ax[1].plot(ks, sils, "o-");     ax[1].set_title("Silhouette")
plt.show()
```



Hình 11.2: Đường cong inertia (khuỷu tay tại $K = 4$) và hệ số Silhouette theo số cụm.

Hạn chế của K-Means. Phải chọn trước K; giả định cụm có dạng "cầu" và kích thước tương đương; nhạy với ngoại lai (trung bình bị kéo lệch); chỉ dùng được với khoảng cách Euclid (biến thể K-Medoids dùng được khoảng cách bất kỳ).

11.2.3 Mean-shift: dịch chuyển về đỉnh mật độ

K-Means đòi hỏi biết trước số cụm K và luôn tạo ra các cụm dạng khối cầu. Mean-shift cũng biểu diễn mỗi cụm bằng một extbftâm, nhưng tâm ở đây là extbfđỉnh mật độ của dữ liệu thay vì trung bình của các điểm được gán. Nhờ vậy thuật toán tự tìm ra số cụm.

Mean-shift được Fukunaga và Hostetler đề xuất năm 1975 để ước lượng gradient của hàm mật độ, và được Comaniciu và Meer (2002) phổ biến trong thị giác máy tính (phân đoạn ảnh, theo vết đối tượng). Ý tưởng: coi dữ liệu là mẫu từ một hàm mật độ; mỗi **đỉnh** (mode) của hàm mật độ là một cụm.

Thuật toán: với mỗi điểm khởi đầu $\mathbf{x}$, lặp:

$$\mathbf{x} \leftarrow \frac{\sum_i K(\mathbf{x}_i - \mathbf{x})\,\mathbf{x}_i}{\sum_i K(\mathbf{x}_i - \mathbf{x})}, \tag{11.3}$$

tức dịch $\mathbf{x}$ về **trung bình có trọng số** của các điểm trong cửa sổ (nhân K có độ rộng `bandwidth`). Điểm "leo dốc" dần lên đỉnh mật độ gần nhất; các điểm hội tụ về cùng một đỉnh thuộc cùng một cụm.

Listing 11.3: Mean-shift với ước lượng bandwidth

```
from sklearn.cluster import MeanShift, estimate_bandwidth

X, _ = make_blobs(n_samples=500, centers=4, cluster_std=1.0, random_state=42)
bw = estimate_bandwidth(X, quantile=0.2)
ms = MeanShift(bandwidth=bw).fit(X)
print("So cum tim duoc:", len(ms.cluster_centers_))
```

Ưu điểm: không cần chọn số cụm, tìm được cụm hình dạng tùy ý. Nhược điểm: chi phí tính toán lớn và kết quả phụ thuộc nhiều vào `bandwidth`.

11.3 Phân cụm mềm: Fuzzy C-Means

K-Means là thuật toán **phân cụm cứng** (hard clustering): mỗi điểm thuộc về **đúng một** cụm. Trong thực tế, ranh giới giữa các nhóm thường không rõ ràng: một khách hàng có thể vừa giống nhóm "tiết kiệm" vừa giống nhóm "mua sắm thường xuyên"; một điểm ảnh ở biên giữa hai mô trong ảnh y khoa mang đặc điểm của cả hai. **Phân cụm mềm** (soft clustering) gán cho mỗi điểm một **độ thuộc** $w_{ij} \in [0,1]$ vào từng cụm j, với $\sum_j w_{ij} = 1$ (Raschka và cộng sự, 2022, Chương 10). Thuật toán tiêu biểu là **Fuzzy C-Means** (FCM), do Dunn (1973) đề xuất và Bezdek (1981) phát triển hoàn chỉnh trên nền lý thuyết tập mờ của Zadeh (1965).

Định nghĩa 11.3 (Hàm mục tiêu của Fuzzy C-Means)**.** *Với k cụm và **hệ số mờ** $m > 1$ (thường chọn $m = 2$), FCM cực tiểu*

$$J_m = \sum_{i=1}^{n}\sum_{j=1}^{k} w_{ij}^{m}\,\left\|\mathbf{x}_i - \boldsymbol{\mu}_j\right\|^2, \qquad \textit{với } \sum_{j=1}^{k} w_{ij} = 1,\ \ w_{ij} \geq 0. \tag{11.4}$$

Giải bài toán tối ưu có ràng buộc bằng nhân tử Lagrange cho hai công thức cập nhật luân phiên:

$$w_{ij} = \left[\sum_{c=1}^{k}\left(\frac{\|\mathbf{x}_i - \boldsymbol{\mu}_j\|}{\|\mathbf{x}_i - \boldsymbol{\mu}_c\|}\right)^{\frac{2}{m-1}}\right]^{-1}, \tag{11.5}$$

$$\boldsymbol{\mu}_j = \frac{\sum_{i=1}^{n} w_{ij}^{m}\,\mathbf{x}_i}{\sum_{i=1}^{n} w_{ij}^{m}}. \tag{11.6}$$

Thuật toán giống K-Means: khởi tạo tâm, rồi lặp **bước cập nhật độ thuộc** (11.5) và **bước cập nhật tâm** (11.6) đến khi tâm không còn thay đổi đáng kể. Tâm cụm là trung bình có trọng số w_{ij}^m của **mọi** điểm, thay vì chỉ trung bình các điểm thuộc cụm. Hệ số m điều khiển độ "mờ": khi $m \to 1$, độ thuộc tiến về 0 hoặc 1 và FCM trở thành K-Means; khi m tăng, độ thuộc tiến về $1/k$ (mọi điểm thuộc đều mọi cụm).

Ví dụ 11.2 (Một vòng lặp Fuzzy C-Means)**.** *Bảy điểm trên trục số $1, 2, 3, 5{,}5, 8, 9, 10$ — hai nhóm rõ ràng và một điểm $5{,}5$ nằm chính giữa. Chọn $k = 2$, $m = 2$, tâm khởi tạo $\mu_1 = 4$, $\mu_2 = 7$.*

Bước cập nhật độ thuộc. Với $m = 2$, số mũ $\frac{2}{m-1} = 2$, nên với hai cụm: $w_{i1} = \dfrac{1}{1 + (d_{i1}/d_{i2})^2}$. Chẳng hạn điểm $x = 1$ có $d_{11} = 3$, $d_{12} = 6$, nên $w_{11} = \frac{1}{1+0{,}25} = 0{,}8$ và $w_{12} = 0{,}2$.

x_i	$d(x_i, \mu_1 = 4)$	$d(x_i, \mu_2 = 7)$	w_{i1}	w_{i2}
1	3	6	0,8000	0,2000
2	2	5	0,8621	0,1379
3	1	4	0,9412	0,0588
5,5	1,5	1,5	**0,5000**	**0,5000**
8	4	1	0,0588	0,9412
9	5	2	0,1379	0,8621
10	6	3	0,2000	0,8000

Bảng 11.3: Độ thuộc sau bước cập nhật đầu tiên ($m = 2$)

Bước cập nhật tâm. Với $w_{i1}^2 = (0{,}64;\ 0{,}7432;\ 0{,}8858;\ 0{,}25;\ 0{,}0035;\ 0{,}0190;\ 0{,}04)$, tổng bằng $2{,}5815$:

$$\begin{aligned}\textstyle\sum_i w_{i1}^2\, x_i &= 0{,}64 \cdot 1 + 0{,}7432 \cdot 2 + 0{,}8858 \cdot 3 + 0{,}25 \cdot 5{,}5 \\ &\quad + 0{,}0035 \cdot 8 + 0{,}0190 \cdot 9 + 0{,}04 \cdot 10 \approx 6{,}7577, \\ \mu_1 &= \frac{6{,}7577}{2{,}5815} \approx 2{,}6178.\end{aligned}$$

Do dữ liệu đối xứng qua 5,5, $\mu_2 = 11 - \mu_1 \approx 8{,}3822$. Hàm mục tiêu giảm từ $J = 24{,}30$ ở vòng 1 xuống $10{,}20$ ở vòng 2, và thuật toán hội tụ về $\mu_1 \approx 2{,}2861$, $\mu_2 \approx 8{,}7139$. Khi hội tụ, các điểm $1, 2, 3$ có độ thuộc vào cụm 1 lần lượt là $0{,}973$; $0{,}998$; $0{,}985$, còn điểm $5{,}5$ giữ nguyên độ thuộc $0{,}5/0{,}5$.

So sánh với K-Means. Trên cùng dữ liệu, K-Means buộc phải xếp điểm $5{,}5$ vào một cụm, ở đây là cụm trái, nên tâm cụm trái bị kéo lệch thành $\frac{1+2+3+5{,}5}{4} = 2{,}875$, còn tâm cụm phải là 9. Lời giải xếp $5{,}5$ vào cụm phải có cùng tổng bình phương khoảng cách $13{,}1875$, nên lựa chọn của K-Means hoàn toàn tùy thuộc vào khởi tạo. FCM phản ánh trung thực sự mơ hồ này: điểm $5{,}5$ có độ thuộc như nhau vào hai cụm, và vì trọng số $w^m = 0{,}25$ của nó nhỏ, nó ảnh hưởng rất ít đến vị trí tâm.

Listing 11.4: Cài đặt Fuzzy C-Means bằng NumPy và so sánh với K-Means

```
import numpy as np
from sklearn.cluster import KMeans

def fuzzy_cmeans(X, centers, m=2.0, n_iter=100, tol=1e-6):
    X = np.asarray(X, dtype=float).reshape(len(X), -1)
    C = np.asarray(centers, dtype=float).reshape(len(centers), -1)
    for _ in range(n_iter):
        D = np.linalg.norm(X[:, None, :] - C[None, :, :], axis=2)   # (n, k)
        D = np.fmax(D, 1e-12)                                       # tranh
    chia cho 0
        W = 1.0 / ((D[:, :, None] / D[:, None, :]) ** (2 / (m - 1))).sum(axis
    =2)
        Wm = W ** m
        C_new = (Wm.T @ X) / Wm.sum(axis=0)[:, None]                # tam co
    trong so
        converged = np.abs(C_new - C).max() < tol
        C = C_new
```

```
        if converged:
            break
    return C, W

x = np.array([1, 2, 3, 5.5, 8, 9, 10])
centers, W = fuzzy_cmeans(x, centers=[4, 7])
print(centers.ravel().round(4))          # [2.2861 8.7139]
print(W[3].round(4))                     # diem 5.5: [0.5 0.5]

km = KMeans(n_clusters=2, n_init=10, random_state=0).fit(x.reshape(-1, 1))
print(km.cluster_centers_.ravel(), km.labels_)   # [2.875 9.] [0 0 0 0 1 1 1]
```

Thư viện `scikit-fuzzy` cung cấp hàm `cmeans` hoàn chỉnh. Một cách tiếp cận phân cụm mềm khác, có nền tảng xác suất, là **mô hình hỗn hợp Gauss** (`GaussianMixture` trong scikit-learn), với phương thức `predict_proba` trả về xác suất thuộc từng cụm.

11.4 Phân cụm phân cấp (Hierarchical Clustering)

Phân cụm phân cấp bắt nguồn từ ngành phân loại học số (numerical taxonomy) của Sokal và Sneath (1963) và được Johnson (1967) hệ thống hóa. Kết quả không phải một phân hoạch duy nhất mà là một **cây phân cấp** (dendrogram): cắt cây ở độ cao nào sẽ được số cụm tương ứng.
Thuật toán gộp dần (agglomerative, từ dưới lên):

1. Ban đầu mỗi điểm là một cụm.
2. Tìm hai cụm **gần nhau nhất** và gộp chúng lại.
3. Lặp lại đến khi chỉ còn một cụm.

Khoảng cách giữa hai cụm A, B được định nghĩa bởi **tiêu chí liên kết** (linkage):

Liên kết	Công thức	Đặc điểm
Single	$\min_{a\in A, b\in B} d(a,b)$	Tạo cụm dài, hình dạng tùy ý; dễ bị "hiệu ứng chuỗi"
Complete	$\max_{a\in A, b\in B} d(a,b)$	Cụm gọn, đường kính nhỏ
Average	trung bình mọi cặp $d(a,b)$	Cân bằng giữa hai loại trên
Ward (1963)	mức tăng WCSS khi gộp	Cụm kích thước đều; mặc định của scikit-learn

Bảng 11.4: Các tiêu chí liên kết

Ví dụ 11.3 (Single linkage trên dữ liệu một chiều)**.** *Cho 5 điểm trên trục số:* $1, 2, 4, 7, 11$.

1. *Khoảng cách nhỏ nhất là* $d(1,2) = 1 \Rightarrow$ *gộp* $\{1,2\}$ *ở độ cao 1.*
2. *Khoảng cách từ* $\{1,2\}$ *đến 4 là* $\min(3,2) = 2$ *(nhỏ nhất)* $\Rightarrow$ *gộp* $\{1,2,4\}$ *ở độ cao 2.*
3. *Khoảng cách từ* $\{1,2,4\}$ *đến 7 là 3* $\Rightarrow$ *gộp* $\{1,2,4,7\}$ *ở độ cao 3.*
4. *Cuối cùng gộp với 11 ở độ cao 4.*

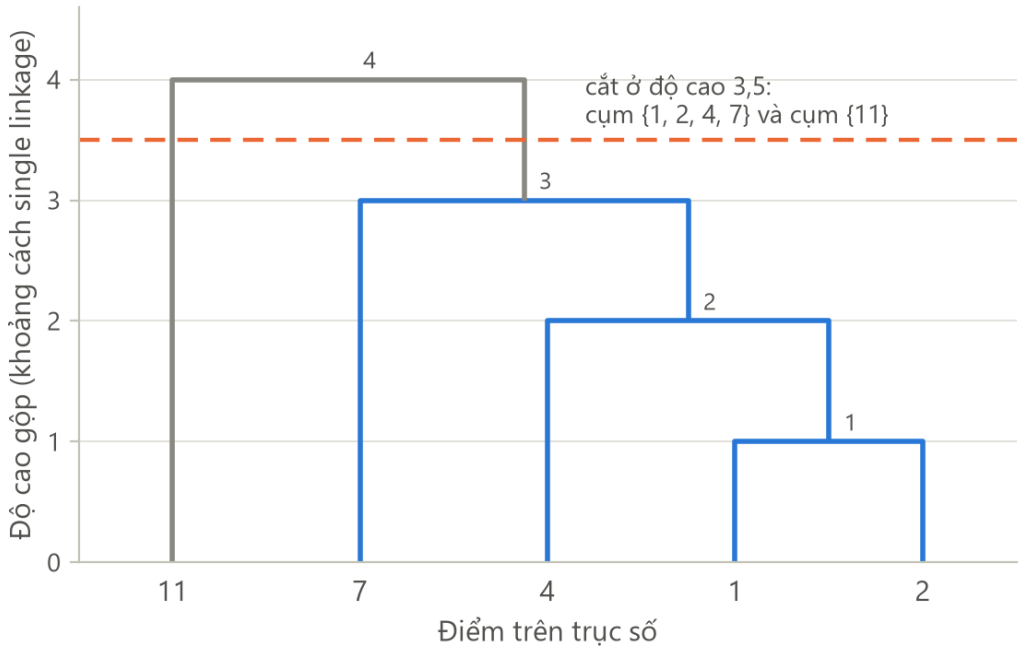
Cắt dendrogram ở độ cao 3,5 được 2 cụm: $\{1,2,4,7\}$ *và* $\{11\}$.

Listing 11.5: Phân cụm phân cấp và vẽ dendrogram

```
import numpy as np
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import linkage, dendrogram
import matplotlib.pyplot as plt

p = np.array([[1], [2], [4], [7], [11]])
print(AgglomerativeClustering(n_clusters=2, linkage="single")
      .fit_predict(p))                          # [0 0 0 0 1]

Z = linkage(p, method="single")
print(Z)        # moi dong: [cum_a, cum_b, do_cao, so_diem]
dendrogram(Z, labels=["1", "2", "4", "7", "11"])
plt.axhline(3.5, linestyle="--")                # duong cat -> 2 cum
plt.show()
```



Hình 11.3: Dendrogram của 5 điểm với single linkage; đường nét đứt ở độ cao 3,5 cắt cây thành 2 cụm.

Ưu điểm: không cần chọn trước K, kết quả trực quan. Nhược điểm: độ phức tạp tối thiểu $O(n^2)$ bộ nhớ, không phù hợp với dữ liệu rất lớn; quyết định gộp không thể hoàn tác.

11.5 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) do Ester, Kriegel, Sander và Xu công bố năm 1996, được trao giải "Test of Time" của hội nghị KDD năm 2014. DBSCAN định nghĩa cụm là **vùng có mật độ điểm cao**, nhờ đó tìm được cụm có **hình dạng bất kỳ** và tự nhận diện **nhiễu**, không cần biết trước số cụm.

Định nghĩa 11.4 (Các khái niệm của DBSCAN)**.** *Với hai tham số ε (`eps`) và MinPts (`min_samples`):*

- ε-***lân cận*** *của* p: $N_\varepsilon(p) = \{q : d(p,q) \le \varepsilon\}$ *(bao gồm cả* p*).*
- ***Điểm lõi*** *(core):* $|N_\varepsilon(p)| \ge$ *MinPts.*
- ***Điểm biên*** *(border): không phải điểm lõi nhưng nằm trong* ε*-lân cận của một điểm lõi.*
- ***Điểm nhiễu*** *(noise): không phải lõi, cũng không phải biên.*

Một cụm là tập lớn nhất các điểm ***liên thông mật độ****: bắt đầu từ một điểm lõi, lan rộng qua các điểm lõi trong lân cận của nhau, kèm theo các điểm biên của chúng.*

Ví dụ 11.4 (Chạy tay DBSCAN)**.** *Cho 9 điểm (đánh số từ 0 theo Python),* $\varepsilon = 2$*, MinPts* $= 3$*:*

Điểm	**Tọa độ**	ε**-lân cận**	$\lvert N_\varepsilon\rvert$	**Loại**
0	(2, 10)	{0}	1	Nhiễu
1	(2, 5)	{1}	1	Nhiễu
2	(8, 4)	{2, 4, 5}	3	**Lõi**
3	(5, 8)	{3, 7}	2	Biên (gần lõi 7)
4	(7, 5)	{2, 4, 5}	3	**Lõi**
5	(6, 4)	{2, 4, 5}	3	**Lõi**
6	(1, 2)	{6}	1	Nhiễu
7	(4, 9)	{3, 7, 8}	3	**Lõi**
8	(3, 7.5)	{7, 8}	2	Biên (gần lõi 7)

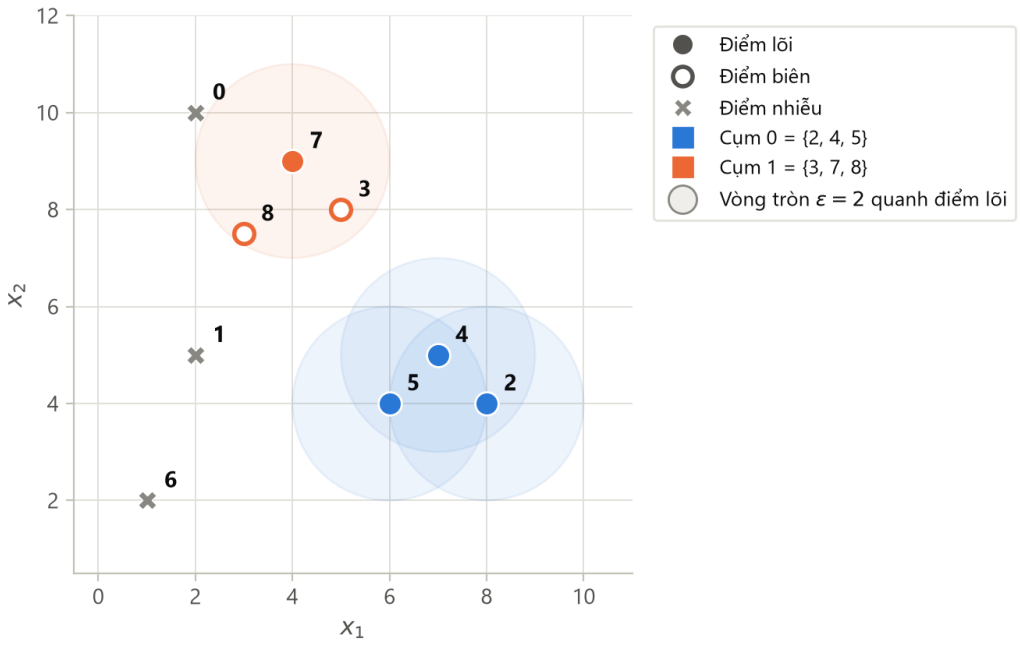
Bảng 11.5: Phân loại điểm của DBSCAN ($\varepsilon = 2$, *MinPts* $= 3$)

Ví dụ $d(2,4) = \sqrt{1^2+1^2} = 1{,}41$, $d(2,5) = 2{,}00$, $d(7,3) = 1{,}41$, $d(7,8) = 1{,}80$, còn $d(3,8) = 2{,}06 > 2$. Kết quả: cụm 0 = {2, 4, 5}, cụm 1 = {3, 7, 8} (lõi 7 với hai điểm biên 3 và 8), nhiễu = {0, 1, 6}.

Listing 11.6: DBSCAN với scikit-learn (ví dụ trên lớp)

```
from sklearn.cluster import DBSCAN

X = [[2, 10], [2, 5], [8, 4], [5, 8], [7, 5],
     [6, 4], [1, 2], [4, 9], [3, 7.5]]
dbscan = DBSCAN(eps=2, min_samples=3).fit(X)
print(dbscan.labels_)              # [-1 -1  0  1  0  0 -1  1  1]  (-1 = nhieu)
print(dbscan.core_sample_indices_)  # [2 4 5 7]
print(dbscan.components_)           # toa do cac diem loi
```



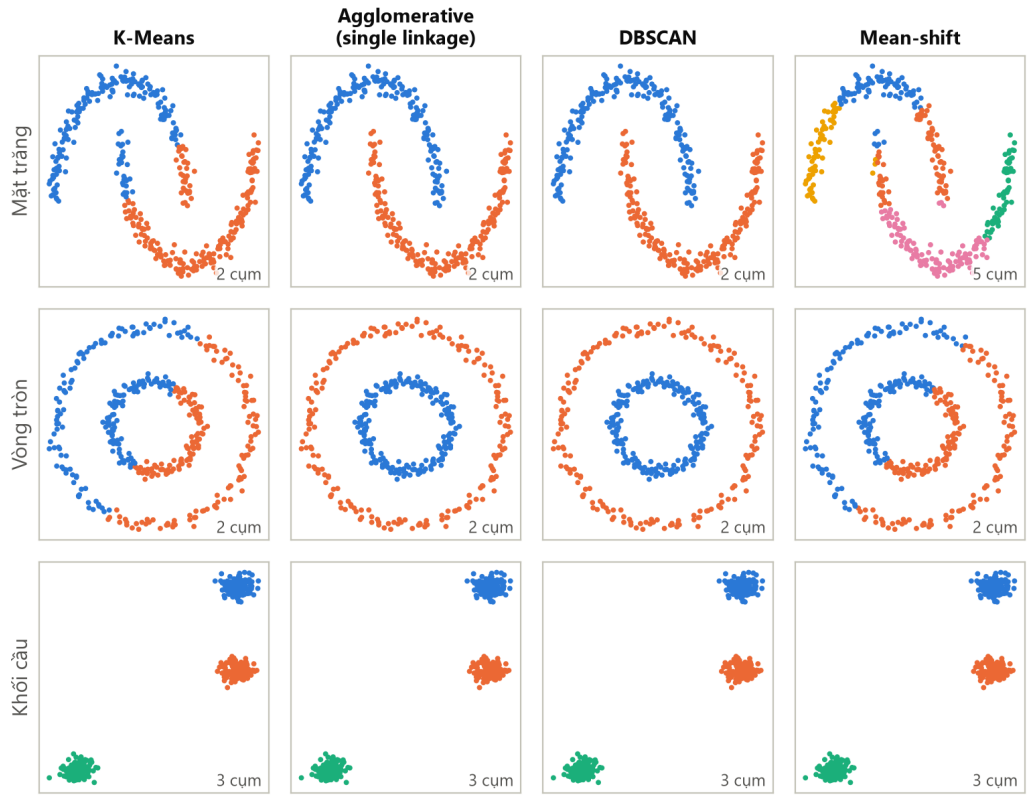
Hình 11.4: Kết quả DBSCAN trên 9 điểm: điểm lõi, điểm biên, điểm nhiễu và vòng tròn bán kính $\varepsilon = 2$ quanh các điểm lõi.

Chọn tham số. Quy tắc kinh nghiệm: *MinPts* $\geq d + 1$ (thường $2d$); để chọn ε, vẽ **đồ thị k-khoảng cách**: khoảng cách từ mỗi điểm đến láng giềng thứ *MinPts*, sắp xếp tăng dần; điểm "gãy" của đồ thị là ứng viên cho ε. DBSCAN gặp khó khi các cụm có **mật độ rất khác nhau** — khi đó dùng OPTICS (Ankerst và cộng sự, 1999) hoặc HDBSCAN.

11.6 So sánh và ứng dụng

Thuật toán	**Cần biết K**	**Hình dạng cụm**	**Phát hiện nhiễu**	**Tham số chính**	**Quy mô dữ liệu**
K-Means	Có	Cầu, đều nhau	Không	K	Rất lớn
Phân cấp	Không (cắt cây)	Tùy liên kết	Không	linkage, ngưỡng cắt	Nhỏ – vừa
DBSCAN	Không	Tùy ý	Có	`eps`, `min_samples`	Vừa – lớn
Mean-shift	Không	Tùy ý	Không	`bandwidth`	Nhỏ – vừa

Bảng 11.6: So sánh các thuật toán phân cụm



Hình 11.5: Kết quả của K-Means, Agglomerative, DBSCAN và Mean-shift trên các tập dữ liệu dạng "mặt trăng", "vòng tròn" và "khối cầu" (đã chuẩn hóa). K-Means và Agglomerative được cho trước số cụm đúng; DBSCAN và Mean-shift tự tìm số cụm.

Listing 11.7: Phân khúc khách hàng: tiền xử lý rồi phân cụm

```
import numpy as np
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# 400 khach hang gia lap thuoc 4 nhom: tuoi, thu nhap (trieu/thang),
# so lan mua trong nam, tong chi tieu (trieu/nam)
rng = np.random.default_rng(0)
centers = [[25, 8, 30, 12], [35, 20, 10, 25], [50, 35, 5, 40], [40, 12, 20, 8]]
data = np.vstack([rng.normal(c, [4, 3, 3, 4], (100, 4)) for c in centers])
customers = pd.DataFrame(data.round(1),
                         columns=["tuoi", "thu_nhap", "so_lan_mua", "chi_tieu"
    ])

kmeans = KMeans(n_clusters=4, n_init=10, random_state=0)
seg = make_pipeline(StandardScaler(), kmeans)       # chuan hoa roi phan cum
customers["segment"] = seg.fit_predict(customers)
print(customers.groupby("segment").mean().round(1))  # chan dung tung phan khuc
#          tuoi  thu_nhap  so_lan_mua  chi_tieu
# segment
```

```
# 0        49.6      34.6       4.9      40.3
# 1        35.1      20.0       9.6      25.1
# 2        24.9       8.0      29.5      12.2
# 3        40.7      11.6      20.3       8.4

# Truc quan hoa tren 2 chieu bang PCA (xem chuong Giam chieu du lieu)
X_std = StandardScaler().fit_transform(customers.drop(columns="segment"))
X2 = PCA(2).fit_transform(X_std)
```

11.7 Tóm tắt chương

- Phân cụm là học không giám sát, dựa trên khoảng cách nên bắt buộc tiền xử lý đúng.
- K-Means lặp hai bước gán – cập nhật để cực tiểu WCSS; dùng K-Means++ và nhiều lần khởi tạo; chọn K bằng elbow và Silhouette.
- Fuzzy C-Means là phân cụm mềm: mỗi điểm có độ thuộc $w_{ij} \in [0, 1]$ vào từng cụm; tâm là trung bình có trọng số w_{ij}^m; hệ số mờ $m \to 1$ thì trở về K-Means.
- Phân cụm phân cấp xây dendrogram bằng cách gộp dần theo tiêu chí liên kết; cắt cây để lấy số cụm.
- DBSCAN tìm cụm theo mật độ, phân loại điểm lõi – biên – nhiễu, tìm được cụm hình dạng tùy ý.
- Mean-shift dịch chuyển điểm về đỉnh mật độ, không cần biết số cụm.

11.8 Bài tập

1. Chạy tay K-Means cho 5 điểm ở ví dụ nhưng khởi tạo tâm là P_1 và P_2. Kết quả cuối có giống ví dụ không? Rút ra nhận xét về vai trò của khởi tạo.
2. Cho 6 điểm 1 chiều: $2, 3, 8, 9, 10, 25$. Vẽ dendrogram với single linkage và complete linkage.
3. Với 9 điểm của ví dụ DBSCAN, chạy lại với $\varepsilon = 3$, *MinPts* $= 3$. Lập bảng phân loại điểm như Bảng 11.5.
4. Sinh dữ liệu `make_moons(n_samples=300, noise=0.05)`, so sánh K-Means ($K = 2$) và DBSCAN. Giải thích sự khác biệt.
5. Với dữ liệu ví dụ Fuzzy C-Means, chạy lại hàm `fuzzy_cmeans` với $m \in \{1{,}1;\ 2;\ 4\}$ và lập bảng độ thuộc của điểm $x = 3$ khi hội tụ. Giải thích vì sao $m \to 1$ cho kết quả gần K-Means.
6. Lượng tử hóa màu: dùng K-Means với $K \in \{4, 16, 64\}$ để nén màu một bức ảnh (mỗi pixel là một điểm trong không gian RGB). So sánh ảnh kết quả.
7. Phân cụm các căn nhà trong Ames Housing theo `Gr Liv Area`, `Overall Qual`, `Year Built` (đã chuẩn hóa). Mô tả đặc điểm và giá trung bình của từng cụm.

Tài liệu tham khảo của chương

1. Ankerst, M., Breunig, M. M., Kriegel, H.-P., & Sander, J. (1999). OPTICS: Ordering Points To Identify the Clustering Structure. *Proceedings of ACM SIGMOD '99*, 49–60.

2. Arthur, D., & Vassilvitskii, S. (2007). k-means++: The Advantages of Careful Seeding. *Proceedings of SODA '07*, 1027–1035.

3. Bezdek, J. C. (1981). *Pattern Recognition with Fuzzy Objective Function Algorithms*. Plenum Press.

4. Comaniciu, D., & Meer, P. (2002). Mean Shift: A Robust Approach Toward Feature Space Analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 24(5), 603–619.

5. Dunn, J. C. (1973). A Fuzzy Relative of the ISODATA Process and Its Use in Detecting Compact Well-Separated Clusters. *Journal of Cybernetics*, 3(3), 32–57.

6. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise. *Proceedings of KDD '96*, 226–231.

7. Fukunaga, K., & Hostetler, L. (1975). The estimation of the gradient of a density function, with applications in pattern recognition. *IEEE Transactions on Information Theory*, 21(1), 32–40.

8. Johnson, S. C. (1967). Hierarchical clustering schemes. *Psychometrika*, 32(3), 241–254.

9. Lloyd, S. P. (1982). Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 129–137.

10. MacQueen, J. (1967). Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, 1, 281–297.

11. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*, Chương 10: Working with Unlabeled Data – Clustering Analysis. Packt Publishing.

12. Rousseeuw, P. J. (1987). Silhouettes: a graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.

13. Ward, J. H. (1963). Hierarchical Grouping to Optimize an Objective Function. *Journal of the American Statistical Association*, 58(301), 236–244.

14. Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3), 338–353.

Phần IV

Thực hành tổng hợp

Chương 12: Thực hành tổng hợp (Labs)

Các chương trước trình bày từng thuật toán riêng lẻ. Chương này đặt chúng vào **quy trình làm việc thực tế** thông qua tám bài thực hành. Bảy bài đầu được xây dựng từ các notebook của lớp học (thư mục `baitap/`); bài cuối cùng ứng dụng Gradient Descent, cài đặt bằng PyTorch. Mỗi bài gồm: mục tiêu, dữ liệu, các bước thực hiện kèm mã nguồn và kết quả thu được khi chạy. Sau khi hoàn thành chương này, người học có khả năng:

- Thực hiện trọn vẹn một quy trình học máy: đọc dữ liệu, tiền xử lý, khảo sát tương quan, huấn luyện nhiều mô hình, tinh chỉnh siêu tham số và đánh giá.
- Đánh giá mô hình đúng cách: chia dữ liệu theo thời gian với chuỗi thời gian và luôn so sánh với một **mô hình cơ sở** (baseline) đơn giản.
- Tìm và sửa lỗi lập trình trong cài đặt thủ công SVD/PCA.
- Cài đặt Gradient Descent theo lô nhỏ bằng PyTorch và kiểm chứng kết quả với lời giải tay.

Notebook	**Nội dung**	**Liên hệ**
`Lab01.ipynb`	Hồi quy giá nhà Ames Housing (Lab 1)	Chương 6, 9
`BTTL.ipynb`	Dự đoán điểm tín dụng ngân hàng (Lab 2)	Chương 8
`Reduce.ipynb`	Nội suy, RobustScaler, SVD, PCA tự cài đặt (Lab 3)	Chương 5, 10
`Untitled0.ipynb`	Dự báo giá cổ phiếu VNM (Lab 4)	Chương 6
`1250080036.ipynb`	Nhận dạng chữ cái viết tay (Lab 5)	Chương 7, 9
`BTTH2.ipynb`	Chữ số viết tay (Lab 6), cảm xúc bình luận tiếng Việt (Lab 7)	Chương 5, 7
—	Gradient Descent với PyTorch (Lab 8)	Chương 6, 7
`KMeans`, `DBScanEx`	Ví dụ phân cụm	Đã dùng ở Chương 11
`PCAExample`, `SVDEXample`	Ví dụ giảm chiều	Đã dùng ở Chương 10
`TFIDF`	Vector hóa văn bản	Đã dùng ở Chương 5

Bảng 12.1: Các notebook trong thư mục `baitap/` và vị trí sử dụng trong giáo trình

Chuẩn bị môi trường. Ngoài bộ cài đặt chuẩn, các bài thực hành cần thêm: `mlxtend` (vẽ ma trận tương quan, ma trận biểu đồ phân tán), `openpyxl` (đọc tệp Excel), `yfinance` (tải dữ liệu chứng khoán), `opencv-python` (đọc ảnh), `pyvi` (tách từ tiếng Việt) và `torch` (PyTorch):

Listing 12.1: Cài đặt thư viện cho các bài thực hành

```
pip install scikit-learn pandas matplotlib seaborn mlxtend openpyxl
pip install yfinance opencv-python pyvi torch
```

Dữ liệu. Mọi tệp dữ liệu được đặt trong thư mục `data/`, cạnh notebook: `AmesHousing.txt` (Lab 1, tải tại `http://jse.amstat.org/v19n3/decock/AmesHousing.txt`), `Banking.xlsx` (Lab 2), thư mục ảnh `alpha2/` (Lab 5) và `handdigits/` (Lab 6), các tệp `Train.csv`, `Dev.csv`, `Test.csv` (Lab 7). Lab 4 tải dữ liệu trực tiếp qua Internet.

12.1 Lab 1 – Hồi quy giá nhà Ames Housing

Mục tiêu. Dự đoán giá bán `SalePrice` từ năm đặc trưng: chất lượng tổng thể (`Overall Qual`, thang 1–10), tình trạng (`Overall Cond`), diện tích sàn ở (`Gr Liv Area`), diện tích tầng hầm (`Total Bsmt SF`) và có điều hòa trung tâm hay không (`Central Air`, Y/N). So sánh hồi quy tuyến tính, hồi quy đa thức và các mô hình cây.

12.1.1 Đọc dữ liệu và tiền xử lý

Listing 12.2: Lab 1 – đọc và tiền xử lý dữ liệu

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

cols = ["Overall Qual", "Overall Cond", "Gr Liv Area",
        "Central Air", "Total Bsmt SF", "SalePrice"]
df = pd.read_csv("data/AmesHousing.txt", usecols=cols, sep="\t")

# Central Air chi co 2 gia tri Y/N -> ma hoa 0/1
df["Central Air"] = LabelEncoder().fit_transform(df["Central Air"])

print(df.isna().sum())          # Total Bsmt SF thieu 1 gia tri
df = df.dropna()                # bo 1 dong: 2930 -> 2929 dong
```

Tham số `usecols` giúp chỉ đọc những cột cần thiết. Vì `Central Air` chỉ có hai giá trị, `LabelEncoder` cho kết quả tương đương mã hóa nhị phân và không tạo thứ tự giả (xem Chương 5). Chỉ một dòng bị thiếu trên 2930 dòng nên xóa dòng là chấp nhận được.

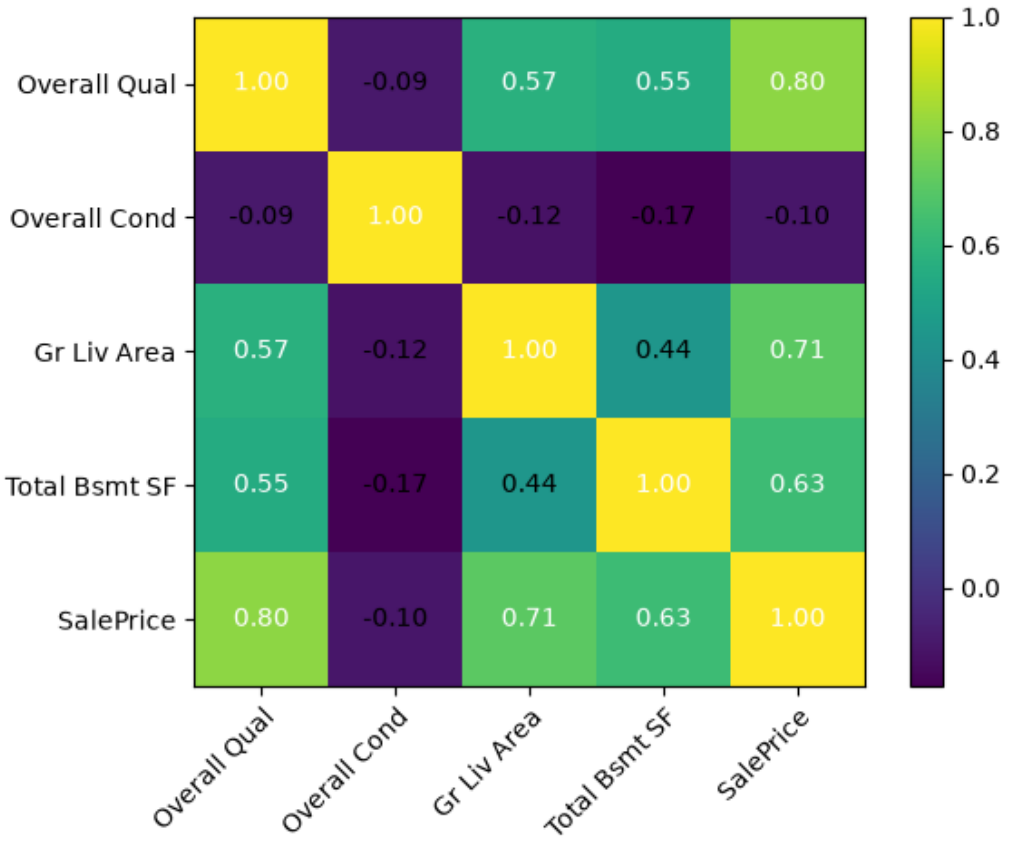
12.1.2 Khảo sát tương quan

Listing 12.3: Lab 1 – ma trận tương quan và ma trận biểu đồ phân tán

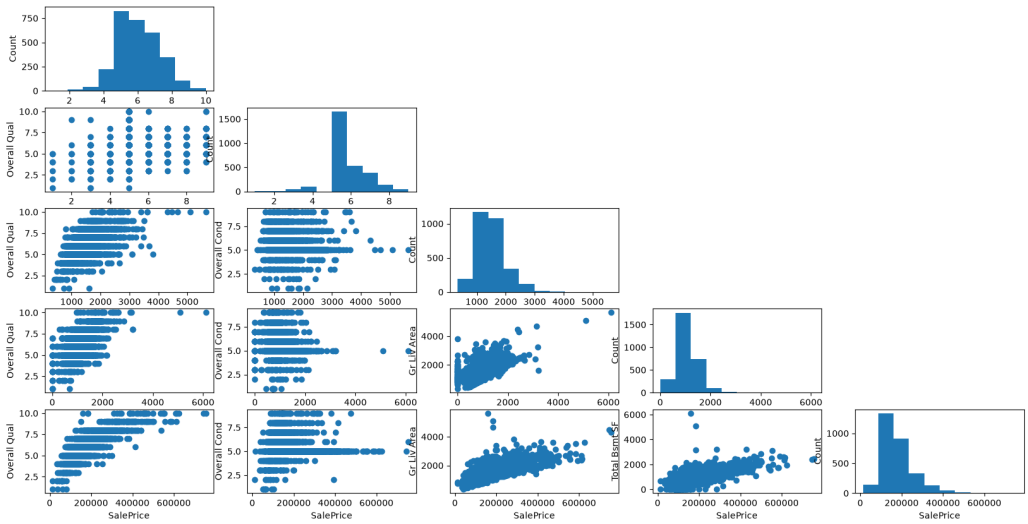
```
import matplotlib.pyplot as plt
from mlxtend.plotting import heatmap, scatterplotmatrix

```

```
cols_con = ["Overall Qual", "Overall Cond", "Gr Liv Area", "Total Bsmt SF", "
    SalePrice"]
heatmap(df[cols_con].corr().values, row_names=cols_con, column_names=cols_con)
plt.show()
scatterplotmatrix(df[cols_con].values, names=cols_con, figsize=(20, 10))
plt.show()
```



Hình 12.1: Ma trận hệ số tương quan Pearson giữa các biến số của Ames Housing (kết quả chạy `Lab01.ipynb`).



Hình 12.2: Ma trận biểu đồ phân tán: đường chéo là histogram của từng biến, các ô còn lại là biểu đồ phân tán từng cặp biến.

Ma trận tương quan cho thấy `SalePrice` tương quan mạnh với `Overall Qual` ($r = 0{,}80$), `Gr Liv Area` ($0{,}71$) và `Total Bsmt SF` ($0{,}63$), nhưng gần như không tương quan với `Overall Cond` ($-0{,}10$). Các đặc trưng cũng tương quan với nhau (chẳng hạn `Overall Qual` và `Gr Liv Area`: $0{,}57$) — dấu hiệu đa cộng tuyến ở mức vừa phải. Ma trận biểu đồ phân tán còn cho thấy quan hệ giữa giá và diện tích có dạng "quạt" (phương sai tăng theo giá trị), và một vài căn diện tích rất lớn nhưng giá thấp — chính là các ngoại lai đã bàn ở Chương 5.

12.1.3 Huấn luyện và so sánh mô hình

Listing 12.4: Lab 1 – huấn luyện các mô hình

```
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import (RandomForestRegressor, AdaBoostRegressor,
                              HistGradientBoostingRegressor)

X = df.drop(columns="SalePrice")
y = df["SalePrice"]
X_train, X_test, y_train, y_test = train_test_split(X, y)   # 2196 / 733 dong

lg = LinearRegression().fit(X_train, y_train)
print(lg.score(X_train, y_train), lg.score(X_test, y_test))  # R^2 train, test

pf = PolynomialFeatures(degree=2)                  # them x_i^2 va x_i*x_j
X_train_pf = pf.fit_transform(X_train)
X_test_pf = pf.transform(X_test)                   # chi transform tap test
lg2 = LinearRegression().fit(X_train_pf, y_train)

dt = DecisionTreeRegressor().fit(X_train, y_train)
grid = GridSearchCV(DecisionTreeRegressor(), param_grid={
```

```
    "max_depth": [5, 6, 7],
    "min_samples_split": [2, 3, 4],
    "min_samples_leaf": [1, 2, 3, 4]})
grid.fit(X_train, y_train)
print(grid.best_score_, grid.best_params_)

rf = RandomForestRegressor().fit(X_train, y_train)
ada = AdaBoostRegressor().fit(X_train, y_train)
hist = HistGradientBoostingRegressor().fit(X_train, y_train)
```

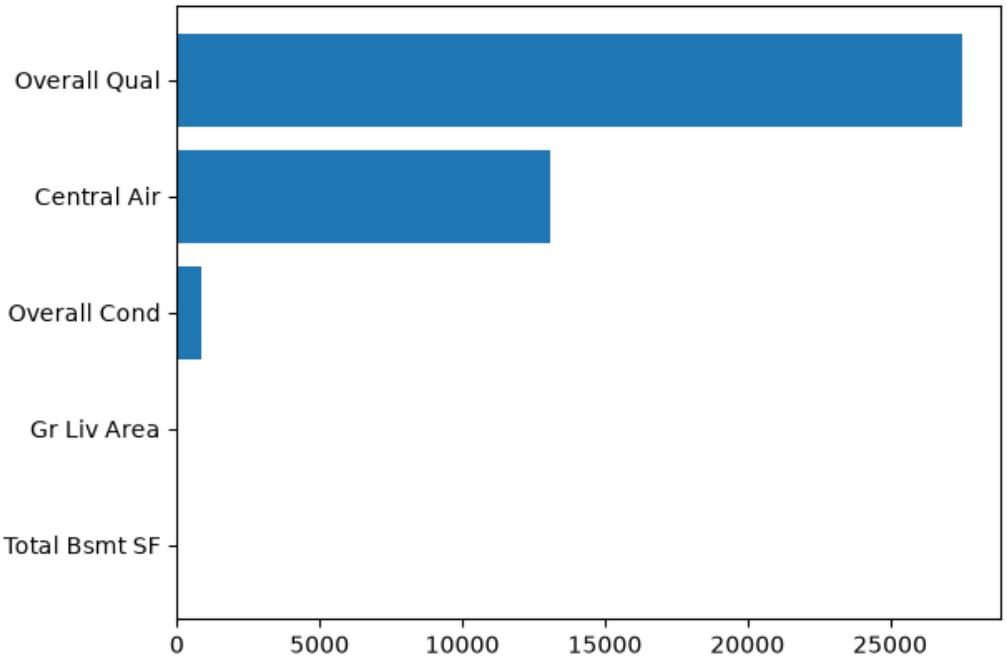
Mô hình	R^2 **train**	R^2 **test**	**Nhận định**
Hồi quy tuyến tính	0,758	0,782	Thiếu khớp nhẹ
Hồi quy đa thức bậc 2	0,848	0,850	Cải thiện rõ, không quá khớp
Cây quyết định (mặc định)	0,995	0,769	**Quá khớp** nặng
Cây quyết định (đã tinh chỉnh)	0,877	0,823	Khoảng cách train/test thu hẹp
Random Forest (mặc định)	0,974	**0,853**	Tốt nhất trên tập test
AdaBoost (mặc định)	0,824	0,779	Kém nhất nhóm cây
HistGradientBoosting (mặc định)	0,903	0,839	Tốt, nhanh

Bảng 12.2: Kết quả Lab 1 khi chạy `Lab01.ipynb`; các con số thay đổi nhẹ giữa các lần chạy vì chia dữ liệu ngẫu nhiên

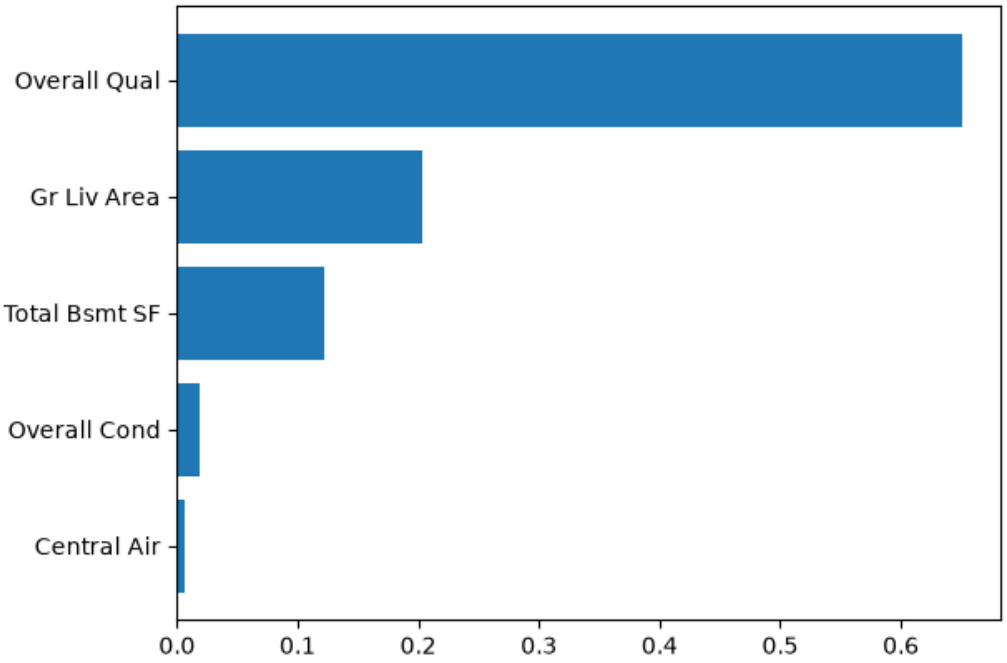
Bảng 12.2 minh họa trọn vẹn đánh đổi bias – variance (Chương 8): mô hình tuyến tính quá đơn giản; thêm đặc trưng bậc hai giúp nắm bắt quan hệ phi tuyến (ví dụ tương tác `Overall Qual` $\times$ `Gr Liv Area`); cây quyết định không giới hạn độ sâu học thuộc tập huấn luyện ($R^2 = 0{,}995$) nhưng tổng quát kém; giới hạn độ sâu bằng `GridSearchCV` hoặc lấy trung bình nhiều cây (Random Forest) khắc phục được điều đó.

Sai số trên tập kiểm tra. Notebook tính MSE của bốn mô hình cây: Random Forest $9{,}77 \times 10^8$, HistGradientBoosting $1{,}07 \times 10^9$, Decision Tree $1{,}09 \times 10^9$, AdaBoost $1{,}47 \times 10^9$. Lấy căn bậc hai để đưa về đơn vị USD: RMSE lần lượt khoảng **31 250**, **32 720**, **33 050** và **38 330** USD — dễ diễn giải hơn nhiều so với MSE.

12.1.4 Mức độ quan trọng của đặc trưng



Hình 12.3: Hệ số hồi quy tuyến tính được vẽ như "mức độ quan trọng" — một cách diễn giải **dễ gây hiểu nhầm** (xem nhận xét bên dưới).



Hình 12.4: Mức độ quan trọng của đặc trưng theo Random Forest (`feature_importances_`): `Overall Qual` chiếm khoảng 65%.

Hồi quy tuyến tính cho hệ số lớn nhất ở `Overall Qual` ($\approx$ 27 512) và `Central Air` ($\approx$ 13 066), trong khi `Gr Liv Area` chỉ có hệ số ≈ 50. Random Forest lại xếp hạng: `Overall`

`Qual` (0,650), `Gr Liv Area` (0,203), `Total Bsmt SF` (0,121), `Overall Cond` (0,019), `Central Air` (0,006). Hai bảng xếp hạng mâu thuẫn ở `Central Air` — lý do được giải thích ngay dưới đây.

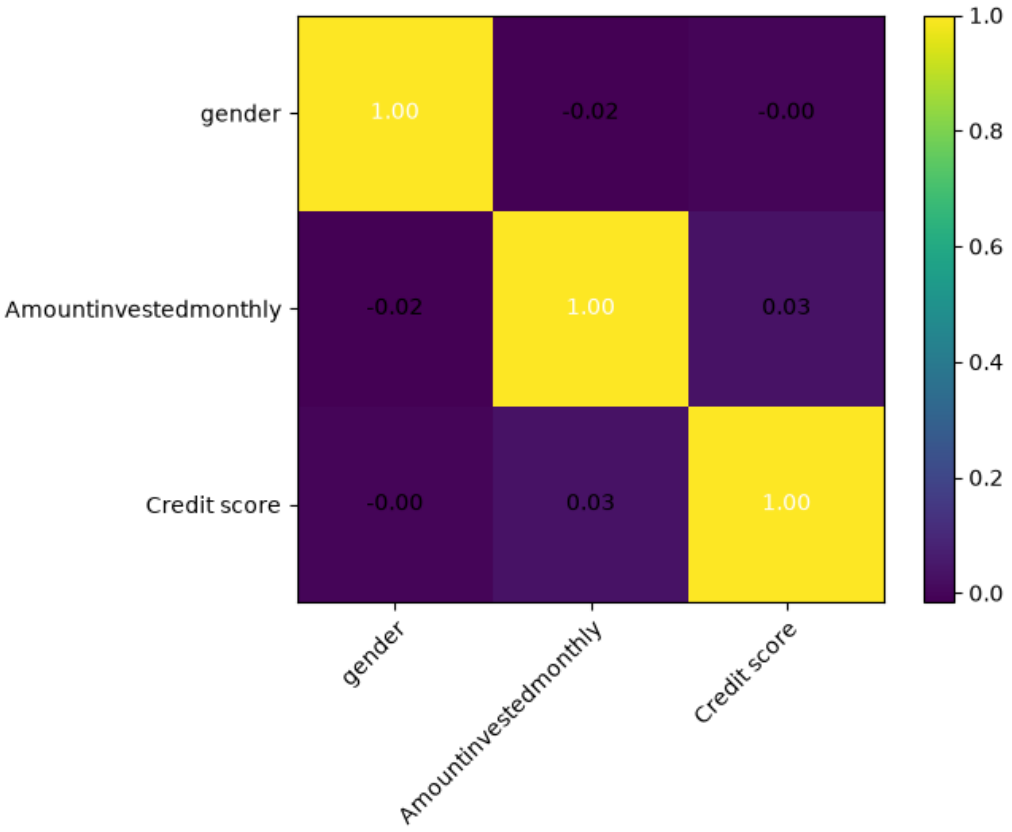
12.2 Lab 2 – Dự đoán điểm tín dụng: khi dữ liệu không có thông tin

Mục tiêu. Áp dụng đúng quy trình của Lab 1 cho tệp `Banking.xlsx`: dự đoán điểm tín dụng (`Credit score`) từ giới tính (`gender`) và số tiền đầu tư hằng tháng (`Amountinvestedmonthly`). Bài này được đưa vào giáo trình chính vì nó **thất bại** — và thất bại theo cách rất đáng học.

Listing 12.5: Lab 2 – đọc dữ liệu từ Excel và chạy cùng quy trình

```
cols = ["gender", "Amountinvestedmonthly", "Credit score"]
df = pd.read_excel("data/Banking.xlsx", usecols=cols)   # can openpyxl
df["gender"] = LabelEncoder().fit_transform(df["gender"])
print(df.isna().sum())        # Credit score thieu 4, Amountinvestedmonthly
    thieu 331
df = df.dropna()              # con 6708 dong

X = df.drop(columns="Credit score")
y = df["Credit score"]
X_train, X_test, y_train, y_test = train_test_split(X, y)   # 5031 / 1677 dong
```



Hình 12.5: Ma trận tương quan của Lab 2: `Credit score` gần như không tương quan với cả hai đặc trưng ($r \approx 0{,}00$ và $0{,}03$).

Mô hình	R^2 **train**	R^2 **test**
Hồi quy tuyến tính	0,001	0,001
Hồi quy đa thức bậc 2	0,001	0,000
Cây quyết định (mặc định)	**0,957**	**−0,881**
Cây quyết định (đã tinh chỉnh)	0,015	−0,032
Random Forest (mặc định)	0,770	−0,297
Random Forest (đã tinh chỉnh)	0,053	−0,007
AdaBoost	−0,141	−0,138
HistGradientBoosting	0,062	−0,023

Bảng 12.3: Kết quả Lab 2 khi chạy `BTTL.ipynb`. Sai số trên tập test: MAE $\approx$ 39 điểm, MSE $\approx$ 2100 (RMSE $\approx$ 46 điểm) với cả bốn mô hình cây.

12.3 Lab 3 – Nội suy, RobustScaler và tự cài đặt SVD, PCA

Mục tiêu. Kiểm chứng bằng tay các phép biến đổi đã học, và quan trọng hơn: học cách **phát hiện lỗi** khi tự cài đặt thuật toán bằng cách đối chiếu với thư viện.

12.3.1 Nội suy tuyến tính giá trị thiếu

Với dữ liệu có **thứ tự** (chuỗi thời gian, chuỗi đo liên tiếp), điền giá trị thiếu bằng trung bình cả cột (mục 5.4) bỏ phí thông tin về vị trí. **Nội suy tuyến tính** nối thẳng hai giá trị đã biết ở hai bên khoảng trống:

$$x_t = x_a + \frac{t-a}{b-a}\,(x_b - x_a), \qquad a < t < b.$$

Ví dụ 12.1 (Nội suy tuyến tính)**.** *Chuỗi* $(75, 74, ?, 84, 86, ?, ?, 95)$*. Ô thứ 3 nằm giữa 74 và 84:* $\frac{74+84}{2} = 79$*. Hai ô thứ 6, 7 nằm giữa 86 và 95, bước tăng* $\frac{95-86}{3} = 3$*: điền 89 và 92.*

Listing 12.6: Nội suy bằng pandas

```
import numpy as np
import pandas as pd

s = pd.Series([75, 74, np.nan, 84, 86, np.nan, np.nan, 95])
print(s.interpolate().tolist())   # [75.0, 74.0, 79.0, 84.0, 86.0, 89.0, 92.0,
    95.0]
```

12.3.2 RobustScaler tính tay

Với $x = (3, 7, 1, 6, 8, 4, 2, 5, 9, 11)$, sắp xếp được $(1, 2, 3, 4, 5, 6, 7, 8, 9, 11)$. NumPy tính tứ phân vị bằng nội suy tuyến tính tại vị trí $(n-1)q$: Q_1 ở vị trí $9 \times 0{,}25 = 2{,}25$ nên $Q_1 = 3 + 0{,}25 \times (4-3) = 3{,}25$; tương tự $Q_3 = 7{,}75$; trung vị $= \frac{5+6}{2} = 5{,}5$. Công thức của RobustScaler cho

$$x' = \frac{x - \text{median}}{Q_3 - Q_1} = \frac{x - 5{,}5}{4{,}5},$$

chẳng hạn $x = 3 \mapsto -0{,}556$ và $x = 11 \mapsto 1{,}222$ — trùng khớp với kết quả của `RobustScaler`. Trung bình của x là $5{,}6$, gần trung vị vì dữ liệu này không có ngoại lai.

12.3.3 Tự cài đặt SVD: một lỗi chỉ số tinh vi

Notebook tính SVD rút gọn cho $\mathbf{X} = \begin{pmatrix} 3 & 2 & 7 \\ 1 & 4 & 6 \end{pmatrix}$ qua trị riêng của $\mathbf{X}^\top\mathbf{X}$ (mục SVD ở Chương 10):

$$\mathbf{X}^\top\mathbf{X} = \begin{pmatrix} 10 & 10 & 27 \\ 10 & 20 & 38 \\ 27 & 38 & 85 \end{pmatrix}, \qquad \lambda \approx (110{,}69;\ 4{,}31;\ 0), \qquad \sigma = \sqrt{\lambda} \approx (10{,}52;\ 2{,}08).$$

Các giá trị suy biến khớp với `TruncatedSVD`, nhưng tọa độ chiếu thì **không**. Nguyên nhân nằm ở một dòng lệnh:

Listing 12.7: Lỗi sắp xếp vector riêng theo dòng thay vì theo cột

```
M = X.T @ X
eigv, eigvec = np.linalg.eigh(M)          # vector rieng nam o CAC COT
orders = np.argsort(eigv)[::-1]

```

```
sigma = eigvec[orders][:, :2]          # SAI: sap xep lai cac DONG
sigma = eigvec[:, orders][:, :2]       # DUNG: sap xep lai cac COT
reduce = X @ sigma
```

`np.linalg.eigh` trả về vector riêng theo **cột**; `eigvec[orders]` hoán vị các **dòng**, tạo ra một ma trận không còn là các vector riêng. Kết quả sai là $\begin{pmatrix} -4{,}76 & 3{,}28 \\ -5{,}95 & 0{,}82 \end{pmatrix}$; kết quả đúng là $\begin{pmatrix} -7{,}75 & 1{,}40 \\ -7{,}12 & -1{,}53 \end{pmatrix}$, trùng với `TruncatedSVD` (sai khác về dấu là bình thường, xem Chương 10).

12.3.4 Tự cài đặt PCA: hai lỗi chồng lên nhau

Với cùng ma trận $\mathbf{X}$, trung bình cột $\bar{\mathbf{x}} = (2;\ 3;\ 6{,}5)$ và dữ liệu đã chuẩn hóa tâm là $\pm(1;\ -1;\ 0{,}5)$. Ma trận hiệp phương sai (chia $n-1=1$) có đúng một trị riêng khác 0 là $\lambda_1 = 4{,}5$, vector riêng $\mathbf{w}_1 = \left(\frac{2}{3};\ -\frac{2}{3};\ \frac{1}{3}\right)$, và tọa độ chiếu $z = \pm 1{,}5$ — đúng như `PCA(n_components=1)`.

Notebook lại cho $z = (6{,}43;\ 6{,}54)$ do hai lỗi:

1. Lặp lại lỗi sắp xếp dòng `eigenvectors[orders]` như ở mục trên.
2. Chiếu **dữ liệu gốc** `X` thay vì dữ liệu đã chuẩn hóa tâm `M`. PCA phải chiếu $\mathbf{X} - \bar{\mathbf{x}}$.

Ngoài ra notebook chia ma trận hiệp phương sai cho $n = 2$ (được $\lambda = 2{,}25$), trong khi scikit-learn chia cho $n-1$ (được `explained_variance_` $= 4{,}5$). Đây không phải lỗi, nhưng cần biết để đối chiếu đúng.

Listing 12.8: PCA tự cài đặt – phiên bản đúng

```
X = np.array([[3, 2, 7], [1, 4, 6]], dtype=float)
M = X - X.mean(axis=0)                 # 1) chuan hoa tam
S = M.T @ M / (X.shape[0] - 1)         # 2) chia n - 1 nhu scikit-learn
eigenvalues, eigenvectors = np.linalg.eigh(S)
orders = np.argsort(eigenvalues)[::-1]
W = eigenvectors[:, orders[:1]]        # 3) chon theo COT
print(eigenvalues[orders][:1])         # [4.5]
print(M @ W)                           # [[ 1.5] [-1.5]]  (co the doi dau)
```

Bài học. Khi tự cài đặt một thuật toán, **luôn đối chiếu với thư viện** trên một ví dụ nhỏ. Giá trị suy biến đúng không chứng minh được vector đúng; chỉ khi *toàn bộ* đầu ra khớp (sai khác dấu được chấp nhận) mới có thể tin cài đặt của mình.

12.4 Lab 4 – Dự báo giá cổ phiếu VNM: ảo ảnh $R^2 = 0{,}998$

Mục tiêu. Dự báo giá đóng cửa ngày mai của cổ phiếu Vinamilk (mã `VNM`, sàn HOSE) từ giá đóng cửa của 7 phiên gần nhất, bằng hồi quy tuyến tính. Đây là cách biến bài toán chuỗi thời gian thành bài toán hồi quy bằng **đặc trưng trễ** (lag features) đã giới thiệu ở Chương 6.

Listing 12.9: Lab 4 – tải dữ liệu và tạo cửa sổ trượt 7 phiên

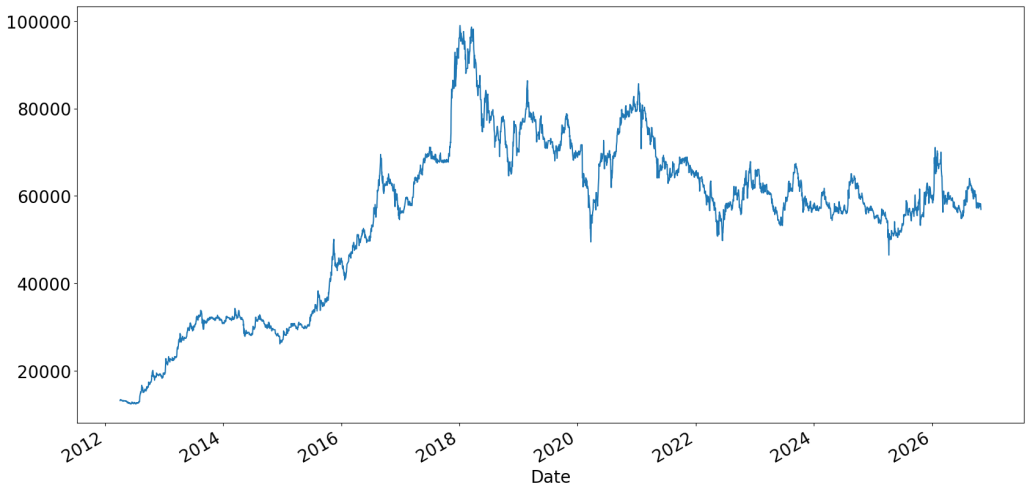
```
import yfinance as yf
import pandas as pd
from sklearn.linear_model import LinearRegression
```

```
from sklearn.model_selection import train_test_split

df = yf.Ticker("VNM.VN").history(period="20y")   # can ket noi Internet
close = df["Close"].dropna().values              # bo phien thieu gia (NaN)

window_size = 7
n = len(close) - window_size
data = {f"x{i}": close[i: n + i] for i in range(window_size)}   # 7 phien truoc
data["y"] = close[window_size:]                                  # phien ke tiep
df2 = pd.DataFrame(data)

X, y = df2.drop(columns="y"), df2["y"]
X_train, X_test, y_train, y_test = train_test_split(X, y)        # !!! xao tron
lg = LinearRegression().fit(X_train, y_train)
print(lg.score(X_train, y_train), lg.score(X_test, y_test))      # ~0.998 ~0.998
```



Hình 12.6: Giá đóng cửa (đã điều chỉnh) của cổ phiếu VNM theo thời gian, tải bằng `yfinance`.

Mô hình đạt $R^2 \approx 0{,}998$ trên cả tập train lẫn tập test. Một kết quả "quá đẹp để là thật" — và đúng là không thật.

12.4.1 Đánh giá lại: chia theo thời gian và so sánh với mô hình cơ sở

1. Rò rỉ dữ liệu do xáo trộn chuỗi thời gian. `train_test_split` mặc định **xáo trộn** các dòng, nên tập test gồm những ngày nằm xen giữa các ngày huấn luyện. Mô hình được "nhìn thấy" giá của ngày hôm trước *và* ngày hôm sau một điểm test — điều không thể có khi dự báo thật. Với chuỗi thời gian phải chia theo thứ tự: huấn luyện trên quá khứ, kiểm tra trên tương lai (`shuffle=False` hoặc `TimeSeriesSplit`).

2. Phải so sánh với mô hình cơ sở ngây thơ. Mô hình "giá ngày mai bằng giá hôm nay" (naive forecast) không cần học gì cả. Chạy lại notebook với dữ liệu VNM từ 03/04/2012 đến 02/11/2026 (3773 phiên) và chia theo thời gian, 75% đầu để huấn luyện:

Cách đánh giá	R^2 **trên tập test**
Hồi quy tuyến tính, chia **xáo trộn** (như notebook)	0,998
Hồi quy tuyến tính, chia **theo thời gian**	0,947
Mô hình ngây thơ: $\hat{y}_{t+1} = y_t$	0,947
Hồi quy tuyến tính dự báo **tỉ suất sinh lời** ngày mai	$-0{,}001$

Bảng 12.4: Lab 4 đánh giá lại (dữ liệu tải ngày 02/11/2026; con số thay đổi theo thời điểm tải)

Khi chia đúng cách, mô hình chỉ ngang bằng mô hình ngây thơ. Hệ số hồi quy giải thích vì sao: trọng số của phiên gần nhất là $0{,}988$, sáu phiên còn lại gần 0 — mô hình đã tự "phát hiện" ra rằng dự báo tốt nhất là *lấy giá hôm nay*.

3. R^2 cao trên giá là chỉ số gây hiểu nhầm. Giá cổ phiếu là chuỗi có xu hướng, giá hôm nay đã gần bằng giá hôm qua, nên bất kỳ mô hình nào "chép lại" giá gần nhất cũng có R^2 rất cao. Đại lượng có ý nghĩa đầu tư là **tỉ suất sinh lời** $r_t = (y_t - y_{t-1})/y_{t-1}$. Dự báo r_{t+1} từ 7 tỉ suất trước đó cho $R^2 \approx 0$: các biến động giá trong quá khứ gần như không chứa thông tin về biến động ngày mai — phù hợp với **giả thuyết thị trường hiệu quả** dạng yếu (Fama, 1970).

Listing 12.10: Lab 4 – đánh giá đúng: chia theo thời gian và so sánh với mô hình ngây thơ

```
import numpy as np
from sklearn.metrics import r2_score

cut = int(0.75 * len(df2))                          # 75% dau = qua khu
X_tr, X_te = X.iloc[:cut], X.iloc[cut:]
y_tr, y_te = y.iloc[:cut], y.iloc[cut:]

lg = LinearRegression().fit(X_tr, y_tr)
print("R2 hoi quy  :", lg.score(X_te, y_te))
print("R2 ngay tho :", r2_score(y_te, X_te["x6"]))   # x6 = gia phien gan nhat
print("He so       :", lg.coef_.round(3))

# Du bao ti suat sinh loi thay vi gia
ret = pd.Series(close).pct_change().dropna().values
m = len(ret) - window_size
Xr = np.column_stack([ret[i: m + i] for i in range(window_size)])
yr = ret[window_size:]
c = int(0.75 * len(yr))
print("R2 loi suat :", LinearRegression().fit(Xr[:c], yr[:c]).score(Xr[c:], yr[
    c:]))
```

4. Một số lưu ý khác. Cột `Close` của `yfinance` là giá đã điều chỉnh theo cổ tức và chia tách cổ phiếu, nên giá năm 2012 thấp hơn giá giao dịch thực tế khi đó. Dữ liệu tải về thay đổi theo ngày chạy, vì vậy kết quả của mỗi người sẽ khác nhau đôi chút; hãy lưu dữ liệu ra tệp CSV để tái lập được thí nghiệm.

12.5 Lab 5 – Nhận dạng chữ cái viết tay từ điểm ảnh

Mục tiêu. Phân loại ảnh chữ cái viết tay thường a–z bằng KNN, cây quyết định và Random Forest; đọc ảnh bằng OpenCV, biến ảnh thành vector đặc trưng, tinh chỉnh siêu tham số và đánh giá bài toán nhiều lớp.

Dữ liệu. Thư mục `alpha2/` có một thư mục con cho mỗi chữ cái, tổng cộng 1144 ảnh của 26 lớp (đúng 44 ảnh mỗi lớp). Tập kiểm tra gồm 286 ảnh. Đoán ngẫu nhiên đều giữa 26 lớp chỉ đạt khoảng $1/26 \approx 3{,}8\%$; đây là mốc để so sánh các kết quả dưới đây.

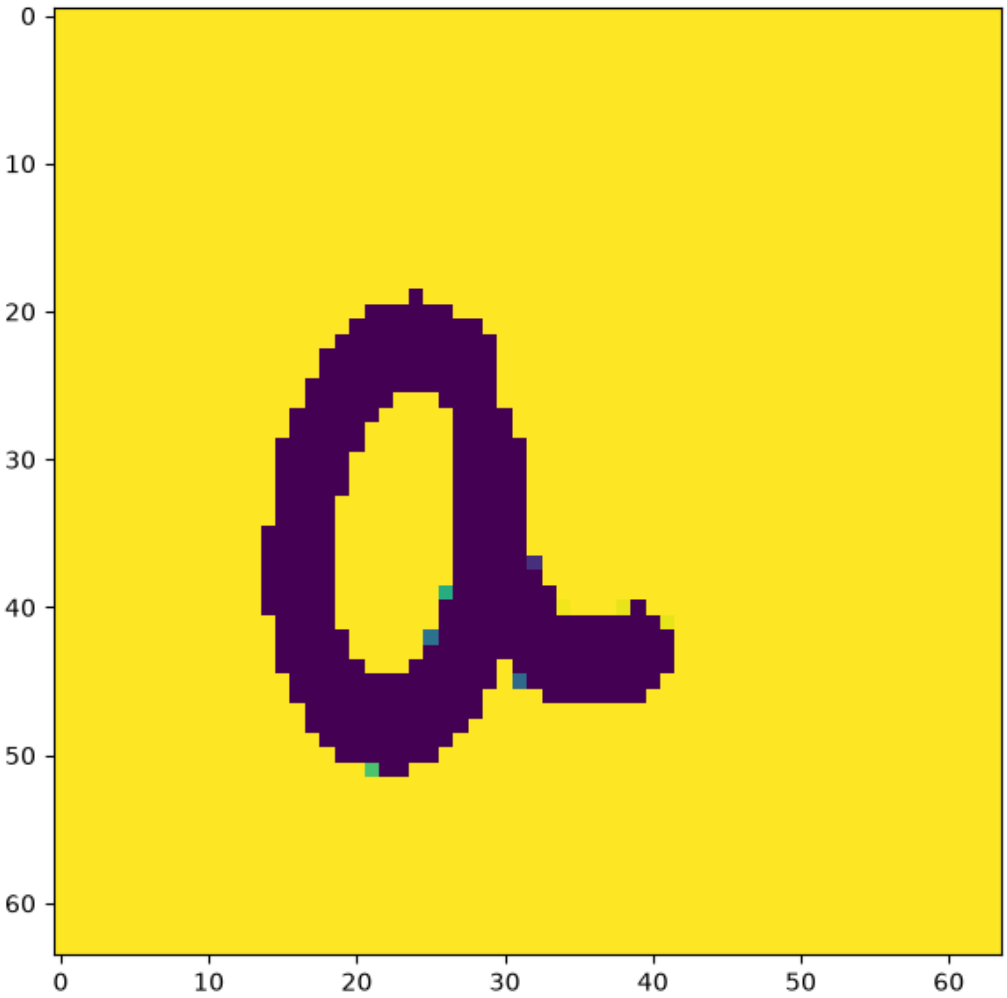
12.5.1 Từ ảnh đến ma trận đặc trưng

Ảnh xám là ma trận số nguyên độ sáng. Các mô hình của chương trước nhận mỗi mẫu là một vector, nên mỗi ảnh được đưa về cùng kích thước 64×64 rồi "duỗi phẳng" thành vector $64 \times 64 = 4096$ chiều.

Listing 12.11: Lab 5 – đọc ảnh và tạo ma trận đặc trưng

```
# pip install opencv-python
from os import listdir
import cv2
import numpy as np

X, y = [], []
for label in listdir("data/alpha2"):
    for name in listdir(f"data/alpha2/{label}"):
        img = cv2.imread(f"data/alpha2/{label}/{name}", cv2.IMREAD_GRAYSCALE)
        img = cv2.resize(img, (64, 64))
        X.append(img.flatten())          # ma tran 64 x 64 -> vector 4096 chieu
        y.append(label)
X, y = np.array(X), np.array(y)
print(X.shape, y.shape)                  # (1144, 4096) (1144,)
```

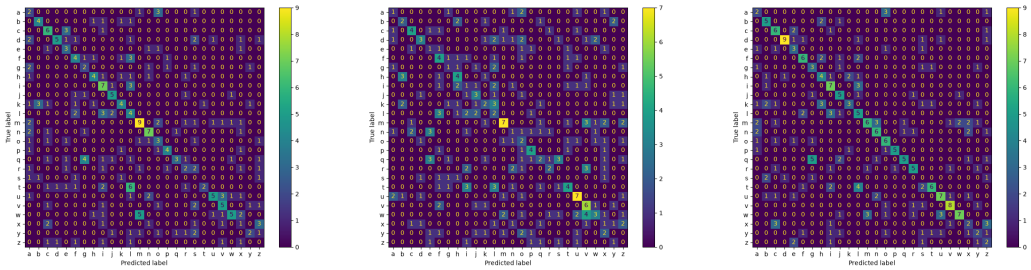


Hình 12.7: Một ảnh chữ a sau khi đưa về 64×64 điểm ảnh; màu do `plt.imshow` tự gán cho độ sáng (kết quả chạy notebook).

12.5.2 Kết quả

Mô hình	Train	Test	Precision	Recall	F1
KNN mặc định	0,621	0,315	—	—	—
Cây quyết định (`max_depth=30`)	1,000	0,234	0,228	0,223	0,218
Random Forest (`max_depth=50`)	1,000	0,343	0,381	0,335	0,333
Random Forest (500 cây, `max_depth=20`)	1,000	**0,437**	0,450	0,431	0,424

Bảng 12.5: Kết quả Lab 5 khi chạy notebook; precision, recall, F1 lấy trung bình macro trên tập kiểm tra. Notebook không tính ba độ đo này cho KNN thật (xem nhận xét 1)

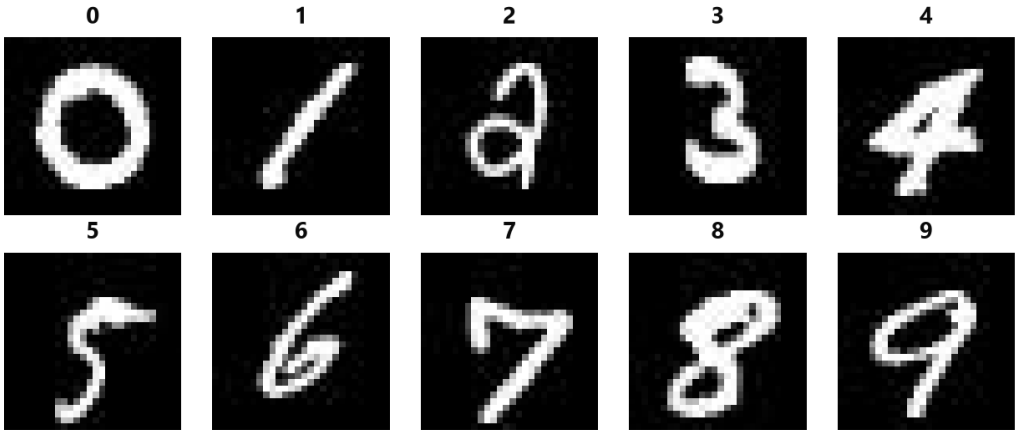


Hình 12.8: Ma trận nhầm lẫn của ba mô hình trong notebook, từ trái sang phải: mô hình được đặt tên `knn` (thực chất là Random Forest, xem nhận xét 1), cây quyết định và Random Forest tốt nhất.

Mô hình tốt nhất đúng 43,7%, gấp hơn 11 lần đoán ngẫu nhiên, nhưng vẫn còn xa mức dùng được. Ma trận nhầm lẫn bên phải cho thấy lỗi có quy luật: chữ `q` hay bị nhận thành `g` (5 ảnh), `t` thành `l` (4 ảnh), `i` thành `l` (3 ảnh), `a` thành `o` (3 ảnh) — đều là những cặp có hình dạng gần nhau.

12.6 Lab 6 – Nhận dạng chữ số viết tay

Mục tiêu. Áp dụng đúng quy trình của Lab 5 cho bài toán chữ số, để thấy cùng một thuật toán cho kết quả khác hẳn khi dữ liệu thay đổi. Thư mục `data/handdigits/` gồm 42 000 ảnh xám 28×28 của 10 chữ số, mỗi lớp có từ 3795 đến 4684 ảnh. Ảnh được duỗi phẳng thành vector 784 chiều.



Hình 12.9: Mỗi chữ số một ảnh mẫu trong thư mục `handdigits`.

Listing 12.12: Lab 6 – đọc ảnh, chia dữ liệu phân tầng và so sánh các mô hình

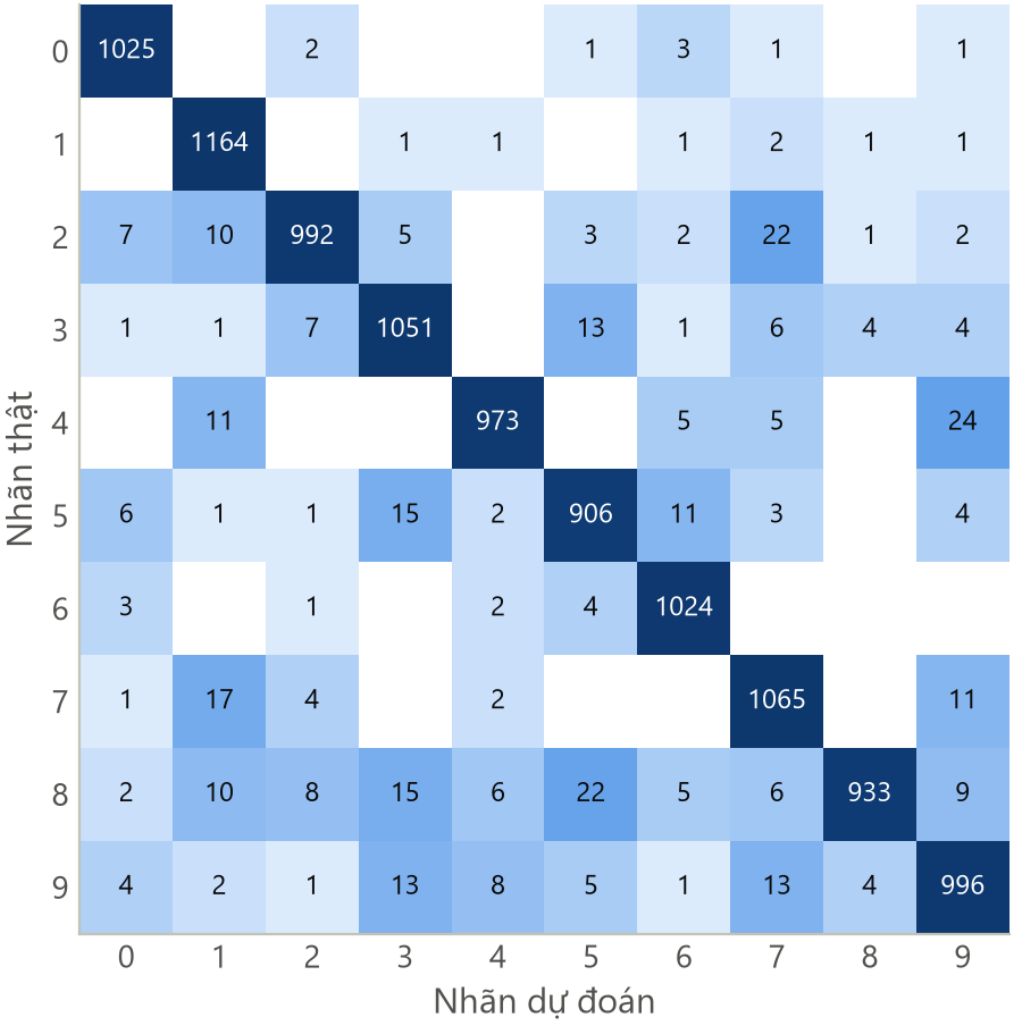
```
from os import listdir
import cv2
import numpy as np
from sklearn.decomposition import PCA
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier

X, y = [], []
for num in listdir("data/handdigits"):
    for name in listdir(f"data/handdigits/{num}"):
        img = cv2.imread(f"data/handdigits/{num}/{name}", cv2.IMREAD_GRAYSCALE)
        X.append(img.flatten())
        y.append(int(num))
X, y = np.array(X), np.array(y)                        # (42000, 784) (42000,)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, stratify=y,
                                          random_state=42)
models = {
    "Dummy": DummyClassifier(strategy="most_frequent"),
    "KNN": KNeighborsClassifier(n_jobs=-1),
    "Decision Tree": DecisionTreeClassifier(max_depth=12, random_state=42),
    "Random Forest": RandomForestClassifier(n_jobs=-1, random_state=42),
    "PCA + KNN": make_pipeline(PCA(n_components=50, random_state=42),
                               KNeighborsClassifier(n_jobs=-1)),
}
for name, m in models.items():
    p = m.fit(X_tr, y_tr).predict(X_te)
    print(name, round((p == y_te).mean(), 3), round(f1_score(y_te, p, average="
    macro"), 3))
```

Mô hình	**Accuracy**	**F1 macro**
Dummy (luôn đoán lớp đông nhất)	0,112	0,020
Decision Tree (`max_depth=12`)	0,839	0,837
Random Forest (mặc định)	0,956	0,956
KNN ($K = 5$)	0,965	0,964
PCA (50 thành phần) + KNN	**0,970**	**0,970**

Bảng 12.6: Kết quả Lab 6 trên 10 500 ảnh kiểm tra, `random_state=42`



Hình 12.10: Ma trận nhầm lẫn của KNN trên tập kiểm tra; độ đậm theo thang log để thấy rõ các ô lỗi nhỏ.

KNN sai 371 trên 10 500 ảnh. Các lỗi nhiều nhất đều hợp lý về hình dạng: 4 nhận thành 9 (24 ảnh), 8 thành 5 (22), 2 thành 7 (22), 7 thành 1 (17). Chữ số 8 có recall thấp nhất (0,918), chữ số 1 cao nhất (0,994).

12.7 Lab 7 – Phân tích cảm xúc bình luận tiếng Việt

Mục tiêu. Phân loại bình luận sản phẩm điện thoại thành tích cực hoặc tiêu cực, vận dụng Count và TF-IDF ở Chương 5, Naive Bayes và hồi quy logistic ở Chương 7.

Dữ liệu. Ba tệp `Train.csv`, `Dev.csv`, `Test.csv` lần lượt có 7786, 1112 và 2224 bình luận. Nhãn được suy ra từ số sao: từ 4 sao trở lên là tích cực, còn lại là tiêu cực. Tập kiểm tra có 62,8% tích cực, nên một mô hình luôn đoán "tích cực" đã đạt accuracy 0,628; đó là mốc phải vượt qua.

Tách từ. Tiếng Việt viết cách theo âm tiết, nên "điện thoại" sẽ bị đếm thành hai từ nếu không xử lý. Hàm `tokenize` của thư viện `pyvi` nối các âm tiết của một từ bằng dấu gạch dưới: `điện_thoại`, `màn_hình`. Từ điển thu được từ tập huấn luyện có 8681 từ.

Listing 12.13: Lab 7 – tách từ, vector hóa và so sánh mô hình

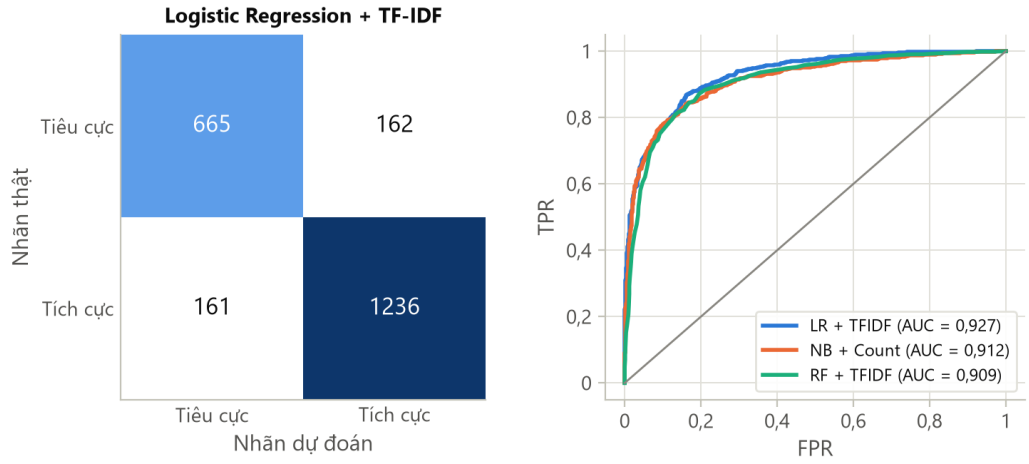
```
# pip install pyvi
import pandas as pd
from pyvi.ViTokenizer import tokenize
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.naive_bayes import MultinomialNB

train, dev, test = (pd.read_csv(f"data/{n}.csv") for n in ["Train", "Dev", "
    Test"])
for d in (train, dev, test):
    d["y"] = (d["n_star"] >= 4).astype(int)     # 1 = tich cuc, 0 = tieu cuc
    d["tokens"] = d["comment"].apply(tokenize)

for vec in [CountVectorizer(), TfidfVectorizer()]:
    A = vec.fit_transform(train["tokens"])      # fit chi tren tap huan luyen
    D, B = vec.transform(dev["tokens"]), vec.transform(test["tokens"])
    for model in [MultinomialNB(), LogisticRegression(max_iter=2000)]:
        model.fit(A, train["y"])
        p = model.predict(B)
        print(type(vec).__name__, type(model).__name__,
              round(model.score(D, dev["y"]), 3), round(model.score(B, test["y"
    ]), 3),
              round(f1_score(test["y"], p, pos_label=0), 3),
              round(roc_auc_score(test["y"], model.predict_proba(B)[:, 1]), 3))
```

Vector hóa	Mô hình	Train	Dev	Test	F1 lớp tiêu cực	AUC
Count	Naive Bayes	0,881	0,856	0,835	0,785	0,912
Count	Random Forest	1,000	0,861	0,845	0,780	0,911
Count	Logistic Regression	0,955	0,865	0,835	0,780	0,915
TF-IDF	Naive Bayes	0,845	0,802	0,800	0,668	0,911
TF-IDF	Random Forest	1,000	0,855	0,840	0,774	0,909
TF-IDF	Logistic Regression	0,902	**0,871**	**0,855**	**0,805**	**0,927**

Bảng 12.7: Kết quả Lab 7. Mô hình được chọn theo cột Dev; cột Test chỉ để báo cáo



Hình 12.11: Trái: ma trận nhầm lẫn của mô hình tốt nhất trên tập kiểm tra. Phải: đường ROC của ba mô hình.

Hệ số của hồi quy logistic cho biết từ nào đẩy dự đoán về mỗi phía, và kết quả rất dễ hiểu. Các từ tiêu cực mạnh nhất gồm *tệ, thất_vọng, chán, đơ, kém, chậm*; các từ tích cực mạnh nhất gồm *tốt, ok, tuyệt_vời, đẹp, mượt, trâu*. Riêng *chê* lại mang trọng số tích cực vì thường đi trong cụm "miễn chê", cho thấy giới hạn của mô hình túi từ: nó không biết thứ tự và ngữ cảnh.

12.8 Lab 8 – Ứng dụng Gradient Descent: từ tính tay đến PyTorch

Mục tiêu. Cài đặt lại ví dụ mini-batch Gradient Descent đã tính tay ở Mục 6.4.2 (hồi quy) và Mục 7.4.2 (phân lớp) bằng PyTorch, một thư viện tính toán có khả năng **tự động tính đạo hàm** (automatic differentiation, gọi tắt là autograd). Qua đó, người học kiểm chứng công thức gradient đã suy ra bằng tay và làm quen với những lỗi thường gặp khi tự viết vòng lặp huấn luyện.

12.8.1 Tensor và autograd: kiểm chứng ví dụ tính tay

Tensor của PyTorch là mảng nhiều chiều giống `ndarray` của NumPy (Chương 4), có thêm hai khả năng: chạy được trên GPU và **ghi lại các phép toán** đã thực hiện. Khi một tensor được khai báo với `requires_grad=True`, mọi phép toán dùng đến nó được ghi thành một **đồ thị tính toán** (computational graph). Việc tính giá trị của hàm mất mát đi theo chiều xuôi của đồ thị và được gọi là **lan truyền xuôi**. Lệnh `backward()` đi theo chiều ngược lại, áp dụng quy tắc dây chuyền (đạo hàm của hàm hợp) để tính đạo hàm, gọi là **lan truyền ngược**, rồi ghi đạo hàm của hàm mất mát theo từng tham số vào thuộc tính `.grad`.

Đoạn mã dưới đây chạy lại **đúng** ví dụ mini-batch Gradient Descent cho phân lớp ở Mục 7.4.2: năm mẫu, hai đặc trưng, `batch_size` $= 3$, khởi tạo $b = 0{,}3$, $w_1 = 0{,}2$, $w_2 = 0{,}1$ và tốc độ học $\alpha = 0{,}1$. Điểm khác biệt là ta **không viết công thức gradient**: chỉ cần khai báo mô hình và hàm mất mát, PyTorch tự tính phần còn lại.

Listing 12.14: Mini-batch Gradient Descent cho hồi quy logistic với autograd của PyTorch

```
1 import torch
2
```

```
X = torch.tensor([
    [1.0, 3.0],
    [2.0, 1.0],
    [1.0, 1.0],
    [7.0, 8.0],
    [6.0, 4.0]
])
y = torch.tensor([0.0, 0.0, 0.0, 1.0, 1.0])
b = torch.tensor(0.3, requires_grad=True)       # can tinh dao ham
w = torch.tensor([0.2, 0.1], requires_grad=True)
loss_func = torch.nn.BCEWithLogitsLoss()        # sigmoid + entropy cheo
batch_size = 3
alpha = 0.1
for start in range(0, len(X), batch_size):
    X_batch = X[start:min(start + batch_size, len(X))]
    y_batch = y[start:min(start + batch_size, len(X))]
    z = b + X_batch @ w                         # lan truyen xuoi
    loss = loss_func(z, y_batch)
    loss.backward()                             # lan truyen nguoc
    with torch.no_grad():                       # cap nhat ngoai do thi
        w -= alpha * w.grad
        b -= alpha * b.grad
    print('Gradient', b.grad, w.grad)
    w.grad.zero_()                              # xoa gradient cu
    b.grad.zero_()
    print('Weight', b.item(), w.tolist())
```

Listing 12.15: Kết quả chạy đoạn mã trên

```
Gradient tensor(0.6752) tensor([0.9052, 1.1352])
Weight 0.23247984051704407 [0.10948067158460617, -0.013518474996089935]
Gradient tensor(-0.2967) tensor([-1.9260, -1.7689])
Weight 0.2621545195579529 [0.30207714438438416, 0.16337350010871887]
```

Bốn dòng kết quả trùng khớp với lời giải tay ở Mục 7.4.2. Lô 1 cho gradient $(0{,}6752;\ 0{,}9052;\ 1{,}1352)$ và trọng số mới $(0{,}2325;\ 0{,}1095;\ -0{,}0135)$. Lô 2 cho gradient $(-0{,}2967;\ -1{,}9260;\ -1{,}7689)$ và trọng số mới $(0{,}2622;\ 0{,}3021;\ 0{,}1634)$. Các chữ số lẻ ở cuối dòng `Weight` xuất hiện vì tensor lưu số thực 32 bit. Sự trùng khớp này khẳng định công thức gradient $\frac{1}{B}\mathbf{X}_B^\top\big(\sigma(\mathbf{z}) - \mathbf{y}\big)$ suy ra bằng tay ở Chương 7 là đúng. Bảng 12.8 đối chiếu từng dòng lệnh với bước tính tay tương ứng.

Dòng lệnh	Ý nghĩa	Bước tính tay tương ứng
`requires_grad=True`	Đánh dấu tensor là tham số cần tính đạo hàm	Các ẩn số b, w_1, w_2
`z = b + X_batch @ w`	Lan truyền xuôi, đồng thời ghi đồ thị tính toán	$z_i = b + w_1 x_{i1} + w_2 x_{i2}$
`BCEWithLogitsLoss()`	Áp dụng sigmoid, tính entropy chéo rồi lấy trung bình trên lô	$L = -\frac{1}{B}\sum_i \big[y_i \ln \hat{y}_i + (1-y_i)\ln(1-\hat{y}_i)\big]$
`loss.backward()`	Lan truyền ngược, ghi đạo hàm vào `.grad`	$\nabla L = \frac{1}{B}\mathbf{X}_B^\top(\hat{\mathbf{y}} - \mathbf{y})$
`w -= alpha * w.grad` trong `torch.no_grad()`	Cập nhật tham số, không ghi phép cập nhật vào đồ thị	$\mathbf{w} \leftarrow \mathbf{w} - \alpha\nabla L$
`w.grad.zero_()`	Xóa gradient của lô vừa xong	Mỗi lô tính gradient lại từ đầu

Bảng 12.8: Đối chiếu mã PyTorch với các bước tính tay

Ở đây hệ số chặn b được khai báo thành một tham số riêng, nên `X` không cần cột toàn số 1 như cách viết ma trận ở Chương 6 và 7; hai cách viết tương đương nhau. Biểu thức `min(start + batch_size, len(X))` chỉ giúp đọc rõ ý, vì phép cắt lát của Python tự dừng ở cuối dãy: lô cuối cùng tự động chỉ còn 2 mẫu.

Vì sao phải gọi `zero_()`? Sau mỗi lần gọi `backward()`, PyTorch **cộng dồn** gradient vào `.grad` chứ không ghi đè. Nếu bỏ hai dòng `zero_()`, gradient dùng để cập nhật ở lô 2 là tổng gradient của hai lô: $(0{,}6752 - 0{,}2967;\ 0{,}9052 - 1{,}9260;\ 1{,}1352 - 1{,}7689) = (0{,}3785;\ -1{,}0208;\ -0{,}6337)$. Trọng số sau epoch khi đó sai thành $(0{,}1946;\ 0{,}2116;\ 0{,}0499)$ thay vì $(0{,}2622;\ 0{,}3021;\ 0{,}1634)$. Chương trình vẫn chạy và không báo lỗi; đây là lỗi phổ biến nhất của người mới dùng PyTorch. Việc cộng dồn là một thiết kế có chủ ý: khi bộ nhớ GPU không chứa nổi một lô lớn, ta chia lô thành nhiều phần, gọi `backward()` cho từng phần rồi mới cập nhật một lần (kỹ thuật tích lũy gradient — gradient accumulation).

Vì sao cập nhật trong `torch.no_grad()`? Phép cập nhật trọng số không phải một phần của mô hình. Nếu để PyTorch ghi phép cập nhật vào đồ thị tính toán, đồ thị sẽ phình to qua mỗi bước. Hơn nữa, PyTorch không cho phép sửa trực tiếp (in-place) một tensor lá đang cần tính đạo hàm ở ngoài khối `no_grad()`.

Vì sao dùng `BCEWithLogitsLoss` thay cho sigmoid rồi `BCELoss`? Hai cách cho cùng kết quả về mặt toán học nhưng khác nhau về **ổn định số**. Xét một mẫu có nhãn $y = 0$ mà mô hình cho $z = 20$, tức là dự đoán sai rất nặng. Trong số thực 32 bit, $\sigma(20) \approx 1 - 2 \times 10^{-9}$ bị làm tròn thành đúng 1, nên $\ln(1 - \sigma(z)) = \ln 0$. Bảng 12.9 cho thấy hậu quả.

z (nhãn $y = 0$)	`BCELoss(sigmoid(z))`		`BCEWithLogitsLoss()(z)`	
	Mất mát	Gradient	Mất mát	Gradient
5	5,0067	0,9933	5,0067	0,9933
20	100	0	20	1
200	100	0	200	1

Bảng 12.9: Mất mát và gradient theo z khi nhãn $y = 0$, tính bằng số thực 32 bit

Với $z = 20$, `BCELoss` chặn mất mát ở 100 và trả về gradient bằng 0: mô hình đang sai nghiêm trọng nhưng không nhận được tín hiệu nào để sửa. `BCEWithLogitsLoss` nhận trực tiếp z và tính theo công thức tương đương $\ln(1 + e^z) - yz$, được sắp xếp để không bao giờ tràn số. Nhờ đó gradient luôn bằng $\sigma(z) - y$, đúng như công thức ở Chương 7.

Hồi quy: chỉ cần đổi hàm mất mát. Thay `loss_func` bằng `torch.nn.MSELoss()` và thay `y` bằng giá trị hồi quy $(4, 3, 1, 9, 7)$. Đoạn mã khi đó in ra gradient $(-3{,}8667;\ -5{,}3333;\ -8{,}1333)$ rồi $(5{,}8667;\ 39{,}3267;\ 39{,}9733)$, với trọng số cuối $(0{,}1000;\ -3{,}1993;\ -3{,}0840)$. Các số này trùng với lời giải tay ở Mục 6.4.2, kể cả hiện tượng phân kỳ sau lô 2. Hệ số 2 trong gradient của MSE tự xuất hiện từ đạo hàm của hàm bình phương, không cần khai báo.

12.8.2 Từ vòng lặp thủ công đến `torch.optim`

Hai dòng cập nhật trong `torch.no_grad()` và hai dòng `zero_()` lặp lại trong mọi chương trình huấn luyện, nên PyTorch đóng gói chúng thành **bộ tối ưu** (optimizer). Đoạn mã sau dùng lại `X`, `y` và `loss_func` ở trên, và cho đúng các trọng số đã tính:

Listing 12.16: Thay phần cập nhật thủ công bằng torch.optim.SGD

```
b = torch.tensor(0.3, requires_grad=True)
w = torch.tensor([0.2, 0.1], requires_grad=True)
opt = torch.optim.SGD([b, w], lr=0.1)
for start in range(0, len(X), batch_size):
    Xb, yb = X[start:start + batch_size], y[start:start + batch_size]
    opt.zero_grad()                                  # = w.grad.zero_()
    loss = loss_func(b + Xb @ w, yb)
    loss.backward()
    opt.step()                                       # = w -= alpha * w.grad
    print(round(b.item(), 4), [round(v, 4) for v in w.tolist()])
# 0.2325 [0.1095, -0.0135]
# 0.2622 [0.3021, 0.1634]
```

Khi cần, có thể thay `SGD` bằng các bộ tối ưu khác của `torch.optim` như `Adam` mà không phải sửa phần còn lại của vòng lặp.

12.9 Tóm tắt chương

- Lab 1: đa thức bậc 2 và Random Forest cho $R^2 \approx 0{,}85$ trên Ames Housing; cây không giới hạn độ sâu quá khớp.
- Lab 2: khi đặc trưng không tương quan với mục tiêu, mọi mô hình đều thất bại; R^2 âm nghĩa là thua mô hình đoán trung bình.
- Lab 3: nội suy tuyến tính cho dữ liệu có thứ tự; khi tự cài đặt SVD/PCA, vector riêng nằm theo **cột**, PCA phải chuẩn hóa tâm, và luôn đối chiếu với thư viện.
- Lab 4: xáo trộn chuỗi thời gian gây rò rỉ dữ liệu; R^2 cao trên giá chỉ phản ánh tính liên tục của chuỗi; mô hình phải thắng được dự báo ngây thơ mới có giá trị.
- Lab 5: duỗi phẳng ảnh 64×64 cho 4096 đặc trưng trên 858 mẫu huấn luyện; độ chính xác tốt nhất đạt 43,7%.

– Lab 6: cùng quy trình đạt 0,97 trên chữ số nhờ có nhiều dữ liệu hơn hẳn; PCA vừa giảm chi phí vừa cải thiện KNN.

– Lab 7: tách từ tiếng Việt, TF-IDF và hồi quy logistic cho kết quả tốt nhất.

– Lab 8: autograd của PyTorch tái hiện đúng từng con số của ví dụ mini-batch Gradient Descent tính tay ở Chương 6 và 7. Khi tự viết vòng lặp: xóa gradient sau mỗi bước, cập nhật trong `torch.no_grad()`, dùng `BCEWithLogitsLoss` thay cho sigmoid rồi `BCELoss`.

12.10 Bài tập

1. (Lab 1) Chạy lại Lab 1 với 5 lần chia dữ liệu khác nhau (`random_state` = 0–4). Báo cáo trung bình và độ lệch chuẩn của R^2 test cho từng mô hình. Thứ hạng các mô hình có ổn định không?

2. (Lab 1) Thay `SalePrice` bằng log(SalePrice) và chuẩn hóa đặc trưng bằng `StandardScaler`. Sau đó so sánh hệ số hồi quy với mức độ quan trọng hoán vị của Random Forest, tính bằng hàm `permutation_importance`.

3. (Lab 1) Bổ sung các cột `Neighborhood` (One Hot), `Year Built`, `Garage Cars` vào pipeline ở Chương 5. R^2 cải thiện bao nhiêu?

4. (Lab 2) Thay hai đặc trưng hiện tại bằng các cột khác của `Banking.xlsx` (sau khi đã loại bỏ họ tên, email, SSN). Dùng `DummyRegressor` làm mô hình cơ sở; đặc trưng nào thực sự có ích?

5. (Lab 4) Đánh giá mô hình VNM bằng `TimeSeriesSplit(n_splits=5)` với các cửa sổ 3, 7, 14, 30 phiên. Lần nào thắng được mô hình ngây thơ? Lặp lại với một mã cổ phiếu khác.

6. (Lab 3) Viết hàm `my_pca(X, k)` trả về tọa độ chiếu và tỉ lệ phương sai giải thích; kiểm tra trên tập `load_iris` rằng kết quả trùng với `PCA(k)` (sai khác dấu từng cột).

7. (Lab 5) Thử số thành phần PCA bằng 20, 40, 80 cho PCA + KNN, rồi liệt kê 5 cặp chữ bị nhầm nhiều nhất.

8. (Lab 6) Vẽ độ chính xác của PCA + KNN theo số thành phần 10, 20, 50, 100, 200. Từ đâu độ chính xác không tăng nữa?

9. (Lab 7) Gói `TfidfVectorizer` và `LogisticRegression` vào một `Pipeline`, chọn `C` và `ngram_range` theo tập Dev. Thêm bigram có giúp mô hình hiểu đúng cụm từ "miễn chê" (nghĩa tích cực dù chứa chữ "chê") không?

10. (Lab 8) Chạy đoạn mã autograd với `batch_size` bằng 1 và bằng 5, rồi đối chiếu với bài tập tính tay ở Chương 7. Chạy thêm 200 epoch, có và không xáo trộn dữ liệu (dùng `torch.randperm`); so sánh entropy chéo cuối cùng với Bảng 7.4.

11. (Lab 8) Bỏ hai dòng `zero_()` và chạy lại. Kiểm tra rằng gradient in ra ở lô 2 bằng tổng gradient của hai lô. Lặp lại thí nghiệm ở Bảng 12.9 với nhãn $y = 1$ và $z = -20$.

Tài liệu tham khảo của chương

1. Barocas, S., Hardt, M., & Narayanan, A. (2023). *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press.

2. Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.

3. De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education*, 19(3).

4. Dalal, N., & Triggs, B. (2005). Histograms of Oriented Gradients for Human Detection. *Proceedings of CVPR 2005*, 886–893.

5. Fama, E. F. (1970). Efficient Capital Markets: A Review of Theory and Empirical Work. *The Journal of Finance*, 25(2), 383–417.

6. Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts.

7. LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.

8. Liu, Y. (H.) (2019). *Python Machine Learning By Example* (2nd ed.), Chương 9: Stock Price Prediction with Regression Algorithms. Packt Publishing.

9. Raschka, S., Liu, Y. (H.), & Mirjalili, V. (2022). *Machine Learning with PyTorch and Scikit-Learn*, Chương 9: Predicting Continuous Target Variables with Regression Analysis (bộ dữ liệu Ames Housing). Packt Publishing.

10. Raschka, S. (2018). MLxtend: Providing machine learning and data science utilities and extensions to Python's scientific computing stack. *Journal of Open Source Software*, 3(24), 638.

11. Sarkar, D., Bali, R., & Sharma, T. (2018). *Practical Machine Learning with Python*, Chương 11: Forecasting Stock and Commodity Prices. Apress.